

Aprende. Crea. Domina.

Primera Edición

La Biblia de Shaders en Godot.

Todo lo que necesitas saber sobre shaders para mejorar los gráficos de tu juego, explicado de forma lineal, paso a paso.

IndieDev

GameDev

3DCG



Fabrizio Espíndola.



Jettelly®

La Biblia de Shaders en Godot.

Todo lo que necesitas saber sobre shaders para mejorar los gráficos de tu juego, explicado de forma lineal, paso a paso.

Fabrizio Espíndola

#IndieDev

#GameDev

#3DCG

Primera Edición 2026

La Biblia de Shaders en Godot, versión 0.1.2e.

Publicado por Jettelly Inc. ® Todos los derechos reservados. jettelly.com

77 Bloor St. West, Suite #663, Toronto ON M5S 1M2, Canadá.

Créditos.

Autor.

Fabrizio Espíndola

Diseño.

Pablo Yeber

Ilustración.

Hanbit Park

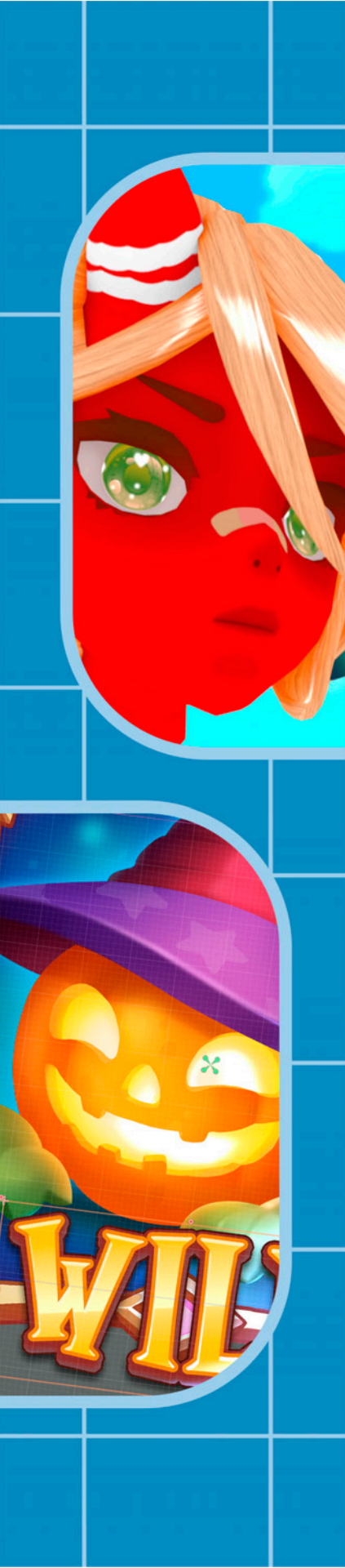
Editor.

Ewan Lee



Contenido.

Introducción.	6
Acerca del autor.	6
Prefacio.	6
Sección de Preguntas y Respuestas (Q&A).	7
Errata.	8
Piratería.	8
 1. Introducción a la composición de un mesh.	9
1.1. Propiedades.	10
1.2. Mesh Data.	13
1.3. Introducción a los espacios.	23
1.4. Configurando un objeto único.	27
1.5. Etapas de procesamiento y Rendering Pipeline.	35
1.6. Trabajando con el espacio global.	41
1.7. Introducción al espacio de tangente.	50
1.8. Implementando el espacio de tangente.	54
1.9. Interpolación lineal entre colores.	67
1.10. Introducción a las matrices.	73
1.11. Matrices internas (Built-Ins).	77
1.12. Implementado matrices personalizadas.	86
Resumen del capítulo.	94
 2. Iluminación y renderizado.	95
2.1. Propiedades de un material.	96
2.2. Introducción al modelo Lambertiano.	99
2.3. Implementando el modelo Lambertiano.	102
2.4. Vectores: Puntos vs. direcciones.	115
2.5. Introducción al modelo Blinn-Phong.	116
2.6. Implementando el modelo Blinn-Phong.	119
2.7. De iluminación lineal a sRGB.	127
2.8. Introducción al efecto Rim (Fresnel).	131
2.9. Implementado el efecto Rim.	134
2.10. Introducción al efecto Anisotrópico.	142



2.11. Implementando el modelo Ashikhmin-Shirley.	151
2.12. Sombreado hemisférico.	168
2.13. Implementando un ShaderInclude.	177
2.14. Implementación de mapa de normales.	182
Resumen del capítulo.	192

3. Figuras procedurales y animación por vértices. 193

3.1. Dibujando con funciones e inecuaciones.	195
3.2. Animación y modificación procedural.	208
3.3. Implementación del personaje procedural en GDSDL. .	213
3.4. Deformación de vértices y animación.	236
3.5. Agregando un efecto especular sobre la UI.	250
3.6. Introducción a las rotaciones.	263
3.7. Implementando una matriz de rotación.	267
3.8. Implementando cuaterniones.	276
Resumen del capítulo.	291

4. VFX avanzados y post-processing. 292

4.1. Post-Procesado y efecto Game Boy.	294
4.2. Portando un efecto desde Shadertoy.	309
4.3. Transparencia y distancia.	313
4.4. Ray Marching y texturas 3D.	329
4.5. Introducción a Stencil Buffer.	348
Resumen del capítulo.	366

Glosario. 367

Agradecimientos. 371

Acerca del autor.

Fabrizio Espíndola es un reconocido artista técnico y desarrollador chileno, originario de Santiago, con amplia experiencia en gráficos computacionales y una profunda pasión por la creación de shaders en entornos como Unity y, más recientemente, Godot.

Es autor de **La Biblia de Shaders en Unity** y de los libros **Visualizando Ecuaciones Vol. 1 y 2**, obras de referencia para desarrolladores y artistas técnicos interesados en dominar los gráficos procedurales. Su enfoque práctico y detallado ha ayudado a más de 20 mil profesionales en todo el mundo a mejorar su comprensión y habilidades en la creación de efectos visuales y sistemas de iluminación para videojuegos.

Con más de 19 mil seguidores en LinkedIn, Fabrizio se ha consolidado como una figura influyente dentro de la comunidad de artistas técnicos y desarrolladores. A través de sus publicaciones comparte conocimientos, tutoriales, assets y análisis sobre las últimas tendencias en gráficos en tiempo real, destacándose por su capacidad para simplificar conceptos complejos y hacerlos accesibles a una audiencia amplia.

Entre 2022 y 2025, Fabrizio se desempeñó como Artista Técnico Senior en Rovio, los creadores de Angry Birds, participando en el desarrollo de Angry Birds Bounce, título publicado en Apple Arcade en 2025. Previamente, colaboró con empresas como DeNA y Osmo, donde consolidó su experiencia en la creación y optimización de assets para distintos tipos de aplicaciones. En la actualidad, se dedica a tiempo completo a la creación de contenido educativo a través de Jettelly Inc.

Prefacio.

Hasta el año 2020, el mundo de los shaders seguía siendo, para muchos desarrolladores, un territorio poco explorado y difícil de abordar. Esta falta de material accesible y estructurado fue la principal motivación para comenzar a escribir sobre este tema. Como resultado de ese proceso nació **La Biblia de Shaders en Unity**, un proyecto que permitió a numerosos desarrolladores obtener una visión más clara y profunda sobre los gráficos en videojuegos y el funcionamiento interno del renderizado en tiempo real.

La Biblia de Shaders en Godot continúa esa misma línea, pero con un objetivo adicional: apoyar y fortalecer el ecosistema del desarrollo de videojuegos independientes. Godot

es un motor open source que promueve el acceso libre al conocimiento y la tecnología, valores que se alinean directamente con la filosofía detrás de este libro. La posibilidad de estudiar, modificar y comprender cada parte del pipeline gráfico convierte a Godot en una herramienta ideal para aprender shaders desde sus fundamentos.

Este libro fue escrito con la convicción de que Godot seguirá evolucionando y consolidándose como un motor de videojuegos relevante para las futuras generaciones. A través de sus capítulos, se busca no solo enseñar técnicas específicas de shaders, sino también fomentar una comprensión sólida de los principios matemáticos y gráficos que permiten a los desarrolladores crear efectos visuales de manera consciente, flexible y creativa.

Este proyecto se inició oficialmente en diciembre de 2024 (Godot 4.1), y su primera edición fue completada en enero de 2026 (Godot 4.6 rc1).

Sección de Preguntas y Respuestas (Q&A).

Si tienes alguna duda mientras lees este libro, puedes acceder a la sección de Q&A disponible en tu cuenta de Jettelly.

Para encontrarla, inicia sesión en jettelly.com, ve a “Mis Cursos” (My Courses), selecciona el libro sobre el cual deseas consultar y navega hasta la sección de Q&A. Allí podrás publicar preguntas relacionadas con el material tratado en este libro.

Este espacio te permite preguntar sobre implementaciones específicas, aclarar conceptos técnicos o solicitar explicaciones más detalladas sobre los temas discutidos. Los autores de Jettelly supervisan activamente esta sección y harán todo lo posible para ayudarte a resolver tus dudas y seguir progresando con el material.

Te animamos a utilizar la sección de Q&A siempre que algo no esté claro. Tus preguntas también pueden ayudar a mejorar futuras actualizaciones del libro y asistir a otros lectores que puedan encontrarse con los mismos desafíos.

Errata.

Al momento de escribir este libro, hemos tomado todas las precauciones necesarias para asegurar la fidelidad de su contenido. Aun así, debemos recordar que somos seres humanos y es muy probable que algunos puntos no estén bien explicados o que se hayan cometido errores ortográficos o gramaticales.

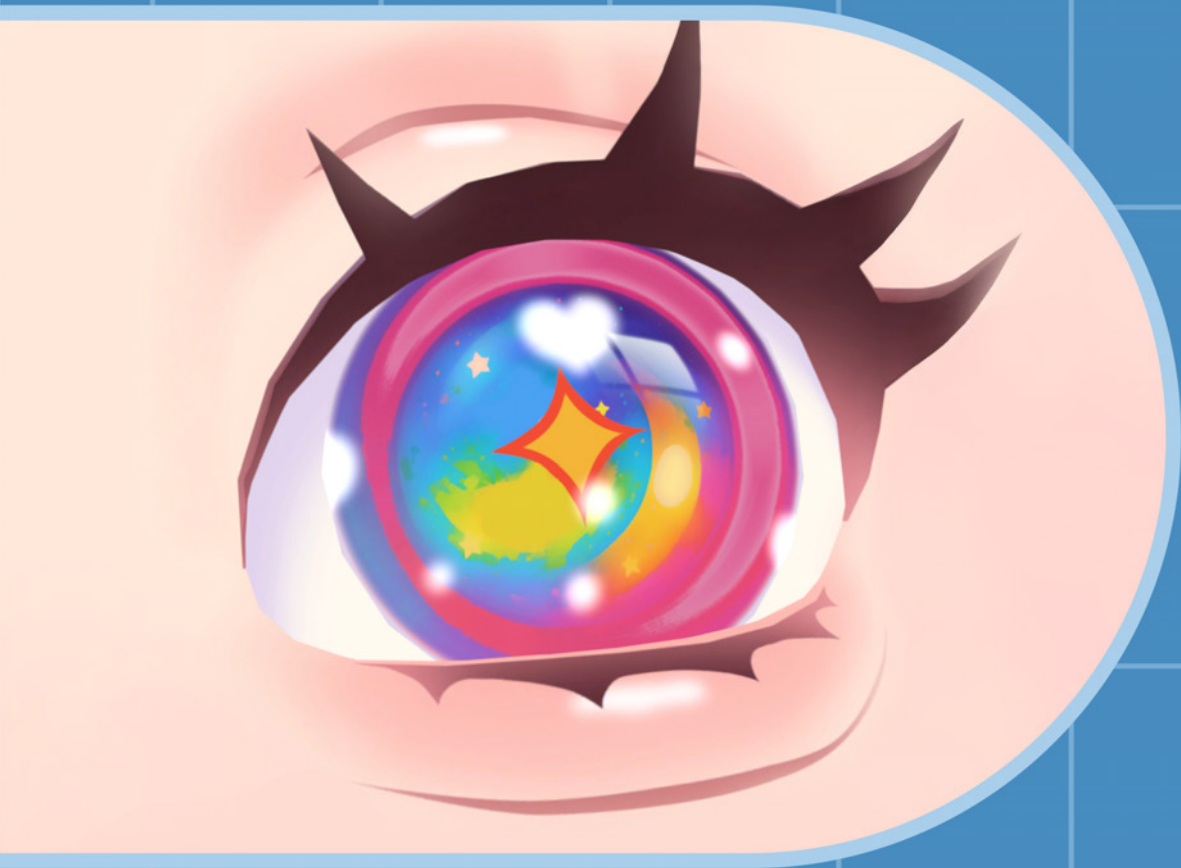
Si encuentras un error conceptual, de código u otro tipo, te agradeceríamos que enviaras un mensaje al correo contact@jettelly.com indicando en el asunto “GSB Errata”. De esta manera, estarás ayudando a otros lectores a reducir su nivel de frustración y contribuirás a mejorar cada versión del libro en futuras actualizaciones.

Asimismo, si consideras que sería útil agregar alguna sección de interés, puedes enviarnos un correo de todas formas y nosotros incluiremos esa información en próximas ediciones.

Piratería.

Antes de copiar, reproducir o facilitar este material sin nuestro consentimiento, recuerda que Jettelly Inc. es un estudio independiente y autofinanciado, por lo que cualquier práctica ilegal podría afectar nuestra integridad como equipo desarrollador.

Este libro se encuentra patentado bajo derechos de autor y tomaremos la protección de nuestras licencias de manera seria. Si encuentras este libro en una plataforma distinta de jettelly.com o detectas una copia ilegal, te solicitamos que te comuniques con nosotros al correo contact@jettelly.com (adjuntando el enlace si fuese necesario). De esta manera, podremos buscar una solución adecuada. Muchas gracias de antemano por tu colaboración. Todos los derechos reservados.



Capítulo 1

Introducción a la composición de un mesh.

En este capítulo daremos nuestros primeros pasos hacia el desarrollo de shaders, los cuales son fundamentales para todo lo que vemos en la pantalla de un computador. Pero, ¿qué es exactamente un shader? En Godot, se trata de un pequeño programa con extensión **.gdshader** que se ejecuta en la GPU, procesando cada píxel visible en la escena. Este tipo de programa nos permite dibujar y modificar visualmente los objetos mediante el uso de coordenadas, colores, texturas y otras propiedades derivadas de la geometría del objeto.

Los shaders aprovechan la arquitectura paralela de las GPU, que están compuestas por miles de núcleos pequeños capaces de ejecutar operaciones simultáneamente, lo que los hace ideales para tareas gráficas intensivas como el renderizado, los efectos visuales o la iluminación dinámica.

Antes de comenzar a escribir nuestros propios shaders, exploraremos los componentes que los conforman, así como las propiedades internas y externas que podemos manipular desde el código. Este conocimiento será la base para entender cómo funcionan, cómo interactúan con la escena y cómo podemos aprovecharlos para crear efectos visuales únicos e interesantes.

1.1 Propiedades.

Comenzaremos nuestra exploración atendiendo primero a la definición de un mesh según la documentación oficial de Godot:

“

Un mesh es un tipo de recurso que contiene geometría basada en matrices de vértices, divididas en superficies. Cada superficie contiene una matriz completamente separada y un material utilizado para dibujarla. En términos de diseño, es preferible un mesh con múltiples superficies a una sola superficie, porque los objetos creados en software de edición 3D generalmente contienen múltiples materiales.

https://docs.godotengine.org/en/stable/classes/class_mesh.html

”

Esta definición es consistente con las que encontramos en otras herramientas 3D, como Autodesk Maya o Blender, ya que comparten principios fundamentales en la forma de definir y manipular geometría 3D. En el contexto de desarrollo de shaders y efectos visuales, los

meshes contribuyen la base sobre la cual aplicamos materiales, texturas y todos aquellos recursos que dan vida y personalidad a nuestros modelos en pantalla.

Al analizar la descripción anterior, hallamos dos componentes clave que merecen mayor atención: las matrices de vértices y los materiales, siendo este último imprescindible al momento de renderizar un mesh. En el campo de los gráficos por computadora, debemos considerar tanto la información numérica que define un objeto; su geometría, coordenadas y atributos, como los recursos visuales que nos permiten verlo en pantalla. Comprender esta dualidad entre datos (geometría, coordenadas, atributos) y representación gráfica (materiales, texturas, shaders) es esencial en nuestro rol como artistas técnicos.

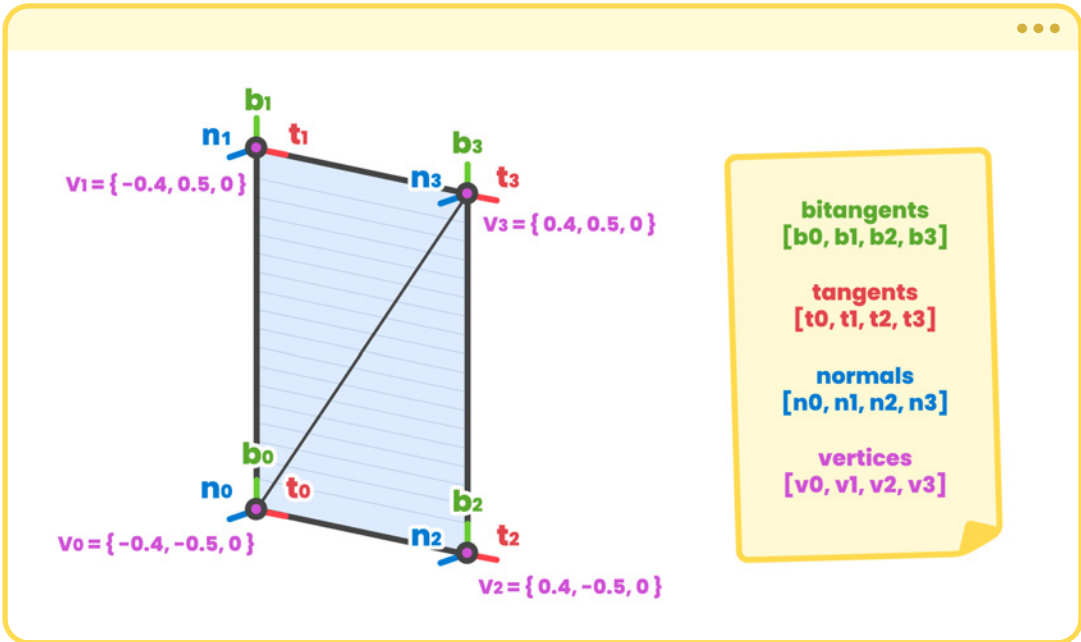
Cuando hablamos de matrices de vértices, nos referimos a la colección ordenada de puntos que definen la forma de un objeto. Un mesh no se limita únicamente a almacenar estos puntos, sino que también incorpora diversas propiedades que, en conjunto, configuran las caras poligonales (triángulos) que dan forma al modelo. Entre estas propiedades se encuentran:

- Vértices.
- Normales.
- Coordenadas UV.
- Bitangentes (o binormales).
- Tangentes.
- Color.

Para comprender la importancia de estas propiedades, imaginemos una hoja de papel. Si alguien nos pidiera que dibujemos en el “frente” de la hoja, ¿cómo determinaríamos cuál es el frente si ambas páginas lucen idénticas? Aquí es donde entran en juego las normales: gracias a ellas sabemos hacia dónde “apunta” la superficie, lo que nos indica qué lado se mostrará al observar el objeto. En aplicaciones 3D, como Maya o Blender, el software suele mostrar únicamente las caras cuyas normales apuntan hacia el observador, ignorando o no renderizando las que están invertidas, por razones de optimización.

Las tangentes y bitangentes también son relevantes, aunque en menor medida que las normales para efectos visuales básicos. Si continuamos con la analogía de la hoja, asumiendo que esta posee cuatro vértices y cuatro normales, también podremos definir cuatro tangentes y cuatro bitangentes asociadas a ellas. Juntas, las normales, tangentes y bitangentes forman un sistema de coordenadas local para la superficie del modelo, imprescindible al aplicar mapas de normales y efectos de iluminación avanzados. Esta base nos resultará fundamental al diseñar, optimizar y perfeccionar nuestros shaders y efectos visuales, ya

que nos permitirá comprender cómo interactúan la geometría y los recursos gráficos en la construcción de entornos y personajes tridimensionales.



(1.1.a Conceptualización de las propiedades de un mesh sobre una hoja de cuaderno)

La tangente suele alinearse con la dirección del eje U de las coordenadas UV, mientras que la bitangente se orienta en el eje V. Juntas, estas definen un sistema de referencia local en cada vértice, conocido como **tangent space**, que resulta esencial al momento de calcular efectos avanzados, como la aplicación de mapas de normales, la simulación de reflejos u otros efectos visuales complejos. Este espacio nos permite interpretar detalles adicionales de las texturas en relación con la superficie del mesh, brindándonos un mayor control sobre el resultado final.

Más adelante en este libro, dedicaremos una sección completa a la creación de efectos visuales utilizando espacio de tangente, las coordenadas UV y el color de los vértices. Por ahora, es importante destacar que, para que las propiedades de un mesh sean visibles en nuestro motor de desarrollo, debemos vincularlo a un proceso de renderizado y a uno o más materiales.

En Godot, la API encargada de procesar la geometría del mesh y mostrarla en la escena es el **RenderingServer**. Sin esta API, el mesh existiría únicamente como un recurso de datos en

la memoria, sin ninguna representación visual ni interacción con la cámara o las luces. Su manifestación en el software toma forma a través de un nodo denominado **VisualInstance3D**, el cual sirve como punto de partida para manipular objetos 3D. Uno de los hijos que más utilizaremos es **MeshInstance3D**, que gestiona las interacciones visuales con el mesh u el material de manera interna, permitiéndonos trabajar de forma más intuitiva sin lidiar directamente con los detalles técnicos del proceso de renderizado.

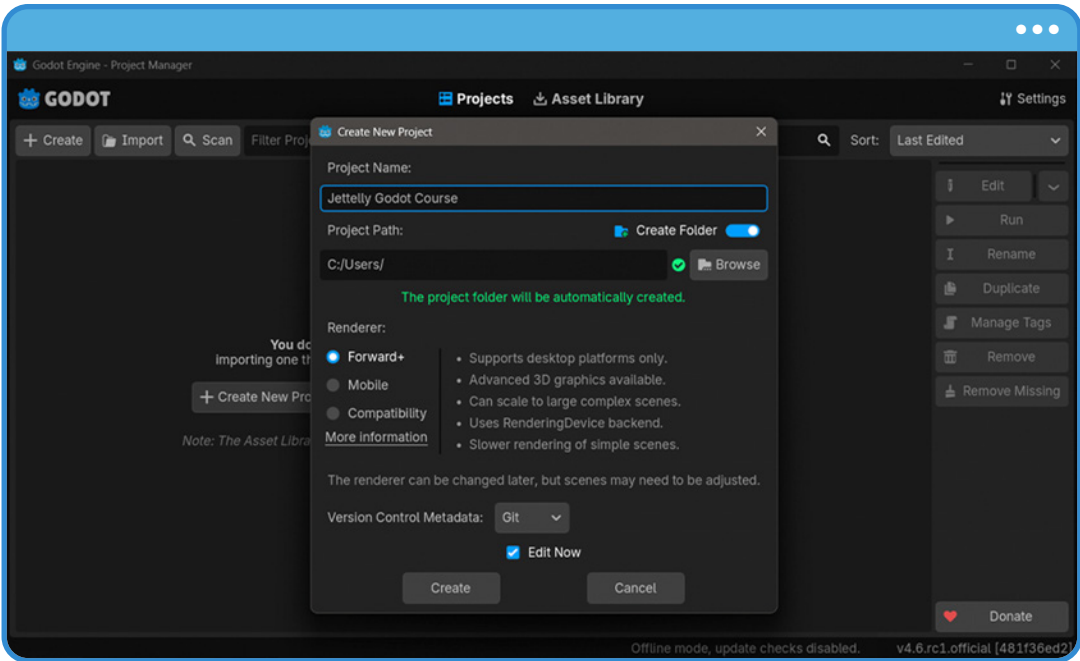
Desde el nodo **MeshInstance3D**, accedemos tanto al mesh como a su material directamente desde el **Inspector**. Un material en Godot es un recurso que define la forma en que la superficie de un objeto se renderiza, estableciendo cómo interactúa con la luz, las sombras, las texturas y otros efectos visuales. En su núcleo, el material contiene un shader, es decir, un programa que corre en la GPU. Godot utiliza una variante simplificada de GLSL (OpenGL Shading Language) para definir estos shaders, facilitando su edición y comprensión.

A través del shader, podemos acceder a las propiedades del mesh. Por ejemplo, la variable predefinida **VERTEX** nos permite manipular la posición de los vértices, lo que abre la posibilidad de alterar la malla directamente desde el shader. Asimismo, contamos con variables como **NORMAL**, **TANGENT** y **BITANGENT**, que facilitan la manipulación de las normales, las tangentes y las bitangentes, respectivamente. Comprender el funcionamiento de estas variables requiere visualizar los valores numéricos incluidos en el Vertex Data, ya que así podremos entender cómo las distintas propiedades trabajan en conjunto para producir el resultado que vemos en pantalla.

1.2 Mesh Data.

En esta sección realizaremos un ejercicio práctico para entender cómo las listas de vértices y sus propiedades se agrupan y, posteriormente, son leídas por la GPU a través de un shader para representar visualmente la geometría en la pantalla. Para ello, crearemos un **QuadMesh** en formato **.obj** mediante un editor de texto (como Notepad).

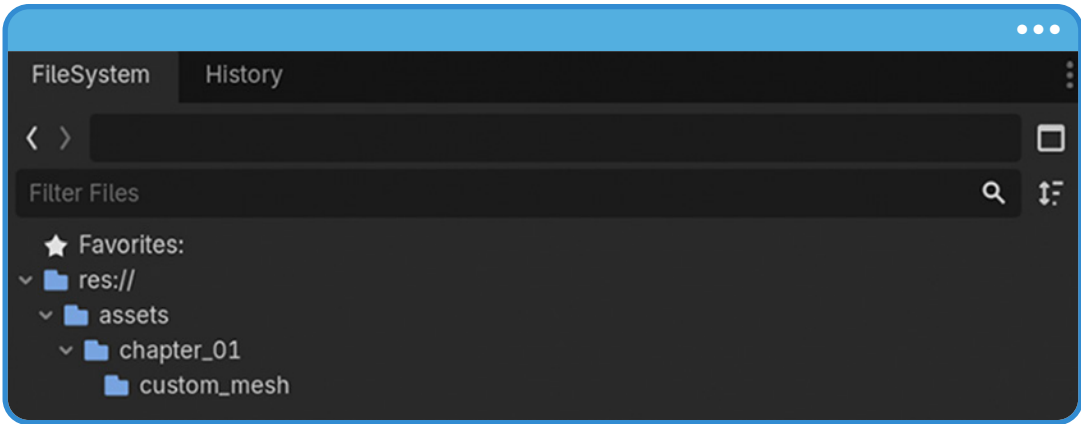
Primero, crearemos un nuevo proyecto en Godot, al que nombraremos **Jettelly Godot Course**, y nos aseguraremos de seleccionar **Forward+** como **Renderer**. Más adelante, profundizaremos en la arquitectura de un **Rendering Pipeline** y las diferencias entre tipos de renderizado. Por el momento, asumiremos que utilizaremos este proyecto a lo largo de todo el proceso de aprendizaje en una computadora de escritorio, con el objetivo de mantener una configuración de referencia coherente mientras avanzamos.



(1.2.a Creación de un proyecto en Godot 4.6 rc1)

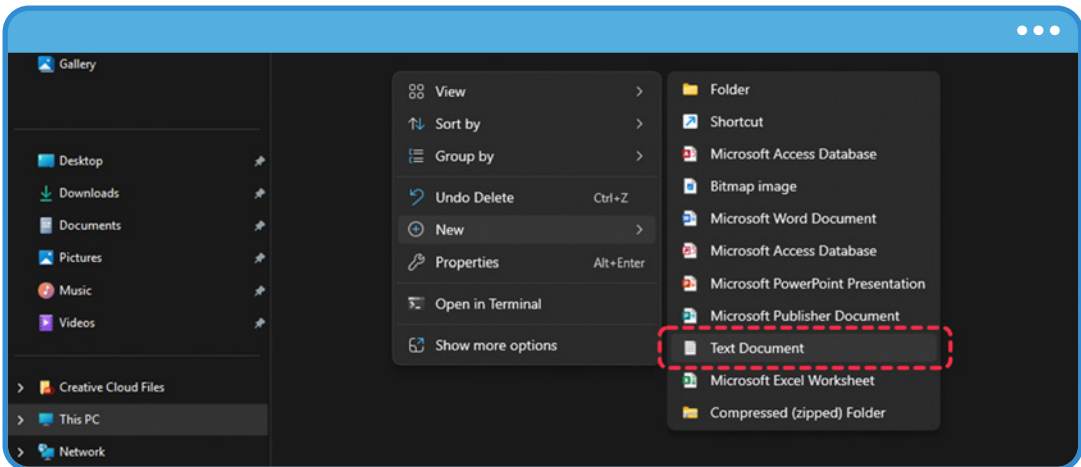
Una vez abierto nuestro proyecto, comenzaremos organizando nuestros recursos. Nos dirigiremos a la ventana **FileSystem** y crearemos nuestra primera carpeta, a la que llamaremos **assets**. Todos los archivos que generemos a partir de este momento los guardaremos en subcarpetas dentro de **assets**, esto nos brindará una mejor estructura de trabajo y facilitará el control de versiones. Por ejemplo, si utilizamos Git, podremos enlazar esta carpeta y hacer **Commits** exclusivos de sus cambios, evitando así modificaciones accidentales en la configuración principal del proyecto.

Dentro de assets, crearemos una nueva carpeta llamada **chapter_01**, y dentro de esta, otra carpeta denominada **custom_mesh**, donde alojaremos los archivos con lo que trabajaremos a continuación.



(1.2.b Estructura inicial del proyecto)

Tal como mencionamos, el **QuadMesh** que utilizaremos en este ejercicio será creado a través de un editor de texto. El proceso es bastante sencillo: nos dirigimos a la carpeta **custom_mesh** en nuestro sistema operativo y creamos un archivo de texto genérico. En Windows, este procedimiento se realiza haciendo clic derecho en la carpeta de destino y seleccionamos **New > Text Document**, tal como se ilustra a continuación:



(1.2.c Creación de un documento de texto en Windows)

A este documento lo nombraremos **custom_quad**, y cambiaremos su extensión de **.txt** a **.obj**. Este formato permite representar geometría 3D de forma estándar. Sin embargo, si regresamos a Godot después de este cambio, es probable que el archivo aparezca con un ícono de advertencia (una "X"), indicando que el archivo se encuentra "corrupto" o incompleto. Esto es normal, ya que aún no hemos agregado la información necesaria para

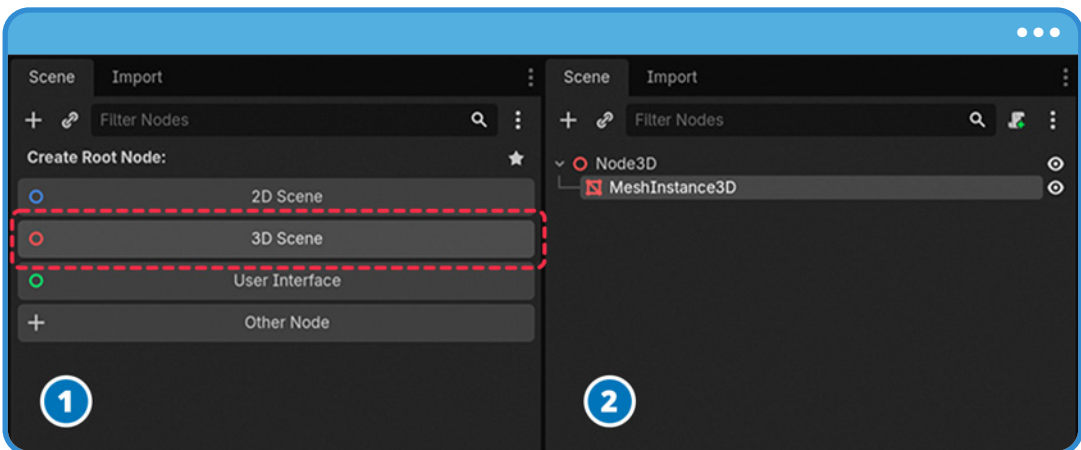
describir la geometría del QuadMesh. En breve, definiremos todos los datos necesarios para que Godot pueda interpretar correctamente el objeto.

Al exportar modelos 3D en formato **.obj**, se agrega también un archivo con extensión **.mlt** (Material Template Library) que contiene información sobre las propiedades de iluminación del objeto. Por el momento, no profundizaremos en este archivo, ya que nos concentraremos en la geometría misma y dejaremos que Godot asigne los valores de iluminación por defecto. Más adelante abordaremos cómo trabajar con materiales y cómo esta librería puede influir en la apariencia final de la geometría.

La estructura básica de un archivo **.obj** suele incluir:

- Definición de la posición de los vértices en local space.
- Definición de las coordenadas UV en uv space.
- Definición de las normales en local space.
- Parámetro de suavizado de ángulos.
- Grupo de caras del modelo.
- Definición de las caras del modelo.

Para visualizar correctamente el **QuadMesh** en Godot, necesitaremos definir esta información. De este modo, la GPU podrá proyectar la geometría sobre la pantalla. Antes de comenzar, crearemos una escena 3D en Godot eligiendo un **Node3D** como nodo padre y agregando un nodo **MeshInstance3D** como hijo. Este nodo nos permitirá asignar el mesh que acabamos de crear y renderizarlo en la escena.



(1.2.d Selección de una escena 3D)

Según la documentación oficial:

“

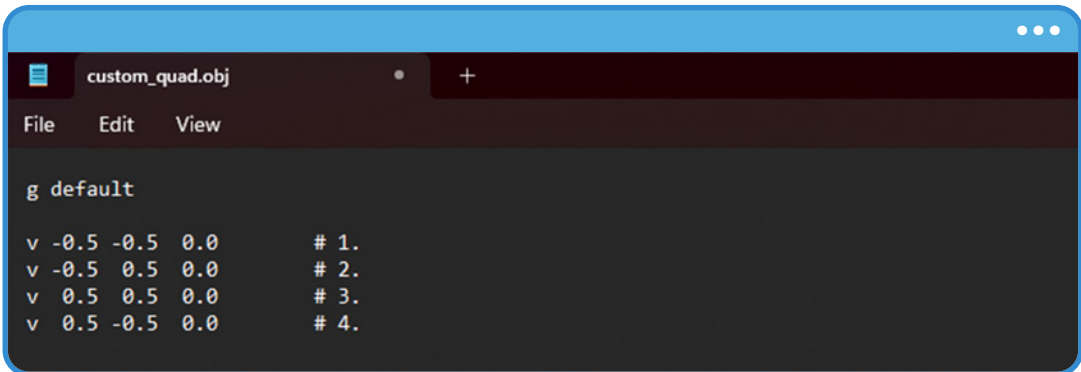
El **MeshInstance3D** es un nodo que toma un recurso **Mesh** y lo agrega al escenario actual creando una instancia de él. Esta es la clase que se usa con más frecuencia para renderizar geometría 3D y se puede usar para instancia un solo **Mesh** en muchos lugares.

https://docs.godotengine.org/en/stable/classes/class_meshinstance3d.html

”

Si seleccionamos el **MeshInstance3D** en la ventana **Scene** y nos dirigimos al **Inspector**, notaremos que su primera propiedad es **Mesh**, la cual nos permite asignar un recurso de malla directamente. Intentar hacerlo en este instante con nuestro **custom_quad.obj** no dará resultados, puesto que no hemos definido la información del objeto y este sigue marcado como corrupto. Para solucionarlo, abrimos el objeto con el editor de texto de preferencia y agregaremos las siguientes propiedades de este.

Dado que Godot trabaja con metros por defecto, utilizaremos un sistema de unidades donde un rango entre -0.5 y 0.5 representará un **QuadMesh** de 1 x 1 metros. Esto nos simplificará la visualización y el ajuste de la geometría eventualmente.

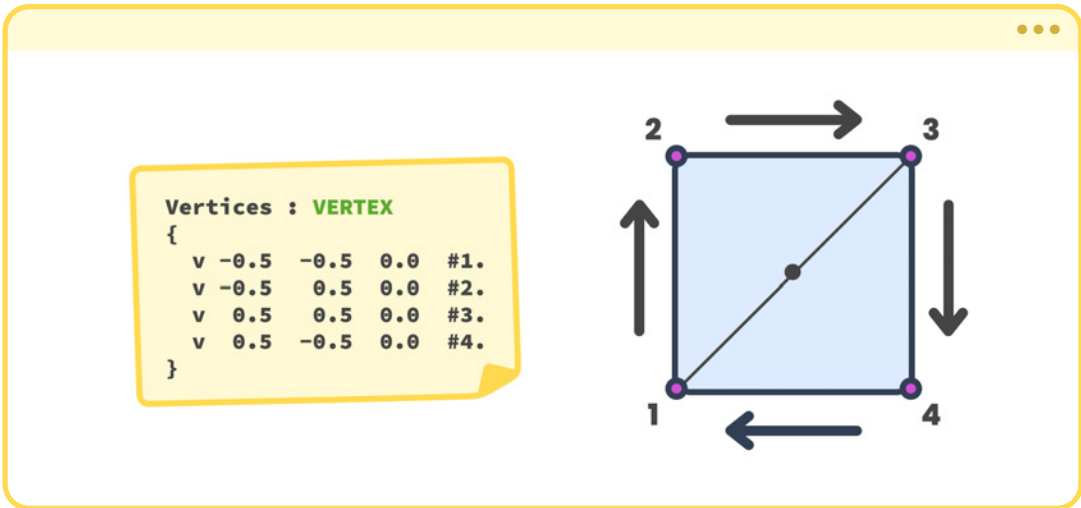


```

custom_quad.obj
File Edit View
g default
v -0.5 -0.5 0.0 # 1.
v -0.5 0.5 0.0 # 2.
v 0.5 0.5 0.0 # 3.
v 0.5 -0.5 0.0 # 4.
  
```

(1.2.e Definición de los vértices para un **QuadMesh**)

En la Figura 1.2.e, se han definido cuatro vértices, uno en cada esquina del **QuadMesh**, siguiendo el sentido de las agujas del reloj por conveniencia. Sin embargo, más adelante, cuando definamos las caras, utilizaremos el orden inverso para cumplir con las convenciones de Godot, donde el eje $+z$ apunta hacia la cámara. En los archivos **.obj**, los vértices se definen utilizando la letra **v**.

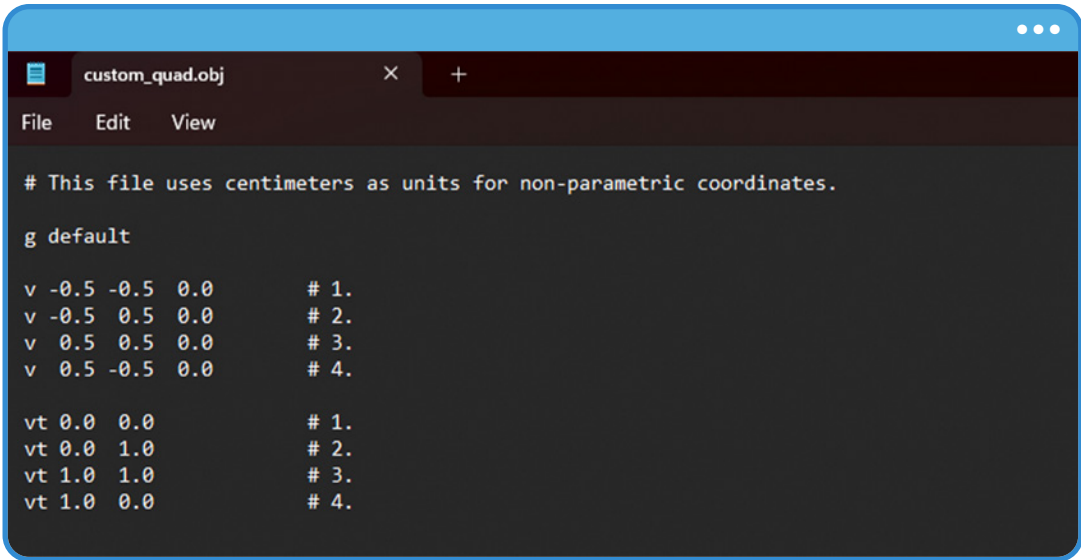


(1.2.f Los vértices del **QuadMesh** han sido definidos en dirección horaria)

La variable incluida **VERTEX** del shader nos dará acceso a estos valores en tiempo de ejecución, lo que eventualmente nos permitirá modificar su posición o manipular la geometría de formas más complejas. Aunque en este ejemplo sólo tenemos cuatro vértices, un modelo realista podría tener miles o millones, organizados en distintas superficies y matrices. Por ejemplo, si tuviéramos un “guerrero 3D” dividido en dos piezas: cuerpo y espada, posiblemente posea una matriz para el cuerpo completo y otra para la espada.

A continuación, definiremos las coordenadas UV, representadas en **.obj** por **vt**. Estas coordenadas, comprendidas entre $[0.0 : 1.0]$, nos permiten mapear texturas sobre la superficie. En la siguiente sección 1.3 de este libro hablaremos más en detalle sobre este tipo de espacio. De momento, para cada vértice del **QuadMesh**, asignaremos una coordenada UV considerando los siguientes valores:

- Vértice #1 (inferior izquierdo): $[0.0, 0.0]$.
- Vértice #2 (superior izquierdo): $[0.0, 1.0]$.
- Vértice #3 (superior derecho): $[1.0, 1.0]$.
- Vértice #4 (inferior derecho): $[1.0, 0.0]$.



```

custom_quad.obj
File Edit View

# This file uses centimeters as units for non-parametric coordinates.

g default

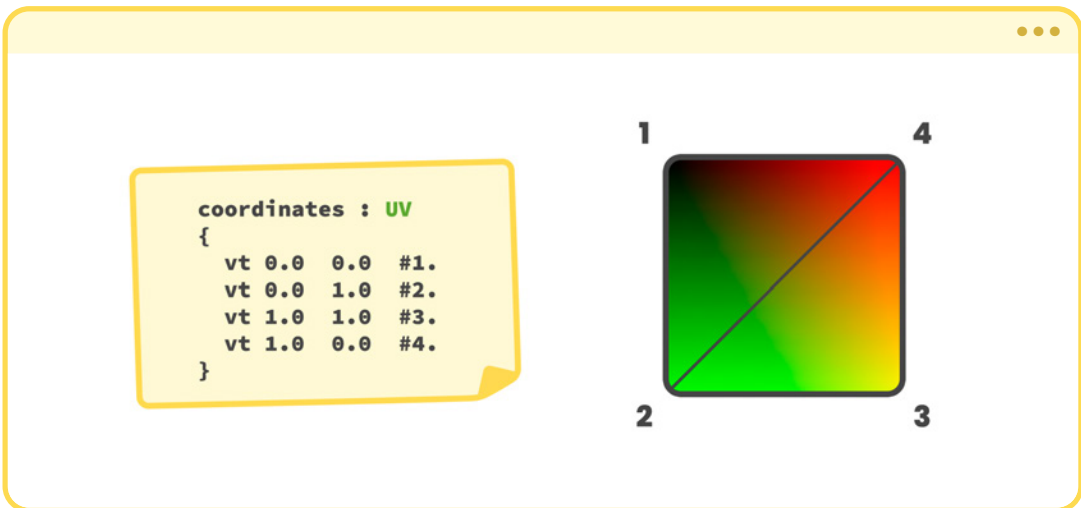
v -0.5 -0.5 0.0      # 1.
v -0.5 0.5 0.0       # 2.
v 0.5 0.5 0.0        # 3.
v 0.5 -0.5 0.0       # 4.

vt 0.0 0.0           # 1.
vt 0.0 1.0           # 2.
vt 1.0 1.0           # 3.
vt 1.0 0.0           # 4.

```

(1.2.g Definición de las coordenadas UV en un **QuadMesh**)

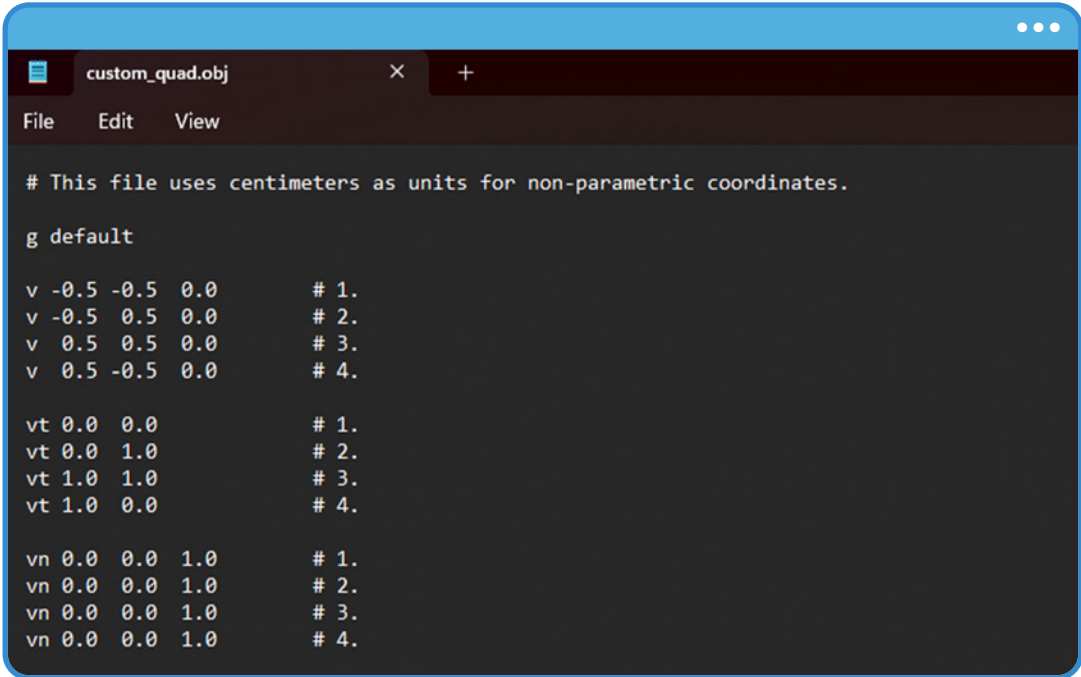
Estas coordenadas son esenciales para la proyección de texturas sobre la superficie del objeto. En el shader, la variable interna **UV** nos permitirá acceder y manipular estos valores, lo que resulta muy útil para generar efectos visuales, mover texturas a lo largo de la superficie, o incluso implementar técnicas de mapeo más complejas.



(1.2.h Visualización de las coordenadas UV)

El siguiente paso es definir las normales, representadas en **.obj** por **vn**. Las normales son vectores que indican la orientación de la superficie y son fundamentales para el cálculo de

la iluminación. En nuestro caso, definiremos las normales de forma que apunten hacia el observador. Esto significa que sus componentes XY serán 0.0, mientras que la componente Z será 1.0, apuntando hacia adelante.



```

custom_quad.obj
File Edit View

# This file uses centimeters as units for non-parametric coordinates.

g default

v -0.5 -0.5 0.0      # 1.
v -0.5  0.5 0.0      # 2.
v  0.5  0.5 0.0      # 3.
v  0.5 -0.5 0.0      # 4.

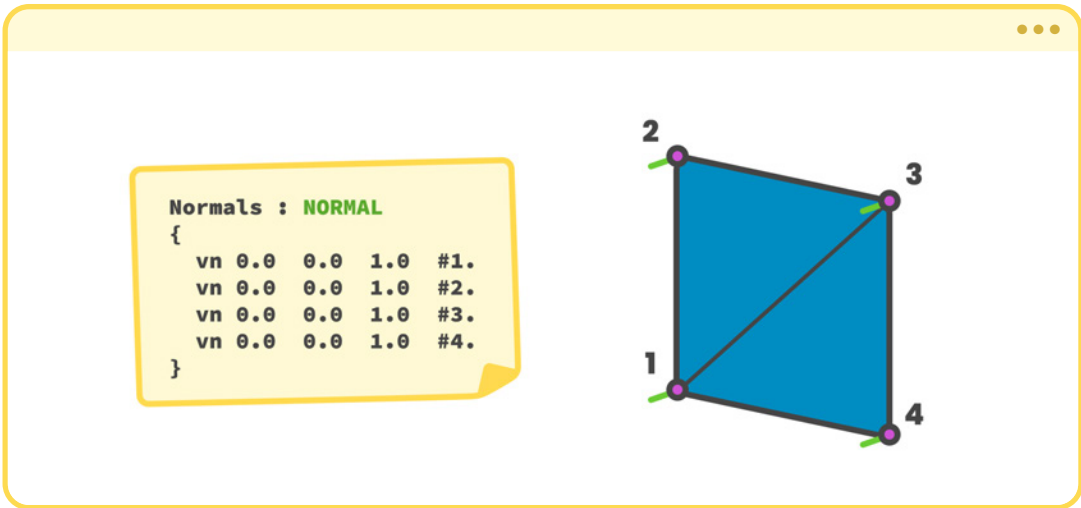
vt 0.0  0.0          # 1.
vt 0.0  1.0          # 2.
vt 1.0  1.0          # 3.
vt 1.0  0.0          # 4.

vn 0.0  0.0  1.0      # 1.
vn 0.0  0.0  1.0      # 2.
vn 0.0  0.0  1.0      # 3.
vn 0.0  0.0  1.0      # 4.

```

(1.2.i Definición de las normales en un Quad)

Cabe destacar que, la componente Z podría ser -1.0 de igual manera. No obstante, se ha determinado un valor positivo considerando los ejes espaciales en Godot y su iluminación direccional por defecto.

(1.2.j) Visualización de las normales normalizadas en un **QuadMesh**

La variable **NORMAL** en el shader nos permitirá acceder a esta información, y mediante su manipulación podremos alterar la forma en que la luz incide sobre la superficie, generando efectos muy variados, desde cambios en la apariencia de la iluminación hasta distorsiones visuales complejas.

Hasta este punto, hemos definido vértices, coordenadas UV y normales, es decir, la base que la GPU necesita para comprender la forma y orientación de la geometría. Aún nos falta especificar cómo se agrupan estos vértices para formar caras poligonales. En un archivo **.obj**, las caras se definen con la letra **f**, indicando el índice del vértice, el índice de la coordenada UV y el índice de la normal, en ese orden, separados por una barra.

```
# This file uses centimeters as units for non-parametric coordinates.

g default

v -0.5 -0.5 0.0      # 1.
v -0.5  0.5 0.0      # 2.
v  0.5  0.5 0.0      # 3.
v  0.5 -0.5 0.0      # 4.

vt 0.0  0.0          # 1.
vt 0.0  1.0          # 2.
vt 1.0  1.0          # 3.
vt 1.0  0.0          # 4.

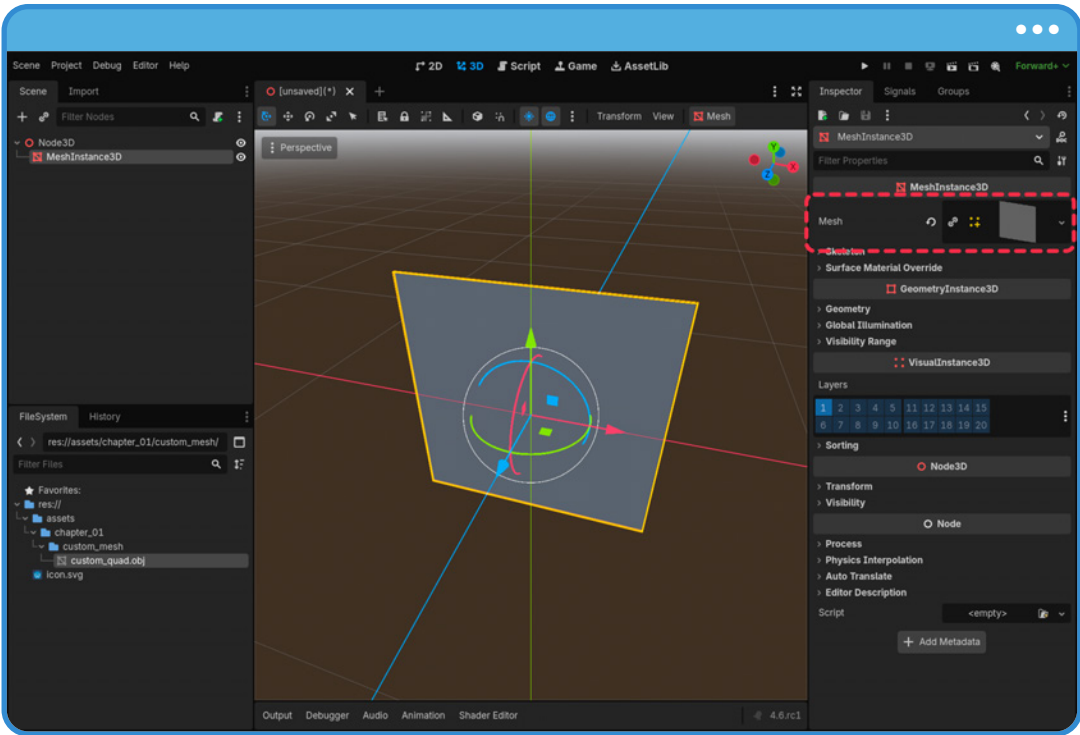
vn 0.0  0.0  1.0      # 1.
vn 0.0  0.0  1.0      # 2.
vn 0.0  0.0  1.0      # 3.
vn 0.0  0.0  1.0      # 4.

s off
g quad_mesh
f 4/4/4 3/3/3 2/2/2 1/1/1
```

(1.2.k Definición de las caras **f** del mesh)

Por ejemplo, una referencia del tipo `[4/4/4]` indica que la cara usa el vértice número 4, la coordenada UV número 4, y la normal número 4. El parámetro `s` indica el suavizado de normales; al estar en `off` no aplica ningún suavizado.

Tras definir las caras, guardaremos los cambios y regresaremos a Godot. Al hacer clic derecho sobre el **custom_quad.obj** en el **FileSystem** y seleccionar **Reimport**, el objeto se actualizará. Ahora podremos asignar el Mesh resultando al nodo **MeshInstance3D** sin problemas.



(1.2.1 Se ha asignado el **custom_quad.obj** como mesh en el nodo)

Con esto, hemos completado la definición básica de un **QuadMesh** a través de un archivo **.obj**, estableciendo sus vértices, coordenadas UV, normales y caras que forman la geometría. En capítulos posteriores iremos profundizaremos en la manipulación de estos datos, cómo se integran con materiales y espacios, y cómo podemos aprovecharlos para generar efectos visuales atractivos en nuestras escenas 2D y 3D.

1.3 Introducción a los espacios.

¿Qué se creó primero: el tiempo o el espacio?

Cuando trabajamos con shaders, uno de los conceptos más desafiantes para los desarrolladores gráficos es el manejo de los espacios. Estos definen cómo se representan y transforman elementos fundamentales del renderizado, como la posición, dirección y movimiento dentro del mundo virtual. Cada espacio tiene un propósito específico y comprender sus diferencias es esencial para desarrollar efectos visuales avanzados y mecánicas complejas.

Entre los espacios más comunes en gráficos computacionales encontramos:

- Espacio UV (uv space o texture space).
- Espacio de objeto (object space o local space).
- Espacio global (world space).
- Espacio de tangente (tangent space).
- Espacio de vista (view space o camera space).
- Espacio clip (clip space).
- Espacio de pantalla (screen space).

Antes de definir cada uno de estos espacios, realizaremos un ejercicio práctico que nos ayudará a comprender su naturaleza utilizando un ejemplo basado en la vida real.

Imaginemos nuestra posición en el mundo. En este momento, nos encontramos en el planeta Tierra, lo que significa que nuestra ubicación es relativa a este. Si decidimos permanecer completamente estáticos, nuestra posición con respecto a la Tierra no cambiará. No importa si el planeta se mueve a través de la Vía Láctea, nuestra posición sigue siendo la misma en relación con la Tierra.

Sin embargo, si quisiéramos calcular nuestra posición con respecto a la galaxia, la situación cambiaría. En este caso, al considerar el desplazamiento de la Tierra en el espacio, nuestra posición en la galaxia variaría con el tiempo. Esto introduce cambios en los valores de posición, donde los componentes XYZ estarían en constante actualización.

Este concepto de referencia relativa es clave para comprender cómo funcionan los diferentes espacios en gráficos computacionales. Dependiendo del sistema de referencia utilizado, una misma posición puede expresarse de manera distinta, lo que afecta directamente cómo interpretamos y manipulamos los datos dentro de un shader.

Este mismo principio se aplica a los diferentes espacios mencionados anteriormente. Por ejemplo, el espacio de objeto describe la posición de los vértices de un mesh en relación con el propio objeto, es decir, con su pivote o centro de masa. Si movemos el objeto en el mundo, sus vértices se desplazan junto con él, pero su posición relativa al centro del objeto permanece inalterada como pudimos ver previamente al definir los vértices en un archivo **.obj**, donde cada posición estaba expresada en función del objeto mismo y no de su entorno.

Pregunta existencial: Los átomos de nuestro cuerpo se mueven con nosotros, lo que podría llevarnos a pensar que están en espacio de objeto. Pero ¿en relación con qué? ¿Existe un centro absoluto en nuestro cuerpo? ¿Cómo “saben” los átomos dónde deben estar en todo momento? Estas preguntas reflejan la naturaleza relativa de los espacios en gráficos computacionales: la posición de un punto solo tiene sentido dentro de un marco de referencia determinado.

Siguiendo con las analogías, el espacio global describe posiciones relativas al mundo. En un software como Godot, este espacio se define dentro de la ventana gráfica (**Viewport**), que representa el mundo del juego. No importa si trabajamos en un entorno bidimensional o tridimensional; la cuadrícula que observamos en la vista de edición corresponde a este espacio global.

Por otro lado, el espacio UV se refiere a las coordenadas utilizadas para mapear texturas sobre una superficie. Sin embargo, para comprender este concepto, primero debemos entender qué es una textura. Desde un punto de vista gráfico, una textura es un conjunto de píxeles organizados en una estructura bidimensional, que juntos forman una imagen que podemos aplicar sobre un objeto.

Desde una perspectiva matemática, una textura se define como:

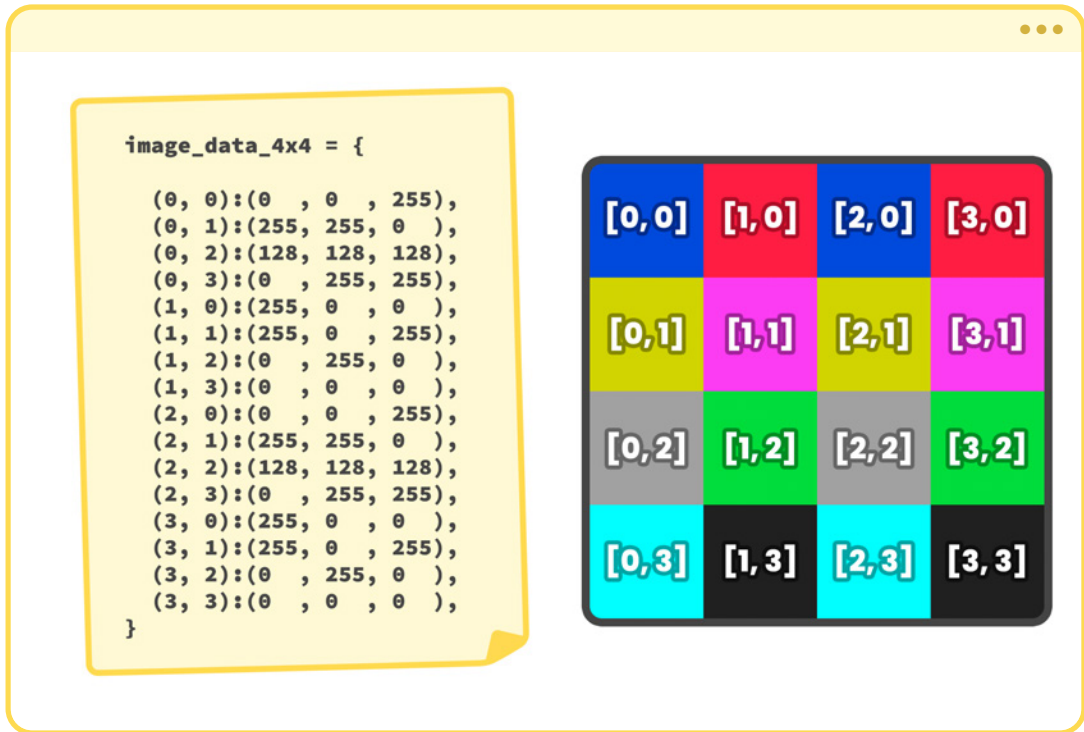
$$T(u, v) \rightarrow R^n$$

(1.3.a)

Donde T corresponde a la función de la textura, uv son sus coordenadas, y R^n define el espacio de salida, es decir, la cantidad de componentes de color. Por ejemplo, si $n = 3$, podríamos estar trabajando con una imagen **.jpg**, la cual contiene tres canales de color RGB. En cambio, si $n = 4$, podríamos referirnos a una imagen **.png**, que incluye un cuarto canal llamado alfa (A) para la transparencia.

Desde una perspectiva programática, una textura no es más que una estructura de datos organizada en una lista de valores. Cada elemento de esta lista representa un píxel con información de color y una posición determinada dentro de la imagen.

Para visualizarlo mejor, consideremos una textura de **4×4 píxeles**:

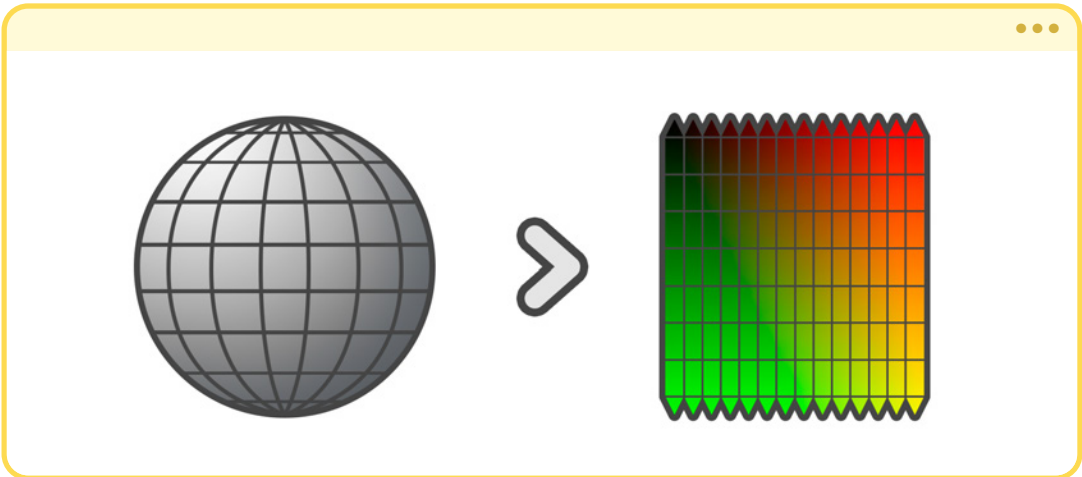


(1.3.b Visualización de la data de una textura)

Si observamos la referencia anterior, podemos notar que a la izquierda se muestra una representación estilo diccionario de una lista llamada `image_data_4x4`. En esta estructura, cada entrada almacena tanto la posición de un píxel como su valor de color en formato RGB. Por otro lado, a la derecha, encontramos la representación visual de estos datos: una textura compuesta por 16 texels (o píxeles en Photoshop) de color.

Es importante destacar que, aunque conceptualmente podemos representar una textura como un diccionario, este enfoque no es eficiente en términos de memoria ni rendimiento. Sin embargo, resulta útil para comprender la naturaleza de una textura y su relación con las coordenadas UV. En términos prácticos, las texturas en gráficos computacionales suelen almacenarse como arreglos unidimensionales de valores numéricos, optimizados para su uso en la GPU. Cada texel dentro de la textura se accede mediante sus coordenadas UV, lo que permite realizar operaciones como muestreo, interpolación y filtrado.

Las coordenadas UV están definidas en un espacio normalizado, donde el origen se encuentra en $(0.0_u, 0.0_v)$ y la esquina opuesta en $(1.0_u, 1.0_v)$. Este sistema de coordenadas permite mapear la información contenida en una textura, como la representación que vimos en la referencia 1.3.b.



(1.3.c Vértices sobre coordenadas UV)

Dado que el espacio UV es relativo a cada vértice del objeto, cuando movemos el modelo en el mundo, sus coordenadas UV permanecen fijas con respecto a su geometría, lo que permite que la textura se proyecte correctamente sobre la superficie. Además, las coordenadas UV están definidas en valores de coma flotante, lo que proporciona una alta precisión en el mapeo de texturas. Esta característica resulta fundamental al momento de definir texturas de alta densidad (Texel Density), permitiendo agregar detalles nítidos a personajes y objetos sin pérdida de resolución.

1.4 Configurando un objeto único.

Durante este capítulo, realizaremos un pequeño ejercicio en **Godot** para comprender mejor el funcionamiento del **espacio global** y su relación con las variables internas `NODE_POSITION_WORLD` y `MODEL_MATRIX`.

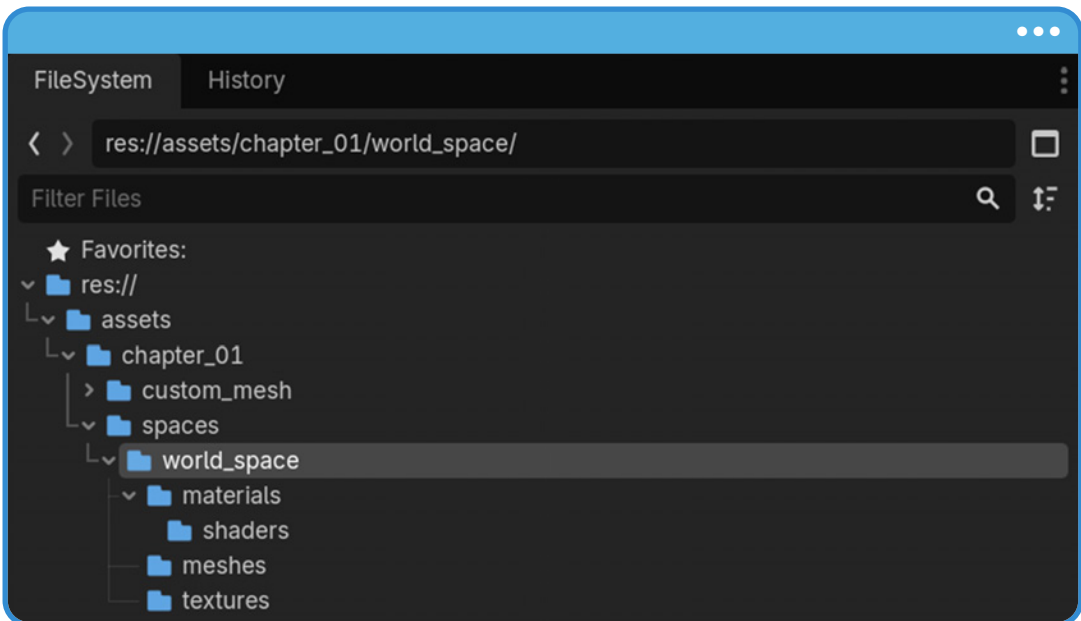
Como primer paso, organizaremos nuestro proyecto para facilitar la implementación. Para ello, seguimos estos pasos:

- 1 Dentro de nuestro proyecto, en **chapter_01**, crearemos una nueva carpeta llamada **spaces**.
- 2 Dentro de **spaces**, añadiremos otra carpeta con el nombre **world_space**.
- 3 Finalmente, dentro de **world_space**, organizaremos el contenido creando las carpetas:
 - a **materials**.
 - b **shaders**.
 - c **meshes**.
 - d **textures**.

Nota

Por asuntos prácticos, incluiremos la carpeta **shaders** dentro de **materials** en cada sección de este proyecto. Esto nos permitirá simplificar la estructura del proyecto y localizar rápidamente los recursos relacionados. No obstante, este paso es opcional.

Si hemos seguido los pasos correctamente, la estructura de nuestro proyecto debería verse de la siguiente manera:



(1.4.a Se ha incluido la carpeta **shaders** dentro de **materials**)

Para demostrar la utilidad de las coordenadas globales, realizaremos el siguiente ejercicio: A través de un **shader**, modificaremos la escala y el color de la textura de un objeto 3D en

función de su posición en el espacio global. Esto nos permitirá observar cómo un mismo objeto cambia dinámicamente a medida que se desplaza en la escena. Como resultado, si duplicamos el objeto y lo movemos a lo largo del entorno, cada instancia mostrará variaciones únicas en su apariencia sin necesidad de modificar su material manualmente.

Asumiendo que ya has descargado el material de apoyo para este libro, durante el ejercicio, utilizaremos el modelo **tree_mesh.fbx**, el cual importaremos a la carpeta **meshes** que creamos previamente. Además, nos aseguraremos de incluir las texturas **checker_tex** y **tree_tex** en la carpeta **textures**.

Nota

Recuerda que este libro posee un paquete descargable donde puedes encontrar todos los archivos necesarios para seguir los ejercicios de cada sección paso a paso. El paquete lo puedes descargar directamente desde tu cuenta de Jettelly, ingresando a <https://jettelly.com/>

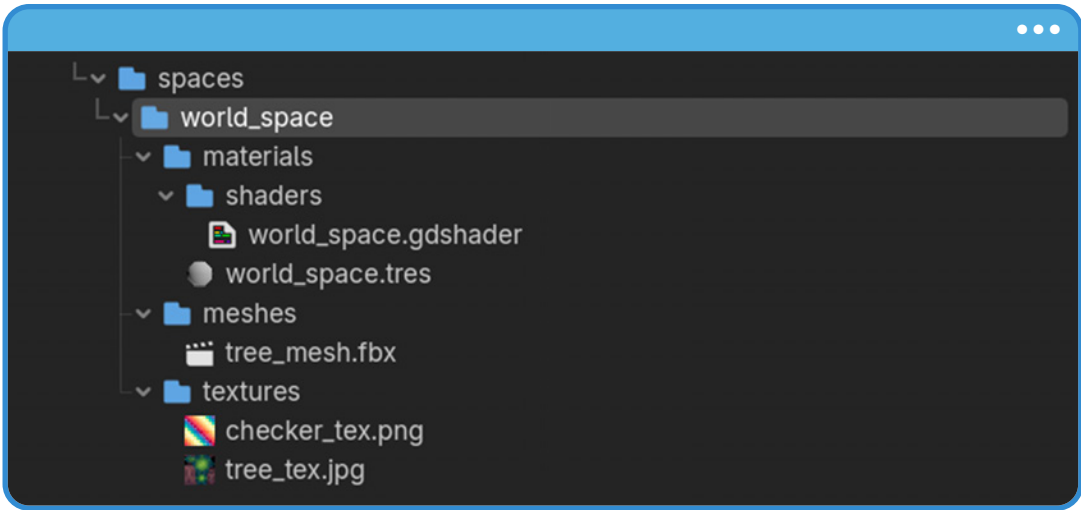
Es importante mencionar que, además del modelo y las texturas, también necesitaremos un **material** y un **shader**. En este último, implementaremos funciones tanto en la etapa de vértices **vertex()** como en la etapa de fragmento **fragment()** para modificar tanto el color del objeto como su escala. Para ello, sobre la carpeta **materials**, presionamos clic derecho y seleccionamos:

➤ Create New > Resource > ShaderMaterial.

Por razones prácticas, nombraremos a este material **world_space**. Repitiendo parte del proceso anterior, nos posicionamos sobre la carpeta **shaders**, presionamos clic derecho y seleccionamos:

➤ Create New > Resource > Shader.

Para este último, emplearemos el mismo nombre del **material**, así será más fácil identificar la conexión entre ambos. Si todo ha marchado correctamente, nuestro proyecto deberá lucir de la siguiente manera:



(1.4.b Se han incluido los archivos descargables al proyecto)

A continuación, comenzaremos con la implementación. Para ello, necesitaremos una única instancia del objeto **tree_mesh**.

Podemos realizar este proceso de la siguiente manera:

- 1 Nos ubicamos sobre el archivo **tree_mesh.fbx**.
- 2 Hacemos **clic derecho** y seleccionamos **New Inherited Scene**.

Esta acción nos permite crear una escena basada en el archivo original, manteniendo una relación de herencia, lo que facilita futuras modificaciones sin alterar el recurso base.

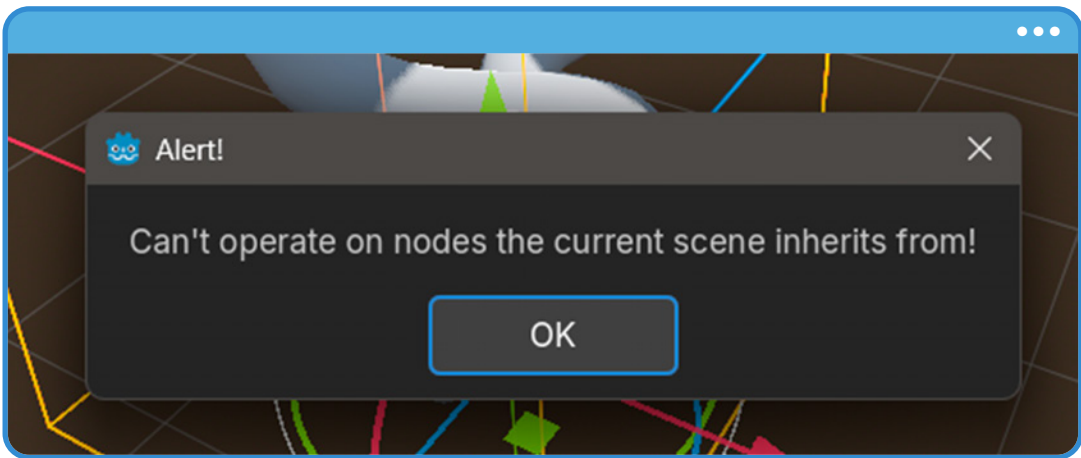
Al hacer esto, se abrirá una nueva escena en la ventana **Scene**, que contendrá los siguientes elementos:

- > Un nodo **Node3D** llamado **tree_mesh**.
- > Un nodo **MeshInstance3D** llamado **geo_tree_013**.

Para demostrar el uso de coordenadas globales, necesitaremos crear varias instancias de **tree_mesh** en diferentes posiciones dentro de la escena 3D.

Sin embargo, al observar el objeto en la ventana de edición, notamos que aparece resaltado en **color amarillo**. Esto indica que el nodo está bloqueado debido a la **herencia**, lo que impide su edición directa.

De hecho, si intentamos renombrar el nodo presionando **F2 (Rename)**, veremos el siguiente mensaje en pantalla:

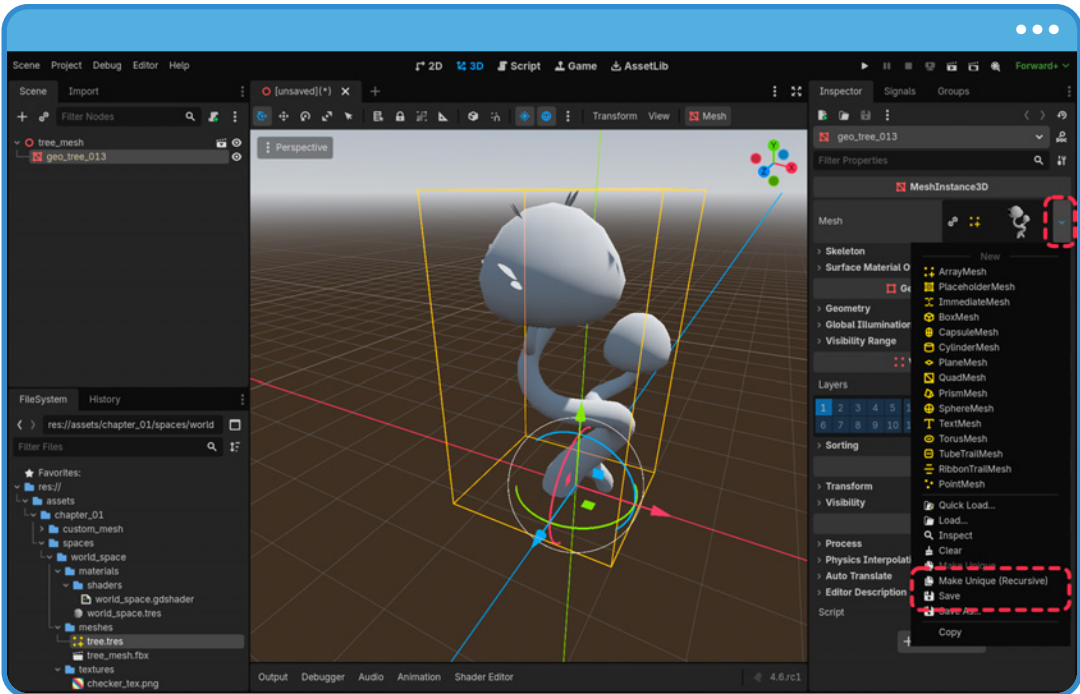


(1.4.c El nodo no se puede editar)

Para poder utilizar y personalizar el nodo según nuestras necesidades, debemos seguir estos pasos:

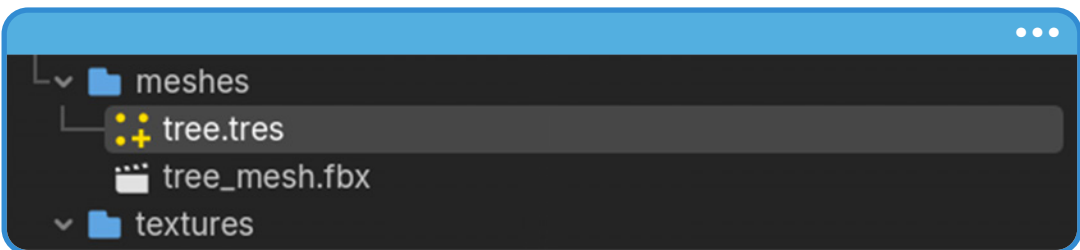
- 1 Seleccionamos el nodo dentro de la escena.
- 2 En el panel **Inspector**, buscamos la propiedad **mesh**.
- 3 Presionamos la opción **Make Unique (o Make Unique Recursive)** para desvincular el recurso heredado y convertirlo en una instancia independiente.
- 4 Guardamos el recurso en la carpeta **meshes** del proyecto.
- 5 Para facilitar su identificación, lo nombraremos **tree**.

De esta manera, tendremos un objeto independiente que podremos modificar libremente sin afectar el recurso original.



(1.4.d Se ha marcado al objeto como único)

Si hemos seguido correctamente todos los pasos, dentro de la carpeta **meshes** encontraremos un nuevo archivo independiente el cual podremos utilizar de manera flexible durante el ejercicio.



(1.4.e Se ha guardado un objeto único)

Durante esta etapa, será necesario crear una nueva escena que contenga múltiples instancias del objeto **tree**. Por lo tanto, nos dirigiremos a la ventana **Scene** e iniciaremos una nueva escena yendo a:

- Menú: Scene > New Scene.

O presionando el atajo **Ctrl+N** en nuestro teclado y, dado que emplearemos un objeto 3D, seleccionamos **3D Scene** como configuración.

Nota

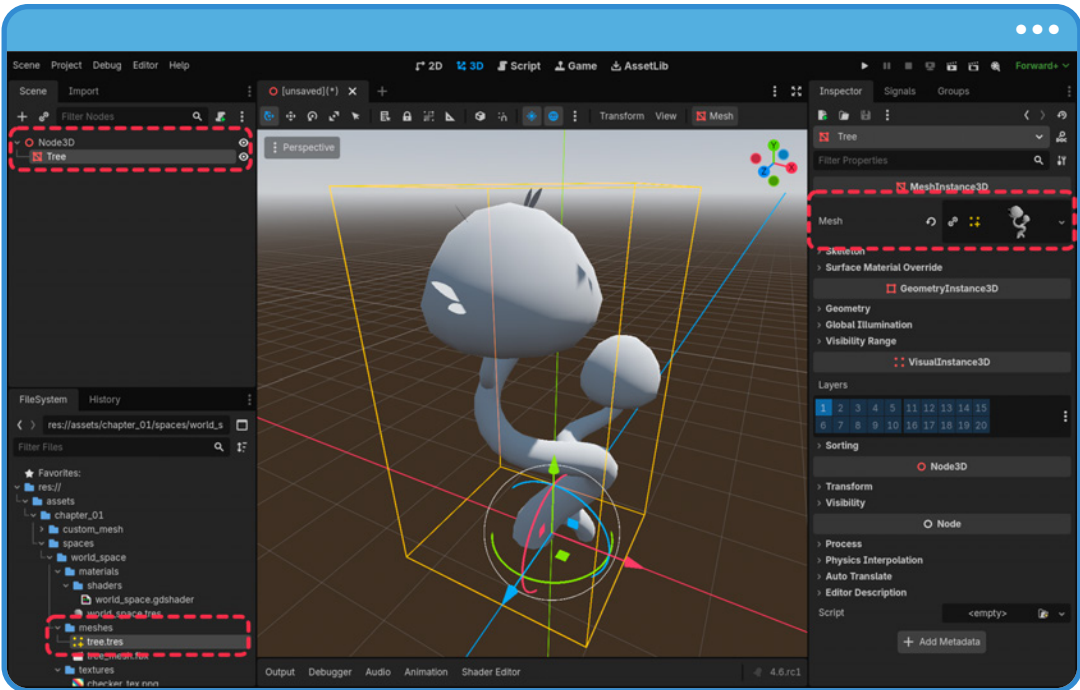
Según la documentación de **Godot**, el **Node3D** es el nodo base para objetos 3D y proporciona propiedades esenciales de transformación y visibilidad. Todos los demás nodos tridimensionales heredan de él. En un proyecto 3D, debemos utilizar un **Node3D** como nodo padre cuando queremos mover, escalar, rotar, mostrar u ocultar sus elementos hijos de manera organizada y eficiente. Para más información, puedes visitar el siguiente enlace: https://docs.godotengine.org/en/stable/classes/class_node3d.html

A continuación, agregaremos un nodo hijo de tipo **MeshInstance3D** siguiendo estos pasos:

- 1 Nos posicionamos sobre el nodo previamente creado.
- 2 Hacemos **clic derecho** y seleccionamos **Add Child Node**.
- 3 En la lista de nodos, buscamos y añadimos un **MeshInstance3D**.

Si bien podríamos utilizar un nodo **MultiMeshInstance3D** junto con un recurso **MultiMesh** para optimizar la creación de múltiples instancias, en este caso no es necesario. Dado que solo emplearemos algunas copias del objeto **tree** para demostrar la utilidad de las coordenadas globales, el uso de **MeshInstance3D** es suficiente y adecuado, ya que permite asignar directamente un recurso de tipo **Mesh**.

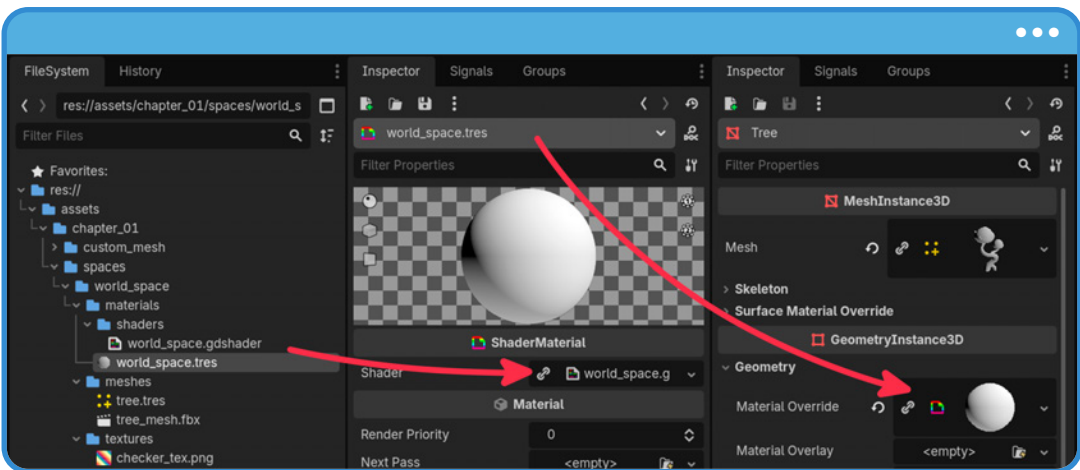
Ahora, simplemente seleccionamos el objeto **tree** y lo asignamos en la propiedad **Mesh** del nodo **MeshInstance3D(Tree)**, tal como se muestra a continuación:



(1.4.f Se ha asignado el objeto **tree** en la propiedad **Mesh**)

Con este último paso, hemos completado la configuración inicial necesaria para comenzar el desarrollo de nuestro **shader**. A continuación, realizamos los siguientes pasos:

- 1 Asignamos el shader **world_space** a su respectivo material.
- 2 Arrastramos el material en la propiedad **Material Override** del objeto **Tree** en la ventana **Scene**.



(1.4.g Se ha asignado el material al objeto **Tree**)

Si hemos seguido los pasos correctamente, el objeto **tree** debería aparecer en color blanco dentro de la escena, lo que indica que el shader se ha aplicado correctamente y está listo para su configuración.

1.5 Etapas de procesamiento y Rendering Pipeline.

Para comenzar esta sección, nos ubicamos sobre el shader **world_space** y lo abrimos haciendo doble clic sobre él. Al realizar esta acción, se abrirá la ventana **Shader Editor** ubicada a un costado de la ventana **Animation**, en la parte central baja de la interfaz.

En Godot, los shaders están estructurados en etapas de procesamiento, y dos de los métodos más comunes en su desarrollo son:

- **vertex()**: Responsable de la manipulación de los vértices antes del rasterizado.
- **fragment()**: Encargado del procesamiento de cada fragmento (píxel) en la etapa final del renderizado.

Estos métodos controlan una serie de procesos que ocurren en la GPU, entre los que se incluyen:

- Transformación de vértices.
- Mapeo de pantalla (Screen Mapping).
- Procesamiento de atributos.
- Procesamiento de color.
- Proyección.
- Cálculo de transparencias.
- Clipping.

De hecho, cuando abrimos un shader, su configuración inicial tiene la siguiente estructura:

```
shader_type spatial;

void vertex() {
    // Called for every vertex the material is visible on.
}

// El código continúa en la página siguiente.
```



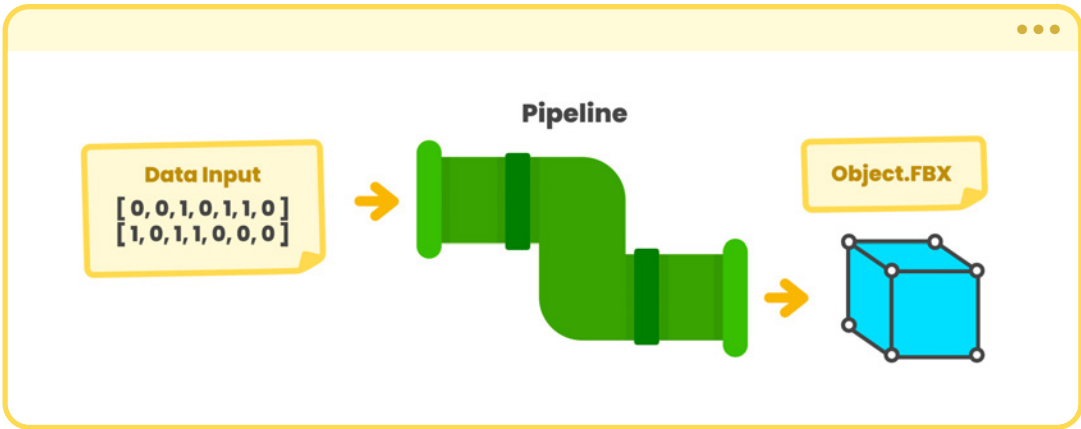
```
void fragment() {  
    // Called for every pixel the material is visible on.  
}  
  
//void light() {  
    // Called for every pixel for every light affecting the material.  
    // Uncomment to replace the default light processing function ...  
//}
```

En el código anterior, podemos notar la presencia de un tercer método llamado **light()**, el cual está comentado por defecto. Este método se ejecuta una vez por cada fuente de luz que afecta al objeto. Sin embargo, en esta sección nos enfocaremos únicamente en los métodos **vertex()** y **fragment()**, por lo que dejaremos el análisis de **light()** para más adelante.

Para entender el funcionamiento de los métodos **vertex()** y **fragment()**, es fundamental conocer la naturaleza del **Render Pipeline**. Una analogía que me gusta utilizar para explicar este concepto es la de las tuberías en Super Mario.

En la sección 1.2 de este libro, desarrollamos un **QuadMesh** utilizando coordenadas definidas en un archivo **.obj**. Como habrás notado, todas las figuras, ya sean bidimensionales, tridimensionales o incluso colores, existen inicialmente solo como datos en la memoria. Esto nos lleva a una pregunta clave: ¿Cómo se transforman estos datos en objetos visibles y coloridos en la pantalla? La respuesta está en el Render Pipeline.

Podemos imaginar este concepto como un proceso en cadena donde un objeto poligonal viaja a través de diferentes etapas hasta ser finalmente renderizado en pantalla. Es como si el objeto estuviera atravesando una serie de tuberías, transformándose paso a paso hasta llegar a su destino final: la pantalla de nuestro ordenador.



(1.5.a Datos siendo transformados en un Cubo tridimensional)

Godot utiliza un Render Pipeline unificado, a diferencia de Unity, donde cada Render Pipeline (URP, HDRP o Built-in RP) tiene sus propias características y modos de renderizado. En Unity, la elección del pipeline afecta directamente las propiedades de los materiales, las fuentes de luz, el color y, en general, todas las operaciones gráficas que determinan la apariencia y optimización de los objetos en pantalla.

En el caso de Godot, su sistema de renderizado está basado en **OpenGL** y cuenta con distintos **modos de rendering**. Estos modos fueron introducidos al comienzo del libro, cuando configuramos nuestro proyecto, y son los siguientes:

- Forward+.
- Forward Mobile.
- Compatibility.

Comenzando con **Forward+**, si prestamos atención a la documentación oficial en Godot, encontraremos la siguiente información:

“

Este es un renderizador en **Forward** que utiliza un enfoque **clusterizado** para la iluminación. La iluminación **clusterizada** emplea un compute shader para agrupar las luces en una cuadrícula 3D alineada con el frustum. Luego, en tiempo de renderizado, los píxeles pueden consultar qué luces afectan la celda de la cuadrícula en la que se encuentran y solo ejecutar los cálculos de iluminación para aquellas luces que puedan influir en ese píxel.

<https://docs.godotengine.org/en/stable/tutorials/rendering/renderers.html>

”

¿Qué significa esto? El término enfoque **clusterizado** se refiere a un método de optimización en los cálculos de iluminación. En lugar de calcular la iluminación para cada píxel u objeto de la escena de manera individual, este método divide el espacio 3D en **clusters** (pequeñas regiones espaciales). Luego, en tiempo de renderizado, cada píxel consulta únicamente las luces que afectan su cluster, evitando cálculos innecesarios y mejorando el rendimiento, especialmente en escenas con iluminación dinámica compleja.

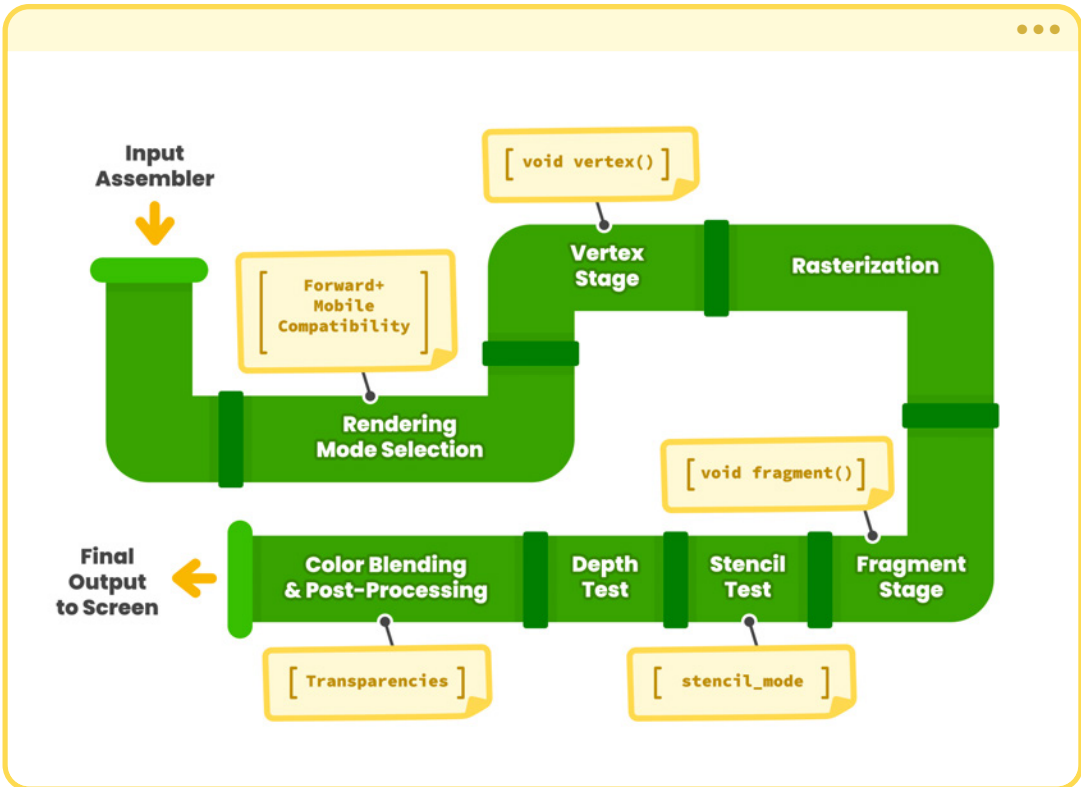
Debido a esta optimización, el modo **Forward+** es utilizado principalmente en proyectos para PC, donde se requiere alta calidad gráfica y múltiples fuentes de luz. En contraste, otros modos de renderizado en Godot están diseñados para diferentes necesidades:

- **Forward Mobile:** Optimizado para dispositivos móviles, prioriza el rendimiento sobre la calidad gráfica.
- **Compatibility:** También basado en Forward Rendering, pero pensado para GPUs antiguas que no son compatibles con Vulkan.

Nota

Dada las especificaciones y limitaciones de OpenGL, no abordaremos en detalle todos los aspectos del sistema de renderizado en Godot. Si deseas obtener más información sobre los distintos modos de rendering y sus características, puedes visitar el siguiente enlace: https://docs.godotengine.org/en/stable/contributing/development/core_and_modules/internal_rendering_architecture.html

Ahora bien, ¿cuál es la relación entre estos modos de renderizado y los métodos **vertex()** / **fragment()**? Para comprenderlo mejor, prestemos atención a la siguiente referencia:



(1.5.b Render Pipeline con algunos de los procesos más importantes)

El ensamblador de entrada (**Input Assembler**) es el componente encargado de recibir y organizar la información de los objetos (meshes) presentes en la escena. Este proceso incluye datos como:

- Vértices
- Índices
- Normales
- Tangentes
- Coordenadas UV
- Entre otros atributos geométricos

Toda esta información se almacena como valores numéricos y se organiza de manera estructurada para que el Render Pipeline pueda interpretarla correctamente y, finalmente, generar los objetos tridimensionales en pantalla.

La elección del modo de renderizado (**Rendering Mode Selection**) determina qué técnica de renderizado se utilizará en el proyecto (Forward+, Forward Mobile o Compatibility). Esta selección tiene un impacto directo en el rendimiento y la calidad visual del renderizado, ya que define cómo se procesan los gráficos y qué API de renderizado se empleará (Vul-

kan, Direct3D, OpenGL ES, etc.). Dependiendo del modo elegido, ciertos efectos gráficos y optimizaciones estarán disponibles o limitados, lo que afecta la forma en que los shaders interactúan con la escena.

La etapa de procesamiento de vértices, representada por el método **vertex()**, es una de las fases fundamentales en un shader. Durante esta fase, la información obtenida del **Input Assembler** se transforma a través de distintos espacios de coordenadas, siguiendo esta secuencia:

- 1 Espacio de objeto (**object space**): Es el espacio local del modelo, donde los vértices están definidos en relación con su propio pivote.
- 2 Espacio global (**world space**): Aquí, los vértices se transforman en relación con la posición del objeto dentro de la escena.
- 3 Espacio de vista (**view space**): Se determina la posición de los vértices con respecto a la cámara, es decir, cómo deben verse desde el punto de vista del observador.
- 4 Espacio clip (**clip space**): Se proyectan los vértices en un sistema de coordenadas homogéneas, donde se preparan para la siguiente fase.
- 5 Espacio de dispositivo normalizado (NDC - Normalized Device Coordinates): Se realiza la división por W, lo que normaliza las coordenadas en un rango entre -1.0 y 1.0 en cada eje.
- 6 Espacio de pantalla (**screen space**): Finalmente, los vértices se asignan a coordenadas de píxeles en la pantalla, donde el objeto es rasterizado y renderizado.

Este proceso permite que la escena tridimensional sea correctamente proyectada en una superficie 2D, transformando los modelos poligonales en imágenes visibles en pantalla.

Una vez transformados los vértices al espacio de pantalla, el siguiente paso en el Render Pipeline es la **rasterización**. Durante esta etapa, las primitivas geométricas generadas a partir de los vértices, triángulos, líneas o puntos, se convierten en fragmentos (posibles píxeles) en la pantalla. Este proceso define qué partes del objeto serán visibles y cómo se distribuirán en la imagen final.

Además, en esta fase se aplica el **Depth Testing** (prueba de profundidad), un mecanismo clave que determina la visibilidad de los fragmentos en función de su distancia relativa a la cámara. Esta prueba compara la profundidad de cada fragmento con la información

almacenada en el **Z-Buffer** (o **Depth Buffer**), descartando aquellos que quedan ocultos detrás de otros objetos.

Después de la rasterización, cada fragmento pasa a la etapa de fragmento **fragment()**, donde se calcula su color final antes de convertirse en un píxel en pantalla. En esta fase se determinan efectos visuales clave como iluminación, texturas, transparencia y reflejos. Además, cada fragmento es procesado de forma independiente en la GPU, lo que permite aplicar cálculos complejos de manera eficiente.

Nota

Cabe destacar que el método **light()** no reemplaza al método **fragment()**, sino que se ejecuta después de este, pero solo en presencia de una fuente de luz que afecte al fragmento actual. Profundizaremos en su funcionamiento a lo largo del capítulo 2.

Finalmente, se aplica la etapa de **Blending** y **Post-Processing**, donde se incorporan efectos visuales antes de renderizar la imagen en pantalla. Aquí se procesan elementos como transparencias, mezcla de colores, bloom, anti-aliasing, corrección de color, etc. Estos efectos refinan la calidad visual y optimizan la apariencia final de la escena.

Ahora que entendemos la relación entre los distintos modos de rendering y los métodos del shader, continuaremos con la implementación de **world_space**.

1.6 Trabajando con el espacio global.

Un atributo clave de nuestro shader es la directiva **shader_type spatial**, la cual indica que el shader se aplicará a objetos tridimensionales.

Nota

Si quisiéramos crear shaders para interfaz de usuario (UI) o sprites, deberíamos usar **shader_type canvas_item**. Para efectos de partículas, en cambio, utilizaríamos **shader_type particles**. Puedes encontrar más información al respecto en el siguiente enlace: https://docs.godotengine.org/en/stable/tutorials/shaders/shader_reference/index.html

Dado que nuestro objeto **Tree** tiene un color blanco por defecto, comenzaremos agregando una **textura** a nuestro shader. Para ello, añadimos la siguiente línea de código:

```

shader_type spatial;

uniform sampler2D _MainTex : source_color;

void vertex()
{
    // Called for every vertex the material is visible on.
}

```

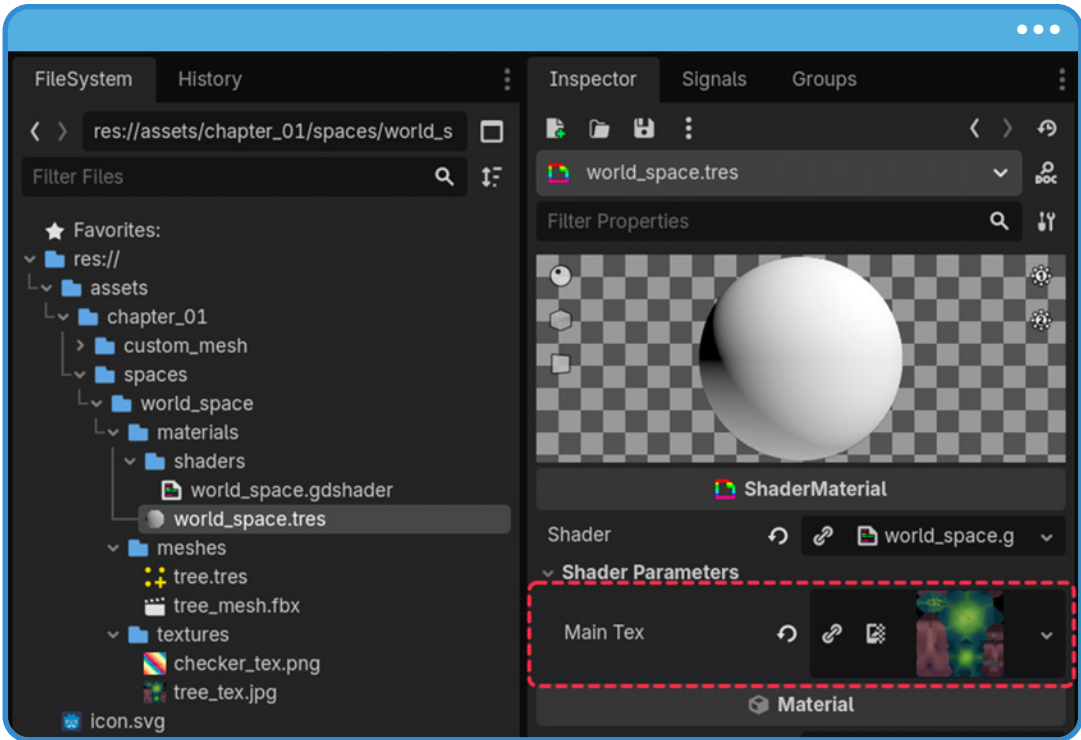
En este código, declaramos una nueva variable de tipo **sampler2D** llamada **_MainTex**, la cual posee el **hint source_color**. Esta variable está definida de manera global, lo que significa que puede ser utilizada en cualquier método dentro del shader.

Un **sampler2D** hace referencia a una textura bidimensional, la cual podemos utilizar en el shader. La palabra clave **uniform** indica que la textura será asignada externamente desde el **material**. Por otro lado, **source_color** define que la textura corresponde a un mapa de albedo (color base), es decir, una textura utilizada para determinar el color principal del material.

Nota

source_color es solo uno de los muchos hints que podemos utilizar en un shader. Por ejemplo, podemos emplear **hint_normal** para definir un mapa de normales. Si deseas ver la lista completa de hints disponibles, puedes consultar el siguiente enlace: https://docs.godotengine.org/en/latest/tutorials/shaders/shader_reference/shading_language.html

De hecho, si guardamos los cambios y seleccionamos el **material** desde la ventana **FileSystem**, podremos ver que en la sección **Shader Parameters** aparece una nueva propiedad llamada **"Main Tex"**. Esta propiedad hace referencia directa a la variable **_MainTex** que acabamos de declarar en el shader, permitiéndonos asignar una textura desde el editor de Godot.



(1.6.a Se ha declarado una nueva propiedad de textura)

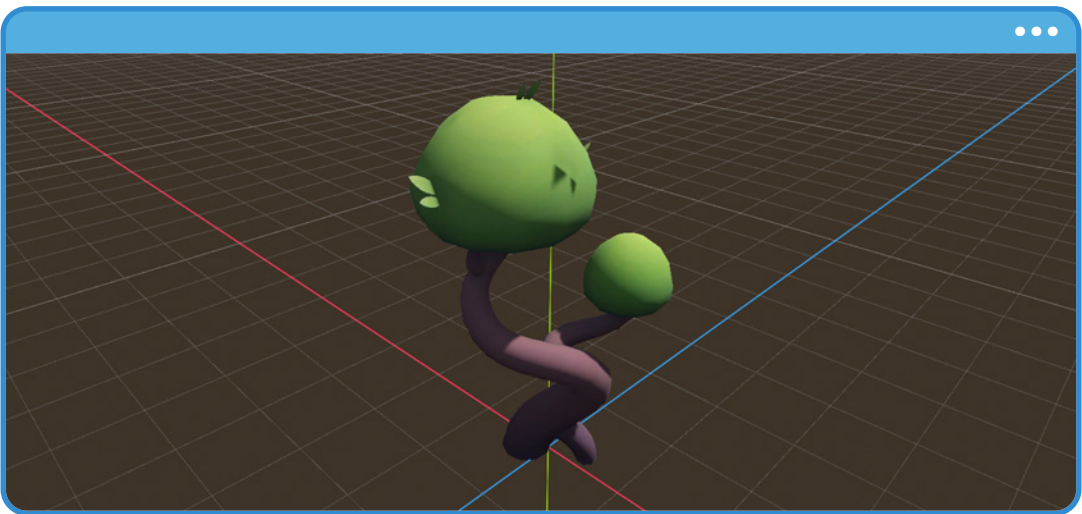
Sin embargo, si asignamos una textura cualquiera a la propiedad, notaremos que el objeto **Tree** en la escena no cambia de color. Esto se debe a que, aún no hemos declarado un vector RGB que procese el color de la textura para cada fragmento.

Dado que necesitamos calcular el color de la textura para cada píxel, será necesario ir la **etapa de fragmento** en nuestro shader y agregar las siguientes líneas de código:

```
void fragment()
{
    vec3 albedo = texture(_MainTex, UV).rgb;
    ALBEDO = albedo;
}
```


Del código anterior, hay varios puntos clave que debemos considerar para interpretar correctamente la operación que estamos realizando:

- La función **texture()**: Esta función devuelve un vector RGBA y recibe dos argumentos: una variable de tipo **sampler2D** (la textura) y un vector bidimensional (las coordenadas **UV**). Básicamente, **texture()** toma las coordenadas UV y aplica un muestreo sobre la textura **_MainTex**, obteniendo el color correspondiente, el cual se almacena en el vector RGB denominado **albedo**.
- Asignación a **ALBEDO**: El valor obtenido de la textura se asigna a la variable **ALBEDO**, que es una variable predefinida en Godot y representa el color base de la superficie del fragmento.
- Proyección de la textura: Como las coordenadas UV se calculan en función de la posición de los vértices en el espacio de objeto, deberíamos ver la textura proyectada correctamente sobre el objeto en la escena.



(1.6.b El objeto **Tree** ahora posee color)

Nota

Recuerda que los texels son los puntos de color dentro de la textura, es decir, las unidades mínimas de color que la conforman. En cambio, los píxeles son los puntos de color que vemos en la pantalla de nuestro ordenador. Ahora bien, ¿por qué no vemos puntos definidos en la textura aplicada al objeto tree? Esto ocurre porque Godot aplica una interpolación lineal automáticamente, suavizando la transición entre texels en lugar de mostrar bordes definidos.

A continuación realizaremos un ejercicio simple: modificaremos el color de la textura según la posición global del objeto **Tree**. Para ello, utilizaremos la variable interna **NODE_POSITION_WORLD**, la cual hace referencia al componente **Transform Position** del objeto en la escena. Dado este vínculo, podemos asumir que su valor de retorno es un vector de tres dimensiones que abarca el área de usabilidad dentro de la escena.

Este enfoque representa un problema potencial: ¿qué sucede si la posición global del objeto **Tree** es (100, 234, 134) o cualquier otro valor mayor a 1.0? En un shader, los colores están definidos en un rango de 0.0 a 1.0. Si los valores de posición exceden este rango, el resultado podría ser un objeto completamente **blanco**, ya que los valores de color fuera de este intervalo se interpretan como máximos.

Para evitar este problema, crearemos una función que limite los valores de retorno. Esta función se encargará de normalizar la posición global del objeto dentro del rango adecuado. Ahora bien, debemos asegurarnos de que esta función se declare antes del método **vertex()**, ¿por qué? Porque la GPU lee los shaders de manera secuencial, de arriba hacia abajo. Si necesitamos utilizar esta función tanto en la etapa de vértices como en la de fragmentos, será imprescindible declararla al inicio del código para que pueda ser utilizada en ambos métodos.

```
uniform sampler2D _MainTex : source_color;

vec3 hash33(vec3 p)
{
    p = fract(p * 0.4567);
    p += dot(p, p.yzx + 1.3456);
    p.x *= p.x;
    p.y *= p.y;
    p.z *= p.z;
    return fract(p);
}

void vertex()
```

¿Qué está ocurriendo en el método `hash33()`? Analicemos su estructura desde un punto de vista matemático. Considerando dos vectores tridimensionales: uno de entrada y otro de salida, si $p = (4.23, 5.52, 3.74)$:

Para la primera línea de código,

$$\text{fract}(p * 0.4567) = (0.931841, 0.520984, 0.708058)$$

(1.6.c)

Nota

El valor **0.4567** corresponde a una constante arbitraria utilizada dentro de la función hash para generar variación pseudoaleatoria. En funciones como `hash33()`, es común multiplicar los componentes de entrada por constantes no enteras con el objetivo de reducir la relación directa entre los valores y evitar patrones visibles en el resultado. El número específico no tiene un significado especial y podría reemplazarse por otras constantes que produzcan suficiente variación.

Si multiplicamos el primer componente del vector p por 0.4567 notaremos que eso da como resultado 1.931841. Sin embargo, en la operación se está empleando la función `fract()` la cual retorna únicamente los valores fraccionarios de un número. Esto aplica para todos los componentes del vector p .

Su definición en para un valor flotante corresponde a:

```
float fract (float v)
{
    return v - floor(v);
}
```

Luego,

$$p \cdot (p_{yzz} + 1.3456) = \sim 4.42184278$$

(1.6.d)

Este último se agrega a cada componente del vector p . Por lo tanto,

$$p = (5.353684, 4.942827, 5.129901)$$

(1.6.e)

Entonces,

$$p_x * p_x = 28.661930$$

$$p_y * p_y = 24.431537$$

$$p_z * p_z = 26.315882$$

(1.6.f)

Finalmente,

$$fract(p) = (0.661930, 0.431537, 0.315882)$$

(1.6.g)

Cabe destacar que el método anterior podría llevar cualquier nombre que deseemos. Sin embargo, se ha denominado **hash33** por la siguientes razones:

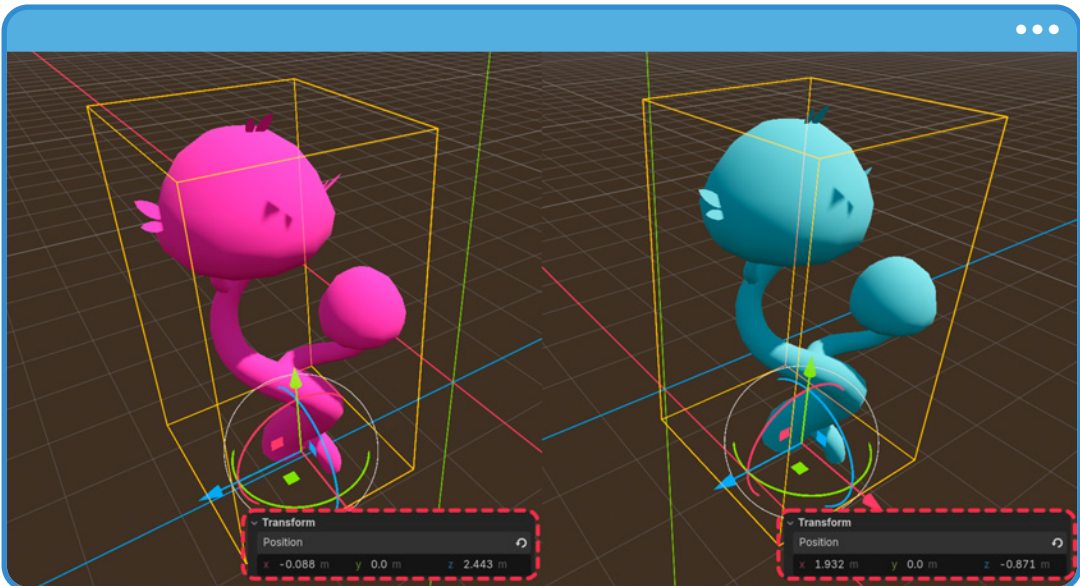
- Devuelve un vector de tres dimensiones como salida (3).
- Genera valores pseudoaleatorios (hash).
- Recibe un vector de tres dimensiones como argumento (3).

Este tipo de métodos se utilizan con frecuencia para generar “ruido” visual. No obstante, al combinarlo con otras funciones, podemos obtener efectos más interesantes, como el que implementaremos a continuación:

```
void fragment()
{
    vec3 random_color_ws = hash33(NODE_POSITION_WORLD);
    vec3 albedo = texture(_MainTex, UV).rgb;
    ALBEDO = random_color_ws;
}
```

Como podemos observar, se ha declarado un nuevo vector de tres dimensiones llamado **random_color_ws**, el cual se ha inicializado con el resultado del método **hash33()**, utilizando como entrada la posición global del objeto **Tree**. Posteriormente, este vector RGB se ha asignado a la variable **ALBEDO**, definiendo así el color base del shader en función de la posición del objeto en el espacio global.

Si guardamos los cambios y movemos el objeto a lo largo de la escena observaremos que cambia de color según la posición en donde se encuentre.



(1.6.h El objeto tree cambia de color según su posición en el mundo)

Dada la resolución de la posición del objeto, es posible que el color parpadee varias veces al moverlo dentro de la escena. Esto ocurre porque los valores de posición cambian de manera continua, generando variaciones constantes en el color.

En algunos casos, este efecto puede ser un problema. Para solucionarlo, podemos limitar el cambio de color y hacer que solo se modifique por cada unidad métrica. Implementar esta solución es sencillo: basta con extender la operación de cálculo de color e incluir la función **floor()**, que según su definición:

“

Devuelve el entero más grande que no sea mayor que un escalar o cada componente de un vector.

<https://registry.khronos.org/OpenGL-Refpages/gl4/html/floor.xhtml>

”

Su definición para un vector tridimensional corresponde a:

```
vec3 floor(vec3 v)
{
    vec3 rv;
    int i;

    for (i=0; i<3; i++) {
        rv[i] = v[i] - fract(v[i]);
    }
    return rv;
}
```

La función **floor()** nos permite eliminar los decimales de un valor y devolver únicamente la parte entera. En términos prácticos, esto significa que la posición del objeto se redondeará hacia abajo, asegurando que el color solo cambie cuando se cruce una unidad métrica completa en la escena.

Por ejemplo, si $p = (1.165, 0.0, 1.712)$,

$$\text{floor}(p) = (1, 0, 1)$$

(1.6.i)

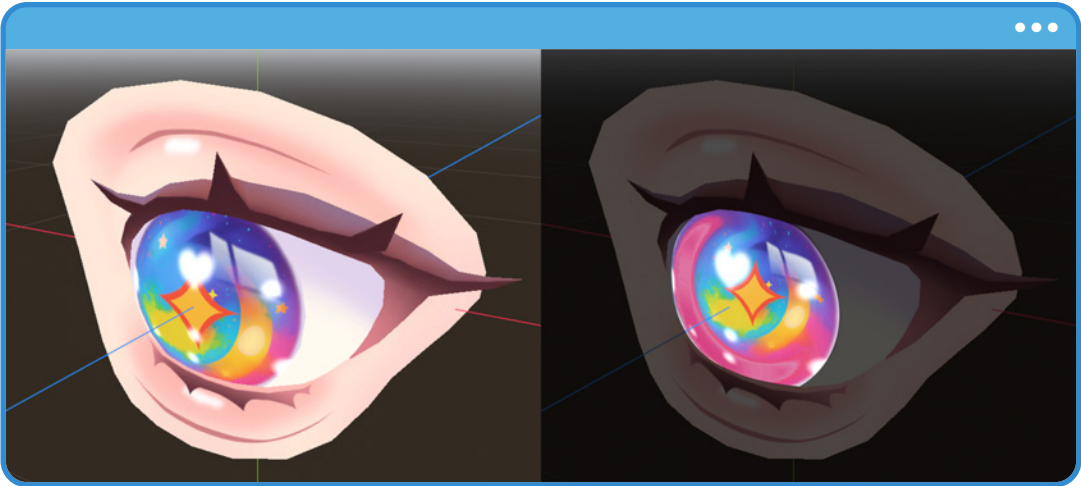
```
void fragment()
{
    vec3 random_color_ws = hash33(floor(NODE_POSITION_WORLD));
    vec3 albedo = texture(_MainTex, UV).rgb;
    albedo += random_color_ws * 0.15;
    ALBEDO = albedo;
}
```

Si guardamos los cambios y movemos el objeto **Tree** una vez más dentro de la escena, notaremos que su color solo cambiará cuando pase de una unidad métrica a otra en la cuadrícula. Esto evita el parpadeo constante y nos permite controlar mejor la variación de color según la posición global del objeto.

1.7 Introducción al espacio de tangente.

¿Te has preguntado cómo se generan esos efectos de profundidad en los ojos de un personaje 3D? Una opción sería utilizar múltiples capas (meshes): una para la córnea, otra más para la pupila, y así sucesivamente, aplicando transparencias entre ellas. Sin embargo, existen métodos más eficientes que permiten simplificar este proceso mediante la interacción entre la dirección de la vista (cámara) y las coordenadas de los vértices del objeto.

Para comprender mejor este concepto, prestemos atención a la siguiente imagen:



(1.7.a El ojo de la derecha posee profundidad)

Es posible que no se perciba con claridad a simple vista; sin embargo, el ojo de la derecha presenta una leve sensación de profundidad que se adapta a la dirección desde la cual observamos la escena. Es decir, si movemos la cámara, notaremos que la pupila del personaje transmite cierto nivel de tridimensionalidad, a pesar de estar compuesta únicamente por una textura bidimensional.

Para lograr este tipo de efectos, es necesario recurrir a un sistema de coordenadas específico conocido como espacio de tangente, el cual está definido por tres vectores ortogonales: tangente, bitangente y normal.

Como se mostró en la Figura 1.1.a (en la primera sección de este capítulo), el espacio tangente puede visualizarse mediante estos tres vectores que definen la orientación de un vértice, actuando como si cada uno tuviera su propio gizmo de transformación. Sin embargo, este espacio no solo puede definirse en la etapa de vértices, sino también por fragmento, permitiendo realizar transformaciones a nivel de píxel.

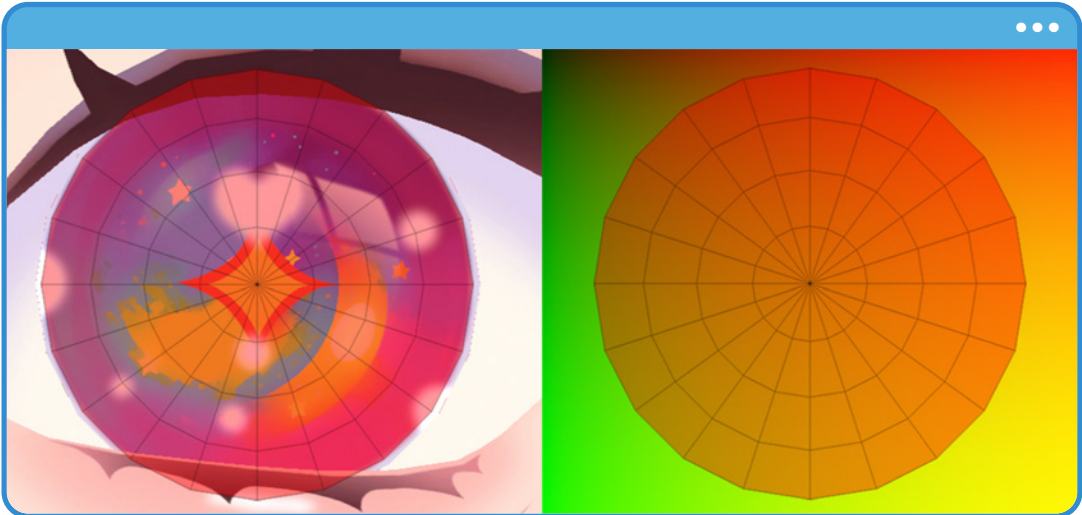
Antes de adentrarnos en la implementación de este efecto, haremos un breve análisis de la topología del objeto 3D que utilizaremos durante el desarrollo. Nos enfocaremos en algunas de sus propiedades más relevantes, particularmente en relación con sus coordenadas UV. Para este propósito, trabajaremos con el modelo **anime_eye.fbx**, incluido en el paquete descargable de este libro, ubicado en la siguiente ruta:

➤ Assets > chapter_01 > spaces > tangent_space > meshes

Si observamos el objeto con atención, notaremos que está compuesto por tres capas diferenciadas:

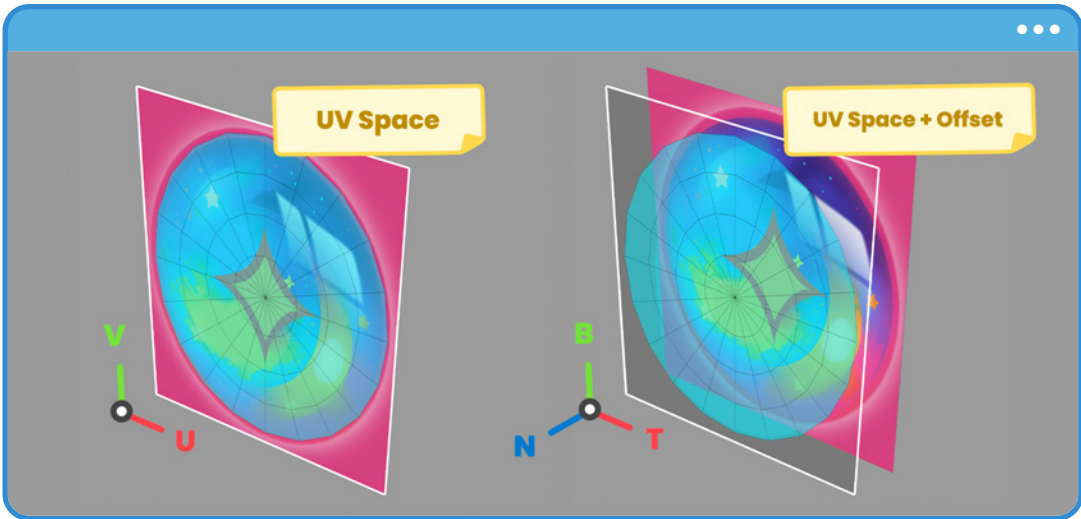
- Eye.
- Skin.
- Pupil.

Aplicaremos nuestro shader sobre la capa correspondiente a la pupila. Por lo tanto, como primer paso, revisaremos sus coordenadas UV. Luego, a través de una comparación, analizaremos cómo el espacio tangente puede facilitarnos la generación de efectos visuales como el desplazamiento en función del punto de vista.



(1.7.b Vértices de la pupila dentro del espacio UV)

Aunque la pupila tenga una forma geométrica similar a un óvalo, en este caso sus coordenadas UV, en términos de mapeo, están contenidas dentro de un espacio cuadrado. Esto es importante porque las coordenadas UV solo permiten desplazamiento en dos ejes: U y V. En cambio, el espacio de tangente introduce una tercera dimensión, lo que nos permite simular efectos de profundidad directamente sobre la textura. Al aplicar un desplazamiento en el espacio tangente, podemos modificar la apariencia de los texels alrededor del área de la pupila. Como resultado, se amplía el campo visual percibido, generando una ilusión de tridimensionalidad sin necesidad de modificar la geometría del modelo.

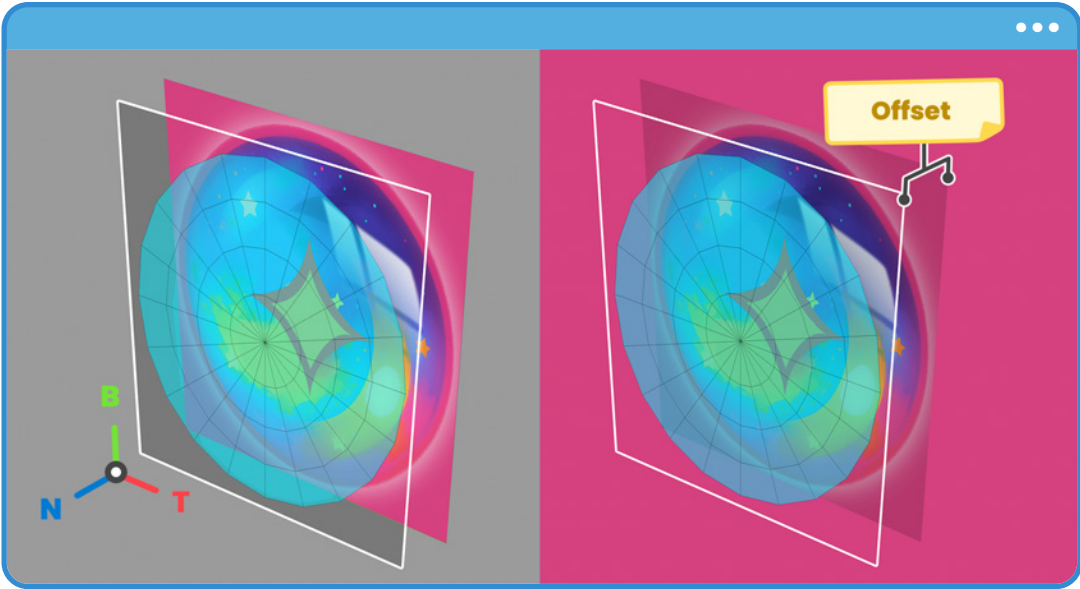


(1.7.c La imagen de la derecha posee un pequeño desplazamiento de las coordenadas)

En Godot, las texturas están configuradas en modo **Repeat** de forma predeterminada. Esto significa que, si aplicamos un desplazamiento considerable para simular profundidad, es probable que los bordes de la textura comiencen a repetirse alrededor del modelo 3D, en este caso, la pupila. Esta repetición afecta negativamente el resultado visual, ya que rompe la ilusión de profundidad y naturalidad que buscamos lograr en el ojo.

Para evitar este efecto no deseado, es recomendable cambiar el modo de repetición de la textura a **Clamp** desde la configuración del shader. Con esta opción, al desplazarnos más allá de los límites originales de la textura, se conserva el color del borde más cercano en lugar de repetir la imagen, lo que permite transiciones más suaves y coherentes en las áreas desplazadas.

Adicionalmente, podemos limitar la intensidad del desplazamiento utilizando una máscara que actúe como zona de influencia. Esta máscara puede ser una textura en escala de grises, diseñada específicamente para delimitar las regiones donde el efecto debe aplicarse, dejando el resto sin modificar.



(1.7.d Se repiten los píxeles de los bordes para obtener resultados precisos)

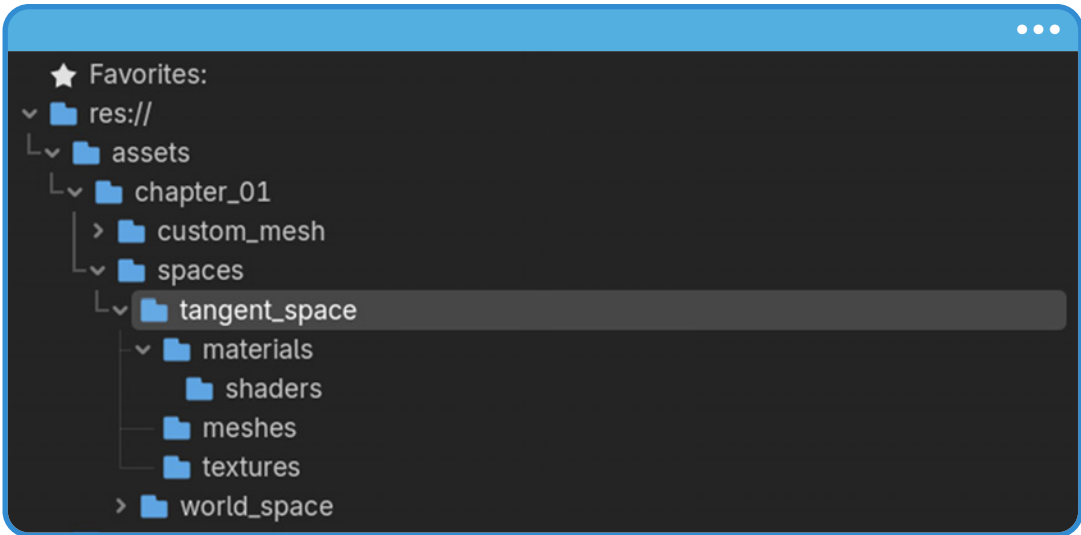
1.8 Implementando el espacio de tangente.

En esta sección realizaremos un nuevo ejercicio práctico con el objetivo de comprender más a fondo el funcionamiento del espacio de tangente y su relación con las variables internas **TANGENT**, **NORMAL** y **BINORMAL**.

Como primer paso, organizaremos nuestro proyecto para facilitar la implementación de las distintas funciones que utilizaremos. Para ello, seguimos los siguientes pasos:

- 1 Dentro del proyecto, navegamos a **chapter_01 > spaces** y crearemos una nueva carpeta llamada **tangent_space**.
- 2 Dentro de **tangent_space**, estructuramos el contenido creando las siguientes subcarpetas:
 - a **materials**.
 - b **shaders**.
 - c **meshes**.
 - d **textures**.

Si hemos seguido correctamente estos pasos, la estructura de nuestro proyecto debería lucir de la siguiente manera:

(1.8.a Se ha creado la carpeta **tangent_space**)

Para demostrar la utilidad del espacio de tangente, realizaremos el siguiente ejercicio: mediante un shader, transformaremos el vector de vista desde el espacio de vista (view space) al espacio de tangente. Para esto, definiremos una función personalizada llamada **view_to_tangent()**, la cual recibirá como argumentos cuatro vectores: la **tangente**, la **bitangente**, la **normal** y el vector de **vista**.

Esta función se encargará de calcular el producto punto entre el vector de vista y cada uno de los tres vectores que conforman la base del espacio tangente. El resultado será un nuevo vector, expresado en coordenadas locales al espacio de tangente, que podrá ser utilizado para generar efectos visuales dependientes de la dirección de la cámara, como desplazamiento, sombreado o profundidad simulada.

Nota

Los archivos necesarios para este ejercicio están disponibles en el paquete descargable asociado a tu cuenta accesible desde: <https://jettelly.com/store/the-godot-shaders-bible>

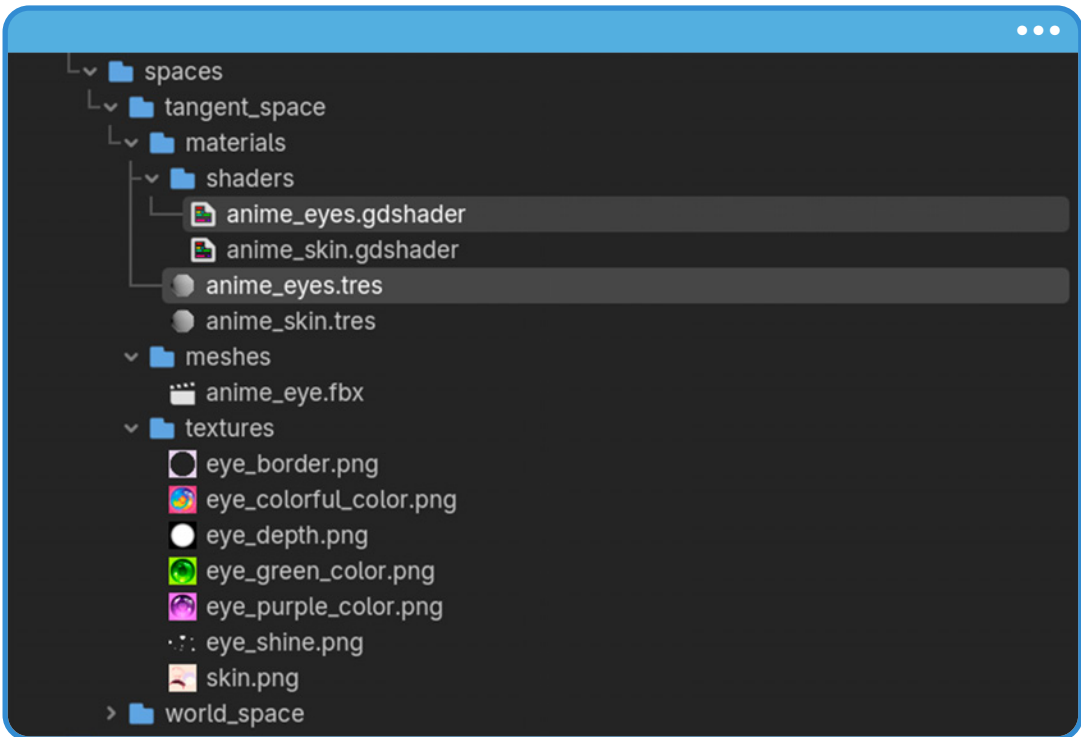
Para iniciar nuestro proceso de desarrollo, comenzaremos creando un nuevo **material** y un **shader**. Primero, sobre la carpeta materials, presionamos clic derecho y seleccionamos:

➤ Create New > Resource > ShaderMaterial.

Por razones prácticas, nombraremos a este material **anime_eyes**. A continuación, repetimos el mismo procedimiento en la carpeta **shaders**: clic derecho y seleccionamos:

➤ Create New > Resource > Shader.

Asignaremos también el nombre **anime_eyes** a este shader para mantener una relación clara entre ambos archivos y facilitar su identificación. Si todo ha salido correctamente, la estructura de nuestro proyecto debería verse de la siguiente manera:



(1.8.b Se han incluido los archivos descargables al proyecto)

Ahora que ya tenemos los archivos necesarios, seleccionamos el objeto **anime_eye.fbx** (ubicado en la carpeta **meshes**) y lo arrastramos al **Viewport** para instanciarlo en la escena. Luego, nos aseguramos de que su **posición** y **rotación** sean (0, 0, 0). Posteriormente, seleccionamos el nodo **anime_eye** en la ventana Scene, presionamos clic derecho > **Make Local**.

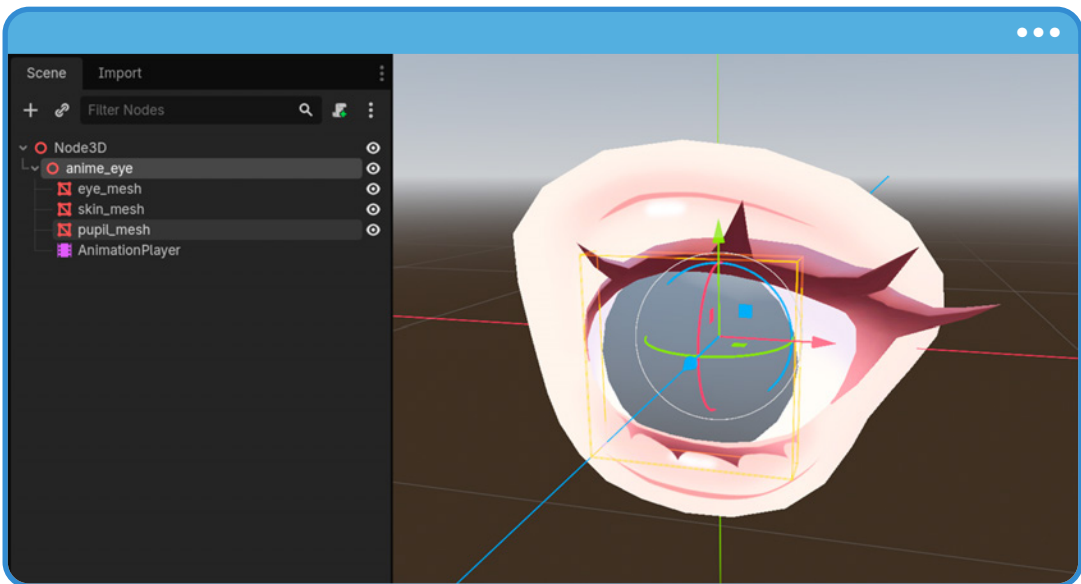
A diferencia de **New Inherited Scene**, que se utiliza para crear una escena derivada del recurso importado y poder personalizarla, esta acción nos permite usar el modelo como instancia dentro de la escena actual, conservando su estructura y las propiedades importadas, pero haciéndolas locales y editables.

Dentro de la ventana **Scene** encontraremos las capas asociadas al modelo 3D: **eye**, **skin** y **pupil**. Asignaremos el material **anime_skin** tanto a la capa **skin** como a eye, mientras que el material **anime_eye** lo aplicaremos directamente a la capa **pupil**, ya que sobre esta desarrollaremos el efecto. Asegúrate de haber vinculado previamente el shader correspondiente dentro del material, utilizando la propiedad **Shader**.

Nota

Recuerda que para asignar un shader a un material: 1) te debes posicionar sobre el material, 2) hacer doble clic sobre el mismo, 3) y luego asignar el shader en la propiedad **Shader** del material.

Si todo ha marchado correctamente, nuestra escena debería lucir de la siguiente manera:



(1.8.c Se han asignado los materiales correspondientes a cada capa 3D)

Antes de continuar, hay un par de cosas que debemos prestar atención respecto al objeto **anime_eye** dentro de nuestra escena. Considerando la configuración de iluminación:

- 1 El material para la piel y el ojo no está siendo afectado por la luz.
- 2 En cambio, el material para la pupila si está siendo afectado por la luz, adquiriendo un tinte grisáceo, cuando en realidad debería mostrarse completamente blanco por defecto.

Adicionalmente, notamos que la pupila no presenta ninguna textura visible. Esto es esperable, ya que aún no hemos declarado un **sampler2D** en nuestro shader para vincular una textura externa.

A continuación, procederemos a implementar estas características iniciales, como se muestra en el siguiente fragmento de código:

```
shader_type spatial;
render_mode unshaded;

uniform sampler2D _MainTex : source_color;

void vertex()
{
    // Called for every vertex the material is visible on.
}

void fragment()
{
    vec3 albedo = texture(_MainTex, UV).rgb;
    ALBEDO = albedo;
}
```

Si prestamos atención al bloque de código anterior, notaremos que en la segunda línea establece el modo de renderizado **unshaded**, el cual, según la documentación oficial en Godot:

“

El resultado es solo albedo. No se produce iluminación ni sombreado en el material, lo que acelera el renderizado.

https://docs.godotengine.org/en/stable/tutorials/shaders/shader_reference/spatial_shader.html

”

En términos prácticos, esto significa que el **render_mode unshaded** desactiva todos los cálculos de iluminación, lo que puede ser muy beneficioso en contextos donde se necesita rendimiento optimizado, como en videojuegos para dispositivos móviles de gama media o baja.

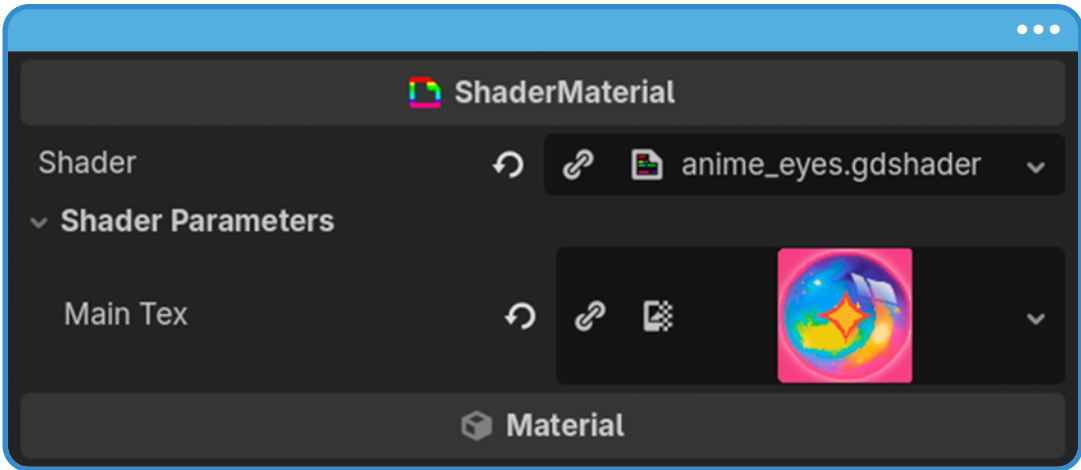
Sin embargo, esta configuración también implica que cualquier efecto de iluminación o profundidad debe ser simulado manualmente desde el shader. En este caso particular, esa limitación no representa un problema, ya que nuestro objetivo es visualizar de forma clara el efecto de desplazamiento aplicado en el espacio de tangente. Por esta razón, el uso del modo **unshaded** es totalmente apropiado: nos permite enfocarnos exclusivamente en el comportamiento visual que queremos lograr, sin distracciones derivadas de cálculos adicionales. Esto simplifica tanto la implementación como la comprensión del concepto que estamos explorando.

Nota

Los shaders de tipo **spatial** en Godot permiten configurar múltiples modos de renderizado. Uno de ellos es **cull_disabled**, que habilita la renderización de ambas caras, interna y externa, de un objeto 3D. Para más detalles sobre los distintos modos disponibles, puedes consultar la documentación oficial en el siguiente enlace: https://docs.godotengine.org/en/stable/tutorials/shaders/shader_reference/spatial_shader.html

Con esta pequeña implementación, ya podemos dirigirnos al material **anime_eyes** e incluir una textura en la propiedad **Main Tex**. Para el ejercicio, puedes utilizar cualquiera de las dos texturas incluidas en el proyecto:

- eye_colorful_color.
- eye_green_color.
- O eye_purple_color.



(1.8.d **Main Tex** es igual a **eye_colorful_color**)

Si los ajustes se han realizado correctamente, el objeto **pupil** ahora debería mostrar la textura que hemos asignado. Sin embargo, esta configuración aún no presenta ningún tipo de desplazamiento, ya que, si observamos el vector RGB **albedo**, vemos que estamos utilizando las coordenadas **UV** como base para proyectar la textura sobre la superficie.

Para generar un efecto de desplazamiento, necesitamos realizar una serie de pasos clave:

- Definir el espacio de tangente para la geometría de la pupila.
- Transformar la dirección de la cámara **VIEW** al espacio de tangente, utilizando el producto punto con cada uno de los vectores que lo componen.
- Distorsionar las coordenadas UV utilizando el resultado de esta transformación, logrando así una nueva proyección dinámica que reacciona a la vista del usuario.

Comenzaremos declarando un método que nos permitirá transformar la vista al espacio de tangente, como se muestra a continuación:

```
vec3 view_to_tangent(vec3 tangent, vec3 bitangent, vec3 normal, vec3 view)
{
    float t = dot(tangent, view);
    float b = dot(bitangent, view);

    // El código continúa en la página siguiente.
```

```

float n = dot(normal, view);
return normalize(vec3(t, b, n));
}

void fragment() { ... }

```

¿Qué hace este método? Para comprender su funcionamiento, primero debemos analizar la función `dot()`. Esta función, ampliamente utilizada en lenguajes de programación, calcula el producto punto entre dos vectores. En este caso específico, se trata de vectores tridimensionales, y el resultado es un valor escalar que se asigna en la variables `t`, `b` y `n` en función del vector base (**tangente**, **bitangente** y **normal**).

Comencemos revisando su definición algebraica para vectores en tres dimensiones:

$$A \cdot B = \sum_{i=1}^3 A_i B_i$$

(1.8.e)

Que es lo mismo decir,

$$A \cdot B = (A_1 * B_1) + (A_2 * B_2) + (A_3 * B_3)$$

(1.8.f)

La implementación equivalente en GLSL se puede representar de la siguiente manera:

```

float dot(vec3 a, vec3 b)
{
    return a.x * b.x + a.y * b.y + a.z * b.z;
}

```

En este contexto, lo que hace `view_to_tangent()` es proyectar el vector `view` (dirección desde el fragmento hacia la cámara) sobre la base ortonormal formada por los vectores

tangent, **bitangent** y **normal**. El resultado es un nuevo vector expresado en espacio de tangente, lo cual nos permitirá trabajar en ese sistema de coordenadas para aplicar efectos visuales como desplazamiento, sombreado, o simulación de profundidad.

Ahora, considerando la primera operación que se realiza dentro del método, tanto el vector **tangent** como **view** deben encontrarse en espacio de vista. Pero ¿cómo sabemos con certeza que ambos están expresados en ese sistema de coordenadas?

Según la documentación oficial en Godot, en la sección **Fragment Built-ins**:

“

VIEW: Vector normalizado desde la posición de fragmento (o vértice) hasta la cámara (en espacio de vista). Es el mismo para cámaras en perspectiva y ortogonales.

”

En términos simples, **VIEW** es un vector que apunta desde el punto que estamos procesando hacia la cámara, y está expresado en el espacio cuya referencia es la cámara misma. Este vector se interpola automáticamente desde los vértices durante el proceso de rasterización, y es especialmente útil para efectos que dependen del punto de vista, como el desplazamiento que estamos desarrollando.

Por otra parte, la documentación también señala:

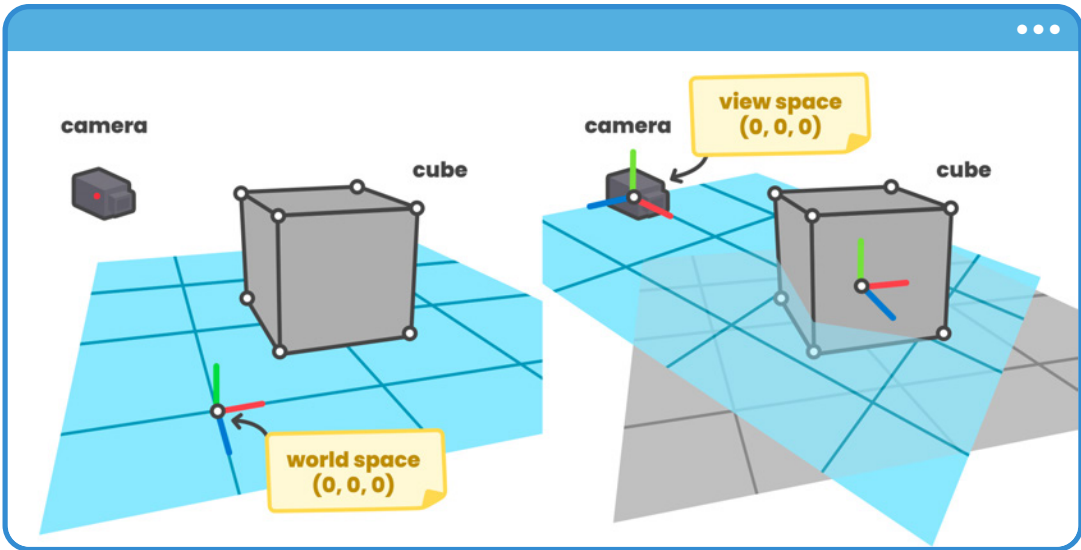
“

TANGENT: Tangente que provienen de la función **vertex()**, en espacio de vista. Si **skip_vertex_transform** está habilitado, es posible que no esté en ese espacio.

”

Dado que en nuestro caso no hemos aplicado ninguna transformación personalizada sobre la tangente dentro del método **vertex()**, podemos asumir que la variable interna **TANGENT** ya se encuentra en espacio de vista, al igual que **VIEW**.

Esta afirmación nos lleva a una siguiente pregunta: ¿cómo podemos visualizar este espacio? Para responder a esta pregunta, analizaremos la siguiente referencia visual:



(1.8.g Diferencia entre espacio global (izquierda) y espacio de vista (derecha))

Comenzando con la imagen de la izquierda, podemos observar que el cubo representado está definido en espacio global, lo que significa que su posición y orientación están determinadas respecto al punto de origen en el **Viewport**. En cambio, en la imagen de la derecha, el cubo se encuentra definido en espacio de vista. Esto implica que su posición y orientación ya no están expresadas con respecto al mundo, sino desde la perspectiva de la cámara. Por lo tanto, su rotación no corresponde exactamente a $(0, 0, 0)$, ya que su sistema de referencia ha cambiado.

Este mismo principio se aplica a los vectores **tangente**, **bitangente** y **normal** utilizados en cada fragmento dentro de la función **view_to_tangent()**. Todos ellos deben estar en el mismo espacio de referencia, en este caso, el espacio de vista, para que las operaciones vectoriales, como el producto punto, tengan sentido matemático y generen resultados válidos.

Nota

Desde el punto de vista de un Artista Técnico, no es estrictamente necesario tener presente cada fórmula matemática al momento de implementar estos efectos. Lo más importante es comprender qué hace cada función y asegurarse de que los vectores involucrados estén expresados en el mismo espacio antes de combinarlos en cualquier operación.

Ahora que comprendemos cómo transformar la dirección de la vista al espacio de tangente, podemos aplicar este conocimiento para distorsionar las coordenadas **UV** dentro de la función **texture()**. Esto nos permitirá visualizar el efecto de desplazamiento sobre la textura de la pupila.

Para ello, declararemos un nuevo vector que almacenará el resultado de la transformación. Luego, multiplicaremos este vector por un valor escalar de desplazamiento y lo aplicaremos directamente a las coordenadas **UV**, como se muestra a continuación:

```
vec3 view_to_tangent(vec3 tangent, vec3 bitangent, vec3 normal, vec3 view)
{ ... }

void fragment()
{
    float offset = 0.5;
    vec3 view_tangent_space = view_to_tangent(-TANGENT, BINORMAL, NORMAL,
        VIEW);
    view_tangent_space *= offset;

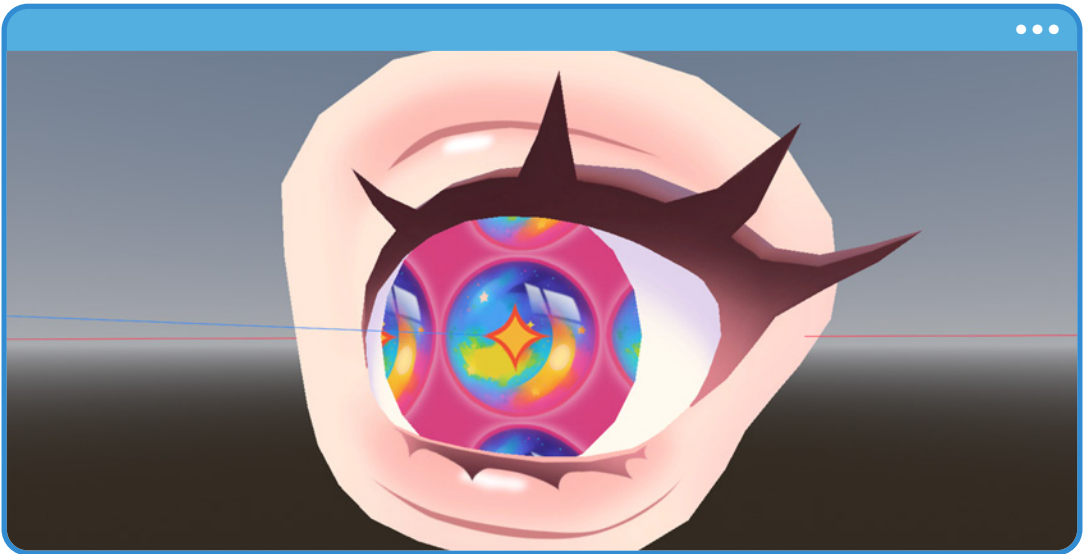
    vec3 albedo = texture(_MainTex, UV + view_tangent_space.xy).rgb;
    ALBEDO = albedo;
}
```

En el código anterior, ocurren tres acciones clave:

- 1 Se declara e inicializa un valor escalar llamado **offset**, con un valor de 0.5. Este valor se utiliza únicamente con fines demostrativos, ya que al finalizar este ejercicio lo ajustaremos a un máximo de 0.3 para obtener un resultado más controlado.
- 2 Se declara e inicializa un nuevo vector tridimensional llamado **view_tangent_space**, el cual almacena el resultado del método **view_to_tangent()**, que transforma el vector de vista al espacio de tangente.
- 3 Se toman los dos primeros componentes XY del vector **view_tangent_space** y se suman las coordenadas **UV** originales dentro de la función **texture()**, generando así un desplazamiento visual en la textura.

Si prestamos atención al primer argumento de la función `view_to_tangent()`, notaremos que la variable **TANGENT** está negada. Aunque esto pueda parecer poco intuitivo al principio, existe una razón lógica detrás: el sistema de coordenadas espaciales en Godot utiliza una convención de mano derecha (right-handed). En este sistema, el eje *z* apunta hacia adelante, por lo que es necesario invertir la dirección del vector tangente para que la base tangente-bitangente-normal (TBN) se construya correctamente. En cambio, motores como Unity utilizan un sistema de mano izquierda (left-handed), donde el vector **TANGENT** puede utilizarse sin invertir su dirección.

Al sumar los dos primeros componentes del vector `view_tangent_space` a las coordenadas UV, estamos proyectando la dirección de la vista sobre el plano definido por la tangente y la bitangente, que están alineadas con el espacio UV. Dado que este vector representa la dirección hacia la cámara en el espacio de tangente, el resultado visual es un desplazamiento dinámico de la textura que simula profundidad sobre la superficie del objeto. Esta técnica modifica la percepción visual sin alterar la geometría real del modelo, logrando un efecto convincente con bajo costo computacional.



(1.8.h La textura **Main Tex** se encuentra en modo repetida)

Si regresamos al **Viewport**, podremos observar dos aspectos principales:

- 1 La pupila ahora muestra un efecto de profundidad de manera efectiva.
- 2 Sin embargo, la textura aparece repetida, generando un resultado visual no deseado.

Aunque una posible solución sería desactivar manualmente el modo **Repeat** en la textura **eye_colorful_color**, en este caso adoptaremos un enfoque más adecuado para entornos controlados por shaders. Dado que estamos implementado este comportamiento directamente en el código del shader, utilizaremos el hint **repeat_disable** justo al **sampler2D** correspondiente a **_MainTex**. Esto evitará que las coordenadas UV se repitan automáticamente al exceder el rango estándar $[0.0 : 1.0]$.

Además, para permitir un mayor control desde el editor, declararemos una nueva variable uniforme llamada **_Offset**, con un rango ajustable entre $[0.0 : 0.3]$. Esta variable nos permitirá modificar dinámicamente la intensidad del desplazamiento directamente desde el Inspector.

El shader actualizado quedaría de la siguiente manera:

```
uniform sampler2D _MainTex : source_color, repeat_disable;
uniform float _Offset : hint_range(0.0, 0.3, 0.05);

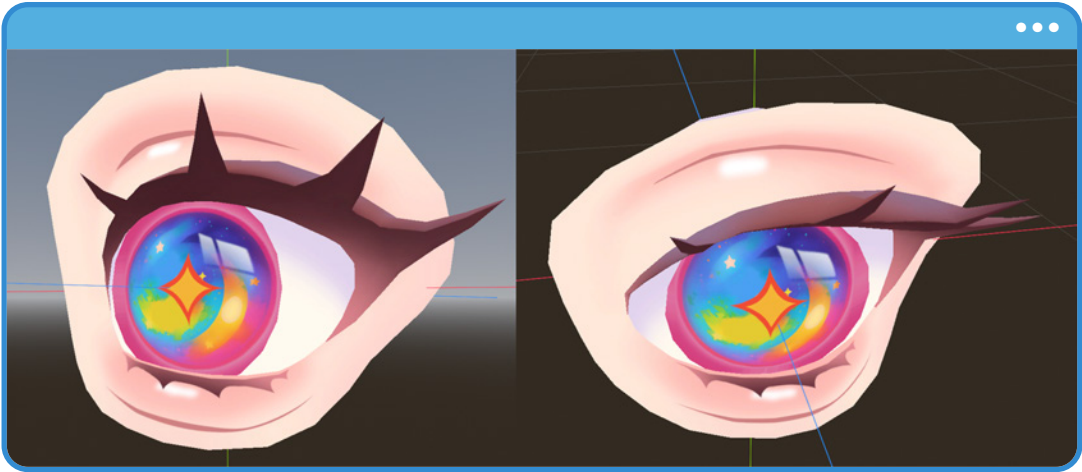
void vertex() { ... }

vec3 view_to_tangent(vec3 tangent, vec3 bitangent, vec3 normal, vec3 view)
{ ... }

void fragment()
{
    float offset = _Offset;
    vec3 view_tangent_space = view_to_tangent(-TANGENT, BINORMAL, NORMAL,
        ↳ VIEW);
    view_tangent_space *= offset;

    vec3 albedo = texture(_MainTex, UV + view_tangent_space.xy).rgb;
    ALBEDO = albedo;
}
```

Con un valor de desplazamiento igual a 0.2, la pupila debería lucir de la siguiente manera:



(1.8.i Efecto de profundidad en la pupila)

Hasta este punto, podríamos considerar que nuestro efecto está completo, ya que cumple correctamente su función principal. Sin embargo, a pesar de que el desplazamiento genera una sensación de profundidad, el resultado aún se percibe plano, debido a la falta de una capa de comparación o contraste visual. Por esta razón, en la siguiente sección dedicaremos unos minutos a mejorar el efecto que hemos desarrollado, incorporando elementos adicionales que enriquecerán la percepción visual y aumentarán el realismo del resultado final.

1.9 Interpolación lineal entre colores.

El efecto de desplazamiento está funcionando correctamente. No obstante, se aplica de forma uniforme sobre toda la textura, cuando en realidad lo que necesitamos es que afecte únicamente a la pupila, es decir, al área circular central.

Para corregir este comportamiento, simplemente podemos agregar una máscara: una nueva textura en la que el área que deseamos desplazar esté en blanco, mientras que el resto se encuentre en negro. ¿Por qué utilizar estos colores? Porque, en términos matemáticos, el color blanco representa un valor de 1.0, mientras que el negro representa 0.0. Por lo tanto, si multiplicamos este valor por nuestro desplazamiento, obtendremos un control gradual del efecto: se aplicará completamente en las zonas blancas (la pupila) y no se aplicará en las zonas negras (el resto del ojo).

Para implementar esta mejora, extenderemos nuestro shader agregando una nueva textura uniforme llamada **_Depth**, como se muestra a continuación:

```
uniform sampler2D _MainTex : source_color, repeat_disable;
uniform sampler2D _Depth : source_color, repeat_disable;
uniform float _Offset : hint_range(0.0, 0.3, 0.05);

void vertex() { ... }

vec3 view_to_tangent(vec3 tangent, vec3 bitangent, vec3 normal, vec3 view)
{ ... }

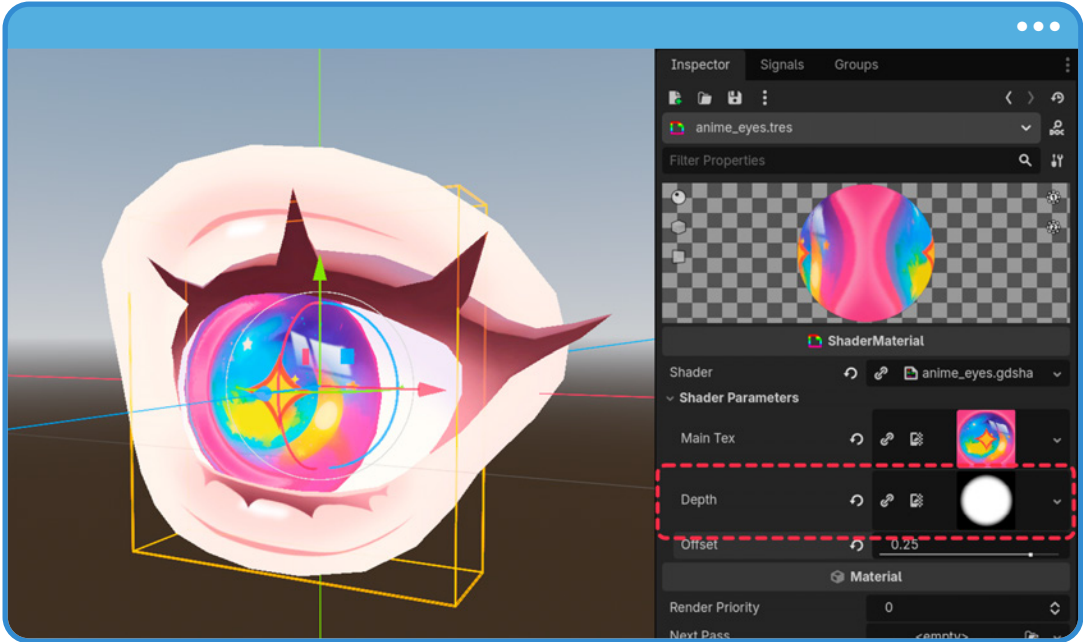
void fragment()
{
    float depth = texture(_Depth, UV).r;
    float offset = _Offset * depth;
    vec3 view_tangent_space = view_to_tangent(-TANGENT, BINORMAL, NORMAL,
        ↳ VIEW);
    view_tangent_space *= offset;

    vec3 albedo = texture(_MainTex, UV + view_tangent_space.xy).rgb;
    ALBEDO = albedo;
}
```

Como podemos observar, se ha introducido una nueva textura llamada **_Depth**, cuya función es actual como máscara de influencia para el desplazamiento. Esta textura debe estar compuesta por valores blancos en la región que deseamos afectar; la pupila, y negros en el resto. Como ya sabemos, al trabajar con imágenes en escala de grises, el canal rojo R es suficiente para representar la intensidad en cada punto de la textura.

En el método **fragment()**, accedemos a ese valor mediante la función **texture(_Depth, UV).r**, almacenando su resultado en la variable **depth**. Este valor representa qué tanto queremos desplazar la textura en ese fragmento específico. Luego, multiplicamos ese valor por el desplazamiento global definido en **_Offset**, generando así una diferencia controlada por el fragmento, que será mayor a la pupila y cero en las zonas circundantes.

De esta forma, el desplazamiento ya no se aplica uniformemente sobre toda la superficie, sino únicamente en aquellas áreas que definimos a través de la máscara. Para poder ver esto en acción, nos aseguraremos de asignar la textura **eye_depth** a la propiedad **Depth** desde el **Inspector**.



(1.9.a Se ha asignado la textura **eye_depth** a la propiedad **Depth**)

Una vez asignada la máscara correctamente, notaremos que se genera un borde de color alrededor de la pupila, lo cual enfatiza visualmente el efecto de desplazamiento. Este color corresponde a las regiones de la textura que se encuentran fuera del área blanca definida en la máscara, es decir, fuera de la pupila, y refleja cómo la distorsión afecta a los texels limítrofes.

Ahora es un buen momento para experimentar con otras capas de color que complementen y enriquezcan el efecto visual. Por ejemplo, podríamos incorporar brillos adicionales o detalles decorativos que refuercen la sensación de profundidad y dirección. Para lograrlo, agregaremos dos nuevas texturas globales a nuestro shader, como se muestra a continuación:

```
uniform sampler2D _MainTex : source_color, repeat_disable;
uniform sampler2D _Depth : source_color, repeat_disable;
uniform sampler2D _Border : source_color, repeat_disable;
uniform sampler2D _Shine : source_color, repeat_disable;
uniform float _Offset : hint_range(0.0, 0.3, 0.05);

void vertex() { ... }
```

Para las texturas **_Border** y **_Shine**, utilizamos las texturas **eye_border** y **eye_shine** respectivamente, las cuales deben ser asignadas desde el **Inspector**. Una vez aplicadas y realizados los ajustes correspondientes en el shader, podremos visualizar el efecto directamente en el **Viewport**, lo que nos permitirá iterar y ajustar con rapidez.

Con las texturas ya declaradas, el siguiente paso consiste en mezclarlas con el color base de la pupila. Para lograr esto, utilizaremos una interpolación lineal mediante la función **mix()**. Esta función, común en GLSL, permite combinar dos valores (en este caso, colores o vectores) en función de un tercer valor escalar que actúa como factor de mezcla.

La definición básica de **mix()** para un vector tridimensional es la siguiente:

```
vec3 mix(vec3 a, vec3 b, float w)
{
    return a + w * (b - a);
}
```

En este contexto:

- El primer argumento **vec3 a** es el valor original (por ejemplo, el color base de la textura **albedo**),
- El segundo; **vec3 b**, es el valor que deseamos introducir (por ejemplo, el borde o brillo),
- El último; **float w**, es un valor entre 0.0 y 1.0 que define qué tanto influye **b** sobre **a**.

Cuando **w = 0.0**, el resultado será completamente **a**. Cuando **w = 1.0**, el resultado será completamente **b**. Y para cualquier valor intermedio, se obtiene una transición suave entre

ambos. Esto nos permitirá combinar efectos adicionales de forma controlada y visualmente coherente sobre la superficie del ojo.

Continuemos ahora extendiendo nuestro shader, implementando en primera instancia la textura **_Border**, tal como se muestra a continuación:

```
void fragment()
{
    float depth = texture(_Depth, UV).r;
    float offset = _Offset * depth;
    vec3 view_tangent_space = view_to_tangent(-TANGENT, BINORMAL, NORMAL,
        VIEW);
    view_tangent_space *= offset;

    vec3 albedo = texture(_MainTex, UV + view_tangent_space.xy).rgb;
    vec4 border = texture(_Border, UV);
    vec3 lerp_ab = mix(albedo, border.rgb, border.a);

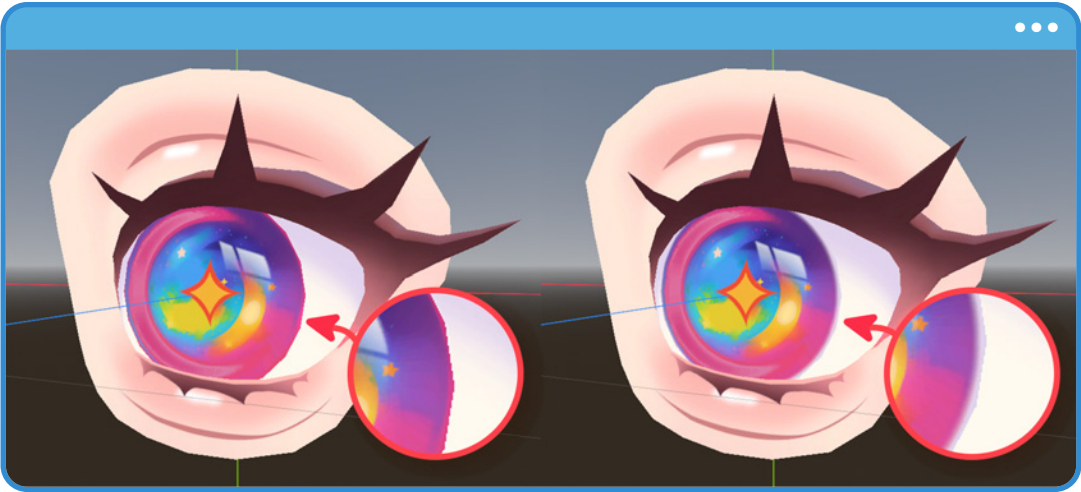
    ALBEDO = lerp_ab;
}
```

Antes de revisar el resultado en el **Viewport**, hagamos un breve análisis de lo que está ocurriendo en el código. Si observamos la variable **border**, notaremos que es un vector de cuatro componentes (RGBA), lo cual es relevante para la operación siguiente.

En la línea posterior, se declara un vector llamado **lerp_ab**, el cual se obtiene mediante una interpolación lineal entre **albedo** (el color base de la textura) y **border.rgb** (el color de la textura decorativa). Esta interpolación se controla utilizando el canal alfa de la textura **border**, es decir, **border.a**.

En términos gráficos, esto significa que:

- Cuando **border.a = 0.0**, el color resultante es completamente **albedo**.
- Cuando **border.a = 1.0**, el color resultante es completamente **border.rgb**.
- Para valores intermedios, se obtiene una mezcla suave entre ambos colores.



(1.9.b Comparación entre solo **albedo** e interpolación lineal entre dos capas)

En el **Viewport**, ahora podremos observar un gradiente sutil entre la pupila y el resto del ojo. Este gradiente, tanto en forma como en color, proviene de la textura **eye_border**, la cual ha sido adaptada estéticamente para integrarse con la composición general. Te recomendamos experimentar con otros colores en esta textura para observar el principio de interpolación lineal en acción y comprobar cómo varía el resultado final.

Solo nos resta aplicar la última capa visual: la textura **eye_shine**. Para ello, repetiremos el mismo proceso utilizando otra función **mix()**, como se muestra a continuación:

```
void fragment()
{
    float depth = texture(_Depth, UV).r;
    float offset = _Offset * depth;
    vec3 view_tangent_space = view_to_tangent(-TANGENT, BINORMAL, NORMAL,
        VIEW);
    view_tangent_space *= offset;

    vec3 albedo = texture(_MainTex, UV + view_tangent_space.xy).rgb;
    vec4 border = texture(_Border, UV);
    vec4 shine = texture(_Shine, UV);
```

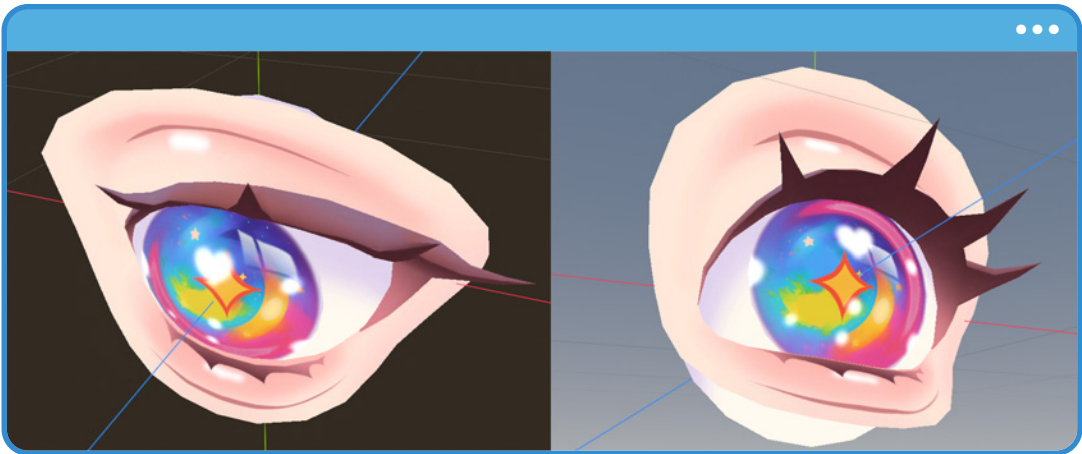
// El código continúa en la página siguiente.

```
vec3 lerp_ab = mix(albedo, border.rgb, border.a);
vec3 lerp_abs = mix(lerp_ab, lerp_ab + shine.rgb, shine.a);

ALBEDO = lerp_abs;
}
```

Como puede observarse, en esta etapa se ha aplicado una segunda interpolación. Partimos del resultado previo (**lerp_ab**, que mezcla el color base con el borde), y lo combinamos con la suma de **lerp_ab + shine.rgb**. Esta suma refuerza el brillo, permitiendo que se superponga suavemente sobre el color original según la intensidad definida en el canal alfa de la textura **eye_shine**.

Finalmente, si regresamos al **Viewport**, podremos visualizar el efecto completo: el desplazamiento aplicado sobre la pupila, junto con las distintas capas decorativas de bordes y brillos, todas controladas mediante máscaras y principios de interpolación lineal.



(1.9.c Texturas funcionando en conjunto)

1.10 Introducción a las matrices.

Uno de los conceptos que veremos con frecuencia en la creación de shaders son las **matrices**. Una matriz es una estructura compuesta por elementos numéricos organizados en filas y columnas, que obedecen ciertas reglas aritméticas y algebraicas. En el contexto del desarrollo gráfico, las matrices son fundamentales para realizar transformaciones espaciales como **traslación**, **rotación** y **escalado**.



(1.10.a A la izquierda, una matriz matemática de 3x3 dimensiones. A la derecha, una matriz de 3x3 dimensiones en **.gdshader**)

En los shaders, las matrices se emplean principalmente para transformar la posición de los vértices de un espacio a otro. Por ejemplo, la razón por la que podemos observar un objeto tridimensional en una pantalla bidimensional se debe a que su posición ha sido transformada mediante una serie de matrices, comúnmente conocidas como **MODEL_MATRIX**, **VIEW_MATRIX** y **PROJECTION_MATRIX**.

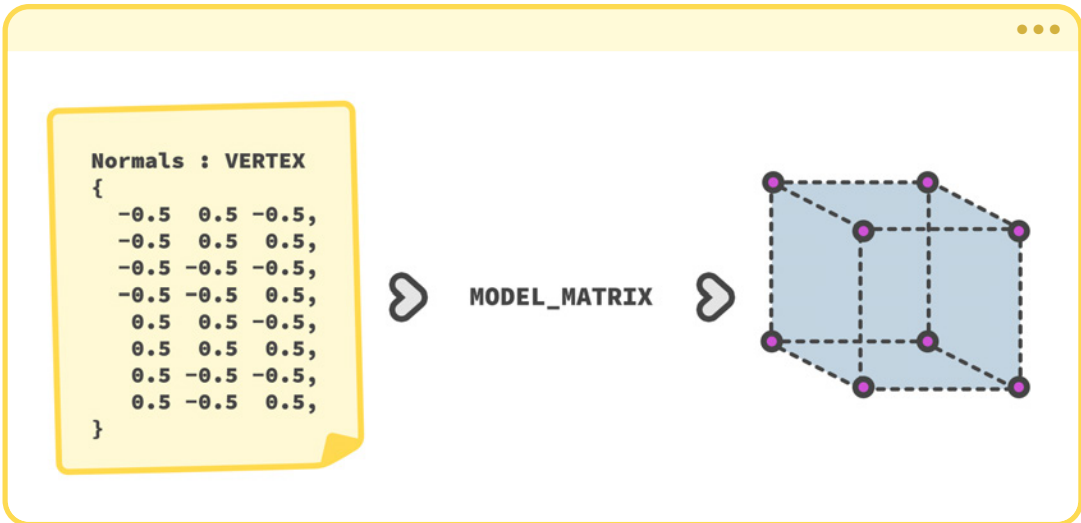
Este proceso ocurre automáticamente durante el pipeline de renderizado. Sin embargo, también podemos utilizar estas matrices directamente dentro de un shader para aplicar transformaciones personalizadas, modificar posiciones dinámicamente o incluso crear efectos visuales dependientes de la cámara, como el efecto **Billboard**.

¿Pero cómo ocurre este proceso? Tomemos como ejemplo una primitiva: un cubo. Sabemos que, por defecto, su información consiste únicamente en datos numéricos. No existe un cubo tridimensional real dentro del computador, y que dichos datos están definidos en espacio local (local space). Por lo tanto, el primer paso natural es transformar esa información a espacio global (world space), utilizando la matriz **MODEL_MATRIX**. Según la documentación oficial de Godot:

“

Transformación de model/local space a world space.

”



(1.10.b Los vértices en espacio local han sido transformados a espacio global)

Una vez que los datos están en espacio global, el cubo puede posicionarse dentro de una escena. Sin embargo, aún no puede ser visualizado, ya que no existe una referencia de observación (como si el cubo existiera, pero no pudieras verlo porque tienes los ojos cerrados). Para eso, debemos transformar su posición desde el espacio global al espacio de vista (view space), usando la matriz **VIEW_MATRIX**, la cual:

“

Transformación desde world space a view space.

”



(1.10.c Los vértices en espacio global han sido transformados a espacio de vista)

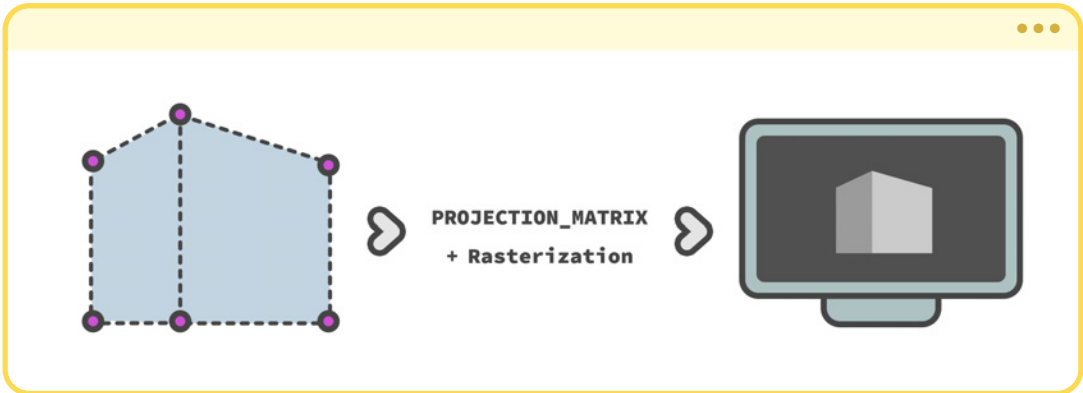
En este punto, el objeto ya puede ser representado desde la perspectiva de la cámara. No obstante, para proyectarlo en la pantalla del computador, es necesario pasar sus coordenadas desde el espacio de vista a un espacio proyectado. Esto lo logramos mediante la matriz **PROJECTION_MATRIX**, que, según la documentación oficial de Godot:

“

Transformación desde view space a clip space.

”

Este espacio proyectado (clip space) es el paso previo al proceso de rasterización, el cual finalmente convierte los datos en los píxeles que vemos en la pantalla. Por ende, podemos asumir que todo este proceso de transformación ocurre en el método **vertex()**.



(1.10.d El cubo ha sido proyectado en una pantalla bidimensional)

Nota

En Godot podemos encontrar una matriz llamada **MODELVIEW_MATRIX**. Esta matriz corresponde a la combinación de ambas transformaciones: la matriz de modelo (**MODEL_MATRIX**) y la matriz de vista (**VIEW_MATRIX**). Por lo tanto, transforma un punto desde el espacio local del objeto hasta el espacio de vista. https://docs.godotengine.org/en/stable/tutorials/shaders/shader_reference/spatial_shader.html

Entender el mundo de las matrices siempre ha sido complejo. Por esa razón, en las siguientes secciones dedicaremos unos minutos a implementar cada una de estas matrices, con el fin de comprender mejor su funcionamiento. Además, crearemos matrices personalizadas que nos permitirán transformar los vértices de nuestros objetos de manera dinámica desde el Inspector.

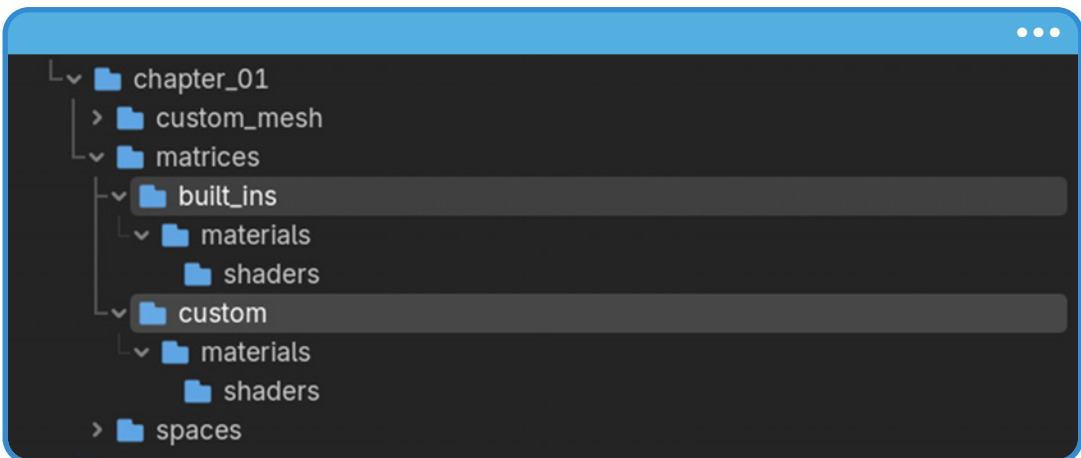
1.11 Matrices internas (Built-Ins).

En esta sección realizaremos un nuevo ejercicio práctico con el objetivo de comprender más a fondo el funcionamiento de las matrices internas **MODEL_MATRIX**, **VIEW_MATRIX** y **PROJECTION_MATRIX**, así como su relación con la variable **VERTEX**. Además, desarrollaremos un efecto conocido como **Billboard** empleando la matriz **INV_VIEW_MATRIX**.

Para facilitar la implementación de las distintas funciones que utilizaremos, organizaremos nuestro proyecto siguiendo estos pasos:

- 1 Dentro del proyecto, navegamos a **chapter_01** y crearemos una nueva carpeta llamada **matrices**.
- 2 Considerando que esta sección estará dividida en dos partes, nos aseguraremos de crear dos nuevas carpetas dentro de **matrices**:
 - a **built_ins**.
 - b **custom**.
- 3 Finalmente, dentro de cada carpeta, estructuramos su contenido creando las siguientes subcarpetas:
 - a **materials**.
 - b **shaders**.

Si hemos seguido correctamente estos pasos, la estructura de nuestro proyecto debería lucir de la siguiente manera:



(1.11.a Se han agregado las carpetas **built_ins** y **custom** al proyecto)

Como se mencionó previamente, desarrollaremos el efecto **Billboard** a través de un shader. Este efecto se caracteriza por orientar automáticamente un objeto 3D (o QuadMesh) para que siempre apunte hacia la cámara, sin importar su posición en el mundo, como el símbolo de interrogación del Item Box en Mario Kart. Ese símbolo siempre está orientado hacia la cámara, sin importar donde te encuentres en el juego.

En nuestro caso, implementaremos su comportamiento directamente en el método **vertex()**, manipulando la orientación del objeto en función de la posición de la cámara. Para ello, iniciaremos creando algunas carpetas que nos ayudarán a mantener nuestro proyecto ordenado.

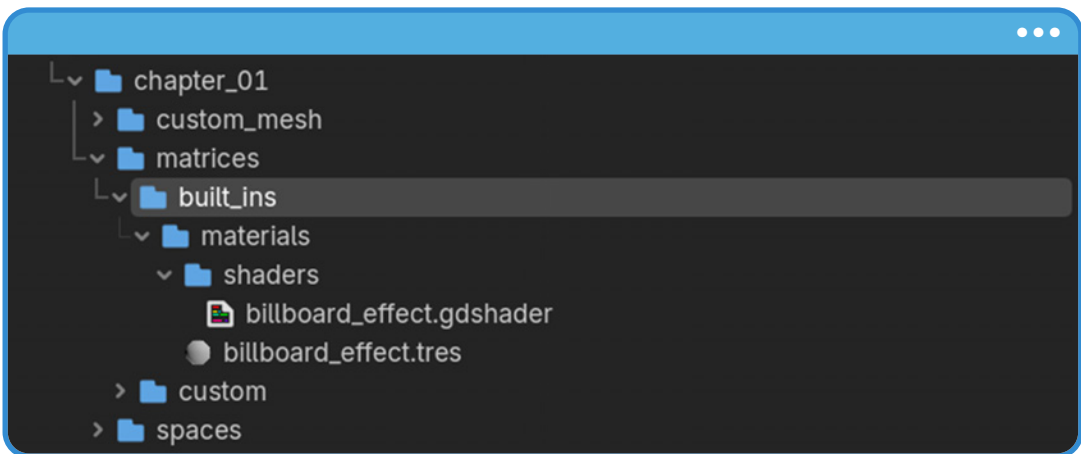
Posicionándonos sobre la carpeta **built_ins > materials**, presionamos clic derecho y seleccionamos:

➤ Create New > Resource > ShaderMaterial.

Por razones prácticas, nombraremos a este material **billboard_effect**. Repitiendo el proceso anterior, nos posicionamos sobre la carpeta **shader**, presionamos clic derecho y seleccionamos:

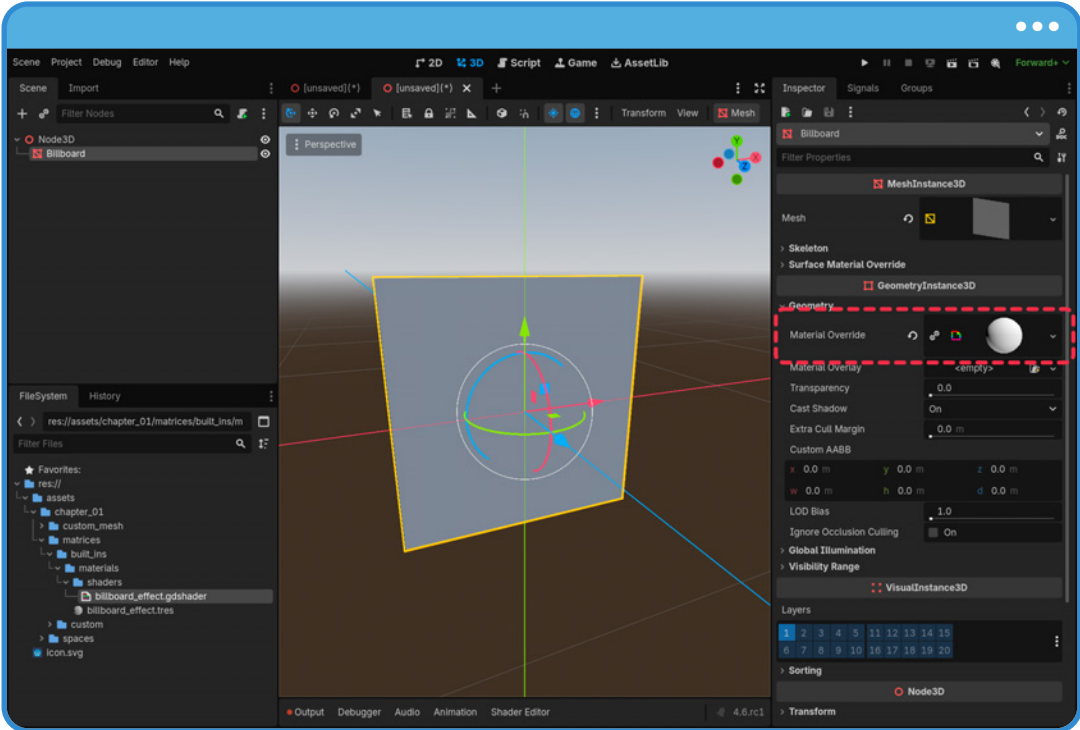
➤ Create New > Resource > Shader.

Como de costumbre, nos aseguraremos de utilizar el mismo nombre del **material**, de modo que sea más fácil identificar la conexión entre ambos. Si se han seguido los pasos correctamente, nuestro proyecto debería lucir de la siguiente manera:



(1.11.b Se ha agregado tanto un material como un shader al proyecto)

Para iniciar el proceso de implementación, crearemos una nueva escena 3D y nos aseguraremos de incluir un nodo **MeshInstance3D** como hijo, al cual nombraremos **Billboard**. A este nodo le asignaremos un **QuadMesh** como **Mesh**. Además, tras haber creado el shader **billboard_effect** y su respectivo material, nos aseguraremos de asignar el material en la propiedad **Material Override** del objeto **Billboard**.



(1.11.c Se ha incluido un **QuadMesh** en la escena)

Comenzaremos prestando atención a nuestro shader **billboard_effect**, en especial al método **vertex()**. Como podemos observar, no hay código aparente dentro del campo del método. Sin embargo, como ya sabemos, se están realizando operaciones matemáticas de forma automática, tales como las multiplicaciones de matrices. Si consultamos la documentación oficial de Godot, encontraremos la siguiente información:

“

Vertex Built-ins: Los usuarios pueden sobrescribir la transformación `modelview` y `projection` utilizando la variable interna **POSITION**. Si **POSITION** está escrita en cualquier lugar en el shader, siempre será usada, por ende, el usuario tendrá la responsabilidad de asegurar que sus valores son siempre aceptables. Cuando se utiliza **POSITION**, los valores provenientes de **VERTEX** son ignorados y no se realiza la proyección.

”

Para ver esto en práctica, emplearemos el modo de render `skip_vertex_transform`. Este desactiva la transformación automática que realiza Godot en el método `vertex()`, otorgándonos control total sobre el posicionamiento y la proyección de los vértices. No obstante, antes de aplicar esta configuración, nos aseguraremos de configurar y asignar una textura al shader, para evitar que el `QuadMesh` se vea completamente blanco o vacío durante el ejercicio.

```
shader_type spatial;
render_mode unshaded;

uniform sampler2D _MainTex : source_color;

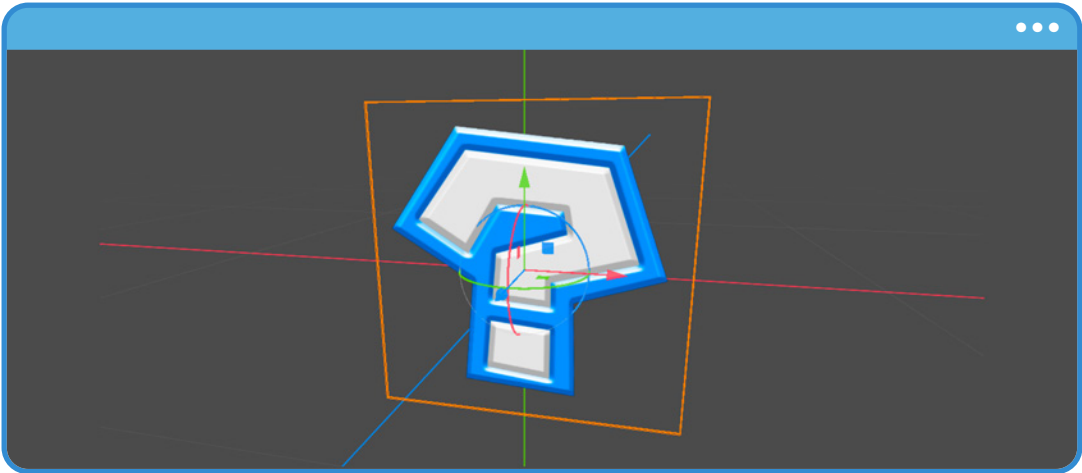
void vertex() { ... }

void fragment()
{
    vec4 albedo = texture(_MainTex, UV);
    ALBEDO = albedo.rgb;
    ALPHA = albedo.a;
}
```

En el fragmento de código anterior, se ha declarado una nueva variable **albedo**, de tipo **vec4** (RGBA o XYZW), la cual almacena el resultado de samplear la textura `_MainTex` utilizando las coordenadas **UV**. Como ya sabemos, esta textura será asignada posteriormente desde el Inspector. A diferencia de los ejemplos de secciones anteriores, aquí no solo se utiliza el color RGB de la textura, sino también su canal alfa, el cual se asigna directamente a la variable interna **ALPHA**.

La función de **ALPHA** es controlar la transparencia del fragmento. Esto permite que los píxeles de la textura sean completamente opacos, translúcidos o totalmente transparentes, según el valor del canal alfa definido en la imagen. Para este ejercicio, utilizaremos la textura **item_box_symbol_tex**, incluida en el paquete de descarga adjunto a este libro.

Si guardamos nuestro shader y asignamos la textura, el objeto **Billboard** en nuestra escena debería lucir de la siguiente manera:



(1.11.d Se ha asignado la textura **item_box_symbol_tex**)

A continuación, agregaremos la propiedad **skip_vertex_transform** a nuestro shader, como se muestra en la siguiente línea de código:

```
shader_type spatial;
render_mode unshaded;
render_mode skip_vertex_transform;

uniform sampler2D _MainTex : source_color;

void vertex() { ... }
```

Al realizar este paso, nuestro QuadMesh va a desaparecer del **Viewport**. Como podemos deducir, esto se debe a que no se está llevando a cabo ninguna transformación de vértices; por ende, será necesario realizar dicho proceso manualmente. Para comenzar, transformaremos

los vértices del mesh desde el espacio de modelo (local o object space) al espacio de vista (view space), empleando la matriz **MODELVIEW_MATRIX**, como se muestra a continuación:

```
render_mode skip_vertex_transform;

uniform sampler2D _MainTex : source_color;

varying vec3 vertex_os;

void vertex()
{
    vertex_os = VERTEX;
    VERTEX = (MODELVIEW_MATRIX * vec4(vertex_os, 1.0)).xyz;
}
```

Hay algunas propiedades que debemos tomar en cuenta para interpretar correctamente el código anterior. Comenzando con la palabra clave **varying**, esta se utiliza para definir una variable global compartida entre los métodos **vertex()** y **fragment()**. En este caso, el vector **vertex_os** podrá ser accedido tanto en el vértice como en el fragmento, permitiendo intercambiar datos entre ambas etapas del shader, si es necesario.

En la primera línea de código, dentro del método **vertex()**, el vector **vertex_os** recibe el valor de la variable **VERTEX**. Por lo tanto, podemos asumir que los valores de cada vértice están en espacio de objeto, de ahí el sufijo **_os** en el nombre de la variable.

Nota

Se considera una buena práctica colocar no solo utilizar nombres descriptivos para las variables en un shader, sino también indicar el espacio en el que se encuentran. De esta manera, sabremos que **variable_os** está en object space; **variable_ws**, en world space; **variable_vs**, en view space, y **variable_ss**, en screen space.

A continuación, **VERTEX** adquiere su nueva posición transformada al multiplicar la matriz **MODELVIEW_MATRIX** por un vector de cuatro dimensiones. En esta operación, los primeros tres componentes XYZ corresponden a la posición original del vértice (es decir, **vertex_os**), mientras que el cuarto componente W se establece en 1.0.

¿A qué se debe este valor? El número 1.0 representa un vector de posición en el espacio homogéneo. En gráficos por computadora, se utilizan vectores de cuatro dimensiones **vec4** para permitir transformaciones afines mediante la multiplicación por matrices 4x4. Establecer $W = 1.0$ indica que el vector representa una posición en el espacio, y por lo tanto será afectado por traslaciones, rotaciones y escalado. En cambio, si $W = 0.0$, el vector se considera una dirección, lo que significa que no será afectado por traslaciones, solo por rotaciones y escalado. Por esta razón, dado que los vértices representan posiciones en el espacio tridimensional, debemos asegurarnos de que W sea igual a 1.0.

Podemos obtener el mismo resultado separando las matrices de transformación, como se muestra a continuación:

```
void vertex()
{
    vertex_os = VERTEX;
    vec4 vertex_ws = MODEL_MATRIX * vec4(vertex_os, 1.0);
    vec4 vertex_vs = VIEW_MATRIX * vertex_ws;
    VERTEX = vertex_vs.xyz;
}
```

En este fragmento, como podemos observar, el proceso de transformación sólo involucra las matrices **MODEL_MATRIX** y **VIEW_MATRIX**. Sin embargo, la proyección aún se realiza automáticamente como parte de la transformación. Para llevar a cabo el proceso completo, incluyendo la proyección, debemos utilizar la variable interna **POSITION**, como se muestra a continuación:

```
void vertex()
{
    vertex_os = VERTEX;
    vec4 vertex_ws = MODEL_MATRIX * vec4(vertex_os, 1.0);
    vec4 vertex_vs = VIEW_MATRIX * vertex_ws;
    vec4 vertex_proj = PROJECTION_MATRIX * vertex_vs;
    POSITION = vertex_proj;
}
```


No obstante, si decidimos seguir este camino, debemos asegurarnos de que **POSITION** contenga valores válidos en todo momento, lo cual añade un nivel adicional de responsabilidad y carga de trabajo.

Sabiendo esto, ¿cómo podríamos aplicar un efecto Billboard directamente en el shader? La idea principal de este efecto es que el objeto siempre mire hacia la cámara, sin importar su orientación original en el mundo.

Para lograrlo, podemos construir una matriz personalizada que combine la posición del objeto en el mundo con la orientación de la cámara. Esto lo conseguimos extrayendo las primeras tres filas de la matriz **INV_VIEW_MATRIX**, la cual, según la documentación oficial de Godot:

“

Transformación desde view space a world space

”

Con esto mantenemos los vértices en espacio global, pero orientados desde el punto de vista de la cámara. Si combinamos esta matriz con la última fila de la matriz **MODEL_MATRIX**, que contiene la posición del objeto en espacio global, obtendremos un objeto que apunta siempre a la cámara.

A continuación se muestra cómo aplicar esta lógica en el método **vertex()**, manteniendo la implementación previa:

```
void vertex()
{
    vertex_os = VERTEX;

    mat4 BILLBOARD_MATRIX = mat4(
        INV_VIEW_MATRIX[0],
        INV_VIEW_MATRIX[1],
        INV_VIEW_MATRIX[2],

        // El código continúa en la página siguiente.
```

```

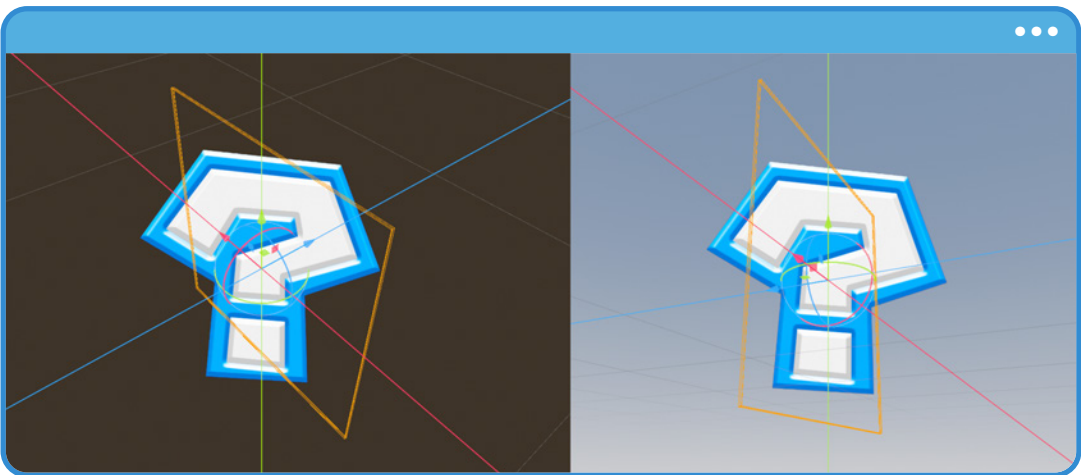
    MODEL_MATRIX[3]
);

vec4 vertex_ws = BILLBOARD_MATRIX * vec4(vertex_os, 1.0);
vec4 vertex_vs = VIEW_MATRIX * vertex_ws;
vec4 vertex_proj = PROJECTION_MATRIX * vertex_vs;
POSITION = vertex_proj;
}

```

Como podemos observar en el ejemplo anterior, se ha declarado una nueva matriz denominada **BILLBOARD_MATRIX**, la cual reorienta el objeto para que siempre mire hacia la cámara. Esta matriz se construye tomando las tres primeras filas de **INV_VIEW_MATRIX**, que representan la orientación de la cámara en el espacio global. Esto mantiene los vértices en su respectivo espacio, pero reorientados desde el punto de vista de la cámara. La cuarta fila proviene de **MODEL_MATRIX**, lo que garantiza que la posición del objeto en el mundo se mantenga intacta.

Si los cambios se han realizado correctamente, nuestro objeto Billboard debería apuntar hacia la cámara en el **Viewport**, independientemente de su posición en la escena.



(1.11.e El objeto Billboard siempre apunta hacia la cámara)

1.12 Implementado matrices personalizadas.

Hasta este punto, hemos puesto el foco en las matrices internas que permiten la visualización de un objeto 3D en la escena. Pero, ¿cómo podríamos crear matrices por nuestra cuenta? Según la documentación oficial de Godot, podemos utilizar los siguientes tipos de dato para definir matrices:

- **mat2**, para matrices bidimensionales 2x2.
- **mat3**, para matrices tridimensionales 3x3.
- **mat4**, para matrices tetradimensional 4x4.

Cada una de estas define un espacio distinto y se emplean en distintos casos. Por ejemplo, podríamos definir una matriz para escalar, deformar o rotar los vértices de nuestros modelos, tanto en 2D como en 3D. Ahora, si bien podríamos utilizar una matriz **mat3** en un videojuegos de dos dimensiones, lo correcto (pensando en términos de usabilidad y optimización) sería emplear una matriz **mat2**, ya que solo se requiere transformar los ejes de movimiento xy.

¿Cómo se declara e inicializa una matriz en lenguaje de shader? Según la documentación de Godot, podemos declarar e inicializar matrices de la siguiente manera:

```
mat2 m2 = mat2(vec2(1.0, 0.0), vec2(0.0, 1.0));
mat3 m3 = mat3(vec3(1.0, 0.0, 0.0), vec3(0.0, 1.0, 0.0), vec3(0.0, 0.0, 1.0));
mat4 identity = mat4(1.0);
```

Estas declaraciones crean matrices de identidad de 2x2, 3x3 y 4x4, respectivamente. Podemos representarlas de forma visual de la siguiente manera:

Para una matriz de dos dimensiones:

$$m_2 = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$$

(1.12.a)

Para una matriz de tres dimensiones:

$$m_3 = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

(1.12.b)

Para definir una matriz de cuatro dimensiones, simplemente debemos extender la matriz de tres dimensiones agregando una fila y columna adicional:

$$m_4 = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

(1.12.c)

Nota

Al escribir `mat4 identity = mat4(1.0)`, Godot inicializa automáticamente la diagonal principal con 1.0 y el resto con ceros, creando una matriz de identidad, fundamental para transformaciones lineales sin alterar el vector original.

A modo de ejercicio, podríamos crear una matriz de rotación para visualizar el efecto gimbal. Sin embargo, dejaremos este ejercicios para el capítulo 3, donde hablaremos e implementaremos cuaterniones en **.gdshader**. De esta forma, será más sencillo comparar y entender ambas implementaciones. Por ahora, nos enfocaremos en desarrollar una pequeña matriz personalizada para transformar únicamente los vértices de nuestro objeto.

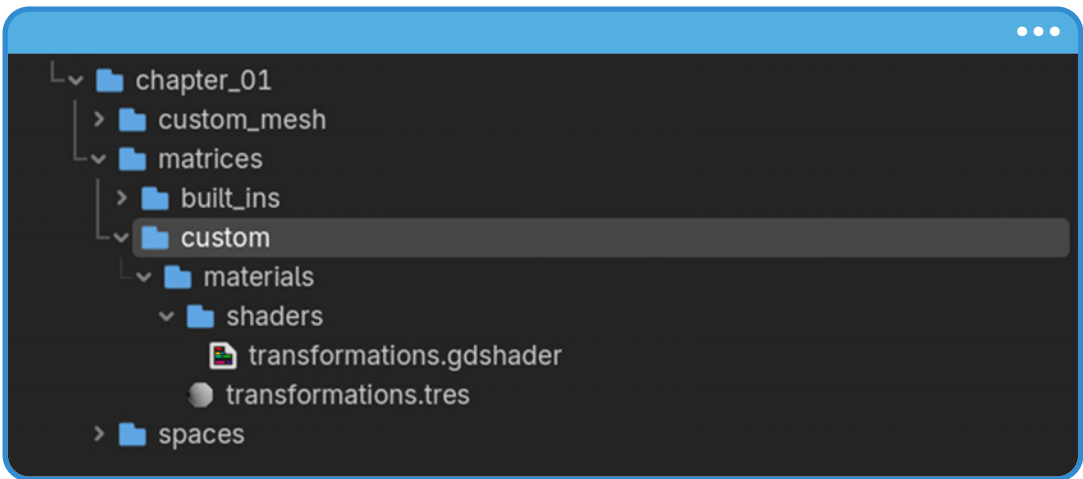
Para ello, iremos al **FileSystem** y, dentro de la carpeta **custom > materials > shaders**, crearemos un nuevo shader, seleccionando:

➤ Create New > Resource > Shader.

Por razones prácticas, nombraremos este shader como **transformations**. Luego, repitiendo el proceso anterior, nos posicionamos sobre la carpeta **materials**, presionamos clic derecho y seleccionamos:

➤ Create New > Resource > ShaderMaterial.

Nos aseguraremos de emplear el mismo nombre para el material, de modo que sea más fácil identificar la conexión entre ambos recursos. Si los pasos se han llevado a cabo correctamente, nuestro proyecto debería verse de la siguiente manera:



(1.12.d Se han incluidos un material y un shader al proyecto)

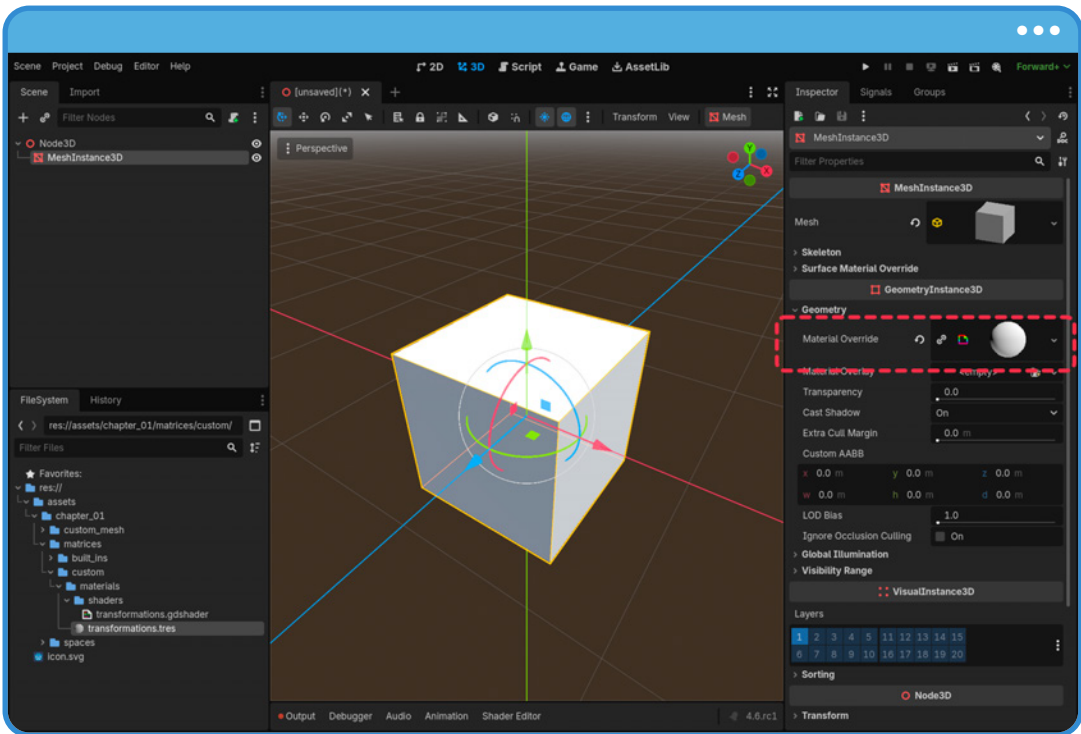
Para el ejercicio que llevaremos a cabo a continuación, podemos utilizar cualquier modelo 3D que tengamos disponible en nuestro computador. Sin embargo, se recomienda utilizar un modelo primitivo; como un **BoxMesh** genérico.

La transformación que implementaremos no solo consistirá en escalar los vértices del objeto, sino que además aplicaremos una transformación lineal denominada **shearing** (cizallamiento), la cual inclina la forma de un objeto moviendo sus vértices en una dirección fija. La magnitud de este movimiento depende de la posición de los vértices a lo largo de otro eje. Este tipo de transformación no altera el volumen del objeto, pero modifica su forma al distorsionar sus ángulos rectos.

Iniciaremos preparando la escena de la siguiente manera:

- Configuraremos una nueva escena 3D.
- Incluiremos un **MeshInstance3D** como hijo.
- Asignaremos un BoxMesh al **MeshInstance3D**.
- Aplicamos el material **transformations**.

Si los pasos se han realizado correctamente, nuestra escena debería verse de la siguiente manera:



(1.12.e Se ha asignado el material **transformations** al **BoxMesh**)

Dado que nuestro objeto es tridimensional, iniciaremos el ejercicio definiendo una matriz de 3x3 dimensiones e inicializándola como una matriz de identidad. Esto nos permitirá evitar cualquier transformación visible en el modelo 3D en esta primera instancia. Para ello, iremos a nuestro shader **transformations** e incluiremos el siguiente macro, posicionándolo sobre el método **vertex()**.

```

shader_type spatial;

#define TRANSFORMATION_MATRIX mat3(vec3(1, 0, 0), vec3(0, 1, 0), vec3(0, 0,
↳ 1))

void vertex()
{
    VERTEX = VERTEX * TRANSFORMATION_MATRIX;
}

```

Como podemos observar en el ejemplo, se ha definido un nuevo macro llamado **TRANSFORMATION_MATRIX**, el cual será reemplazado por una matriz de identidad de 3x3 dimensiones. Cabe destacar que una matriz también puede definirse mediante una función o a través de la declaración de una nueva variable dentro del método **vertex()**. Sin embargo, en este caso se ha optado por utilizar la directiva **#define**, ya que mejora la legibilidad, la consistencia y facilita la reutilización de la matriz a lo largo del desarrollo.

Nota

Es recomendable utilizar la directiva **#define** cuando: 1) deseamos definir bloques de código constante o reutilizable, 2) buscamos una mejor organización del código y menos duplicación, 3) queremos modificar valores de forma global con una única edición. Sin embargo, es preferible utilizar variables constantes **const** o uniformes si no se requiere un comportamiento de sustitución como el que ofrece un macro. https://docs.godotengine.org/en/stable/tutorials/shaders/shader_reference/shader_preprocessor.html

La matriz utilizada anteriormente no posee argumentos, pero también podríamos escribirla de la siguiente manera:

```

shader_type spatial;

#define TRANSFORMATION_MATRIX(s) mat3(vec3(s, 0, 0), vec3(0, s, 0),
↳ vec3(0, 0, s))

// El código continúa en la página siguiente.

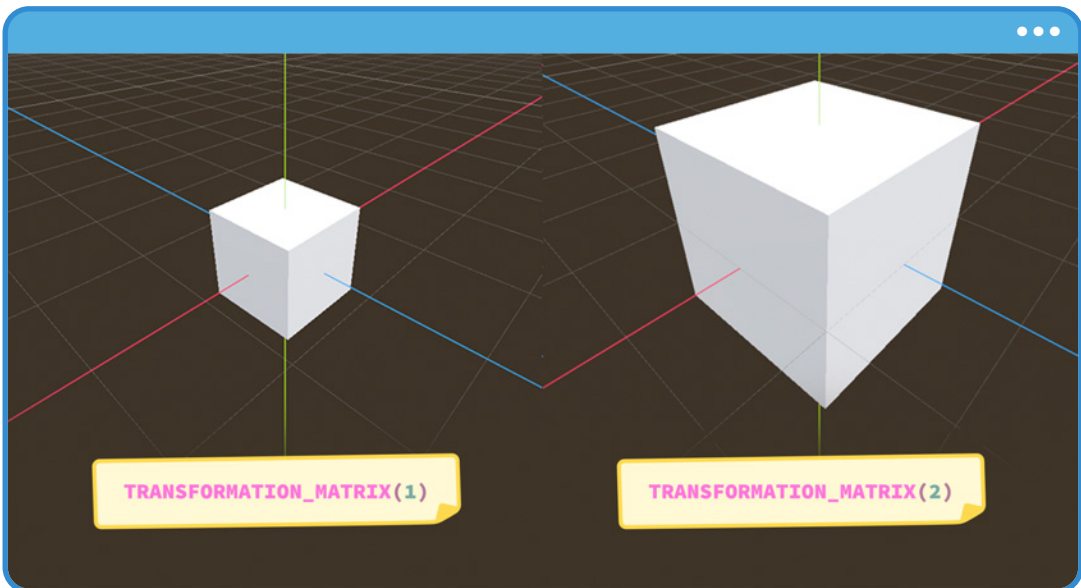
```

```
void vertex()
{
    VERTEX = VERTEX * TRANSFORMATION_MATRIX(1);
}
```

En este caso, se ha definido un macro con un argumento: **s**, el cual modifica los valores de la matriz de identidad. Esta configuración es perfecta, por ejemplo, para aplicar una transformación de escala al objeto 3D. Basta con modificar el valor del argumento para aumentar o disminuir el tamaño del objeto en tiempo real.

- Si **s** = 1, el **BoxMesh** mantiene su tamaño original.
- Si **s** = 2, el **BoxMesh** tendrá el doble del tamaño en todos los ejes.
- Si **s** = 0.5, el **BoxMesh** se reducirá a la mitad.

Este enfoque permite hacer pruebas rápidas y controlar visualmente el efecto de la matriz sin modificar varias líneas de código. Más adelante, podríamos reemplazar este valor por un uniform para permitir su edición desde el **Inspector**.



(1.12.f Se ha aplicado una escala al **BoxMesh**)

Ahora bien, ¿por qué el **BoxMesh** se escala cuando modificamos el valor de la matriz? Esto se debe a que estamos alterando directamente las posiciones originales de cada vértice

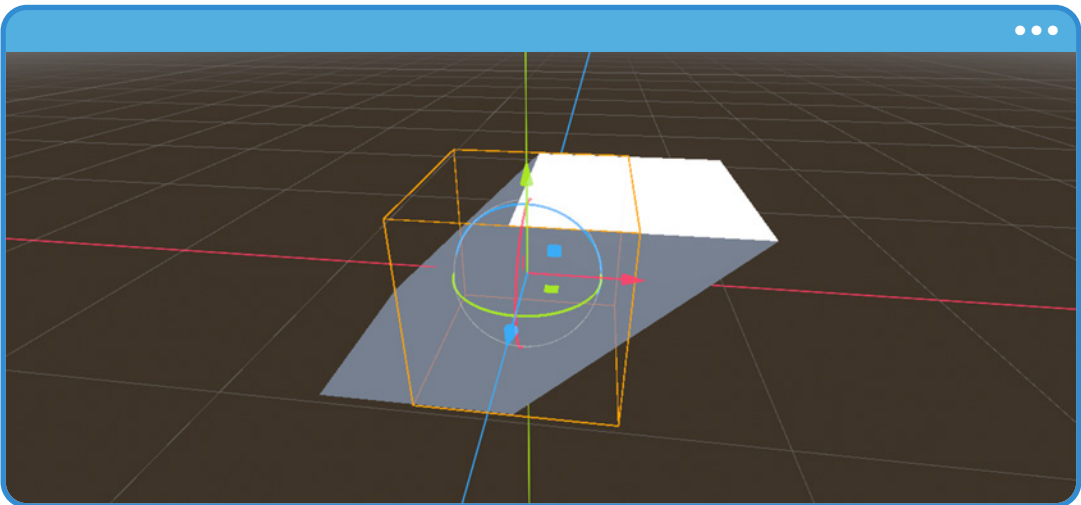
del objeto 3D, antes de que estos sean transformados al espacio global mediante la matriz **MODEL_MATRIX**. En otras palabras, la transformación se aplica en espacio de objeto, justo en el punto donde el vértice aún conserva su posición local. Esto nos permite modificar la geometría del objeto de forma directa, sin necesidad de afectar su transformación global o jerarquía de escena.

A la matriz de transformación también podemos agregar una operación de **shearing**. Para ello, simplemente debemos introducir nuevos valores en la matriz, modificando la manera en que un eje afecta al otro.

Por ejemplo, si deseamos aplicar **shearing** sobre el eje x en función del eje y , podríamos modificar la matriz de la siguiente manera:

```
#define TRANSFORMATION_MATRIX(s) mat3(vec3(s, 1, 0), vec3(0, s, 0), vec3(0, 0, s))
```

En este caso, se ha introducido un nuevo valor 1 en el segundo componente del primer vector tridimensional de la matriz. Esto significa que el eje x ahora está influenciado por el valor del eje y , lo que inclina la forma del objeto a medida que se incrementa la posición vertical de cada vértice.

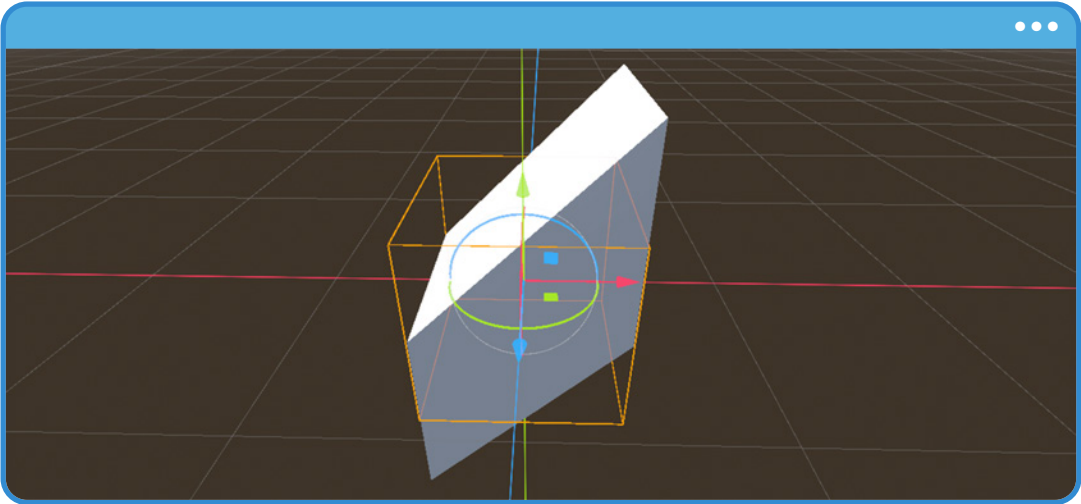


(1.12.g Shearing sobre la coordenada x)

Lo mismo aplica para otras combinaciones de ejes, como x y z . Por ejemplo, si deseamos generar **shearing** sobre la coordenada y , simplemente debemos agregar un valor numérico en el primer componente del segundo vector tridimensional dentro de la matriz, como se muestra a continuación:

```
#define TRANSFORMATION_MATRIX(s) mat3(vec3(s, 0, 0), vec3(1, s, 0), vec3(0, 0,
s))
```

Esta configuración introduce una influencia del eje x sobre el eje y . Es decir, el desplazamiento horizontal de cada vértice afecta su posición vertical, inclinando el objeto diagonal hacia costado, de manera proporcional a su posición en el eje x .



(1.12.h Shearing sobre la coordenada y)

Lo cual es útil para crear efectos visuales como deformaciones animadas, simulación de peso, inclinación dinámica o incluso vibraciones controladas. La clave está en entender cómo un eje afecta al otro dentro de la estructura matricial. Ahora, haz el ejercicio, te invitamos a modificar sus valores y obtener resultados que se adapten a las necesidades de tus proyectos.

Resumen del capítulo.

En este capítulo se adquirieron los fundamentos técnicos necesarios para comprender cómo los objetos 3D son procesados y transformados dentro del sistema de renderizado de Godot, estableciendo la base conceptual para el desarrollo de shaders más avanzados.

Se aprendió a analizar la composición de un mesh, comprendiendo sus propiedades principales y la estructura de sus datos internos, lo que permitió entender cómo la geometría es interpretada por la GPU. Asimismo, se estudiaron los espacios de coordenadas y su rol dentro del pipeline gráfico, junto con la correcta configuración de un objeto único para trabajar de manera controlada en dichos espacios.

También se comprendieron las etapas de procesamiento y el Rendering Pipeline, identificando el recorrido que siguen los datos desde el mesh hasta el fragmento final renderizado en pantalla. En este contexto, se profundizó en el uso del espacio global y se introdujo el espacio de tangente, entendiendo su importancia en el desarrollo de efectos visuales y su implementación práctica.

Posteriormente, se aprendieron los fundamentos de la interpolación lineal entre colores como una de las operaciones matemáticas básicas utilizadas en shaders. Finalmente, se adquirieron los conceptos esenciales sobre matrices, abordando tanto las matrices internas proporcionadas por el motor como la creación e implementación de matrices personalizadas, consolidando una base sólida para la manipulación espacial y las transformaciones geométricas utilizadas a lo largo del libro.



Capítulo 2

Iluminación y renderizado.

En este capítulo continuaremos nuestros estudios sobre shaders, esta vez con un enfoque orientado a la iluminación y el renderizado. Pero, ¿cómo se calcula exactamente la luz dentro de un video juego? ¿Cuáles son las fórmulas matemáticas que permiten simular la interacción entre la luz y los objetos?

Cada vez que aplicamos un material sobre una superficie, ya sea en 2D o en 3D, estamos activando una serie de propiedades que van mucho más allá de una simple textura. Estas propiedades incluyen el cálculo de normales, la oclusión, el reflejo especular, la atenuación de la luz, y otros fenómenos físicos que contribuyen a la apariencia final del objeto en pantalla.

Comprender cómo funcionan estos procesos es esencial para poder modificarlos, optimizarlos o incluso reinventarlos mediante shaders personalizados. A lo largo de este capítulo analizaremos cómo la GPU interpreta la luz, cómo se comportan los diferentes tipos de iluminación en Godot, y qué herramientas tenemos a nuestra disposición para controlar el aspecto final de una escena.

Antes de comenzar con los ejemplos prácticos, revisaremos los principios básicos del modelo de iluminación más utilizado en gráficos en tiempo real, el modelo **Lambertiano**, así como las funciones que nos permiten acceder a la información como la dirección de la luz, la normal del objeto o la posición de la cámara. Este conocimiento será la base para crear efectos visuales avanzados como la iluminación estilo toon, halftone u otro material complejo.

2.1 Propiedades de un material.

Cada vez que deseamos asignar un nuevo material a nuestros objetos 3D, Godot nos pregunta qué tipo de material deseamos crear. Por ejemplo, si hacemos clic sobre el campo **<empty>** en la propiedad **Material Override** de un **MeshInstance3D**, veremos que se despliegan tres opciones:

- 1 **StandardMaterial3D.**
- 2 **ORMMaterial3D.**
- 3 **ShaderMaterial.**

Hasta ahora, hemos trabajado principalmente con el tercero en la lista, es decir, **Shader-Material**, mientras desarrollamos nuestros propios shaders. Sin embargo, ¿para qué sirven los primeros dos? O mejor aún, ¿en qué se diferencian? iniciemos comparándolos. Esto nos permitirá entender mejor en qué consisten estas distintas configuraciones y cuándo conviene utilizar cada una.

Según la documentación oficial en Godot, **StandardMaterial3D**:

“

Las propiedades de **StandardMaterial3D** se heredan de **BaseMaterial3D**. **StandardMaterial3D** utiliza texturas independientes para la oclusión ambiental, la rugosidad y los mapas metálicos. Para usar un único mapa ORM para las tres texturas, utilice **ORMMaterial3D**.

”

La principal diferencia entre un **StandardMaterial3D** y un **ORMMaterial3D** radica en el uso del mapa **ORM**, una textura única en formato RGB que encapsula tres texturas diferentes, cada una en uno de sus canales:

- O – Occlusion (Oclusión): se almacena en el canal R.
- R – Roughness (Rugosidad): se almacena en el canal G.
- M – Metallic (Metálico): se almacena en el canal B.

Este enfoque puede resultar algo confuso para quienes se están iniciando en el lenguaje de shaders. Por ejemplo, cuando crearemos una textura en formato .png, esta incluye cuatro canales por defecto: RGBA. Entonces, ¿cómo es posible colocar diferentes texturas en canales individuales de una sola imagen?

La clave está en comprender el tipo de información que representan estas texturas. Todas ellas, oclusión, rugosidad y metalicidad, son texturas en escala de grises. Esto significa que sus cuatro canales contienen el mismo valor, ya que no representan color, sino intensidad. Por lo tanto, basta con almacenar una de ellas en un solo canal (por ejemplo, solo en el rojo para la oclusión), ignorando los demás.

Nota

Aprovechando esta característica, podemos combinar tres texturas (usualmente en escala de grises) en una sola imagen: cada una en un canal distinto. Esta técnica, conocida como Channel Packing, no solo simplifica la gestión de recursos gráficos, sino que también mejora el rendimiento, ya que reduce la cantidad de texturas que la GPU necesita cargar y procesar durante la renderización.

Ahora bien, ¿cuál es la diferencia entre estos dos tipos de materiales y un **ShaderMaterial**? Hagamos una comparación entre los tres para comprender mejor su naturaleza y uso:

StandardMaterial3D,

- Es un material basado en la técnica **PBR** (Physically Based Rendering) utilizado en objetos 3D.
- Se configura fácilmente, ya que ofrece soporte nativo para mapas de normales, rugosidad, metalicidad, oclusión, fresnel, entre muchas otras propiedades.
- También permite transparencias, sombras, refracción, dispersión (scattering) y múltiples coordenadas UV.
- Su shader ya viene predefinido, por lo que no es necesario escribir código para utilizarlo. Sin embargo, puede resultar costoso en términos de rendimiento para la GPU, especialmente en dispositivos de gama baja.

ORMMaterial3D,

- Al igual que el anterior, es un material **PBR** utilizado en objetos 3D, con la diferencia principal de que utiliza una textura combinada del tipo **ORM**.
- Conserva todas las propiedades disponibles en **StandardMaterial3D**, pero ha sido ligeramente optimizado para un mejor rendimiento al reducir la cantidad de texturas requeridas.
- Su shader también está predefinido, por lo que no requiere escritura de código.

ShaderMaterial,

- A diferencia de los anteriores, este material no incluye propiedades predefinidas; en su lugar, ofrece control total al desarrollador para definir las manualmente.
- Puede utilizarse tanto en objetos 2D como 3D, dependiendo cómo se configure.

- Si bien cuenta con una iluminación básica por defecto, para aprovechar todo su potencial es necesario escribir código en el lenguaje de shader de Godot (**.gdshader**).
- Su nivel de optimización depende completamente del contenido del shader: cuanto más complejo sea el código y mayor la cantidad de operaciones realizadas, mayor será su carga para la GPU.

En conclusión, **StandardMaterial3D** y **ORMMaterial3D** son materiales listos para usar incluidos en Godot. Ambos permiten empezar rápidamente en el desarrollo de un proyecto sin necesidad de escribir código, ya que todos sus cálculos de iluminación y efectos visuales básicos están incorporados por defecto.

Por otro lado, **ShaderMaterial** nos brinda un control total sobre lo que se dibuja en pantalla. Esto lo convierte en una herramienta ideal para la creación de efectos visuales personalizados, materiales estilizados o simulaciones complejas. De hecho, con el uso adecuado de shaders, es posible replicar el comportamiento de los materiales anteriores desde cero, escribiendo nuestro propio código.

Eso es precisamente lo que haremos a lo largo de este capítulo: recrearemos muchas de las propiedades incluidas en los materiales por defecto, con un enfoque en efectos visuales personalizados. Esta práctica nos permitirá comprender a fondo los cálculos de iluminación involucrados en la creación de materiales para nuestros proyectos.

2.2 Introducción al modelo Lambertiano.

Uno de los modelos de iluminación más simples (y a la vez más común) que podemos encontrar en gráficos por computadora es el modelo **Lambertiano**, el cual se basa en una observación realizada por **Johan Heinrich Lambert** en el siglo XVIII. Pero, ¿cómo funciona este modelo? Para poder entenderlo, debemos considerar dos variables:

- La dirección de la fuente lumínica.
- La normal de la superficie.

La cantidad de luz que incide sobre una superficie depende del ángulo que existe entre dicha superficie y la dirección de la fuente luminosa. Si la superficie está orientada directamente hacia la luz, recibe la máxima iluminación posible. En cambio, si se encuentra orientada de forma tangencial o completamente alejada, no recibe luz en absoluto.

Veamos su definición matemática para entenderlo mejor:

$$D = \max(0, \mathbf{n} \cdot \mathbf{l}) \cdot k$$

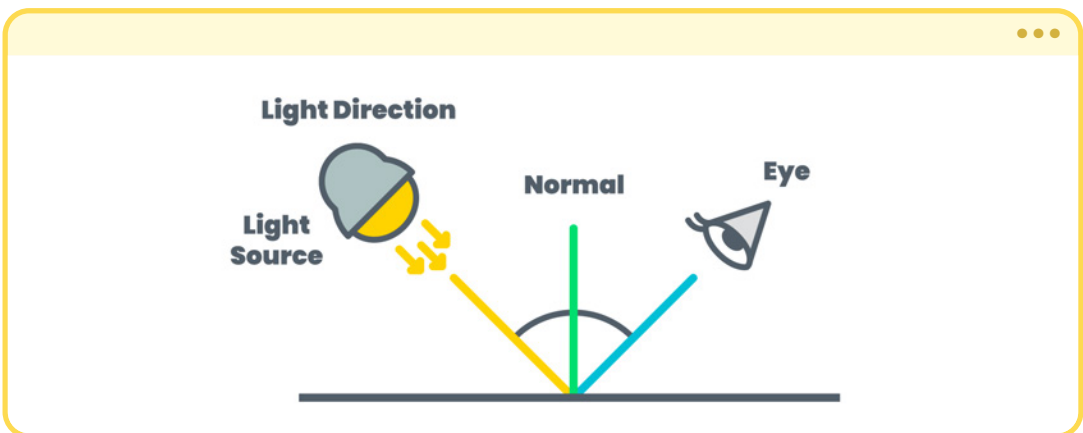
(2.2.a)

Donde:

- D representa la intensidad de la luz difusa que recibe un punto de la superficie.
- k es el coeficiente difuso o el color base de la superficie (es decir, el valor que normalmente se asigna a la variable **ALBEDO** en el método **fragment()**).
- i es la intensidad de la luz.
- $\mathbf{n}$ es la normal de la superficie (vector unitario).
- $\mathbf{l}$ es la dirección de la luz (también un vector unitario).

El producto punto entre $\mathbf{n}$ y $\mathbf{l}$, es decir $(\mathbf{n} \cdot \mathbf{l})$, equivale al coseno del ángulo θ entre ambos vectores. Esto se debe a que, al ser unitarios (su longitud es igual a 1), su producto punto se puede interpretar directamente como $\cos(\theta)$. Esta relación resulta muy útil, tanto en el desarrollo de fórmulas como en la implementación de shaders.

Es cierto que podríamos añadir más elementos a esta ecuación, como el color de la luz, su atenuación por distancia u otros factores. Sin embargo, esta versión simplificada del modelo Lambertiano es suficiente para comprender la lógica fundamental detrás del cálculo de iluminación difusa.



(2.2.b Diagrama ilustrando el sombreado Lambertiano)

Si observamos el diagrama de la Figura 2.2.b, notaremos que en la geometría se incluye la vista (o cámara). Sin embargo, esta no aparece representada en la Ecuación 2.2.a. ¿Por qué razón?

La explicación es sencilla: el modelo **Lambertiano** representa una reflexión puramente difusa. Esto significa que la luz se dispersa de manera uniforme en todas las direcciones, sin depender del ángulo desde el cual se observe la superficie. En otras palabras, la iluminación generada por este modelo es independiente de la posición del observador o la cámara.

Este comportamiento contrasta con los otros modelos de eliminación más complejos, como **Phong** o **Blinn-Phong**, los cuales si tienen en cuenta la dirección de la vista al calcular la reflexión especular de la luz. En esos casos, será necesario incorporar la vista como variable adicional en los cálculos por píxel.

Más allá de la simplicidad de su fórmula, implementar el modelo Lambertiano en Godot resulta sencillo gracias a algunas variables internas que el motor nos proporciona para facilitar este proceso. Por ejemplo:

- D corresponde con la variable **DIFFUSE_LIGHT**.
- n es la normal, representada por **NORMAL**.
- l es la dirección de la luz, representada por **LIGHT**.
- i es intensidad de la luz, pero podemos reemplazarla por **ATTENUATION**.

Finalmente, el término k (el color de la superficie) está representado por la variable **ALBEDO**. Sin embargo, no es necesario incluirlo manualmente en la ecuación, ya que su multiplicación se realiza de forma automática dentro del sistema de renderizado en Godot.

Es importante destacar que los cálculos de iluminación se realizan directamente dentro del método **light()**, el cual se encuentra comentado por defecto. La implementación del modelo Lambertiano en lenguaje **.gdshader** se vería de la siguiente manera:



(2.2.c Se han definido colores para identificar las variables)

El primer paso que debemos llevar a cabo para implementar este modelo es habilitar el método `light()`. En la práctica, este método funciona como una extensión de `fragment()`, ya que también se ejecuta por cada píxel, pero con una diferencia clave: se llama una vez por cada luz que afecte a ese píxel.

Este método está comentado por defecto porque es opcional: solo lo utilizaremos si queremos personalizar cómo la luz interactúa con el material. ¿Y por qué puede resultar costoso para la GPU? Ya sabemos que `fragment()` se ejecuta una vez por cada píxel visible en la pantalla. Por ejemplo, si el juego se ejecuta en una resolución de 1920x1080, `fragment()` se llama unas 2.073.600 veces por cuadro. En cambio, `light()` se ejecuta no solo por cada píxel, sino también por cada luz que lo afecta. Si un píxel está influenciado por 4 luces distintas, el método `light()` se ejecutará 4 veces por el mismo píxel, lo que podría resultar en más de 8 millones de ejecuciones por fotograma dependiendo de la escena.

Por esta razón, es importante usar este método solo cuando sea necesario y con cuidado, ya que puede impactar negativamente en el rendimiento, especialmente en dispositivos con recursos limitados.

2.3 Implementando el modelo Lambertiano.

En esta sección llevaremos a cabo una serie de tareas con el objetivo de comprender no sólo cómo funciona el modelo Lambertiano, sino también cómo utilizar el método `light()` en Godot. Además, exploraremos el modo de renderizado `ambient_light_disabled`, lo que nos permitirá identificar con precisión qué fuentes de luz están afectando a nuestro personaje de forma predeterminada.

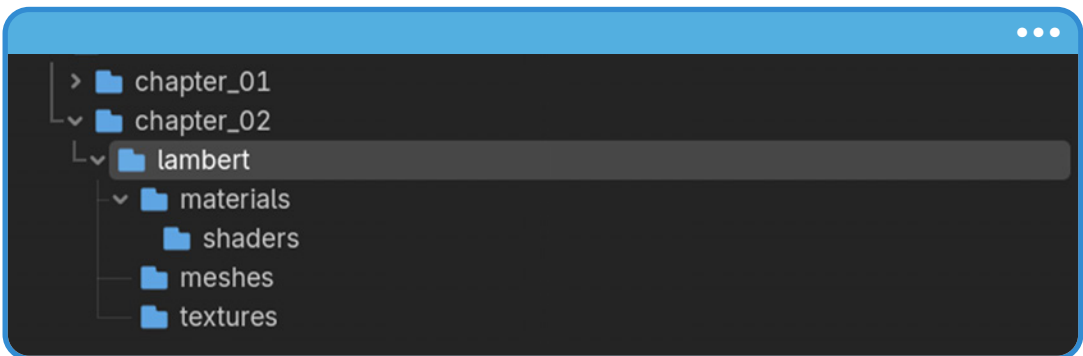
Para lograrlo, realizaremos las siguientes acciones:

- Trabajaremos con el método `light()`.
- Implementaremos el modelo Lambertiano.
- Transformaremos el resultado de iluminación a un estilo toon simple.

Sin embargo, antes de comenzar con la parte técnica, configuraremos nuestro proyecto como lo hemos hecho en secciones anteriores. Para ello, seguiremos estos pasos:

- 1 Dentro de nuestro proyecto, en **assets**, crearemos una nueva subcarpeta llamada **chapter_02**, donde almacenaremos todos los recursos desarrollados en este capítulo.
- 2 Dentro de **chapter_02**, añadiremos otra carpeta con el nombre **lambert**.
- 3 Finalmente, dentro del **lambert**, organizaremos el contenido creando las siguientes carpetas:
 - a **materials**.
 - b **shaders**.
 - c **meshes**.
 - d **textures**.

Si hemos seguido los pasos correctamente, la estructura del proyecto debería verse de la siguiente manera:



(2.3.a Se ha incluido la carpeta **lambert** dentro de **chapter_02**)

A continuación, implementaremos el ejemplo ilustrado en la Figura 2.2.c. Para ello, crearemos un nuevo **shader** y habilitaremos el método `light()` con el fin de poner en práctica las distintas propiedades del modelo Lambertiano.

Durante este ejercicio, utilizaremos el modelo **suzanne.fbx**, el cual deberá importarse dentro de la carpeta **meshes** que creamos previamente. Este archivo está incluido en el paquete descargable adjunto al libro.

Nota

Dado que el modelo utiliza la extensión **.fbx**, será necesario seguir uno de los enfoques descritos previamente. Podemos aplicar el procedimiento explicado en la sección 1.4, abriendo el archivo mediante la opción **New Inherited Scene** y seleccionando posteriormente **Make Unique** para guardar una copia editable del modelo dentro del proyecto. Alternativamente, es posible marcar la instancia como **Make Local**, tal como se explicó en la sección 1.8.

En esta sección, el ejercicio se desarrollará siguiendo el flujo de trabajo descrito en la sección 1.4.

Comenzaremos creando un nuevo **material**. Para ello, sobre la carpeta **materials**, haremos clic derecho y seleccionamos:

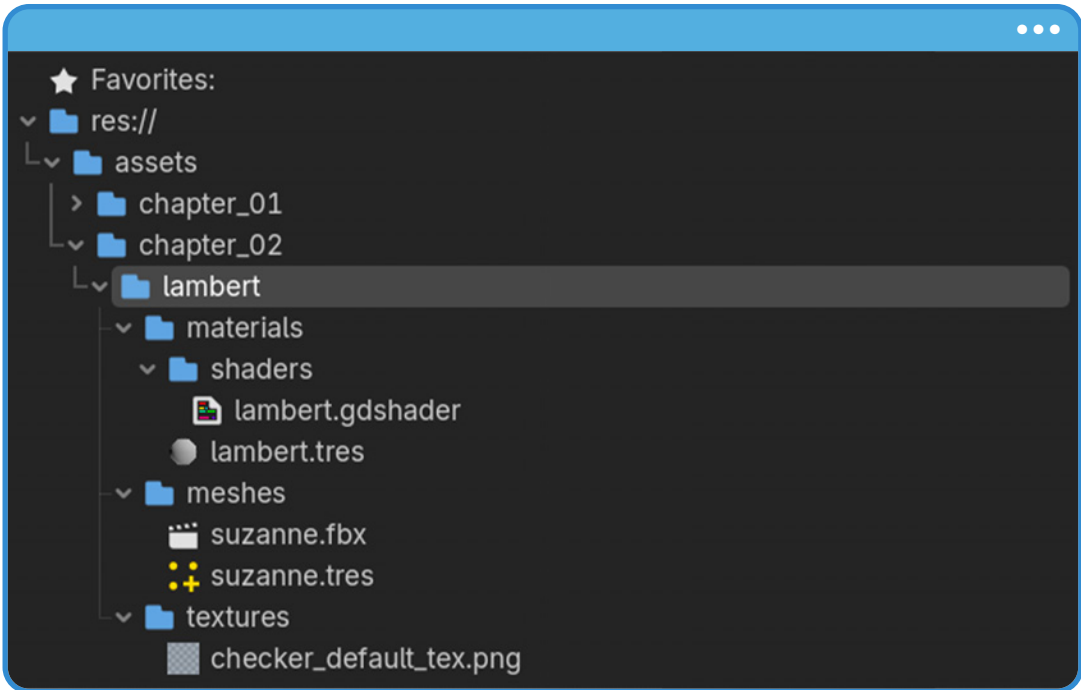
➤ Create New > Resource > ShaderMaterial.

Por razones prácticas, nombraremos a este material como **lamBERT**. A continuación, crearemos el **shader** asociado. En la carpeta **shaders**, haremos clic derecho y seleccionamos:

➤ Create New > Resource > Shader.

Usaremos el mismo nombre (lamBERT) para el shader, de modo que la relación entre ambos recursos sea fácil de identificar y mantener organizada.

Si todo ha sido configurado correctamente, la estructura del proyecto debería verse de la siguiente manera:



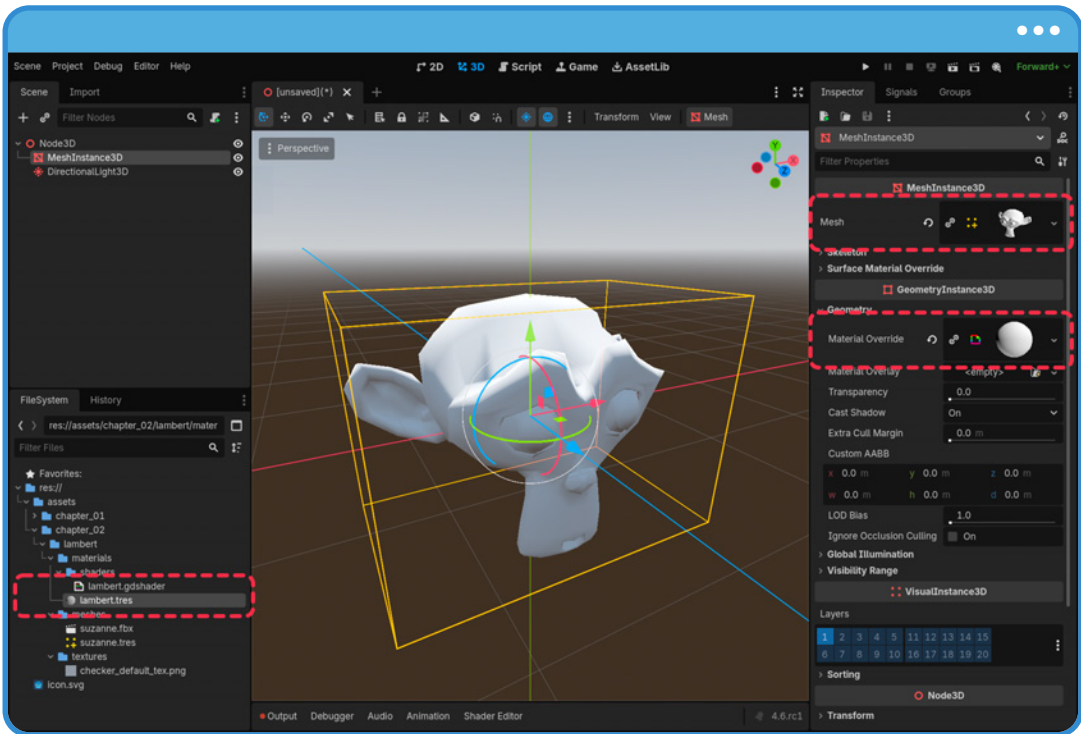
(2.3.b Se han incluido los recursos y se ha marcado a **suzanne** como único)

En este punto podríamos comenzar a editar nuestro shader de forma inmediata. Sin embargo, antes de hacerlo, nos aseguraremos de configurar correctamente la escena. Esto es importante, ya que al habilitar el método `light()` se producirán ciertos cambios en la iluminación del objeto, lo cual nos permitirá analizar cómo funciona la configuración lumínica por defecto en Godot.

Para la escena, utilizaremos un nodo principal **Nodo3D**, y como hijo añadiremos un **MeshInstance3D**, al cual le asignaremos el archivo **suzanne.tres** que guardamos previamente. Como parte del proceso, recordemos lo siguiente:

- El shader **lambert** debe estar asignado a su correspondiente **ShaderMaterial**.
- El **ShaderMaterial** debe estar asignado al mesh, específicamente en la propiedad **Material Override** del **MeshInstance3D**.
- También agregaremos una luz de tipo **DirectionalLight3D** a la escena. Esto nos permitirá observar cómo cambian los resultados de iluminación al modificar la dirección de la luz.

Si todos los pasos se han ejecutado correctamente, la escena debería lucir de la siguiente manera:



(2.3.c Se ha configurado el personaje en la escena)

Como se mencionó anteriormente, el primer paso será habilitar el método `light()` en nuestro shader. Para ello, abrimos el shader **lambert** y retiramos los comentarios del método, permitiendo así que su contenido se ejecute.

A continuación, se muestra la estructura inicial del shader:

```
shader_type spatial;

void vertex() { ... }

void fragment() { ... }

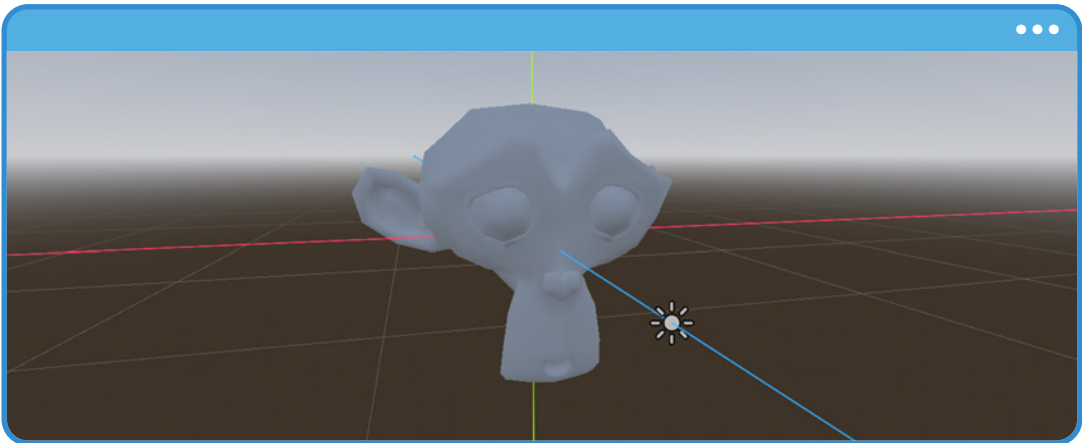
// El código continúa en la página siguiente.
```

```
void light()
{
    // Called for every pixel for every light affecting the material.
    // Uncomment to replace the default light processing function with this...
}
```

Dado que el método **light()** reemplaza el procesamiento de la luz por defecto del motor, al activarlo notaremos un cambio inmediato en la apariencia del objeto en la escena. En particular, la iluminación que afectaba al modelo 3D desaparecerá parcialmente.

Y decimos parcialmente porque, aunque la iluminación directa se elimina, el modelo continúa siendo afectado por la iluminación ambiental que Godot aplica por defecto en todas las escenas. Esta iluminación proviene del entorno y no depende de luces específicas colocadas en la escena, como **DirectionalLight3D**.

Este comportamiento nos servirá para diferenciar claramente los efectos de la iluminación ambiental y los efectos generados por la luz direccional al implementar el modelo Lambertiano.



(2.3.d El personaje posee iluminación de ambiente)

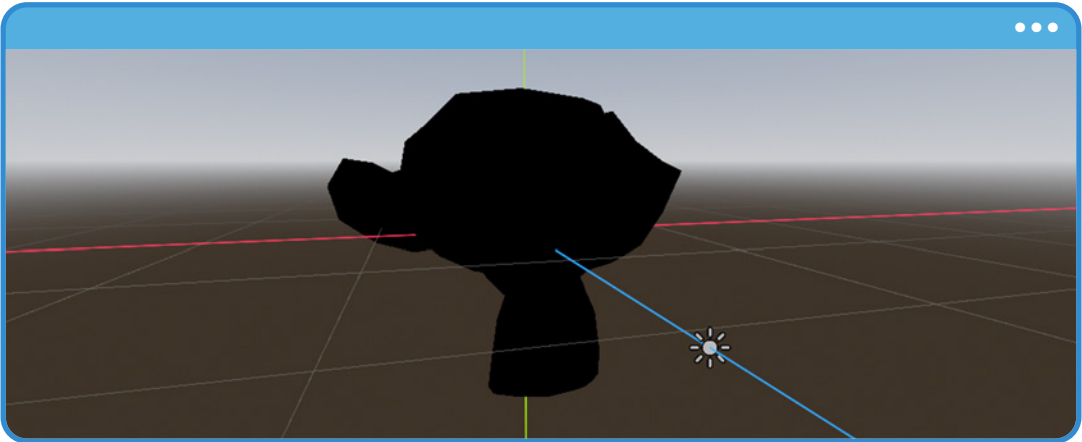
Si quisiéramos deshabilitar la luz de ambiente, debemos emplear el modo de renderizado **ambient_light_disabled**. Esta directiva le indica al motor que omita por completo cualquier contribución lumínica proveniente del entorno (es decir, la iluminación global que se aplica automáticamente si no se especifica otra fuente).

Al incluir esta línea, nuestro personaje se verá completamente oscuro. Esto es perfectamente lógico: no hay fuentes de luz activas en la escena que afecten visualmente al modelo. Sin embargo, esta condición es bastante útil para las pruebas de iluminación que realizaremos a continuación, ya que nos permite observar exclusivamente el resultado del cálculo realizado dentro del método `Light()`.

El código base de nuestro shader, con la directiva aplicada, se verá así:

```
shader_type spatial;  
render_mode ambient_light_disabled;  
  
void vertex() { ... }
```

Después de aplicar este modo de renderizado y con una luz direccional activa en la escena, deberíamos ver que el modelo aparece completamente negro.



(2.3.e Ninguna fuente de luz está afectando a nuestro modelo 3D)

Ahora que nuestro personaje cuenta con una configuración adecuada, procederemos a implementar la función presentada en la Figura 2.2.c, como se muestra a continuación:

```
void light()
{
    float i = ATTENUATION;
    vec3 l = LIGHT;
    vec3 n = NORMAL;
    float D = max(0.0, dot(n, l)) * i;

    DIFFUSE_LIGHT += vec3(D);
}
```

Nota

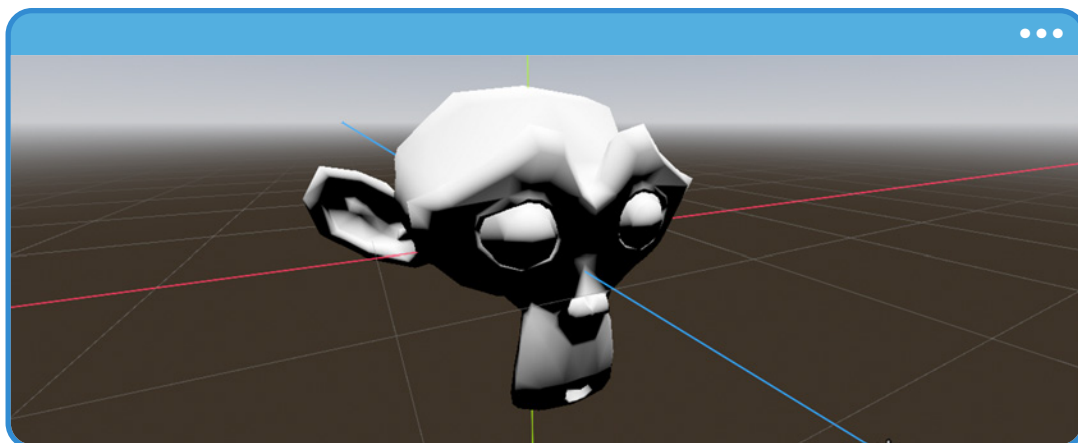
A esta operación podríamos añadir también el color de la luz utilizando la variable **LIGHT_COLOR**, tal como se indica en la documentación oficial de Godot sobre **Light Built-ins**: https://docs.godotengine.org/en/stable/tutorials/shaders/shader_reference/spatial_shader.html

Es importante destacar que los nombres utilizados en esta función (**i**, **l**, **n**, **D**) corresponden a analogías empleadas tradicionalmente al implementar el modelo Lambertiano. Sin embargo, si estuviéramos trabajando en un entorno de producción, sería preferible usar nombres más descriptivos o, directamente, operar con las variables internas proporcionadas por Godot. Esto evita la declaración de variables temporales que solo se utilizan una vez, lo cual puede mejorar la legibilidad y eficiencia del código.

Una versión más concisa y directa del mismo cálculo podría verse así:

```
DIFFUSE_LIGHT += vec3(max(0.0, dot(NORMAL, LIGHT)) * ATTENUATION);
```

No obstante, para fines educativos y de comprensión, en este capítulo utilizaremos nombres intermedios para reforzar la asociación con el modelo matemático original. Una vez que guardemos los cambios y volamos a la escena, deberíamos observar un cambio inmediato en la iluminación de suzanne: ahora responde únicamente a la dirección y presencia de la luz direccional, según el modelo Lambertiano implementado.

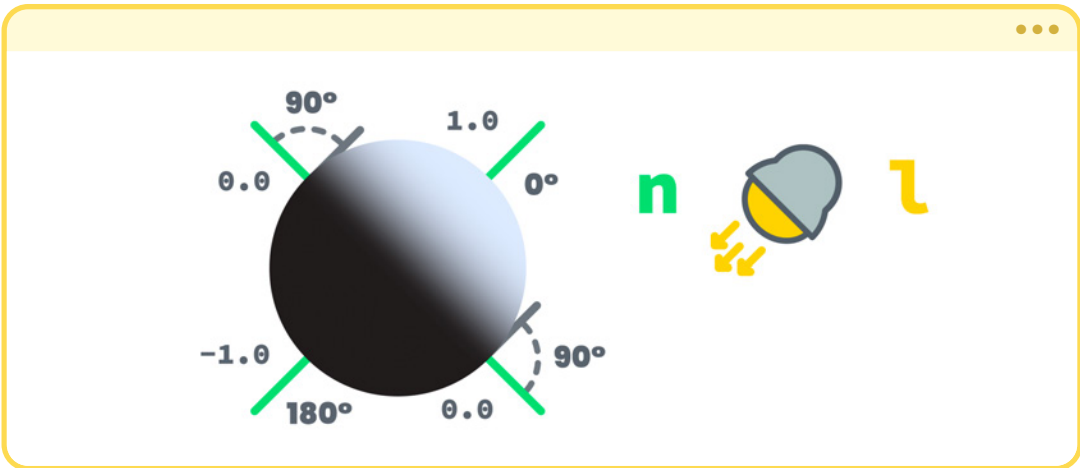


(2.3.f Se ha aplicado el modelo Lambertiano a suzanne)

Como podemos observar, el modelo 3D presenta únicamente dos tonos: luz y sombra (blanco y negro). Este resultado es correcto desde un punto de vista matemático, ya que refleja fielmente el comportamiento del modelo que estamos aplicando en su forma básica.

Ahora bien, ¿por qué este efecto genera una sensación de volumen en el personaje? La clave está en el ángulo que se forma entre las normales de la superficie y la dirección de la luz. A medida que este ángulo varía en cada punto del modelo, también lo hace la intensidad de la luz que incide sobre él. Esta variación suave es lo que produce el sombreado gradual en las formas curvas, como las mejillas o la frente de suzanne, y nos permite percibir profundidad y relieve, incluso en ausencia de color.

Analicemos este principio con mayor detalle en la siguiente figura, donde se ilustra gráficamente la relación entre el ángulo θ , la normal de la superficie y la dirección de la fuente de luz.



(2.3.g Se ha aplicado conceptualmente el modelo Lambertiano a una esfera)

Una esfera, al igual que cualquier modelo 3D, posee un vector normal por cada uno de sus vértices. Sin embargo, con fines ilustrativos, en la figura anterior se han destacado solo cuatro de estas normales. Cada una de ellas apunta en una dirección distinta, determinada por la curvatura y volumen de la superficie.

Si observamos con atención, notaremos que entre las normales y la dirección de la luz forma un ángulo que puede variar desde 0° hasta 180° . Esta relación angular es fundamental para el modelo Lambertiano, ya que determina cuánta luz recibe una determinada zona del objeto.

Recordemos cómo se comporta la función `dot()` en este contexto:

- Devuelve 1.0 (blanco), si el ángulo entre la normal y la luz es de 0° (la luz incide perpendicularmente a la superficie).
- Devuelve 0.0 (negro), si el ángulo es de 90° (la luz incide tangencialmente).
- Devuelve -1.0 (negro saturado, sin utilidad en iluminación difusa), si el ángulo es de 180° (la luz apunta en dirección opuesta).

Sin embargo, el modelo Lambertiano utiliza la función `max(0.0, dot(n, l))` (el mínimo es cero), lo que significa que cualquier valor negativo es descartado, ya que la luz que incide desde “detrás” de la superficie no tiene efecto sobre ella.

Dado que la luz difusa no puede tener valores negativos en la realidad, permitir resultado menores a cero podría producir artefactos visuales, especialmente en las zonas donde la luz debería dejar de tener efecto. Por esa razón, se utiliza **max()**, asegurando que la contribución lumínica sea siempre cero o mayor.

En algunos casos, esta función se reemplaza por **clamp()**, la cual no solo limita el mínimo, sino también el máximo. Este puede ser útil cuando queremos asegurarnos de que el resultado se mantenga dentro de un rango determinado, por ejemplo entre 0.0 y 1.0.

La función **max()** para vectores de tres dimensiones se puede definir de la siguiente manera:

```
vec3 max(vec3 a, vec3 b)
{
    return vec3(
        a.x > b.x ? a.x : b.x,
        a.y > b.y ? a.y : b.y,
        a.z > b.z ? a.z : b.z
    );
}
```

Teniendo esto en consideración, una forma de generar un efecto visual más estilizado es limitar la cantidad de gradiente que se genera entre las zonas iluminadas (blanco) y en sombra (negro). Podemos lograr esto utilizando la función **smoothstep()**, la cual realiza una interpolación suavizada entre dos valores.

La definición de esta función para valores flotantes es la siguiente:

```
float smoothstep(float edge0, float edge1, float x)
{
    float t = clamp((x - edge0) / (edge1 - edge0), 0.0, 1.0);
    return t * t * (3.0 - 2.0 * t);
}
```

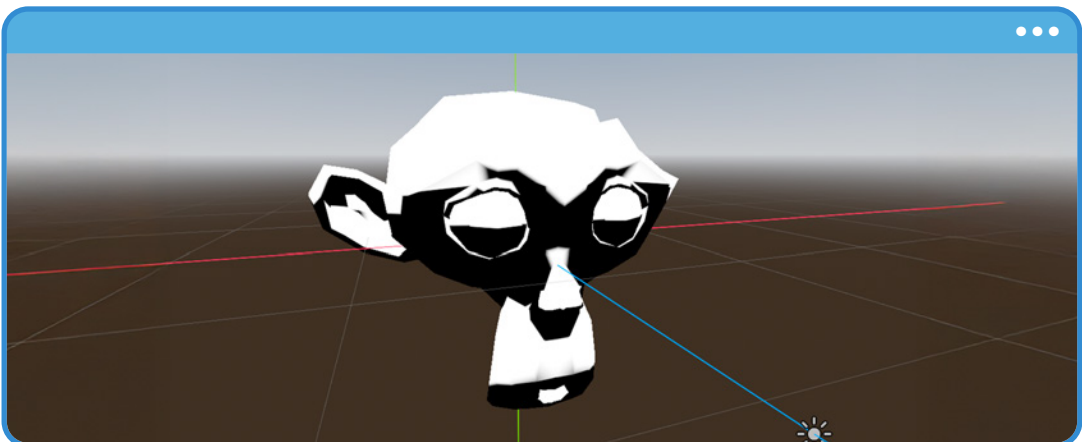
Esta función toma tres argumentos: un valor de entrada **x** y dos límites **edge0** y **edge1**. El resultado es una transición suave entre 0.0 y 1.0, útil para crear transiciones más controladas o para simular zonas de corte difuso en los gradientes de iluminación.

Podríamos aplicar **smoothstep()** directamente sobre el resultado del modelo Lambertiano, con el fin de reducir la franja de valores intermedios y así forzar un contraste más marcado entre luz y sombra. Esto se traduce en una apariencia más estilizada, similar a un efecto toon básico.

```
void light()
{
    float i = ATTENUATION;
    vec3 l = LIGHT;
    vec3 n = NORMAL;
    float D = max(0.0, dot(n, l)) * i;
    D = smoothstep(0.0, 0.05, D);

    DIFFUSE_LIGHT += vec3(D);
}
```

Esta operación genera una iluminación más binaria, donde la transición entre zonas claras y oscuras se produce de forma más abrupta pero aún suavizada, generando un efecto visual ideal para estilos artísticos.



(2.3.h Gradiente suavizado)

Otro efecto interesante que podríamos implementar es modificar el tono de iluminación que recibe nuestro personaje. Para ello, podemos utilizar la función `mix()`, interpolando suavemente entre dos colores y generar un nuevo vector tridimensional en función de un valor de mezcla.

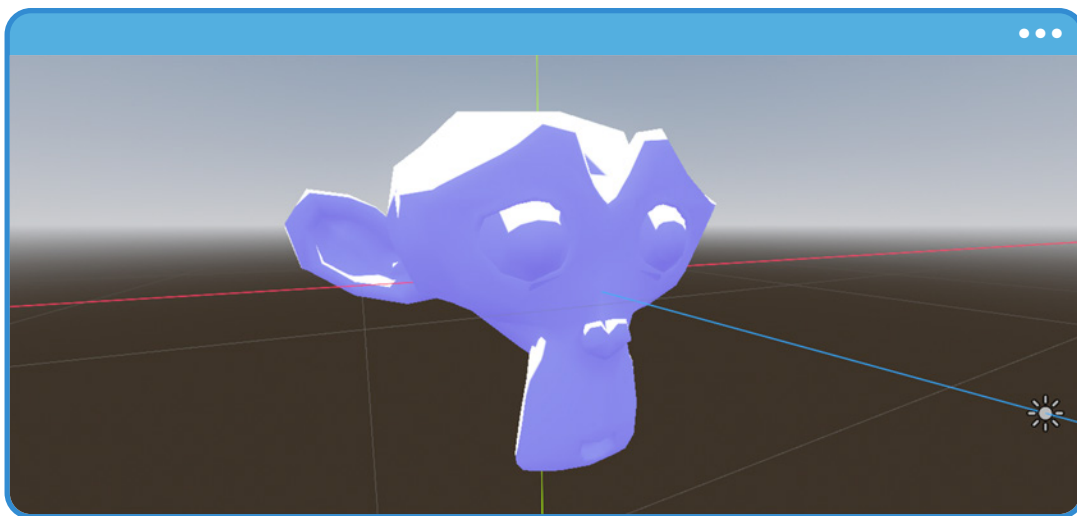
```
void light()
{
    float i = ATTENUATION;
    vec3 l = LIGHT;
    vec3 n = NORMAL;
    float D = max(0.0, dot(n, l)) * i;

    D = smoothstep(0.0, 0.05, D);
    vec3 diffuse = mix(vec3(0.042, 0.023, 0.534), vec3(1.0), D);

    DIFFUSE_LIGHT += diffuse;
}
```

En este caso, la variable **D** se utiliza como factor de mezcla entre dos colores: un tono azulado oscuro `vec3(0.042, 0.023, 0.534)` y el blanco puro `vec3(1.0)`. A medida que **D** aumenta, es decir, a medida que la superficie del objeto recibe más luz directa, el color resultante se aproxima progresivamente al blanco.

Por el contrario, en las zonas menos iluminadas (donde **D** es cercano a cero), el color conserva su tono azulado. Este comportamiento genera un efecto de sombreado estilizado y personalizado. Ideal para escenas con estética artística o que requieran un look no fotorrealista (non-photorealistic rendering).



(2.3.i La sombra resultante posee un color azulado)

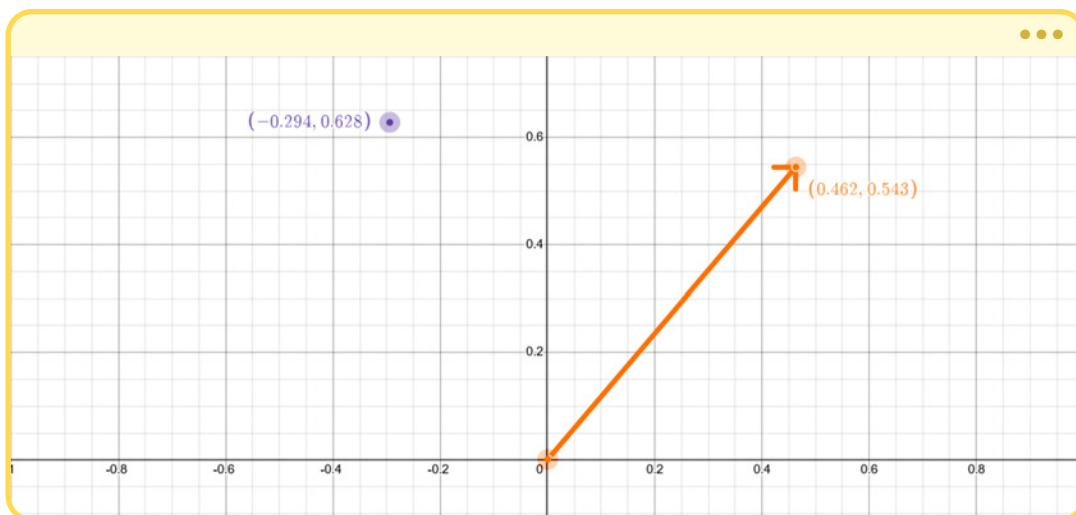
2.4 Vectores: Puntos vs. direcciones.

Hasta este punto, hemos trabajado únicamente con la implementación del modelo Lambertiano. Si has seguido los pasos correctamente, probablemente te encuentres ajustando los valores para obtener distintas tonalidades de iluminación en el renderizado de Suzanne. Pero este proceso genera una pregunta fundamental: ¿por qué ha funcionado? ¿Por qué es posible observar cambios en la tonalidad del personaje al modificar ciertos vectores?

Una pieza clave que no fue mencionada explícitamente en la sección anterior es que, en el contexto de la iluminación, los vectores que manipulamos, como la dirección de la luz o la normal, no representan puntos en el espacio, sino direcciones.

¿Cuál es la diferencia entre un punto y una dirección? Un punto define una posición específica en el espacio tridimensional. Por ejemplo, un vértice indica dónde se encuentra una esquina del modelo en coordenadas locales o globales.

En cambio, una dirección representa una orientación sin una posición absoluta. Es decir, indica hacia dónde “apunta” algo, pero no tiene un origen o ubicación fija. Vectores como **LIGHT**, **VIEW** y **NORMAL** son direcciones unitarias: nos dicen hacia dónde se encuentra la fuente de luz, hacia dónde está mirando la cámara o cómo está orientada la superficie en ese fragmento.



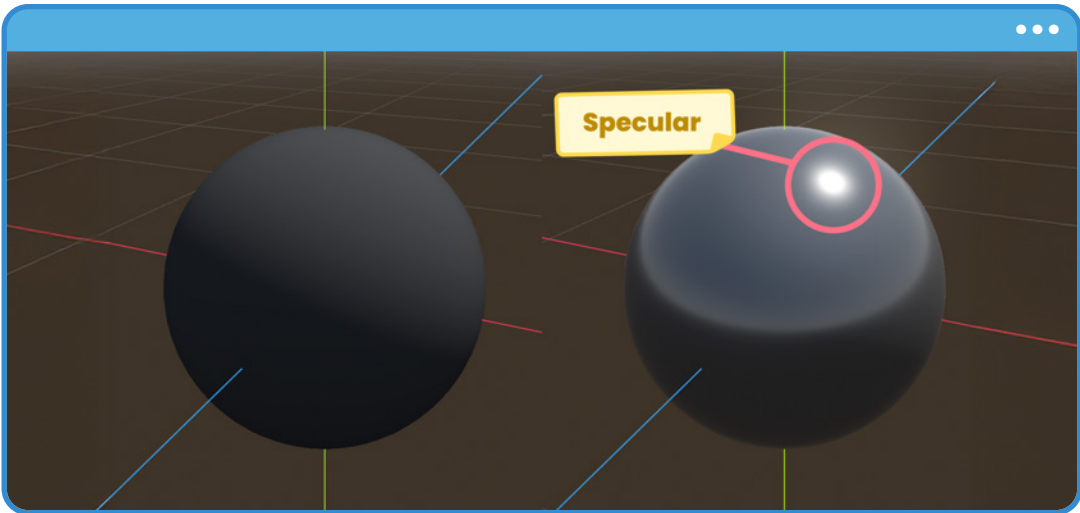
(2.4.a Diferencia entre un punto (purpura) y una dirección (naranja))

Hacer esta distinción es crucial dado que durante este capítulo trabajaremos con direcciones para el cálculo de iluminación. Cuando calculamos el producto punto entre la normal y la dirección de la luz, no estamos comparando posiciones en el espacio, sino la relación angular entre dos orientaciones. Es esta relación la que determina cuánta luz incide sobre una superficie, y por tanto, qué tan iluminada o sombreada se verá.

2.5 Introducción al modelo Blinn-Phong.

Durante esta sección hablaremos sobre el efecto **Specular**, aquel que encontramos en la propiedad **Metallic > Specular** de un material **StandardMaterial3D**. Este nos permite renderizar superficies con reflejos brillantes, simulando cómo la luz se refleja en materiales pulidos. Para poder entenderlo, debemos tomar en consideración tres propiedades principalmente:

- La dirección de la fuente lumínica.
- La normal de la superficie.
- La dirección de la vista (cámara).



(2.5.a A la izquierda, Lambert; a la derecha, Specular)

Este modelo, ampliamente utilizado en gráficos por computadora, proviene del trabajo de **Bui Tuong Phong**, quien propuso un método para agregar brillo especular a una superficie en función de la orientación de sus normales.

Según el modelo de **Phong** original, si deseamos simular la reflectancia especular, debemos realizar la siguiente operación matemática:

$$S = \max(0, \mathbf{r} \cdot \mathbf{v})^m$$

(2.5.b)

Aquí, $\mathbf{r}$ es el vector de reflexión de la luz, $\mathbf{v}$ es la dirección del observador (cámara), $\mathbf{n}$ es la normal, y m es el exponente de brillo, que determina la concentración del reflejo. Un valor bajo en m , por ejemplo $m = 8$, produce un brillo amplio y difuso, mientras que un valor alto, es decir $m = 256$ produce un reflejo pequeño y muy concentrado.

La implementación del modelo de Phong en lenguaje **.gdshader** se vería de la siguiente manera:

$$S = \max(0.0, r \cdot v)^m$$

```
void light()
{
    vec3 n = NORMAL;
    vec3 v = VIEW;
    vec3 l = LIGHT;
    vec3 r = reflect(-l, n);
    float m = 64.0;

    float s = pow(max(0.0, dot(r, v)), m);
    SPECULAR_LIGHT = vec3(s);
}
```

(2.5.c Se han definido colores para identificar cada variable)

Sin embargo, en gráficos en tiempo real como Godot, es común emplear una variante optimizada conocida como el modelo **Blinn-Phong**. En lugar de calcular el vector de reflexión, este modelo utiliza un vector intermedio denominado **halfway** h , que representa el punto medio entre la dirección hacia la luz, y la dirección hacia la cámara. Lo cual nos lleva a una nueva fórmula especular, similar en apariencia a la del modelo Lambertiano (Figura 2.2.a), pero con un enfoque distinto.

$$S = \max(0, n \cdot h)^m (n \cdot l > 0)$$

(2.5.d)

Su implementación en lenguaje **.gdshader** se ve de la siguiente manera:

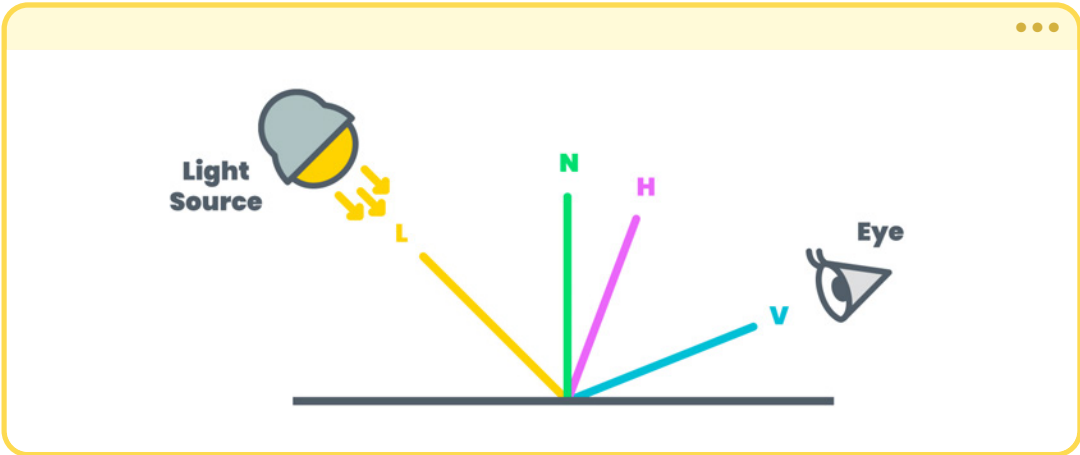
$$S = \max(0, n \cdot h)^m (n \cdot l > 0)$$

```
void light()
{
    vec3 n = NORMAL;
    vec3 l = LIGHT;
    vec3 v = VIEW;
    vec3 h = normalize(l + v);
    float m = 64.0;

    float s = pow(max(0.0, dot(n, h)), m);
    s *= float(dot(n, l) > 0.0);
    SPECULAR_LIGHT = vec3(s);
}
```

(2.5.e Se han definido colores para identificar cada variable)

Cuanto más alineada esté la normal del objeto con el vector h , más intenso será el brillo especular. Esto genera reflejos suaves y realistas que dependen del ángulo de observación, a diferencia del modelo Lambertiano, que no considera la posición de la cámara.



(2.5.f Diagrama ilustrando el modelo Blinn-Phong)

2.6 Implementando el modelo Blinn-Phong.

En esta sección, implementaremos el modelo de iluminación **Blinn-Phong** dentro del método `light()`, utilizando las dos configuraciones que analizamos previamente en las Figuras 2.5.b y 2.5.d. Además, aprenderemos a mejorar el aspecto de los brillos especulares transformando la iluminación desde el espacio lineal hacia sRGB, lo cual es esencial para lograr una apariencia visual más precisa en pantalla.

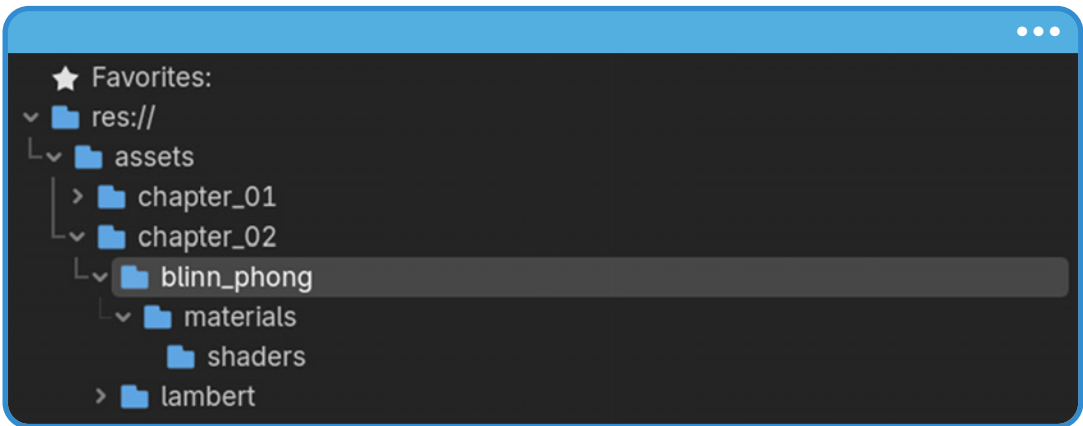
Para ello, seguiremos estos pasos:

- Partiremos desde el shader Lambertiano que ya tenemos implementado.
- Añadiremos ambas variantes de la ecuación especular.
- Aplicaremos la conversión de color desde lineal a sRGB para realzar los brillos.

Antes de comenzar con la codificación, organizaremos nuestro proyecto para mantener una estructura clara y coherente. Realizaremos lo siguiente:

- 1 Dentro de **chapter_02**, crearemos una nueva carpeta llamada **blinn_phong**.
- 2 En su interior, añadiremos las siguientes subcarpetas:
 - a **materials**.
 - b **shaders**.

En esta ocasión, no será necesario importar nuevos modelos ni texturas, ya que trabajaremos con los recursos utilizados en la sección anterior. Si hemos seguido estos pasos correctamente, la estructura de nuestro proyecto debería verse de la siguiente manera:



(2.6.a Se ha incluido la carpeta **blinn_phong** en el proyecto)

Como mencionamos anteriormente, continuaremos trabajando sobre el shader **lambert** que creamos en la sección 2.3. No obstante, para mantener la integridad del shader original, lo duplicaremos antes de hacer cualquier modificación. Para ello, nos posicionamos sobre el archivo del shader, hacemos clic derecho y seleccionamos la opción **Duplicate** (también podemos usar el atajo **Ctrl + D**).

Una vez duplicado, renombraremos el archivo como **blinn_phong** y lo moveremos a la siguiente ruta del proyecto:

- chapter_02 > blinn_phong > materials > shaders.

En cuanto a la escena, volveremos a utilizar la configuración basada en el modelo **suzanne**. Siguiendo el mismo criterio, duplicaremos el archivo **lamBERT.tscn**, lo renombraremos como **blinn_phong.tscn** y lo guardaremos en:

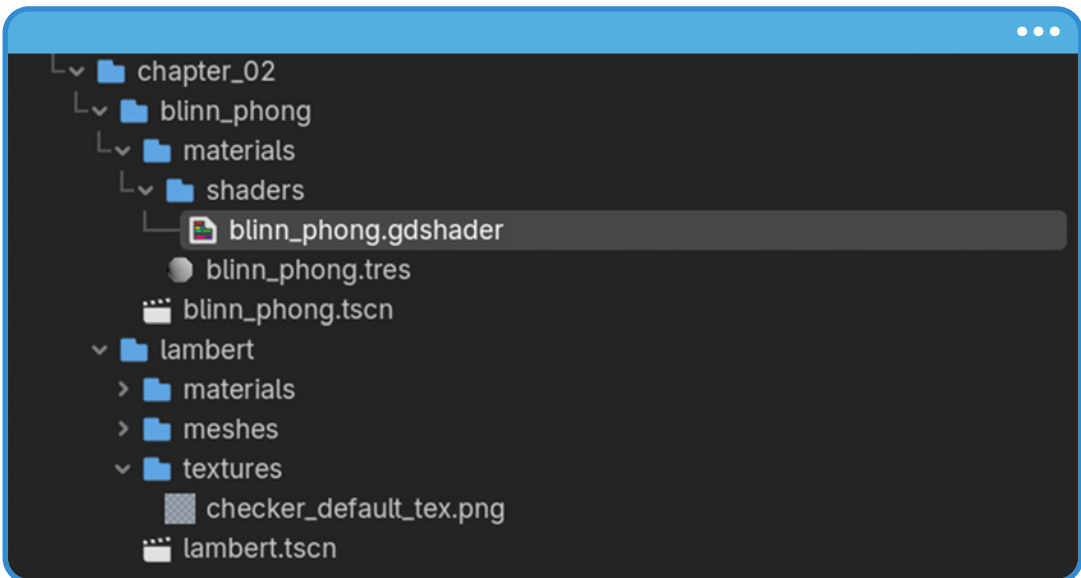
➤ chapter_02 > blinn_phong.

A continuación, crearemos un nuevo material. Para ello, nos posicionamos sobre la carpeta **materials**, hacemos clic derecho y seleccionamos:

➤ Create New > Resources > ShaderMaterial.

Asignaremos también el nombre **blinn_phong** a este material, de manera que todos los elementos estén correctamente vinculados y sea fácil identificar la relación entre ellos dentro de esta sección.

Si hemos seguido todos los pasos correctamente, la estructura del proyecto debería verse de la siguiente manera:



(2.6.b Se ha duplicado el material y la escena **lamBERT**)

Nota

Una vez creado el material y el shader, debemos asegurarnos de asignar el **shader** al **material**, y luego aplicar este material al modelo 3D en la escena. Solo así podremos visualizar los cambios realizados en el código durante la renderización final.

Como ya contamos con la escena configurada con el modelo **suzanne**, procederemos ahora a editar el shader **blinn_phong**. Iniciaremos reestructurando la lógica del modelo **Lambertiano** para encapsularla dentro de una función independiente. Esta práctica no solo mejora la legibilidad del código, sino que también facilita su reutilización.

```
float lambert(float i, vec3 l, vec3 n)
{
    return max(0.0, dot(n, l)) * i;
}

void light()
{
    float i = ATTENUATION;
    vec3 l = LIGHT;
    vec3 n = NORMAL;

    float d = lambert(i, l, n);

    DIFFUSE_LIGHT += vec3(d);
}
```

Si prestamos atención al código, notaremos que la lógica del modelo Lambertiano se mantiene intacta. La única diferencia es que ahora hemos encapsulado la operación principal (el producto punto entre la normal y la dirección de la luz, atenuado por la intensidad) dentro de un método separado llamado **lambert()**.

Nota

En esta nueva versión del shader, se han eliminado las propiedades **_Shadow**, **_Highlight** y **_Smoothness**, ya que nos concentraremos únicamente en la implementación del modelo Blinn-Phong.

Este cambio mejora la modularidad del código, lo que nos permitirá reutilizar esta función en otras partes del shader o modificarla fácilmente si más adelante decidimos ajustar el comportamiento del modelo.

A continuación, implementaremos el modelo de Phong, definido en la ecuación ilustrada en la Figura 2.5.b. Para ello, crearemos una nueva función llamada **phong()**, que calculará el componente especular a partir del vector de vista, la dirección de la luz, la normal de la superficie y el exponente de brillo:

```
float phong(vec3 v, vec3 l, vec3 n, float m)
{
    vec3 r = reflect(-l, n);
    return pow(max(0.0, dot(r, v)), m);
}
```

Nota

Aunque Godot proporciona la variable interna **SPECULAR** dentro del método **fragment()** para calcular la especularidad de forma sencilla, implementar esta función manualmente nos permite comprender en detalle cómo funciona el modelo y, lo más importante, personalizar su comportamiento según nuestras necesidades.

Con ambas funciones listas, podemos actualizar el método **light()** para incluir el cálculo especular del modelo Phong:

```
28 void light()
29 {
30     float i = ATTENUATION;
31     vec3 l = LIGHT;
32     vec3 n = NORMAL;
33     vec3 v = VIEW;
34
35     float d = lambert(i, l, n);
36     float s = phong(v, l, n, 64.0);
```



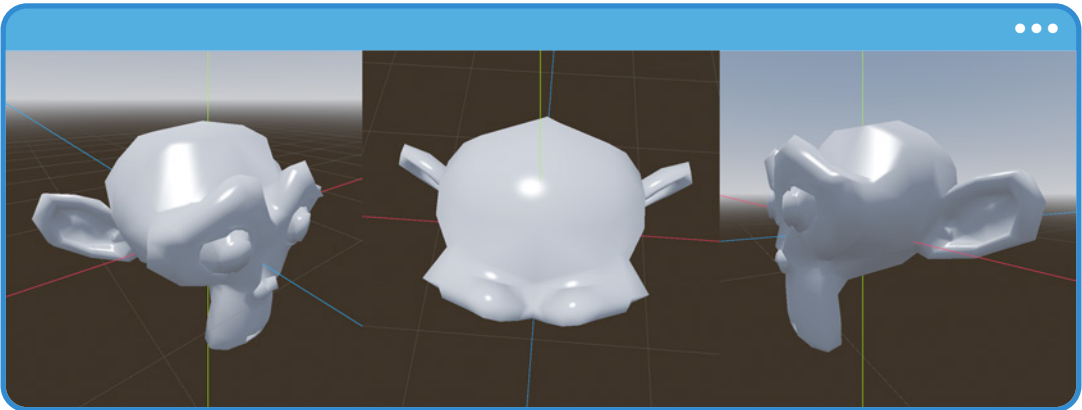
```

37
38     DIFFUSE_LIGHT += vec3(d) * 0.3;
39     SPECULAR_LIGHT += vec3(s);
40 }

```

En este fragmento de código, observamos que en la línea 33, declaramos una nueva variable **v** que almacena la dirección hacia la cámara. Esta dirección es necesaria para calcular el componente especular mediante el método **phong()**. El resultado se almacena en la variable escalar **s** (línea 36) y luego se suma a la salida **SPECULAR_LIGHT** (línea 39).

Este enfoque modular nos permitirá en la siguiente sección reemplazar el modelo Phong por su variante Blinn-Phong, manteniendo el resto de la lógica intacta.



(2.6.c Especificidad Phong con $m = 64.0$)

Nota

En la línea 38, hemos multiplicado el valor de la luz difusa por 0.3 para reducir su intensidad y asegurar que el componente especular sea claramente visible. Además, es importante mencionar que Godot habilita la iluminación ambiental por defecto. Si deseamos desactivarla para realizar pruebas más controladas, podemos usar el macro **ambient_light_disabled**.

Ahora llevaremos a cabo la implementación del modelo **Blinn-Phong**. Como se ha mencionado anteriormente, este modelo modifica el cálculo especular al utilizar un vector intermedio conocido como **halfway** (o half-vector), que representa la dirección promedio entre la luz y la vista. Esta técnica mejora el rendimiento en algunas plataformas y genera resultados visuales más suaves, especialmente en materiales con brillo difuso.

Para implementarlo, declararemos una nueva función llamada `blinn_phong()`, que corresponde a la ecuación presentada en la Figura 2.5.d:

```
float blinn_phong(vec3 v, vec3 l, vec3 n, float m)
{
    vec3 h = normalize(l + v);
    float s = pow(max(0.0, dot(n, h)), m);
    s *= float(dot(n, l) > 0.0);
    return s;
}
```

La línea `vec3 h = normalize(l + v)` calcula el half-vector como la suma normalizada entre la dirección de la luz (**LIGHT**) y la dirección de la vista (**VIEW**). Posteriormente, usamos el producto punto entre `n` (**NORMAL**) y `h` para obtener la intensidad especular, elevada al exponente `m`, que controla la nitidez del brillo. Multiplicamos el resultado por 1.0 o 0.0 según si el ángulo entre `n` y `l` es positivo, lo que evita brillos no deseados en zonas en sombra.

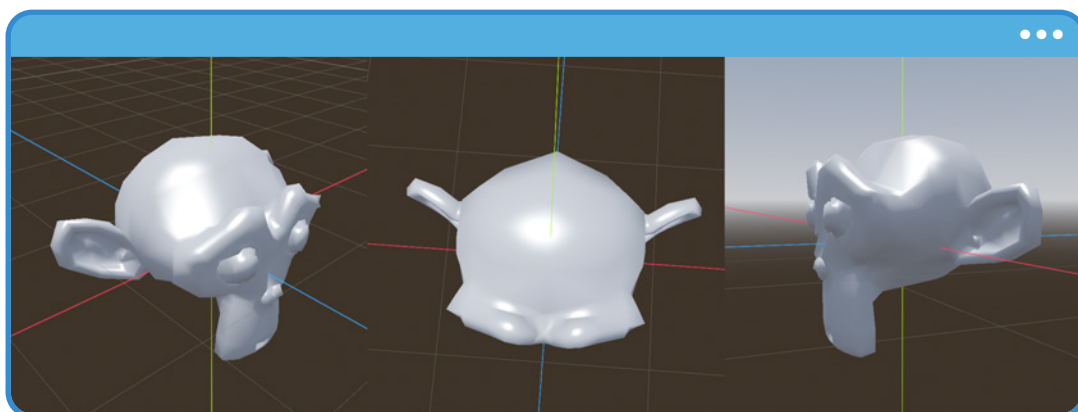
Para aplicar este modelo, simplemente reemplazamos el método `phong()` por `blinn_phong()` dentro del método `light()`. El nuevo código queda de la siguiente manera:

```
36 void light()
37 {
38     float i = ATTENUATION;
39     vec3 l = LIGHT;
40     vec3 n = NORMAL;
41     vec3 v = VIEW;
42
43     float d = lambert(i, l, n);
44     //float s = phong(v, l, n, 64.0);
45     float s = blinn_phong(v, l, n, 64.0);
46
47     DIFFUSE_LIGHT += vec3(d) * 0.3;
48     SPECULAR_LIGHT += vec3(s);
49 }
```

Como podemos ver en la línea 45, donde se declara `s`, simplemente reemplazamos la llamada al modelo Phong por `blinn_phong()`. A pesar de utilizar una lógica diferente internamente, ambos modelos comparten los mismos argumentos:

- Dirección de vista $\mathbf{v}$,
- Dirección de luz $\mathbf{l}$,
- Normal $\mathbf{n}$,
- Y el exponente especular m .

Esta modificación produce un resultado visual similar al de Phong, pero con diferencias sutiles en la forma y distribución del brillo especular



(2.6.d Especificidad Blinn-Phong con $m = 64.0$)

Si comparamos los resultados visuales de las Figuras 2.6.c y 2.6.d, notaremos que ambos modelos producen un efecto similar. Sin embargo, en el caso del modelo Blinn-Phong, el reflejo especular se percibe ligeramente más amplio y suave. Esta diferencia se debe al uso del half-vector en lugar del vector reflejado, lo cual distribuye la intensidad del brillo de manera distinta sobre la superficie.

Ahora bien, independientemente del modelo que empleemos, Phong o Blinn-Phong, es importante recordar que ambos trabajan en un espacio de iluminación lineal. Desde un punto de vista matemático, este comportamiento es correcto. No obstante, el ojo humano no percibe la luz de forma lineal: somos más sensibles a las variaciones de oscuridad que a las de brillo.

Por esta razón, cuando trabajamos con iluminación realista o queremos lograr un resultado visual más llamativo, es recomendable transformar la salida del shader desde espacio lineal a **sRGB**, que se aproxima mejor a la percepción humana. Esta conversión nos permite realzar los brillos y mejorar la fidelidad visual de los materiales.

Dado que esta sección ya ha cubierto una gran cantidad de contenido, abordaremos el tema del espacio de color y la conversión a sRGB con más detalle en la siguiente sección.

2.7 De iluminación lineal a sRGB.

Cuando hablamos de **sRGB** (Standard Red Green Blue), nos referimos al estándar internacional **IEC 61966-2-1, enmienda 1 (2003)**, el cual define de forma precisa el comportamiento del espacio de color sRGB. Este estándar describe no solo el rango de colores (gamut) que puede representarse en una pantalla, sino también cómo estos colores deben almacenarse, interpretarse y reproducirse de forma coherente entre distintos dispositivos.

Hasta este punto del capítulo, hemos explorado diversos espacios y coordenadas. Pero ¿qué significa exactamente espacio de color? En términos simples, se refiere a tres aspectos fundamentales:

- Qué colores pueden representarse.
- Cómo se almacenan esos colores en memoria (encoding).
- Cómo interpretar un color, por ejemplo **vec3(0.143, 0.456, 0.673)**, de forma consistente a en distintos dispositivos y pantallas.

Comprender este concepto es crucial cuando desarrollamos shaders en Godot. Por defecto, las imágenes o texturas que utilizamos (como aquellas exportadas desde Photoshop en formato sRGB) ya vienen con una corrección de gamma incorporada. Sin embargo, los cálculos de iluminación en tiempo real deben realizarse en **espacio lineal** para que los resultados sean físicamente correctos.

Por esta razón, es común encontrar conversiones entre el espacio lineal y el espacio sRGB en distintos puntos del flujo de trabajo del shader. De forma práctica, esto implica que:

- Al **leer** colores desde texturas de tipo albedo o diffuse (que están codificadas en sRGB), primero debemos convertir esos valores al espacio lineal.

- Al **escribir** el resultado final en pantalla, justo después de los cálculos de iluminación, debemos convertir los colores nuevamente al espacio sRGB para que se vean correctamente.

Nota

El espacio de color sRGB debe utilizarse únicamente en imágenes de color al momento de exportar desde un software de edición como Photoshop. Las imágenes en escala de grises (como mapas de normales, rugosidad, etc.) deben mantenerse en espacio lineal para preservar la fidelidad de los datos en el rango [0.0 : 1.0].

Para facilitar estas conversiones en nuestros shaders, podemos definir funciones auxiliares que automaticen el proceso. A continuación, presentamos dos funciones que nos permitirán realizar estas transformaciones de manera precisa:

Para convertir desde sRGB a espacio lineal.

```
vec3 to_linear(vec3 srgb)
{
    vec3 a = pow((srgb + 0.055) / 1.055, vec3(2.4));
    vec3 b = srgb / 12.92;
    bvec3 c = lessThan(srgb, vec3(0.04045));
    return mix(a, b, c);
}
```

Para convertir desde espacio lineal a sRGB.

```
vec3 to_sRGB(vec3 linearRGB)
{
    vec3 a = vec3(1.055) * pow(linearRGB.rgb, vec3(1.0/2.4)) - vec3(0.055);
    vec3 b = linearRGB.rgb * vec3(12.92);
    bvec3 c = lessThan(linearRGB, vec3(0.0031308));
    return vec3(mix(a, b, c));
}
```

Estas funciones permiten una transformación precisa entre ambos espacios de color, asegurando que los brillos y sombras en nuestros materiales sean percibidos correctamente por el ojo humano.

Prestemos atención a la siguiente referencia para entender el concepto:



(2.7.a En la parte superior, sRGB; en la parte inferior, lineal)

Para ilustrar la conversión de color desde el espacio **lineal** al espacio **sRGB**, en la Figura 2.7.a hemos tomado como referencia el primer canal de las coordenadas UV ($uv.x$). Este ejemplo permite visualizar cómo se redistribuyen los valores de luminancia al aplicar la corrección gamma: los tonos intermedios se intensifican, mientras que los valores cercanos a negro ocupan una porción más reducida del rango dinámico.

Este mismo principio puede aplicarse a cualquier valor calculado en espacio lineal. En nuestro caso, utilizaremos esta conversión sobre el componente **especular** para obtener una reflexión visualmente más nítida y brillante.

Sin embargo, debemos tener en cuenta que la función `to_sRGB()` retorna un vector de tres componentes (`vec3`), mientras que el método `blinn_phong()` devuelve un valor escalar (`float`). Para resolver esta incompatibilidad, redefiniremos la variable `s` como un vector tridimensional y aplicaremos la conversión de espacio de color posteriormente.

El siguiente fragmento de código ilustra esta modificación:

```

36 vec3 to_sRGB(vec3 linearRGB)
37 {
38     vec3 a = vec3(1.055) * pow(linearRGB.rgb, vec3(1.0/2.4)) - vec3(0.055);
39     vec3 b = linearRGB.rgb * vec3(12.92);
40     bvec3 c = lessThan(linearRGB, vec3(0.0031308));
41     return vec3(mix(a, b, c));
42 }
43
44 void light()
45 {
46     float i = ATTENUATION;
47     vec3 l = LIGHT;
48     vec3 n = NORMAL;
49     vec3 v = VIEW;
50
51     //vec3 d = vec3(lambert(i, l, n));
52     vec3 s = vec3(blinn_phong(v, l, n, 64.0));
53     s = to_sRGB(s);
54
55     DIFFUSE_LIGHT += vec3(0.0);
56     SPECULAR_LIGHT += s;
57 }

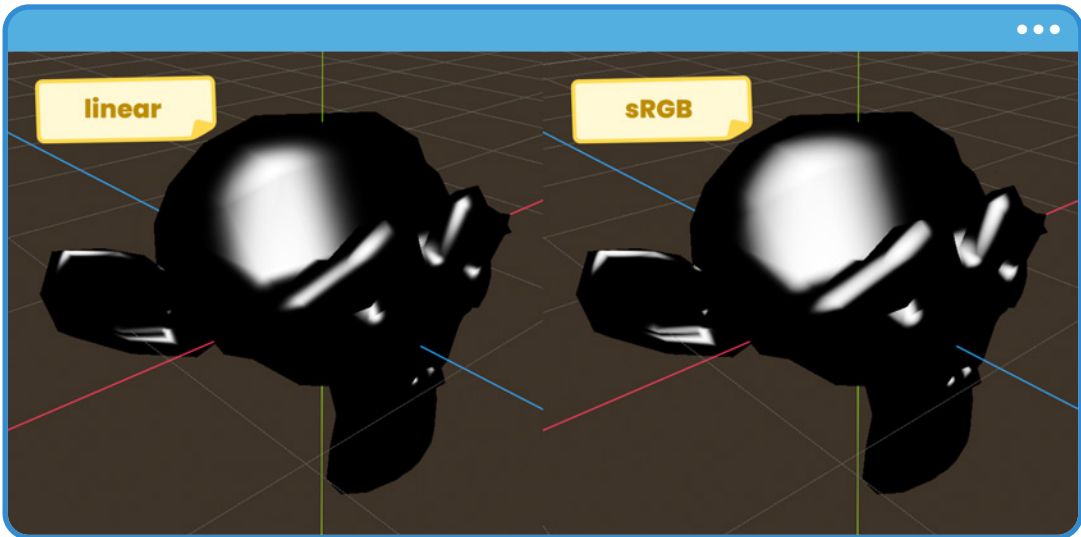
```

Si prestamos atención a la línea 36, observaremos que se ha incluido el método **to_sRGB()** al shader. Luego, en la línea 52, la variable **s** ahora se ha definido como un vector tridimensional, el cual almacena el resultado del método **blinn_phong()**. Posteriormente, en la línea 53, se transforma el valor línea de **s** a sRGB.

Nota

En este ejemplo hemos desactivado la luz difusa (**DIFFUSE_LIGHT += vec3(0.0)**) para centrarnos exclusivamente en el efecto de la especularidad. Asimismo, recomendamos desactivar la iluminación ambiental utilizando el macro **ambient_light_disabled**, lo cual nos permitirá observar con mayor claridad el impacto de la conversión sRGB sobre el brillo.

Si todo ha sido implementado correctamente, el resultado visual mostrará una especularidad más intensa y perceptible, que responde mejor al comportamiento esperado por el ojo humano.



(2.7.b A la izquierda, especularidad lineal; a la derecha, especularidad sRGB)

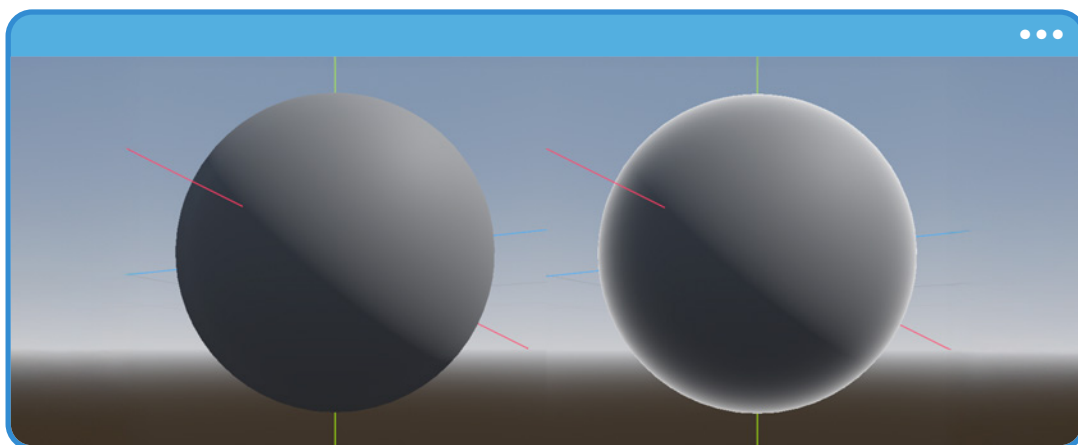
Si bien en este ejemplo aplicamos la conversión de espacio lineal a sRGB únicamente sobre el componente especular, lo cierto es que también podríamos extender esta transformación al componente difuso, ya que este también se encuentra en espacio lineal.

Cada juego tiene su propio estilo visual y necesidades gráficas particulares. Por eso, te animamos a experimentar con estas conversiones y ajustar los valores para lograr un resultado que se alinee con la dirección artística de tu proyecto.

2.8 Introducción al efecto Rim (Fresnel).

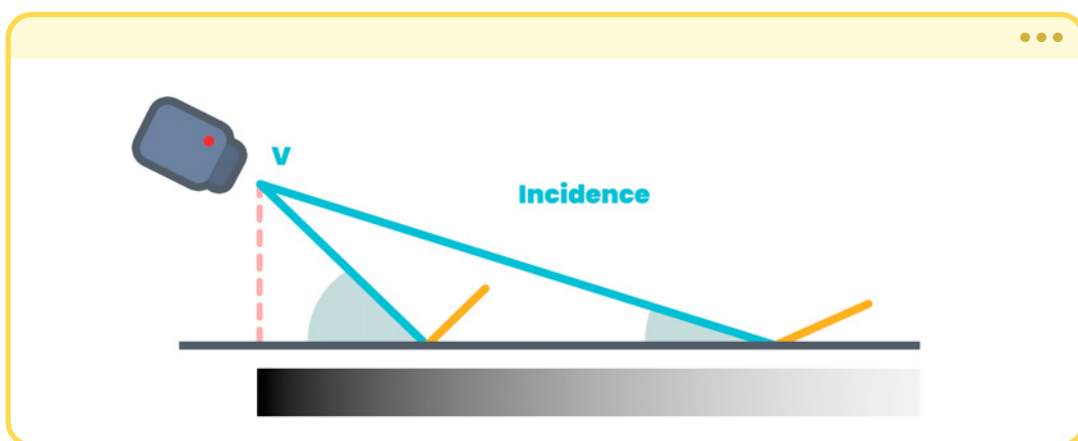
El efecto **Rim**, también conocido como efecto **Fresnel** (en honor al físico francés **Augustin-Jean Fresnel**), es un fenómeno óptico que describe cómo varía la cantidad de luz reflejada sobre una superficie según el ángulo de incidencia. En términos prácticos, este efecto se manifiesta como un borde iluminado que aparece en los extremos de un objeto cuando lo observamos en ángulos oblicuos.

Si estuviéramos utilizando un **StandardMaterial3D**, podríamos activar este efecto de forma sencilla mediante la propiedad **Rim**, la cual viene deshabilitada por defecto. Sin embargo, en esta sección aprenderemos a implementarlo manualmente dentro de un shader, lo cual nos brindará mayor control creativo y comprensión sobre su funcionamiento interno.



(2.8.a A la izquierda, Lambert; a la derecha Rim)

¿Cómo funciona exactamente el efecto Fresnel? El principio detrás del efecto **Fresnel** es bastante sencillo: cuando observamos una superficie de forma frontal, es decir, cuando el vector de vista y la normal de la superficie son paralelos (ángulo cercano a 0°), la cantidad de luz reflejada es mínima o incluso inexistente. En cambio, cuando observamos la superficie desde un **ángulo rasante**; cuando el vector de vista es perpendicular a la normal (ángulo cercano a 90°), la reflexión alcanza su valor máximo.



(2.8.b Diagrama ilustrando el efecto Fresnel o Rim)

Este comportamiento genera un brillo perceptible en los bordes de los objetos. Visualmente, se manifiesta como una silueta luminosa alrededor del modelo, que puede utilizarse tanto para acentuar contornos como para crear efectos estilizados, similares a un outline o un halo artístico.

Desde el punto de vista matemático, una forma común de aproximar el efecto **Rim** en shaders es mediante la siguiente ecuación:

$$F = (1.0 - \max(0.0, \mathbf{n} \cdot \mathbf{v}))^m$$

(2.8.c)

Donde $\mathbf{n}$ es la normal de la superficie, $\mathbf{v}$ es el vector vista, normalizado desde el punto de la superficie hacia la cámara, m es el exponente que controla la nitidez del efecto. Cuanto más bajo sea el valor de m , por ejemplo, $m = 1.0$, más suave y extendido será el efecto **Rim**. Por el contrario, un valor más alto, por ejemplo, $m > 5.0$, genera un borde más delgado, marcado y brillante.

Podemos implementar esta función directamente en nuestro **.gdshader** utilizando Godot Shader Language de la siguiente manera:



(2.8.d Se han definido colores para identificar cada variable)

La imagen anterior muestra tanto la ecuación matemática del efecto **Rim** como su implementación directa en código.

Es importante destacar que, en Godot, este efecto ya está implementado por defecto dentro del sistema de materiales. Según la documentación oficial, podemos acceder al valor del efecto Rim desde el método **fragment()** utilizando la variable interna **RIM**.

El comportamiento de **RIM** está influenciado por la propiedad **ROUGHNESS**, la cual modifica la dispersión del borde iluminado. Sin embargo, exploraremos este detalle con mayor profundidad en la siguiente sección, donde veremos cómo integrar y personalizar este efecto manualmente dentro de un shader visual.

2.9 Implementado el efecto Rim.

En esta sección implementaremos el efecto **Rim** utilizando dos enfoques distintos: A través de las variable internas que nos proporciona Godot, y luego, empujando la función personalizada que definimos en la sección anterior, Figura 2.8.d

El objetivo será comprender tanto la forma simplificada que ofrece el motor como los fundamentos técnicos detrás del efecto para aplicarlos de manera más flexible en distintos contextos.

Para lograrlo, realizaremos los siguientes pasos:

- Crearemos una copia del shader **blinn_phong** y lo renombraremos para trabajar desde ese punto.
- También duplicaremos la escena **blinn_phong.tscn**, manteniendo el modelo suzanne como referencia.
- Implementaremos ambos enfoques del efecto **Rim** dentro del shader para comparar sus comportamientos.

Antes de comenzar con la codificación, organizaremos nuestra estructura de carpetas para mantener el proyecto ordenado. Seguiremos los siguientes pasos:

- 1 Dentro de **chapter_02**, crearemos una nueva carpeta llamada **rim_fresnel**.
- 2 Dentro de esa carpeta, agregaremos las siguientes subcarpetas:
 - a **materials**.
 - b **shaders**.

Dado que continuaremos utilizando el modelo suzanne, no será necesario añadir nuevas mallas ni texturas.

A continuación, duplicaremos el shader **blinn_phong** y lo moveremos a la siguiente ruta:

➤ rim_fresnel > materials > shaders.

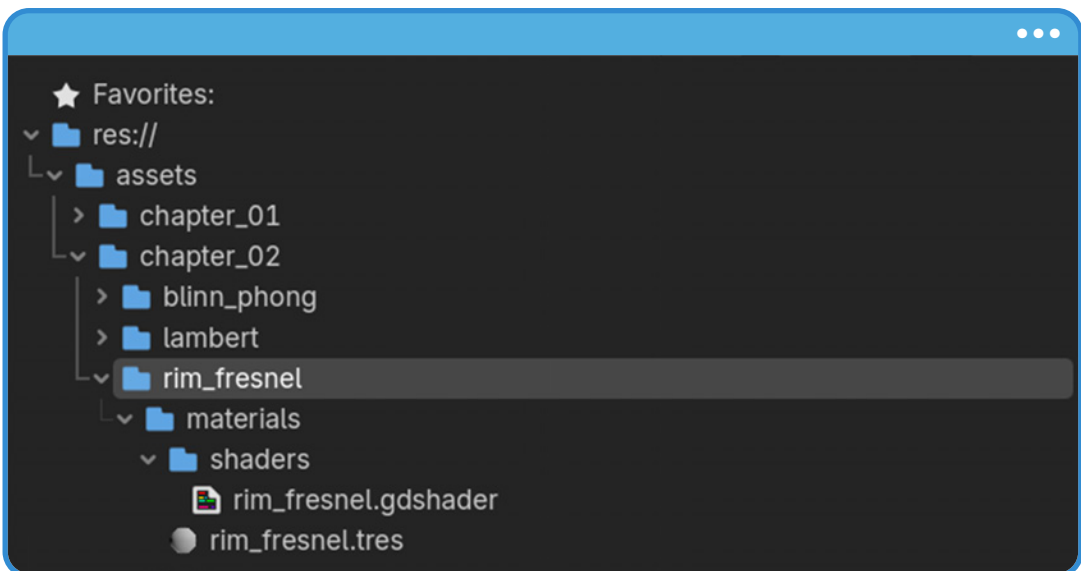
Una vez ahí, nos aseguramos de renombrar el shader como **rim_fresnel** para mantener la consistencia con el nombre de la sección.

Luego, crearemos un nuevo **ShaderMaterial**. Para ello, sobre la carpeta **materials**, hacemos clic derecho y seleccionamos:

➤ Create New > Resource > ShaderMaterial.

Asignaremos también el nombre **rim_fresnel** al material, de modo que su relación con el shader sea clara e inmediata.

Si todos los pasos han sido ejecutados correctamente, la estructura de nuestro proyecto debería lucir de la siguiente manera:



(2.9.a Se han incluido el shader y el material en la sección **rim_fresnel**)

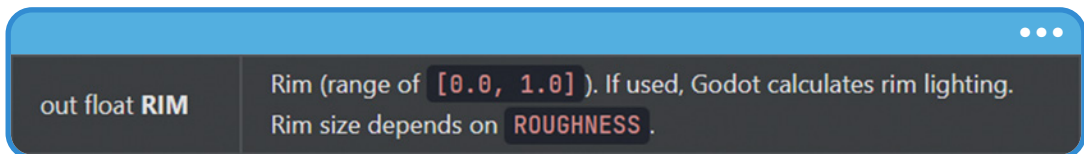
Antes de comenzar con la implementación de las distintas funciones dentro del shader, debemos asegurarnos de que todo esté correctamente conectado. Para ello, asignaremos el shader **rim_fresnel** al material correspondiente, y luego aplicaremos este material al modelo **suzanne** dentro de la escena **rim_fresnel.tscn**. Este paso es fundamental para que podamos visualizar los cambios que realicemos durante el desarrollo de esta sección.

Para entender correctamente cómo funciona el efecto Rim desde la perspectiva de Godot, revisaremos brevemente la documentación oficial, donde se detallan las variables internas disponibles tanto en el método **fragment()** como en **light()**.

Nota

Podemos encontrar esta información en el siguiente enlace: https://docs.godotengine.org/en/stable/tutorials/shaders/shader_reference/spatial_shader.html

En particular, si exploramos la sección **Fragment built-ins**, encontraremos la variable interna **RIM**, la cual representa la intensidad del efecto Rim ya calculado por el motor. Esta variable puede utilizarse directamente dentro del método **fragment()**, como se muestra a continuación:



(2.9.b Variable de salida RIM)

Sin embargo, notaremos que la variable **RIM** no aparece en la sección **Light built-ins** de la documentación. ¿A qué se debe esto?

En Godot, cuando trabajamos solo con el método **fragment()**, somos responsables de asignar directamente los valores a las variables internas como **ALBEDO**, **SPECULAR**, **EMISSION**, etc. En este modo, el motor se encarga automáticamente del cálculo de iluminación, y cualquier modificación (como aplicar el efecto Rim con **RIM**) se refleja de inmediato en pantalla.

Por el contrario, al definir el método **light()**, activamos el modo de iluminación diferida para ese material. Esto cambia el comportamiento del shader: Godot desactiva parte de la lógica automática del método **fragment()** y espera que recalculemos manualmente los

componentes de iluminación. En consecuencia, muchas variables internas como **RIM** dejan de tener efecto directo dentro de **fragment()** si **light()** está presente.

En otras palabras:

- Sin **light()**: el shader realiza todo automáticamente. Solo debemos escribir en **fragment()** y asignar valores a las variables internas.
- Con **light()**: Debemos recalculer explícitamente la iluminación por cada luz que afecta el fragmento. El método **fragment()** pierde parte de su efecto visual.

Nota

En la documentación de **.gdshader**, muchas variables internas aparecen con calificadores como **in**, **out** e **inout**, los cuales indican su comportamiento dentro del pipeline:

- **in vec3 VIEW**: variable de solo lectura en **fragment()**. Contiene la dirección desde el fragmento hacia la cámara.
- **inout vec3 NORMAL**: llega interpolada desde el método **vertex()**, y puede ser leída y modificada en **fragment()** antes de los cálculos de iluminación.

Esto implica que, si utilizamos la variable **RIM** dentro del método **fragment()** mientras el método **light()** está activado, no veremos efecto alguno, ya que el sistema de iluminación diferida lo ignorará.

Para estudiar correctamente cómo funciona **RIM**, lo ideal es comenzar desactivando el método **light()**. De esta manera, podremos observar cómo responde visualmente en el método **fragment()**, sin interferencias.

Procederemos de la siguiente manera:

- Comentaremos temporalmente el contenido del método **light()**.
- Utilizaremos las variables internas **RIM**, **ROUGHNESS** y **RIM_TINT** dentro de **fragment()** para observar cómo se comporta el efecto sobre el modelo **suzanne**.

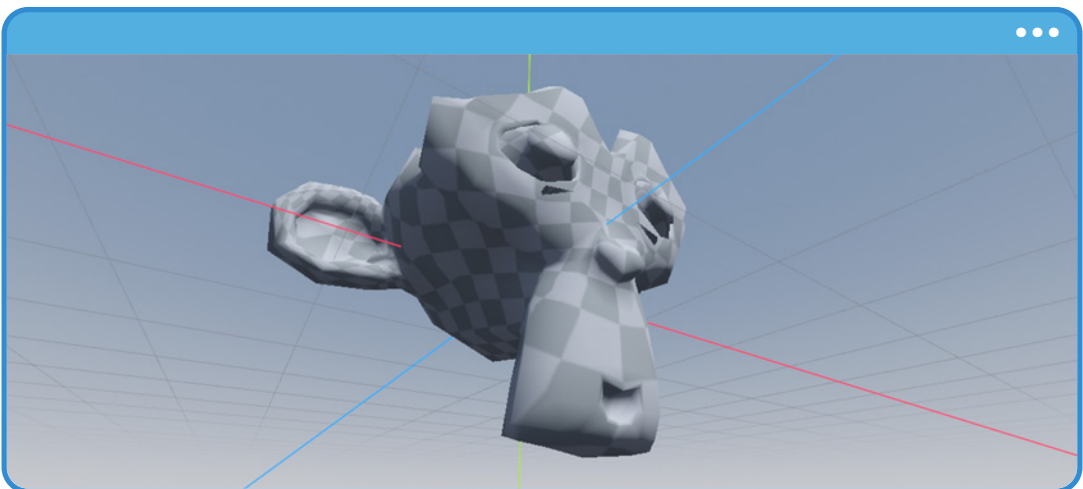
Modificaremos el shader **rim_fresnel** comentando el método **light()** de la siguiente manera:

```
/*
void light()
{
    float i = ATTENUATION;
    vec3 l = LIGHT;
    vec3 n = NORMAL;
    vec3 v = VIEW;

    vec3 d = vec3(lambert(i, l, n));
    vec3 s = vec3(blinn_phong(v, l, n, 128.0));
    s = to_sRGB(s);

    DIFFUSE_LIGHT += d;
    SPECULAR_LIGHT += s;
}
*/
```

Como podemos ver, todo el método **light()** ha sido comentado para que su lógica quede desactivada temporalmente. Una vez que hayamos realizado este cambio, nuestro modelo lucirá de la siguiente manera:



(2.9.c Los cálculos de iluminación provienen de la etapa de fragmento)

Aunque hemos comentado el método `light()`, nuestro modelo sigue mostrando iluminación, principalmente en forma de difusión y luz ambiental. Esto se debe a que, al no definir `light()`, Godot utiliza por defecto su sistema de iluminación automático en el método `fragment()`.

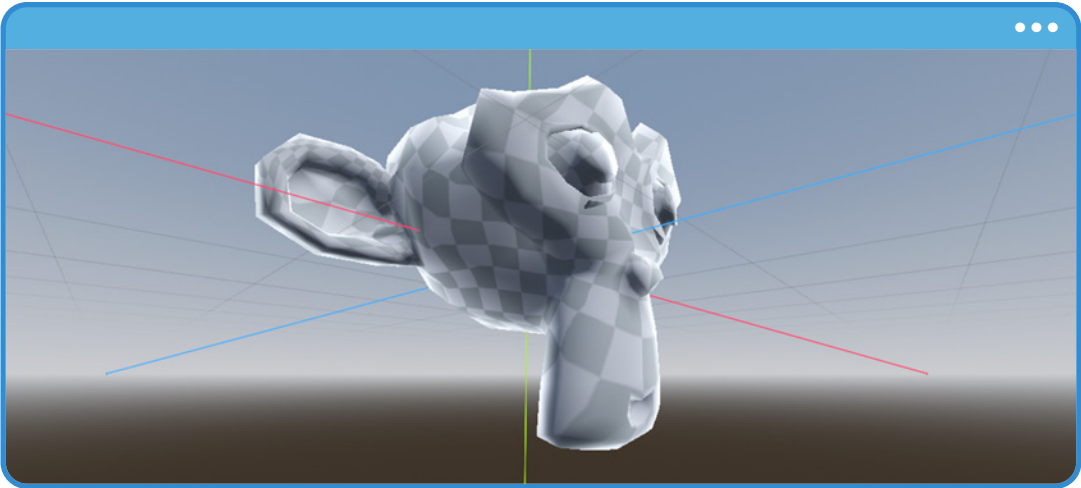
Para habilitar el efecto **Rim** en este contexto, basta con utilizar las variables internas **RIM**, **ROUGHNESS** y **RIM_TINT** dentro del método `fragment()`. A continuación, se muestra una implementación básica:

```
void fragment()
{
    vec3 albedo = texture(_MainTex, UV).rgb;

    ROUGHNESS = 0.8;
    RIM = 1.0;
    RIM_TINT = 0.5;
    ALBEDO = albedo;
}
```

De las tres variables implementadas:

- **RIM** controla la **intensidad** del efecto Rim sobre el modelo: 0.0 significa que el efecto está desactivado, mientras que 1.0 indica que el efecto se aplica al 100
- **ROUGHNESS** influye en el **tamaño** del borde iluminado generado por el efecto Rim. Valores cercanos a 0.0 producen un efecto fino y agudo, mientras que valores cercanos a 1.0 genera un efecto más ancho y suave.
- **RIM_TINT** determina cómo se **mezcla** el efecto Rim con el color del material. con 0.0, el efecto es completamente blanco. En cambio, con 1.0, el efecto se mezcla con el color del material (modo overlay).



(2.9.d Efecto Rim aplicado sobre suzanne en el método **fragment()**)

Como vimos anteriormente, la implementación del efecto **Rim** en el método **fragment()** es bastante sencilla gracias a las variables internas que proporciona Godot. Sin embargo, si en este punto reactivamos el método **light()**, notaremos que el efecto deja de aplicarse. Esto ocurre porque, al definir este método, el motor delega en este último toda la responsabilidad del cálculo de iluminación, y los valores definidos en **fragment()** para propiedades como **RIM**, **ROUGHNESS** o **RIM_TINT** ya no tienen efecto.

Para recuperar el efecto Rim con el método **light()** habilitado, implementaremos nuestra propia versión del cálculo, basada en la ecuación que definimos en la Figura 2.8.d. Para ello:

- Primero, comentaremos las variables **RIM**, **ROUGHNESS** y **RIM_TINT** en el método **fragment()** para asegurarnos de que ya no interfieran en la visualización.
- Luego, nos posicionamos sobre el método **light()** y agregamos el siguiente código:

```
float fresnel(vec3 n, vec3 v, float m, float s)
{
    float f = 1.0 - max(0.0, dot(n, v));
    return s * pow(f, m);
}
```

Aunque en la Figura 2.8.d este método fue nombrado `rim()`, su funcionalidad es la misma. Aquí hemos añadido un nuevo argumento, `s`, que actúa como multiplicador de intensidad, permitiéndonos controlar visualmente el peso del efecto.

A continuación, incorporamos el efecto Fresnel directamente en el método `light()`:

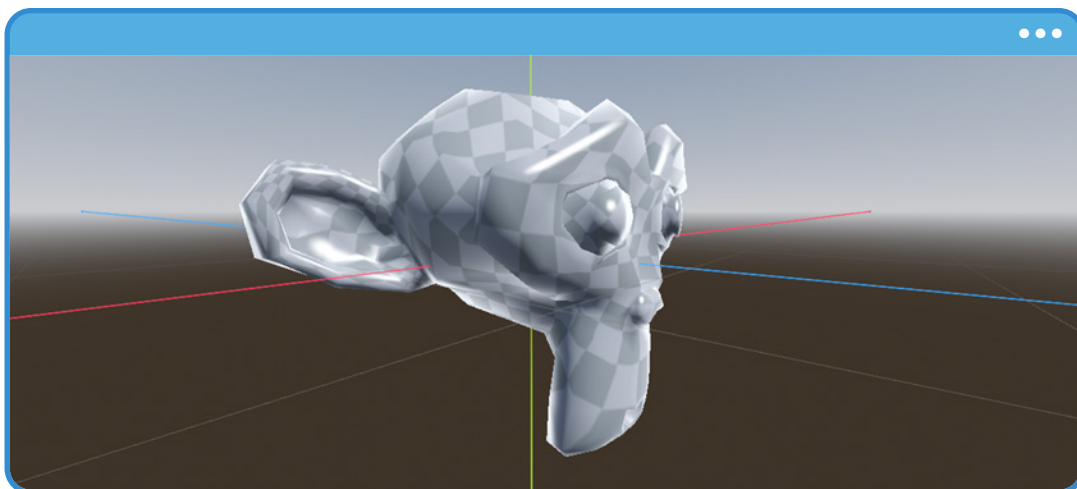
```

55 void light()
56 {
57     float i = ATTENUATION;
58     vec3 l = LIGHT;
59     vec3 n = NORMAL;
60     vec3 v = VIEW;
61
62     float f = fresnel(n, v, 5.0, 2.0);
63     vec3 d = vec3(lambert(i, l, n));
64     d += vec3(f);
65     vec3 s = vec3(blinn_phong(v, l, n, 128.0));
66     s = to_sRGB(s);
67
68     DIFFUSE_LIGHT += d;
69     SPECULAR_LIGHT += s;
70 }

```

Analizando el código, observamos que en la línea 62, declaramos una nueva variable flotante `f`, que almacena el resultado del método **Fresnel**. Posteriormente, en la línea 64, sumamos el valor de `f` al componente difuso `d`, lo que genera el efecto visual característico de borde iluminado.

Los valores `m = 5.0` (exponente) y `s = 2.0` (multiplicador) se han elegido para propósitos de prueba. Te invitamos a experimentar con ellos para obtener distintos estilos.

(2.9.e Efecto Rim en el método `Light()`)

2.10 Introducción al efecto Anisotrópico.

Hasta este punto hemos analizado algunos modelos de reflexión especular con el fin de comprender cómo interpretar e implementar funciones matemáticas dentro del lenguaje **.gdshader**. Sin embargo, los modelos que hemos utilizado representan sólo una pequeña parte del conjunto de modelos existentes.

Cuando deseamos aplicar un modelo de reflexión anisotrópica, la primera pregunta que deberíamos hacer es: ¿cuál de todos los modelos funcionará mejor para mi proyecto? Si bien es cierto que varios modelos devuelven resultados visualmente similares bajo ciertas condiciones, al considerar aspectos como acabado visual, coste computacional u optimización, las diferencias pueden ser significativas.

Algunos modelos comunes de especularidad anisotrópica incluyen:

- Ashikhmin-Shirley.
- Ward.
- Cook-Torrance.
- Phong anisotrópico.
- Blinn-Phong anisotrópico.

Para mantener el enfoque del libro accesible y directo, nos centraremos en explicar dos de ellos: Primero abordaremos el modelo **Phong** anisotrópico, por su simplicidad y bajo costo computacional. Luego, exploraremos el modelo de **Ashikhmin-Shirley**, el cual, si bien es más complejo, ofrece un resultado visual más refinado.

Para ello, comenzaremos analizando la siguiente expresión matemática:

$$A = \frac{(a_u + 1)(a_v + 1)}{8\pi} \cdot (\mathbf{h} \cdot \mathbf{n}) \frac{a_u(\mathbf{h} \cdot \mathbf{t})^2 + a_v(\mathbf{h} \cdot \mathbf{b})^2}{1 - (\mathbf{h} \cdot \mathbf{n})^2}$$

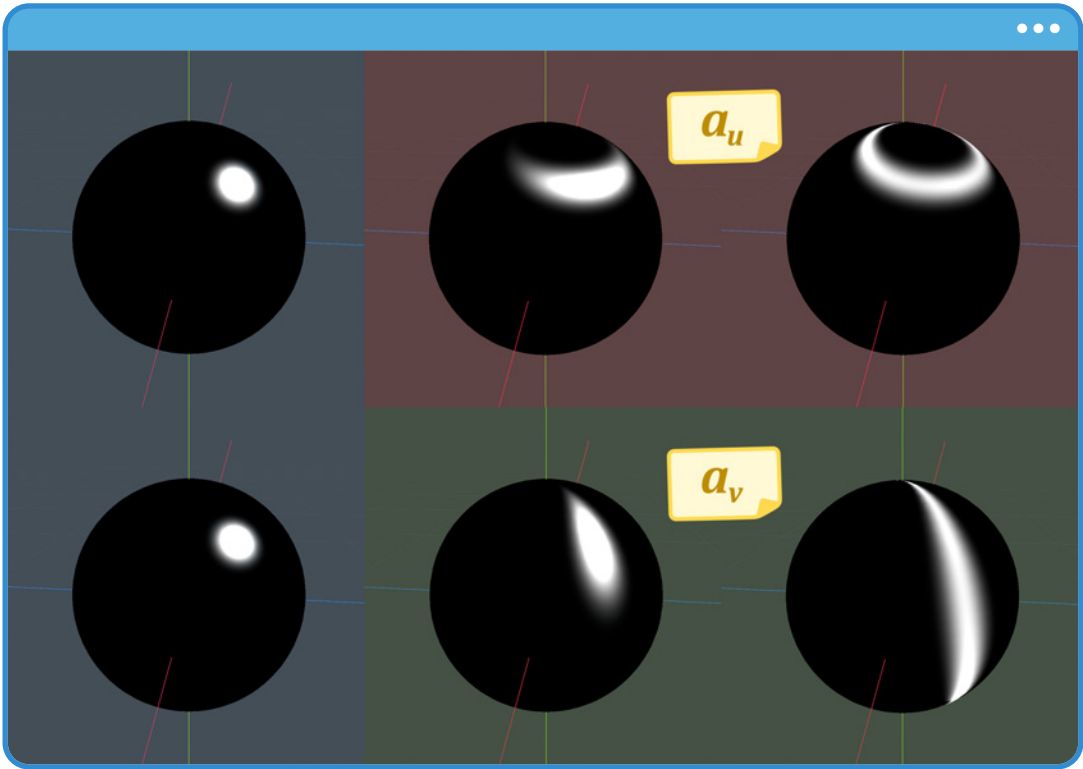
(2.10.a)

Esta ecuación representa a una extensión del modelo clásico de **Phong**, adaptada para reflejar anisotropía mediante exponentes distintos para las direcciones tangente y binormal. En esta fórmula:

- a_u corresponde al exponente en la dirección de la tangente (t).
- a_v corresponde al exponente en la dirección de la binormal (b).
- $\mathbf{h}$ es igual al vector halfway (**LIGHT + VIEW**).
- $\mathbf{n}$ es la normal (**NORMAL**).
- $\mathbf{t}$ es la tangente (**TANGENT**).
- y por último, $\mathbf{b}$ es la binormal (**BINORMAL**).

Es posible que esta ecuación parezca compleja a primera vista, pero su principio es sencillo: Mediante los parámetros a_u y a_v , deformamos la forma circular de la reflexión especular, transformándola en una elipse orientada según la dirección de la superficie. Esta deformación permite simular materiales como el metal cepillado, cabello o telas brillantes, donde la luz refleja la forma direccional.

A continuación, ilustraremos visualmente cómo la variación de a_u y a_v afecta la forma de la reflexión especular.



(2.10.b Deformación de la especularidad anisotrópica en ambas direcciones)

Como se puede observar en la Figura 2.10.b, la variable a_u controla el ancho de la reflexión especular, mientras que a_v controla su alto. Es decir, al incrementar a_u la reflexión se extiende horizontalmente a lo largo de la tangente, y al incrementar a_v , la reflexión se extiende verticalmente siguiendo la dirección de la binormal.

Si quisiéramos implementar la Ecuación 2.10.a en nuestro shader, podríamos hacerlo de la siguiente manera:

$$A = \frac{(a_u + 1)(a_v + 1)}{8\pi} \cdot (\mathbf{h} \cdot \mathbf{n}) \frac{a_u(\mathbf{h} \cdot \mathbf{t})^2 + a_v(\mathbf{h} \cdot \mathbf{b})^2}{1 - (\mathbf{h} \cdot \mathbf{n})^2}$$

```
float phong_anisotropic(vec3 n, vec3 l, vec3 v, vec3 t,
vec3 b, float au, float av)
{
    vec3 h = normalize(l + v);
    float HdotN = dot(h, n);
    float HdotT = dot(h, t);
    float HdotB = dot(h, b);

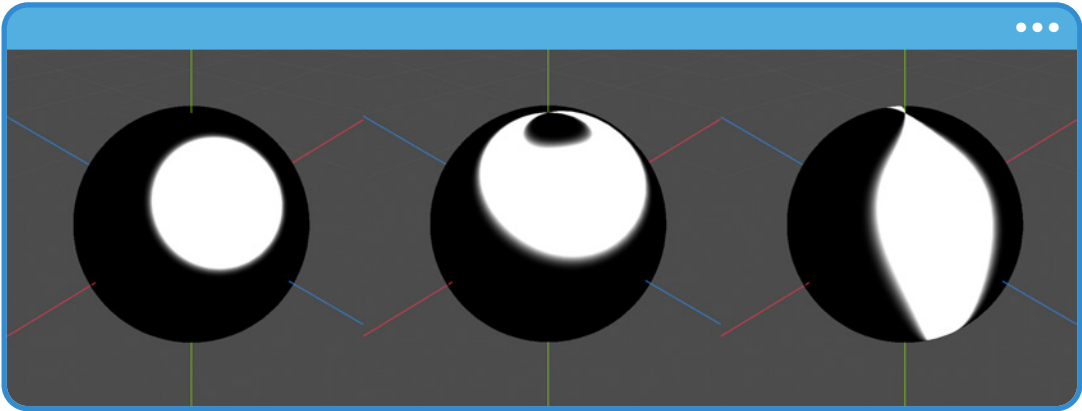
    float exponent = au * pow(HdotT, 2.0) + av * pow(HdotB, 2.0);
    exponent /= 1.0 - pow(HdotN, 2.0);
    float specular = ((au + 1.0) * (av + 1.0)) / 8.0 * PI;
    float a = dot(specular, pow(HdotN, exponent));

    return a;
}
```

(2.10.c Se han definido colores para identificar cada variable)

Este modelo nos permite deformar la forma de la reflexión especular de manera controlada, ajustando los parámetros a_u y a_v para obtener el efecto deseado. Aunque no es un modelo físicamente exacto no conserva la energía de manera precisa, su bajo costo computacional lo hace adecuado para contextos de renderizado estilizado o efectos no realistas.

A continuación, podemos visualizar el efecto sobre una esfera:



(2.10.d Resultado gráfico de la especularidad anisotrópica sobre una esfera)

Como podemos observar en la Figura 2.10.d, el efecto anisotrópico genera una reflexión especular altamente direccional, cuya forma varía en función de los valores asignados a las variables a_u y a_v . Esta deformación del lóbulo especular resulta útil para representar materiales donde la reflexión se distribuye de forma no uniforme.

Sin embargo, la implementación completa presentada anteriormente ser optimizada utilizando una aproximación más simple y eficiente, que produce un resultado visualmente aceptable en contextos donde el rendimiento es prioritario.

Esta versión simplificada se define mediante la siguiente expresión:

$$A = \max(\mathbf{n} \cdot \mathbf{h}, 0.001) a_u (\mathbf{h} \cdot \mathbf{t})^2 + a_v (\mathbf{h} \cdot \mathbf{b})^2$$

(2.10.e)

Lo cual, en términos de implementación luce de la siguiente manera:

$$A = \max(n \cdot h, 0.001) \quad a_u (h \cdot t)^2 + a_v (h \cdot b)^2$$

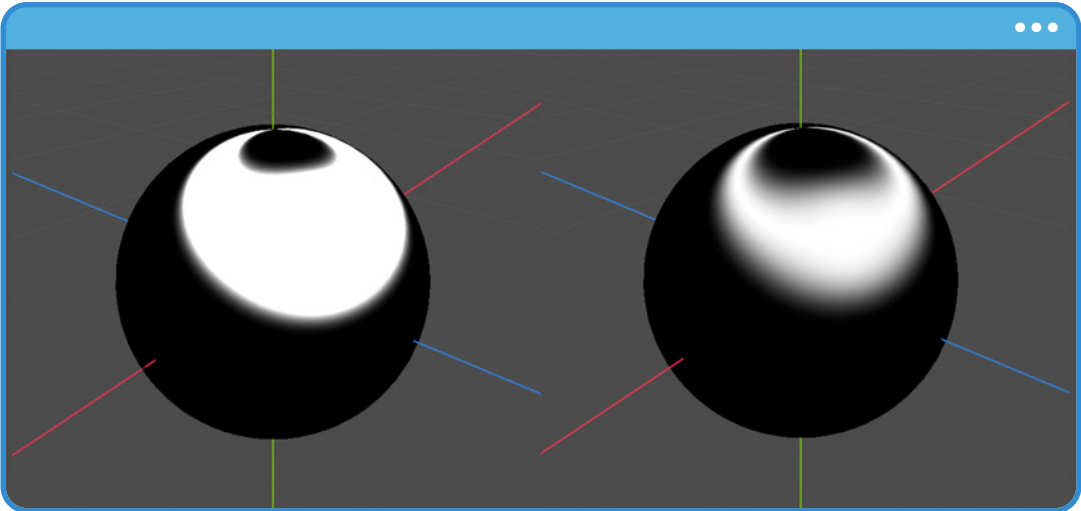
```
float phong_anisotropic_simplified(vec3 n, vec3 l, vec3 v,
vec3 t, vec3 b, float au, float av)
{
    vec3 h = normalize(l + v);
    float HdotT = dot(h, t);
    float HdotB = dot(h, b);
    float NdotH = max(dot(n, h), 0.001);

    float exponent = au * pow(HdotT, 2.0) + av * pow(HdotB, 2.0);
    float a = pow(NdotH, exponent);

    return a;
}
```

(2.10.f Se han definido colores para identificar cada variable)

A diferencia de la primera versión (basada en la Ecuación 2.10.a), esta aproximación entrega una reflexión especular más suavizada y eficiente, sin necesidad de realizar divisiones adicionales. En la práctica, este tipo de simplificación es ideal para efectos estilizados o dispositivos de bajo rendimiento, donde el ahorro computacional tiene un impacto importante.



(2.10.g A la izquierda, modelo Phong clásico; a la derecha, Phong simplificado)

Como se muestra en la Figura 2.10.g, el modelo de Phong clásico (izquierda) genera un lóbulo especular más definido y concentrado, mientras que la versión simplificada (derecha) suaviza el resultado al eliminar el factor de normalización. Aunque ambos modelos son útiles en contextos estilizados, si se requiere una representación más precisa y físicamente fundamentada de la anisotropía, es necesario recurrir a modelos más complejos.

Uno de los modelos más destacados en este sentido es el de **Ashikhmin-Shirley**, cuya formulación matemática se define de la siguiente manera:

$$A = \frac{\sqrt{(a_u + 1)(a_v + 1)} (n \cdot h) \frac{a_u (h \cdot t)^2 + a_v (h \cdot b)^2}{1 - (n \cdot h)^2}}{8\pi (v \cdot h) \max(n \cdot l, n \cdot v)}$$

(2.10.h)

Este modelo representa una evolución del modelo Phong anisotrópico, pero incorpora varios elementos fundamentales que lo acercan al comportamiento real de la luz sobre superficies rugosas:

- El numerador contiene una raíz cuadrada que actúa como factor de normalización para asegurar la conservación de la energía.

- El exponente aplicado a $\mathbf{n} \cdot \mathbf{h}$ considera la orientación del vector Halfway con respecto a las direcciones tangente y binormal, lo que permite una deformación precisa del lóbulo especular.
- El denominador introduce una dependencia del ángulo entre la vista y al luz (a través de $\mathbf{v} \cdot \mathbf{h}$ y el máximo entre $\mathbf{n} \cdot \mathbf{l}$ y $\mathbf{n} \cdot \mathbf{v}$), mejorando la respuesta angular del brillo especular.

Lo interesante de este modelo es que proporciona un equilibrio entre precisión visual y control artístico, permitiendo crear materiales más realistas o con microestructura direccional.

Cabe destacar que el modelo original de **Ashikhmin-Shirley** incluye también una aproximación del efecto Fresnel mediante la fórmula de **Schlick**, definida como:

$$F = r_s + (1 - r_s)(1 - (\mathbf{v} \cdot \mathbf{h}))^5$$

(2.10.i)

Este término mejora la respuesta del material en ángulos rasantes, intensificando la reflexión especular cuando el ángulo de visión se aleja de la normal. Sin embargo, su uso es opcional dependiendo del estilo visual que queramos lograr. Tanto el modelo de **Ashikhmin-Shirley** como el término de Fresnel se pueden implementar dentro de un mismo método en Godot de la siguiente manera:

$$A = \frac{\sqrt{(a_u + 1)(a_v + 1)(n \cdot h)} \frac{a_u(h \cdot t)^2 + a_v(h \cdot b)^2}{1 - (n \cdot h)^2}}{8\pi (v \cdot h) \max(n \cdot l, n \cdot v)} (r_s + (1 - r_s)(1 - (v \cdot h))^5)$$

```
float ashikhmin_shirley_anisotropic(float rs, float au, float av,
vec3 n, vec3 l, vec3 v, vec3 t, vec3 b)
{
    vec3 h = normalize(l + v);
    float NdotH = max(dot(n, h), 0.0001);
    float NdotL = max(dot(n, l), 0.0001);
    float NdotV = max(dot(n, v), 0.0001);
    float VdotH = max(dot(v, h), 0.0001);

    float HdotT = dot(h, t);
    float HdotB = dot(h, b);

    float exponent = au * pow(HdotT, 2.0) + av * pow(HdotB, 2.0);
    exponent /= 1.0 - pow(NdotH, 2.0);

    float a = sqrt((au + 1.0) * (av + 1.0)) * pow(NdotH, exponent);
    a /= (8.0 * PI) * VdotH * max(NdotL, NdotV);
    float f = rs + (1.0 - rs) * pow(1.0 - VdotH, 5.0);
    a *= f;

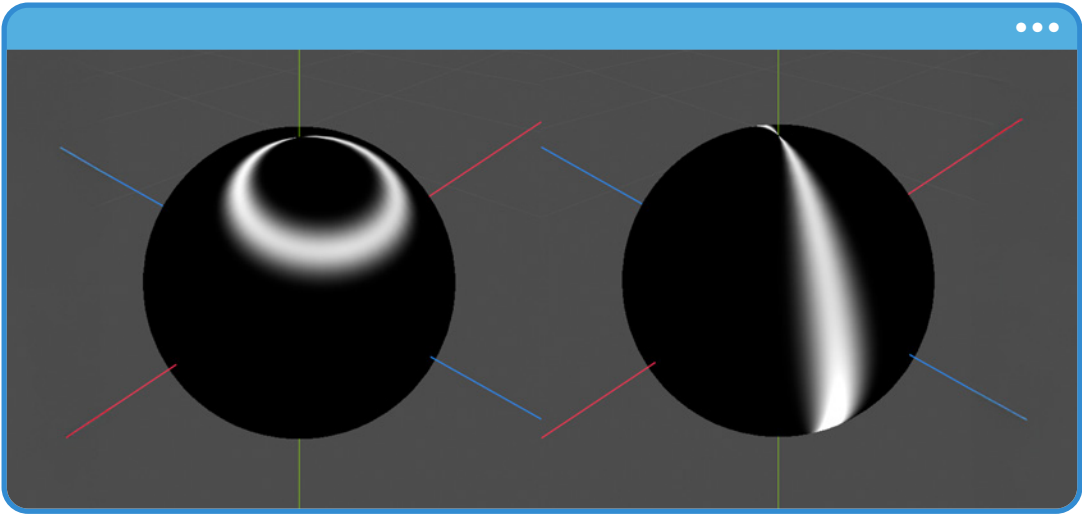
    return a;
}
```

(2.10.j Se han definido colores para identificar cada variable)

En esta implementación:

- r_s representa la reflectancia base.
- a_u y a_v controlan la anisotropía en las direcciones tangencial y binormal, respectivamente.
- Se incluye una corrección para evitar divisiones por cero y mejorar la estabilidad numérica.

El resultado visual generado por esta función se muestra en la siguiente figura:



(2.10.k modelo Ashikhmin-Shirley)

Como podemos observar, este modelo produce un lóbulo especular más definido y direccional, cuya forma depende de la orientación del vector halfway respecto a la tangente y la binormal. Esta precisión lo hace especialmente útil para simular materiales como cabello, telas satinadas, metales cepillados o superficies con orientación estructural y realista.

2.11 Implementando el modelo Ashikhmin-Shirley.

En esta sección pondremos en práctica el modelo **Ashikhmin-Shirley**, aplicándolo sobre el cabello de un personaje de estilo animé. Nuestro objetivo será observar cómo se comporta este modelo en geometrías más complejas, y cómo su componente **anisotrópica** puede acentuar el estilo visual del render, especialmente en materiales como el cabello, que tienden a reflejar la luz de manera direccional.

Comenzaremos organizando nuestro proyecto para mantener una estructura clara y coherente. Realizaremos los siguientes pasos:

- Dentro de la carpeta **chapter_02**, crearemos una carpeta a la que llamaremos **anisotropy**.

- Dentro de esta última, organizaremos el contenido de la siguiente manera:
 - **materials.**
 - **shaders.**
 - **meshes.**
 - **textures.**

A continuación, crearemos el material que aplicaremos al cabello del personaje. Para ello, nos posicionamos sobre la carpeta **materials**, hacemos clic derecho y seleccionamos:

- Create New > Resources > ShaderMaterial.

Nombraremos a este material como **character_hair**.

Luego, creamos el shader correspondiente, posicionándonos sobre la carpeta **shaders**, presionamos clic derecho y seleccionamos:

- Create New > Resources > Shader.

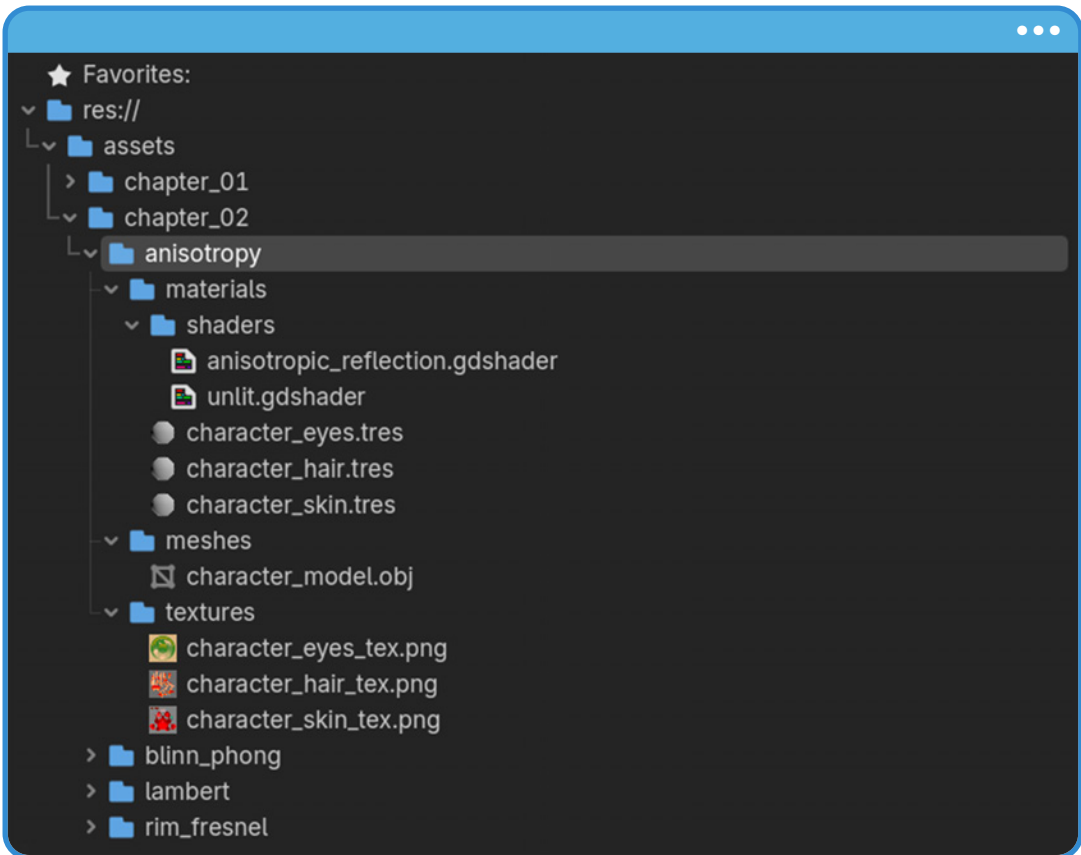
Llamaremos a este archivo **anisotropic_reflection**, manteniendo la coherencia con el nombre de la sección.

Para esta sección utilizaremos un personaje de estilo animé junto con sus respectivas texturas y materiales, los cuales están disponibles como parte del paquete descargable del libro.

Nota

Recuerda que este libro incluye un paquete descargable que contiene todos los archivos necesarios para seguir los ejercicios paso a paso. Puedes descargarlo desde tu cuenta, directamente en <https://jettelly.com/store/the-godot-shaders-bible>

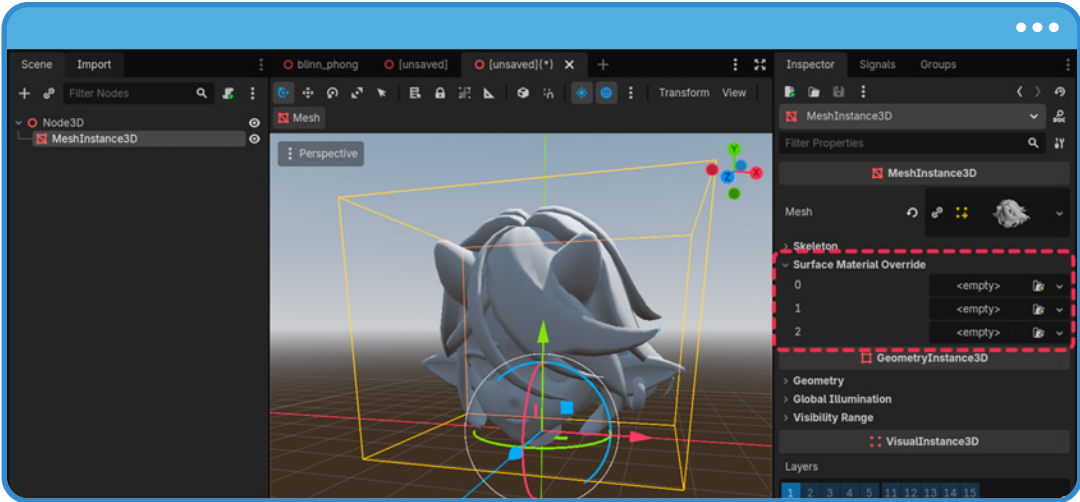
Si todo ha sido configurado correctamente, la estructura de nuestro proyecto debería lucir de la siguiente manera:



(2.11.a Estructura del proyecto)

Iniciaremos el proceso de implementación arrastrando el modelo **character_model** al Viewport. Además, nos aseguraremos de centrar su posición, estableciendo sus valores de transformación en (0, 0, 0).

Un detalle interesante en este modelo es su configuración de materiales a través de la propiedad **Surface Material Override**. Si seleccionamos su nodo **MeshInstance3D**, observaremos en el **Inspector** que cuenta con **tres entradas de material**, enumeradas como 0, 1 y 2.

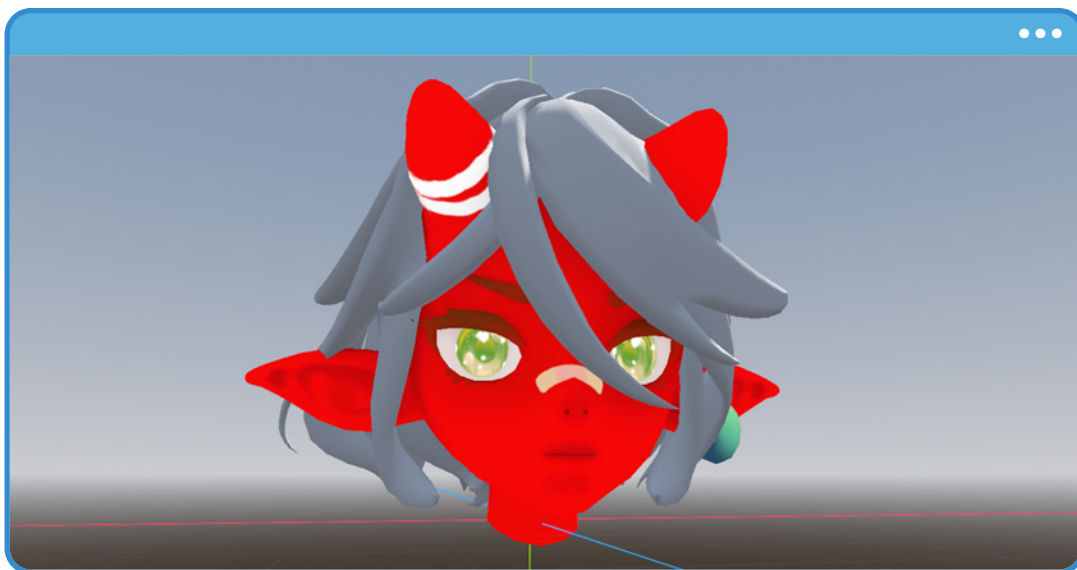


(2.11.b El objeto posee tres entradas de material)

Estas entradas corresponden a los materiales asignados en el software de modelado 3D utilizado para su creación (en este caso, **Maya**):

- El índice 0 hace referencia a los ojos del personaje. Asignaremos aquí el material **character_eyes**.
- El índice 1 corresponde al cabello. En este bloque, asignaremos el material **character_hair**. Cabe destacar que el shader **anisotropic_reflection** debe estar asignado a este material antes de asignar el material al personaje, como lo hemos hecho en secciones anteriores.
- El índice 2 se refiere a la piel del personaje. En este caso, asignaremos el material **character_skin**.

Una vez que hayamos asignado los materiales correctamente, y dado que aún no hemos agregado código al shader **anisotropic_reflection**, el cabello del personaje se verá sin ningún tipo de reflejo anisotrópico. Su apariencia en el **Viewport** será completamente plana, como se muestra a continuación:



(2.11.c Se han asignado los materiales al personaje)

Obviamente, esto cambiará una vez que comencemos a implementar las funciones necesarias en nuestro shader. Para ello, abriremos el archivo **anisotropic_reflection.gdshader** y nos posicionaremos sobre el método **fragment()**.

A continuación, agregaremos las siguientes líneas de texto:

```
shader_type spatial;

uniform sampler2D _MainTex;

void vertex()
{
    // Called for every vertex the material is visible on.
}

void fragment()
{
    vec3 albedo = texture(_MainTex, UV).rgb;
    ALBEDO = albedo;
}

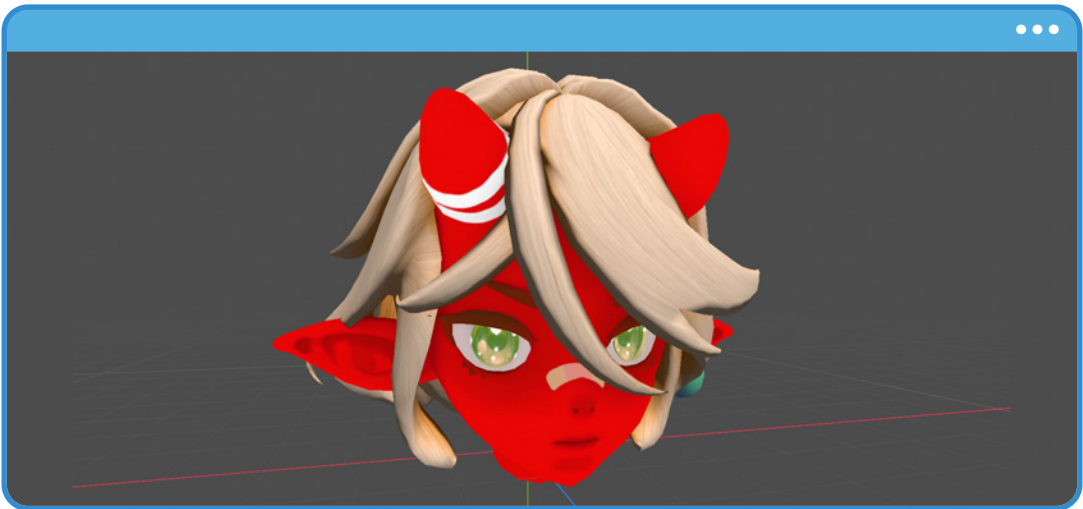
// El código continúa en la página siguiente.
```



```
//void light() {
    // Called for every pixel for every light affecting the material.
    // Uncomment to replace the default light processing function ...
//}
```

Como podemos observar, hemos declarado un nuevo vector tridimensional llamado **albedo**, que almacena el color resultante de muestrear la textura **_MainTex** utilizando las coordenadas **UV** del modelo. Este valor se asigna directamente a la variable interna **ALBEDO**, lo que permite que el color de la textura se aplique correctamente sobre la geometría.

Una vez realizado este cambio, debemos asignar la textura **character_hair_tex** a la propiedad **Main Tex** desde el panel **Inspector**, dentro del material **character_hair**. Esta textura contiene el color base del cabello del personaje.



(2.11.d Se ha asignado la textura al cabello del personaje)

Aunque el resultado actual aún no refleja el efecto que buscamos, nos sirve como punto de partida para comprender la implementación que llevaremos a cabo en esta sección. El modelo **Ashikhmin-Shirley** debe ser implementado dentro del método **light()**, pero antes de volver al shader, necesitaremos agregar una fuente de iluminación a la escena. Para ello, agregaremos una **DirectionalLight3D**, posicionándonos sobre el **Node3D** y seleccionando:

➤ Add Child Node > DirectionalLight3D

Esto nos dará control sobre la dirección e intensidad de la luz, lo cual es fundamental para observar cómo se comporta la reflexión sobre el cabello del personaje.

Volviendo al archivo **.gdshader**, habilitaremos el método **light()** quitando los comentarios que lo cubren. Al hacerlo, notaremos que el cabello del personaje se vuelve **opaco**. Esto ocurre porque los cálculos de iluminación han sido transferidos completamente a este método, y aún no hemos definido ninguna lógica dentro de él. A pesar de esto, el personaje seguirá siendo afectado por la luz ambiental, la cual se gestiona de manera automática.

Para estructurar correctamente nuestra implementación, comenzaremos separando las funciones necesarias en bloques, lo cual facilitará la comprensión y mantenimiento del código. Seguiremos como base la ecuación presentada en la Figura 2.10.j de la sección anterior.

Declaramos la función justo entre los métodos **fragment()** y **light()**. Por motivos prácticos, la llamaremos **ashikhmin_shirley()**:

```
void fragment()
{
    vec3 albedo = texture(_MainTex, UV).rgb;
    ALBEDO = albedo;
}

float ashikhmin_shirley()
{
    return 0.0;
}

void light()
{
}
```

Este modelo requiere varias variables para llevar a cabo sus cálculos. A continuación, listamos los argumentos que pasaremos a la función, los cuales incluyen factores de reflexión, vectores de orientación y propiedades geométricas de la superficie:

- r_s : factor de reflexión.
- a_u : dirección tangencial.
- a_v : dirección binormal.
- n : variable **NORMAL**.
- v : variable **VIEW**.
- l : variable **LIGHT**.
- t : variable **TANGENT**.
- b : variable **BINORMAL**.

Continuaremos incluyendo estas variables como argumentos en la función:

```
float ashikhmin_shirley(float rs, float au, float av, vec3 n, vec3 l, vec3 v,
    vec3 t, vec3 b)
{
    return 0.0;
}
```

En la siguiente etapa, implementaremos el cuerpo de esta función dividiéndolo en tres bloques lógicos, basados en la fórmula matemática presentada en la Figura 2.10.j. Esta división nos permitirá interpretar y desarrollar el modelo paso a paso de forma clara y estructurada.

The diagram illustrates the decomposition of the anisotropic model equation into three components:

- Specular =**
$$A = \frac{\sqrt{(a_u + 1)(a_v + 1)}(n \cdot h)}{8\pi (v \cdot h) \max(n \cdot l, n \cdot v)}$$
- Exponent =**
$$\frac{a_u(h \cdot t)^2 + a_v(h \cdot b)^2}{1 - (n \cdot h)^2}$$
- Fresnel =**
$$(r_s + (1 - r_s)(1 - (v \cdot h))^5)$$

The full equation shown at the top is:

$$A = \frac{\sqrt{(a_u + 1)(a_v + 1)}(n \cdot h)}{8\pi (v \cdot h) \max(n \cdot l, n \cdot v)} \cdot \frac{a_u(h \cdot t)^2 + a_v(h \cdot b)^2}{1 - (n \cdot h)^2} \cdot (r_s + (1 - r_s)(1 - (v \cdot h))^5)$$

(2.11.e Se han separado las funciones del modelo anisotrópico)

Para implementar el modelo, tomaremos como base la ecuación general presentada en la Figura 2.11.e. Una estrategia que podemos utilizar para abordarla es dividirla en bloques más pequeños. Si observamos, por ejemplo, la ecuación de la especularidad, notaremos que aparecen varias operaciones basadas en el producto punto:

- $n \cdot l$ (NdotL).
- $n \cdot v$ (NdotV).
- $n \cdot h$ (NdotH).
- $v \cdot h$ (VdotH).

Dado que algunas de estas variables se utilizan tanto en el cálculo del exponente como en el término de **Fresnel**, lo más eficiente será definir las al inicio de nuestra función:

```
float ashikhmin_shirley(float rs, float au, float av, vec3 n, vec3 l, vec3 v,
    ↪ vec3 t, vec3 b)
{
    vec3 h = normalize(l + v);
    float NdotL = max(dot(n, l), 0.0001);
    float NdotV = max(dot(n, v), 0.0001);
    float NdotH = max(dot(n, h), 0.0001);
    float VdotH = max(dot(v, h), 0.0001);

    return 0.0;
}
```

Hay dos aspectos importantes en este bloque inicial:

- 1 La variable **h** corresponde al vector halfway, calculado como una suma normalizada entre la dirección de la luz y la vista. Su normalización garantiza que su longitud sea igual a 1.0.
- 2 El uso de **max(..., 0.0001)** asegura que evitemos valores cercanos o iguales a 0.0, lo cual podría causar errores visuales o artefactos en el render, especialmente en ángulos rasantes.

A continuación, podemos incorporar el numerador y denominador del modelo, dejando por ahora el exponente sin definir:

```
float ashikhmin_shirley(float rs, float au, float av, vec3 n, vec3 l, vec3 v,
    ↪ vec3 t, vec3 b)
{
    vec3 h = normalize(l + v);
    float NdotL = max(dot(n, l), 0.0001);
    float NdotV = max(dot(n, v), 0.0001);
    float NdotH = max(dot(n, h), 0.0001);
    float VdotH = max(dot(v, h), 0.0001);

    // El código continúa en la página siguiente.
```

```
float exponent; // <-- not initialized

float specular = sqrt((au + 1.0) * (av + 1.0)) * pow(NdotH, exponent);
specular /= (8.0 * PI) * VdotH * max(NdotL, NdotV);

return 0.0;
}
```

Según la Figura 2.11.e, el exponente depende de tres componentes:

- $h \cdot t$ (HdotT).
- $h \cdot b$ (HdotB).
- $n \cdot h$ (NdotH).

Ya hemos definido $(n \cdot h)$ previamente, por lo tanto, únicamente nos queda por agregar $(h \cdot t)$ y $(h \cdot b)$, como se muestra a continuación:

```
float ashikhmin_shirley(float rs, float au, float av, vec3 n, vec3 l, vec3 v,
    ↪ vec3 t, vec3 b)
{
    vec3 h = normalize(l + v);
    float NdotL = max(dot(n, l), 0.0001);
    float NdotV = max(dot(n, v), 0.0001);
    float NdotH = max(dot(n, h), 0.0001);
    float VdotH = max(dot(v, h), 0.0001);

    float HdotT = dot(h, t);
    float HdotB = dot(h, b);

    float exponent = au * pow(HdotT, 2.0) + av * pow(HdotB, 2.0);
    exponent /= 1.0 - pow(NdotH, 2.0);

    float specular = sqrt((au + 1.0) * (av + 1.0)) * pow(NdotH, exponent);
    specular /= (8.0 * PI) * VdotH * max(NdotL, NdotV);

    return 0.0;
}
```

Si bien el modelo **Ashikhmin-Shirley** puede funcionar sin el término de Fresnel, lo incluiremos para completar la ecuación y mejorar la respuesta especular en los bordes del objeto:

```
float specular = sqrt((au + 1.0) * (av + 1.0)) * pow(NdotH, exponent);
specular /= (8.0 * PI) * VdotH * max(NdotL, NdotV);

float f = rs + (1.0 - rs) * pow(1.0 - VdotH, 5.0);
specular *= f;

return specular;
```

Ahora, para poder utilizar esta función dentro del método **light()**, debemos tener en cuenta lo siguiente:

- Las variables **TANGENT** y **BINORMAL** no pueden usarse directamente en **light()**, ya que no están disponibles en ese contexto. Para solucionarlo, declararemos a estas variables como **varying**. Posteriormente, les asignaremos su valor dentro del método **fragment()**.
- Las variables **rs**, **au** y **av** deben declararse como **uniform**, para podemos ajustarlas desde el Inspector.

Procederemos entonces a declarar e inicializar las variables globales y los uniformes necesarios para que nuestro shader pueda utilizar correctamente el modelo anisotrópico dentro de **light()**.

```
1  shader_type spatial;
2
3  uniform sampler2D _MainTex;
4  uniform float _AU : hint_range(0.0, 256.0, 0.1);
5  uniform float _AV : hint_range(0.0, 256.0, 0.1);
6  uniform float _ReflectionFactor : hint_range(0.0, 1.0, 0.1);
7
8  varying vec3 _tangent;
9  varying vec3 _binormal;
```

```

10
11 void vertex() { ... }
15 //Líneas omitidas (12 a 14)
16 void fragment()
17 {
18     _tangent = TANGENT;
19     _binormal = BINORMAL;
20
21     vec3 albedo = texture(_MainTex, UV).rgb;
22     ALBEDO = albedo;
23 }

```

Si prestamos atención al código, observaremos que las propiedades del material han sido declaradas entre las líneas 4 y 6, específicamente. Luego, en las líneas 8 y 9, se declaran las variables **varying** que almacenarán los vectores de tangente y binormal. Estas variables son inicializadas dentro del método **fragment()**, como se muestra en las líneas 18 y 19. De esta manera, nos aseguramos de que los vectores **TANGENT** y **BINORMAL**, que no están disponibles directamente en el método **light()**, puedan ser accedidos mediante **_tangent** y **_binormal**.

Ahora que todo está correctamente configurado, podemos ir al método **light()** y utilizar el modelo **ashikhmin_shirley()**. Para ello, declaramos una nueva variable flotante llamada **aniso**, e inicializamos su valor con el resultado del método. El bloque de código se vería así:

```

void light()
{
    float aniso = ashikhmin_shirley(_ReflectionFactor, _AU, _AV,
    NORMAL, LIGHT, VIEW, _tangent, _binormal);

    SPECULAR_LIGHT += aniso;
}

```

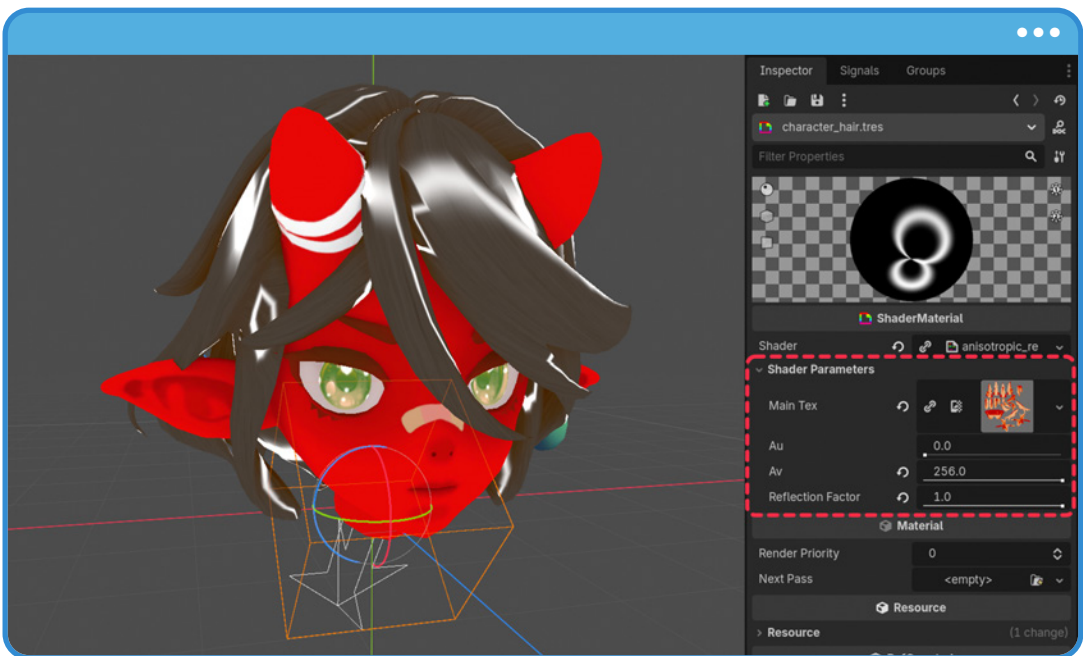
Dado que la reflexión anisotrópica es una forma avanzada de especularidad, se ha asignado el resultado directamente a la salida **SPECULAR_LIGHT**.

Para comprobar cómo afecta esta reflexión a nuestro personaje dentro del Viewport, seguiremos los siguientes pasos:

- 1 Seleccionamos el material **character_hair**.
- 2 Vamos al Inspector.
- 3 Configuramos una sola de las propiedades **Au** o **Av** en su valor máximo (por ejemplo, 256.0).
- 4 La otra propiedad (**Au** o **Av**) debe mantenerse en 0.0.
- 5 Establecemos la variable **Reflection Factor** en 1.0.

Una vez realizados estos ajustes, el cabello del personaje comenzará a mostrar un brillo anisotrópico cuya forma dependerá de la orientación del vector tangente asignado al modelo.

El resultado en el Viewport debería lucir de la siguiente manera:



(2.11.f Se han modificado las propiedades del material)

No obstante, la reflexión anisotrópica presenta algunos problemas en el render, por ejemplo:

- ❶ La reflexión afecta el pelo de manera general, independientemente de la dirección de la luz. Esto es un problema, ya que si miramos la parte inferior del cabello de nuestro personaje, observaremos que los reflejos también se renderizan en esa área.
- ❷ La reflexión se mantiene pareja según la forma del modelo 3D. Sin embargo, esta debería distorsionarse en relación con la textura del cabello.
- ❸ El pelo aún sigue siendo afectado únicamente por la iluminación ambiental, por lo que continúa luciendo opaco.

Abordaremos estos problemas en nuestro shader, comenzando por el primero.

Para evitar que la reflexión afecte las zonas que deberían permanecer en sombra, multiplicaremos el resultado del modelo anisotrópico por el término difuso de **Lambert**, el cual nos resulta útil porque:

- Retorna valores cercanos a 1.0 (blanco) cuando la normal apunta en la misma dirección que la luz.
- Retorna valores cercanos a 0.0 (negro) cuando la normal se opone a la dirección de la luz.

Esto permite atenuar automáticamente los reflejos en aquellas regiones donde no hay iluminación directa.

```
float lambert(float i, vec3 l, vec3 n)
{
    return max(0.0, dot(n, l)) * i;
}

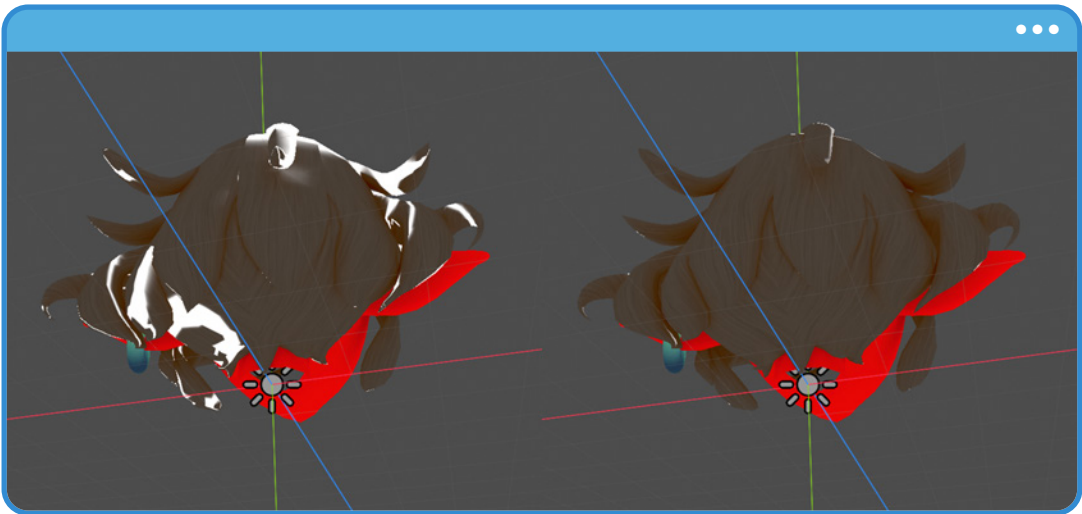
void light()
{
    float aniso = ashikhmin_shirley(_ReflectionFactor, _AU, _AV,
    NORMAL, LIGHT, VIEW, _tangent, _binormal);
    aniso *= lambert(ATTENUATION, LIGHT, NORMAL);

    // El código continúa en la página siguiente.
```

```
SPECULAR_LIGHT += aniso;
}
```

Al implementar el modelo de Lambert en nuestro código, el resultado gráfico se ajustará mejor a las condiciones de iluminación de la escena. Ahora los reflejos especulares se desvanecerán en las zonas en sombra, produciendo un resultado más realista.

El render debería lucir de la siguiente manera:



(2.11.g A la izquierda, **aniso**; a la derecha, **aniso** multiplicado por Lambert)

Como se puede observar en la Figura 2.11.g, al aplicar el modelo de Lambert conseguimos evitar reflexiones en aquellas zonas donde debería haber sombra.

Para solucionar el segundo punto mencionado anteriormente, debemos modular la reflexión anisotrópica utilizando los canales del albedo. Esto se logra declarando un nuevo vector tridimensional dentro del método **light()** y copiando en él los valores actuales de la variable **ALBEDO**:

```

void light()
{
    float aniso = ashikhmin_shirley(_ReflectionFactor, _AU, _AV,
    NORMAL, LIGHT, VIEW, _tangent, _binormal);

    vec3 albedo = ALBEDO;
    aniso *= albedo.r * albedo.g * albedo.b;
    aniso = smoothstep(0.0, 0.1, aniso);
    aniso *= lambert(ATTENUATION, LIGHT, NORMAL);

    SPECULAR_LIGHT += aniso;
}

```

Como podemos observar, el efecto anisotrópico está siendo modulado a través del producto `aniso *= albedo.r * albedo.g * albedo.b`. Esta operación multiplica el valor de la reflexión por la intensidad combinada de los tres canales de color RGB, lo que permite atenuar la especularidad en función del brillo general de la textura.

Para abordar el tercer punto, basta con asignar directamente el valor de `albedo` a la variable de salida `DIFFUSE_LIGHT`, lo que nos permitirá ver los colores de la textura en sus tonos originales:

```

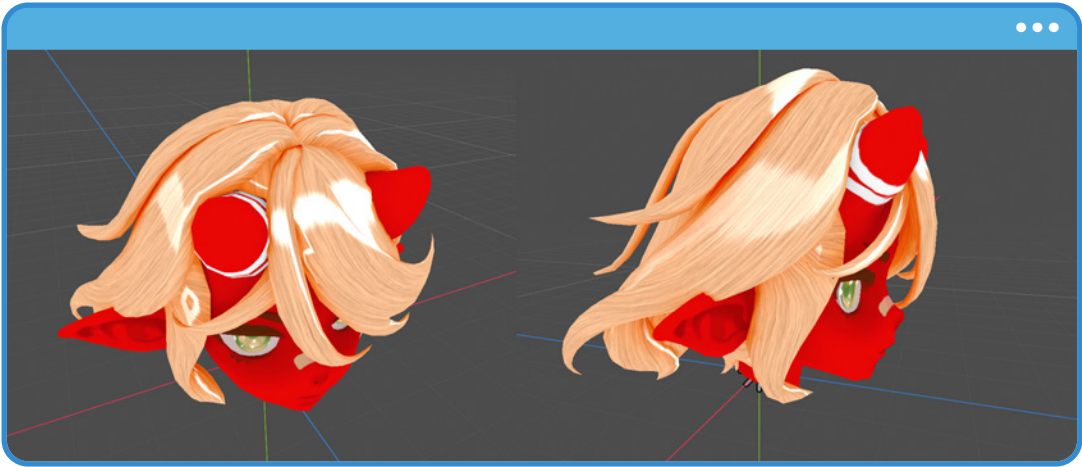
void light()
{
    float aniso = ashikhmin_shirley(_ReflectionFactor, _AU, _AV,
    NORMAL, LIGHT, VIEW, _tangent, _binormal);

    vec3 albedo = ALBEDO;
    aniso *= albedo.r * albedo.g * albedo.b;
    aniso = smoothstep(0.0, 0.1, aniso);
    aniso *= lambert(ATTENUATION, LIGHT, NORMAL);

    DIFFUSE_LIGHT += albedo;
    SPECULAR_LIGHT += aniso;
}

```

Esta configuración genera el siguiente resultado visual,

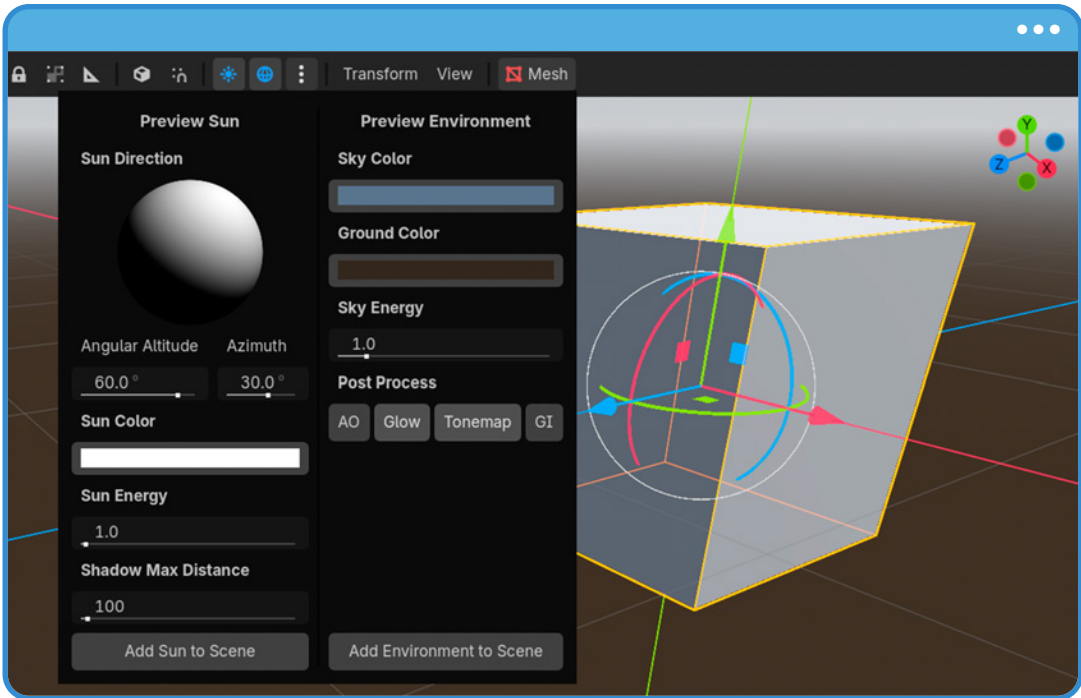


(2.11.h Albedo y especularidad anisotrópica funcionando en conjunto)

2.12 Sombreado hemisférico.

Cuando hablamos de sombreado hemisférico, nos referimos a la luz ambiental, es decir, aquella que simula los rebotes directos e indirectos de los rayos de luz en la escena de manera global. Este tipo de iluminación es fundamental porque nos permite integrar los objetos en el entorno con mayor naturalidad, evitando que se vean planos o desconectados del fondo.

En Godot ya contamos con un modelo que realiza esta operación llamado **World Environment**. A través de él podemos definir la influencia de la iluminación global sobre nuestros personajes y objetos. Incluso, en el Viewport podemos activar o desactivar esta influencia utilizando el botón **Toggle Preview Environment**, lo que nos ayuda a visualizar rápidamente cómo cambia la apariencia de la escena con o sin iluminación ambiental.



(2.12.a World Environment)

Entonces, ¿por qué desarrollar nuestra propia iluminación ambiental? Si Godot ya ofrece un sistema de iluminación global mediante el **World Environment**, ¿por qué querríamos implementar nuestro propio modelo de iluminación ambiental? La razón principal es que necesitamos mayor control cuando buscamos un render estilizado.

Las herramientas que ofrecen los motores de videojuegos por defecto son excelentes y están optimizadas para la mayoría de los casos. Sin embargo, cuando deseamos un estilo visual particular, por ejemplo, algo similar al estilo de DOGWALK, debemos tener control total sobre el resultado del render. Cuantas más técnicas de iluminación dominemos, mejores serán las posibilidades de lograr el estilo visual que queremos para nuestro proyecto.

La lógica detrás del sombreado hemisférico es sencilla y se basa en tres pasos principales:

- 1 Tomar la componente Y de las normales en espacio global.
- 2 Remapear su valor desde el rango $[-1 : 1]$ al rango $[0 : 1]$.
- 3 Usar ese valor como gradiente para una interpolación lineal de colores, definiendo un color para el suelo y otro para el cielo.

De este modo, cada superficie de nuestro modelo recibe un tono basado en la orientación de su normal: las zonas que apuntan hacia arriba se tiñen con el color del cielo, mientras que las que apuntan hacia abajo reciben el color del suelo.

¿Por qué remapear las normales? Sabemos que las normales son vectores de magnitud unitaria, que indican la dirección perpendicular a la superficie. Al analizar su componente Y en espacio global:

- Si el vector apunta hacia arriba, tenemos $(0, 1, 0)$.
- Si apunta en dirección opuesta, hacia abajo, obtendremos $(0, -1, 0)$.

Esto significa que los valores de la componente Y varían entre -1 y 1. Sin embargo, el gradiente de colores que usamos en la interpolación lineal necesita valores que comiencen en 0 y terminen en 1.

$$G = n_y * 0.5 + 0.5$$

(2.12.b)

Con este ajuste, conseguimos que un valor de -1 pase a 0, y un valor de 1 se mantenga en 1, lo que nos permite usar el resultado como factor de interpolación entre el color del suelo y el del cielo.

Para ver esto en práctica comenzaremos organizando nuestro proyecto de la misma manera que en ejercicios anteriores.

- Dentro de la carpeta **chapter_02**, crearemos una nueva carpeta llamada **hemisphere**.
- Dentro de esta última, organizaremos el contenido de la siguiente manera:
 - **materials.**
 - **shaders.**
 - **meshes.**
 - **textures.**

A continuación, crearemos el material que aplicaremos al personaje. Para ello, nos posicionamos sobre la carpeta materials, hacemos clic derecho y seleccionamos:

- Create New > Resources > ShaderMaterial.
- Nombraremos a este material como **hemisphere**.

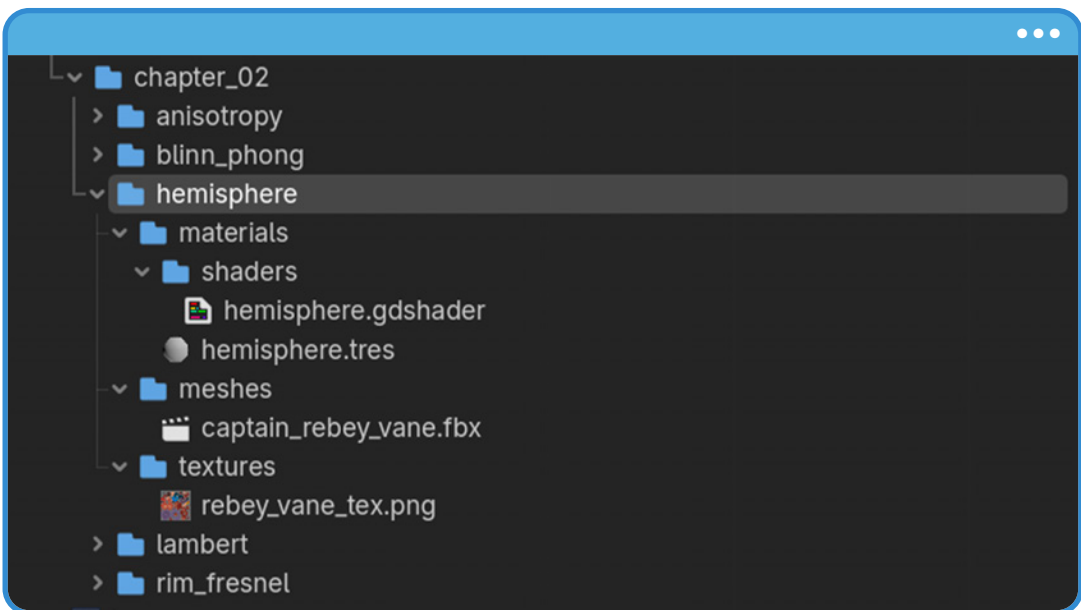
Luego, creamos el shader correspondiente. Posicionándonos sobre la carpeta shaders, presionamos clic derecho y seleccionamos:

- Create New > Resources > Shader.

Llamaremos a este archivo **hemisphere** también, manteniendo la coherencia con el nombre de la sección.

En esta sección utilizaremos a nuestro personaje icónico, **Rebey Vane**, junto con sus texturas y materiales. Todo estos recursos se incluyen en el paquete descargable que acompaña al libro, por lo que podremos seguir los pasos sin inconvenientes.

Si hemos configurado todo correctamente, la estructura de carpetas de nuestro proyecto debería verse de la siguiente manera:



(2.12.c Se ha incluido la carpeta hemisphere en el proyecto)

El primer paso será crear una escena 3D y arrastrar a nuestro personaje **captain_rebey_vanne.fbx** dentro de ella. En la sección 1.4 del capítulo 1 revisamos cómo definir una instancia única de un modelo **.fbx**, en esta ocasión marcaremos al personaje como **Make Local** en la ventana Scene. Como ya sabemos, con esta acción, Godot nos otorgará acceso al **MeshInstance3D** del modelo, lo que nos permitirá asignar el material hemisphere creado previamente en la propiedad **Material Override** del personaje.

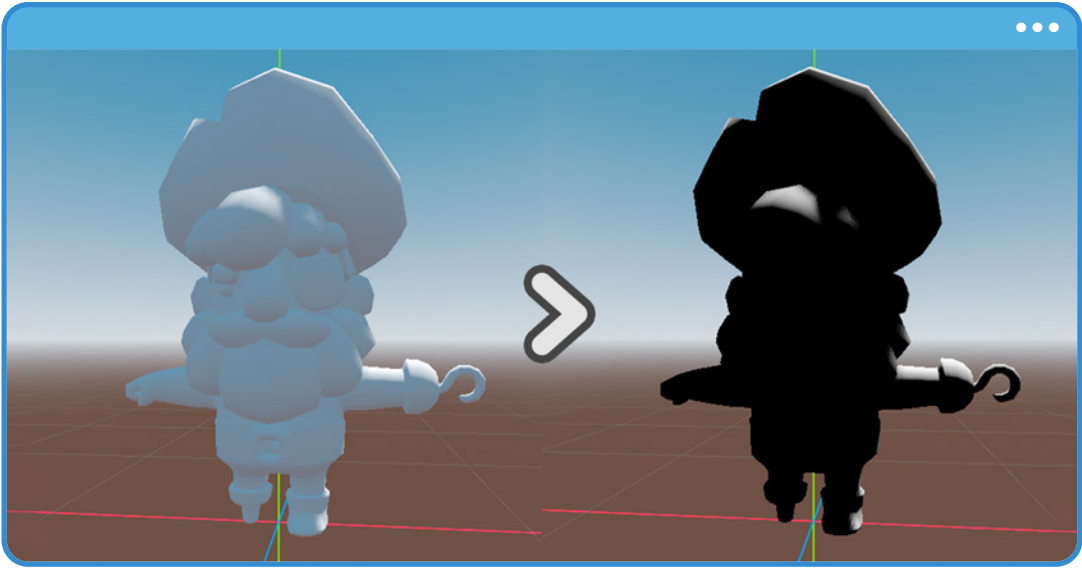
Una vez hecho esto, abriremos el shader hemisphere. En esta sección desarrollaremos nuestro propio modelo de luz ambiental, por lo que comenzaremos declarando algunas propiedades que utilizaremos más adelante. También deshabilitaremos la influencia de la luz ambiental predeterminada activando el modo de renderizado **ambient_light_disabled**, tal como se muestra a continuación:

```
shader_type spatial;
render_mode ambient_light_disabled;

uniform sampler2D _MainTex : source_color;
uniform vec3 _GroundColor : source_color;
uniform vec3 _SkyColor : source_color;
uniform float _Smoothness: hint_range(0.0, 0.5, 0.01);

void vertex() { ... }
```

Al guardar el shader observaremos dos cosas principales: primero, tendremos acceso a las propiedades desde el material en el **Inspector**, y segundo, nuestro personaje se tornará completamente negro porque ya no recibe influencia de la luz ambiental predeterminada.



(2.12.d A la derecha, la luz ambiental ha sido desactivada)

Daremos paso a la implementación de la textura. Primero debemos asegurarnos de asignar la textura **rebey_vane_tex.png** en su propiedad correspondiente dentro del **Inspector**. A continuación, declararemos un vector tridimensional en la etapa de fragmento y lo utilizaremos como valor para **ALBEDO**, de la siguiente manera:

```
void fragment()
{
    vec3 albedo = texture(_MainTex, UV).rgb;
    ALBEDO = albedo;
}
```

Con esta operación, nuestro personaje en la escena mostrará los colores definidos en la textura, aplicados correctamente a cada área del modelo. Sin embargo, seguirá sin recibir influencia de la luz ambiental, lo que provoca que el resultado se vea plano.

Para solucionar parcialmente este problema, habilitaremos el método **light()** y añadiremos un valor constante de 0.5 a **DIFFUSE_LIGHT**:

```
void light()
{
    DIFFUSE_LIGHT += vec3(0.5);
}
```

Hemos escogido el valor 0.5 porque actúa como un punto neutro: no ilumina en exceso ni oscurece demasiado, lo que resulta ideal para visualizar de manera clara el color del albedo en esta etapa.



(2.12.e)

No obstante, reemplazaremos el valor constante de la difusión por la operación que describiremos a continuación. Antes de hacerlo, debemos incluir la función encargada de calcular el sombreado hemisférico:

```
float hemisphere(vec3 n, float s)
{
    float t = n.y * 0.5 + 0.5; // remap [-1, 1] -> [0, 1]
    return smoothstep(0.0 + s, 1.0 - s, t);
}

void light() { ... }
```

En la primera línea del método **hemisphere()** realizamos el remapeo de la componente **n.y**. Como ya sabemos, este proceso transforma el rango original de la normal, que va de -1 a 1, a un rango de 0 a 1. En la línea siguiente devolvemos el resultado del suavizado aplicado al gradiente mediante la función **smoothstep()**, lo que nos permite generar un efecto más estilizado y controlar la transición entre los colores del cielo y el suelo.

Dado que el primer argumento de la función corresponde a la normal del modelo en espacio global y el segundo al nivel de suavizado, podemos implementar el cálculo dentro del método **light()** de la siguiente manera:

```
void light()
{
    vec3 normal_ws = (INV_VIEW_MATRIX * vec4(NORMAL, 0.0)).xyz;

    float gradient = hemisphere(normal_ws, _Smoothness);
    vec3 custom_ambient = mix(_GroundColor, _SkyColor, gradient);

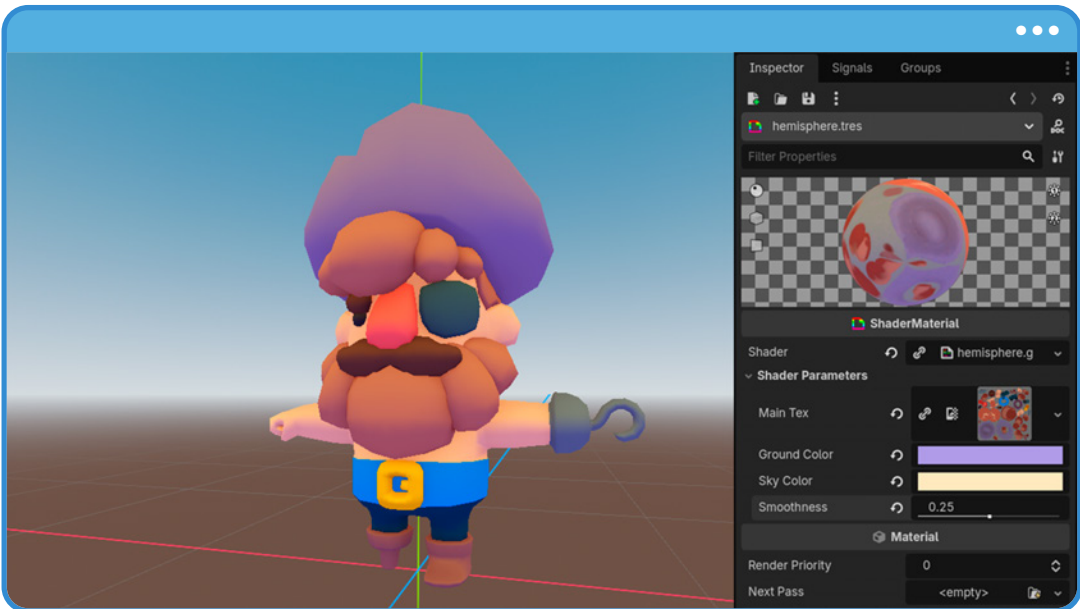
    DIFFUSE_LIGHT += custom_ambient;
}
```

En la primera línea del método **light()** transformamos la normal de espacio de vista a espacio global, ya que, por defecto, las normales en esta etapa se encuentran en espacio de vista. El resultado se almacena en el vector **normal_ws**, que utilizamos como entrada para el cálculo del gradiente. Posteriormente, mezclamos los colores del suelo y del cielo mediante la función **mix()**, y finalmente los añadimos a **DIFFUSE_LIGHT** para aplicar nuestro sombreado hemisférico personalizado.

Nota

Desde el punto de vista de la optimización, sería más conveniente realizar la transformación de las normales dentro del método `fragment()` en lugar de hacerlo en `light()`. De esta manera evitamos repetir el cálculo para cada luz presente en la escena.

Luego declaramos un nuevo escalar llamado **gradient**, que almacena el valor obtenido del sombreado hemisférico. Finalmente, utilizamos este gradiente como tercer argumento en la interpolación lineal que combina los dos colores definidos: el del suelo y el del cielo. El resultado de esta operación se guarda en el vector RGB **custom_ambient**, el cual se suma posteriormente a la luz difusa del modelo.



(2.12.f Se han aplicado dos colores ambientales sobre el pirata)

Podríamos dar por finalizado el ejercicio en este punto, ya que el sombreado hemisférico ha sido implementado correctamente. Sin embargo, en la siguiente sección realizaremos un ejercicio sencillo que nos servirá como introducción a los **ShaderInclude**, lo que nos permitirá organizar mejor el código y mejorar el resultado del efecto.

2.13 Implementando un ShaderInclude.

Supongamos que queremos transformar el color ambiental personalizado de espacio lineal a sRGB. Este proceso es sencillo, ya que en la sección 2.7 introdujimos este concepto. Sin embargo, surge una pregunta importante: ¿cuál es la mejor manera de implementar esta transformación en nuestro shader? ¿Deberíamos copiar y pegar el método nuevamente o reutilizarlo? La respuesta correcta es reutilizarlo, y para ello debemos conocer el concepto de **ShaderInclude**.

Según la documentación oficial en Godot:

“

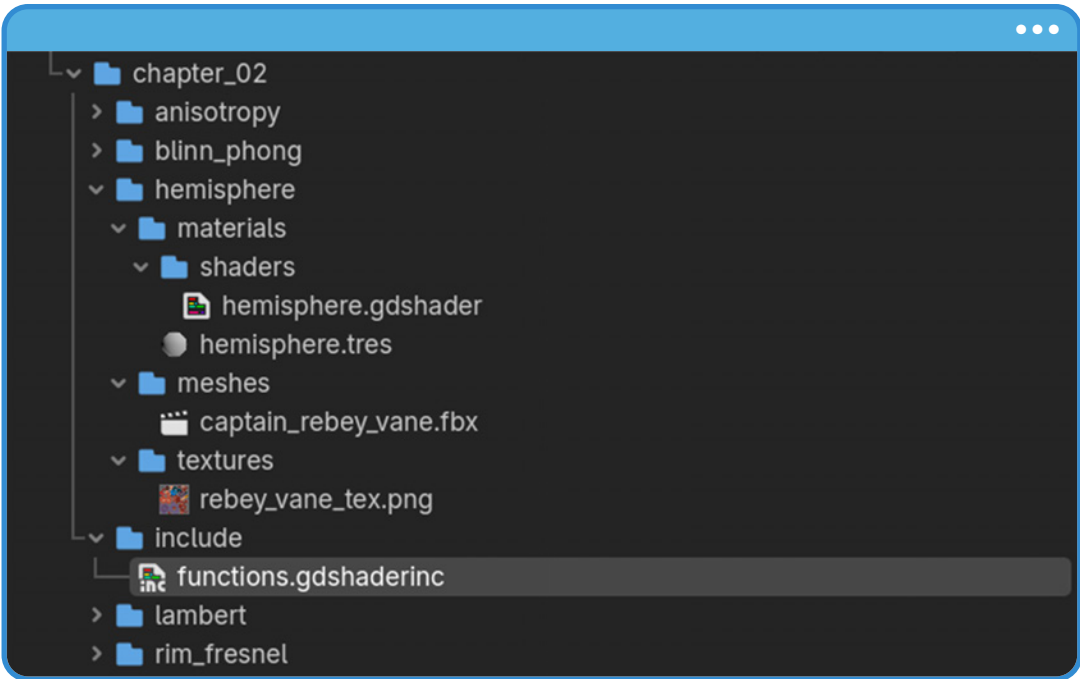
Es un archivo de inclusión de sombreador, guardado con la extensión **.gdshaderinc**. Esta clase permite definir un fragment de sombreador personalizado que puede incluirse en un sombreador mediante la directiva de preprocesador **#include**, seguida por la ruta del archivo.

”

Para entenderlo en la práctica, realizaremos lo siguiente:

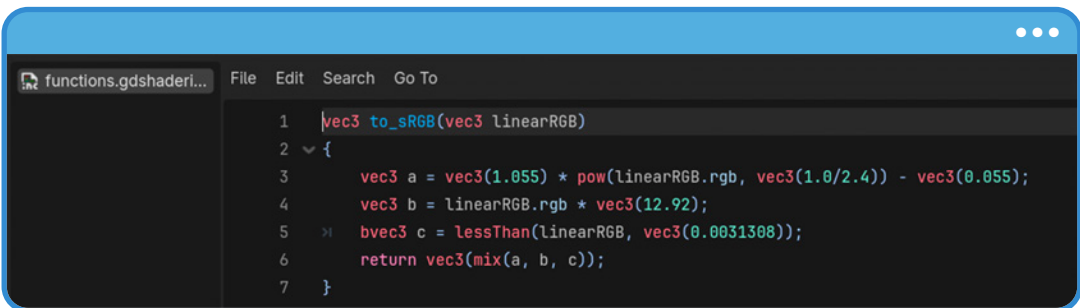
- En nuestro proyecto, dentro de **chapter_02**, crearemos una nueva carpeta llamada **include**.
- Sobre esta carpeta, haremos clic derecho y seleccionamos: Create new > Resource > ShaderInclude.
- A este archivo lo nombraremos como **functions.gdshaderinc**.

Si hemos seguido los pasos correctamente, la estructura de nuestro proyecto debería lucir así:

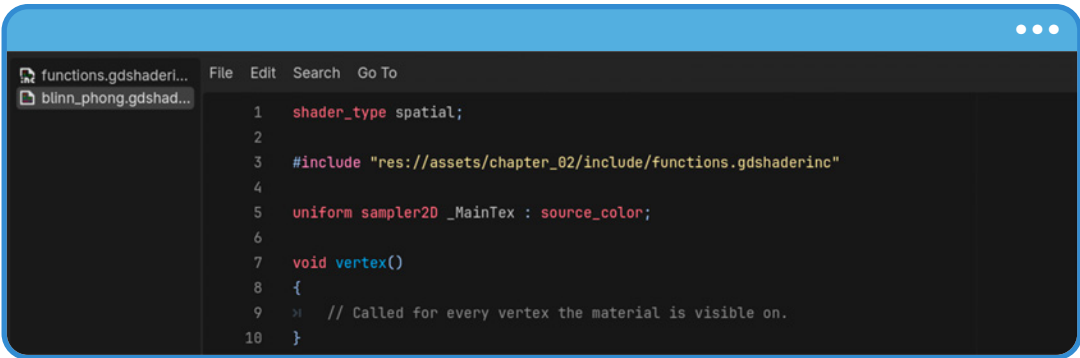


(2.13.a Se ha incluido un ShaderInclude en el proyecto)

A continuación, regresaremos al shader **bling_phong**, donde definimos previamente el método **to_sRGB()**. Lo que haremos será copiar este método completo y pegarlo dentro del archivo **functions**, tal como se muestra a continuación:

(2.13.b Se ha incluido el método **to_sRGB()** en functions)

Ahora podemos realizar la siguiente acción: comentar el método **to_sRGB()** en el shader **bling_phong**. Al hacerlo, nuestro shader dejará de compilar, lo cual es completamente lógico ya que la función ha sido eliminada. Para resolver este problema, simplemente incluiremos nuestro **ShaderInclude** de la siguiente manera:



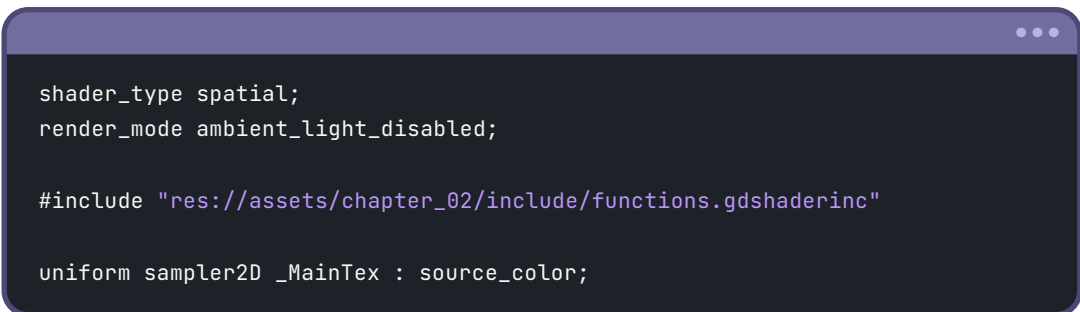
```

1  shader_type spatial;
2
3  #include "res://assets/chapter_02/include/functions.gdshaderinc"
4
5  uniform sampler2D _MainTex : source_color;
6
7  void vertex()
8  {
9      // Called for every vertex the material is visible on.
10 }
```

(2.13.c Se ha incluido la ruta del ShaderInclude)

Podemos observar en la línea 3 del código anterior que incluimos la ruta del **ShaderInclude** dentro del shader **blinn_phong**. A partir de ese momento, reutilizamos el método **to_sRGB()** mediante la directiva de preprocesador **#include**. Este enfoque es especialmente útil porque, si necesitamos reutilizar más funciones, basta con declararlas en **functions.gdshaderinc** e incluirlas en nuestros shaders. Así optimizamos el flujo de trabajo y centralizamos futuras modificaciones en un solo archivo compartido.

Volviendo a nuestro shader hemisphere, agregaremos la ruta del include de la siguiente manera:



```

shader_type spatial;
render_mode ambient_light_disabled;

#include "res://assets/chapter_02/include/functions.gdshaderinc"

uniform sampler2D _MainTex : source_color;
```

Luego, en el método **light()** transformaremos **custom_ambient** de espacio lineal a sRGB:


```

void light()
{
    vec3 normal_ws = (INV_VIEW_MATRIX * vec4(NORMAL, 0.0)).xyz;

    float gradient = hemisphere(normal_ws, _Smoothness);
    vec3 custom_ambient = mix(_GroundColor, _SkyColor, gradient);
    custom_ambient = to_sRGB(custom_ambient);

    DIFFUSE_LIGHT += custom_ambient;
}

```

De esta manera, el resultado visual se aproxima mejor al renderizado estilizado que buscamos, manteniendo el código ordenado y reutilizable.



(2.13.d A la izquierda, lineal; a la derecha, sRGB)

Por último, implementaremos el modelo Lambertiano en el shader para acentuar la diferencia entre luces y sombras.

```

void light()
{
    float NdotL = max(dot(NORMAL, LIGHT), 0.0);
    vec3 lambert = NdotL * ATTENUATION * LIGHT_COLOR;

    DIFFUSE_LIGHT += lambert;

    vec3 normal_ws = (INV_VIEW_MATRIX * vec4(NORMAL, 0.0)).xyz;

    float gradient = hemisphere(normal_ws, _Smoothness);
    vec3 custom_ambient = mix(_GroundColor, _SkyColor, gradient);
    custom_ambient = to_sRGB(custom_ambient);

    DIFFUSE_LIGHT += custom_ambient;
}

```

En este caso, como vemos en el código anterior, se ha duplicado el modelo explicado en la sección 2.2. ¡Ahora es tu turno! Haz el ejercicio, incluye el método **lambert** en el **Shader-include** y luego llámalo desde el método **light()**, ¿cómo lo harías? Asegúrate de que el shader compile y que el resultado gráfico refleja exactamente la operación matemática que estamos llevando a cabo.



(2.13.e Render estilizado con luces y sombras)

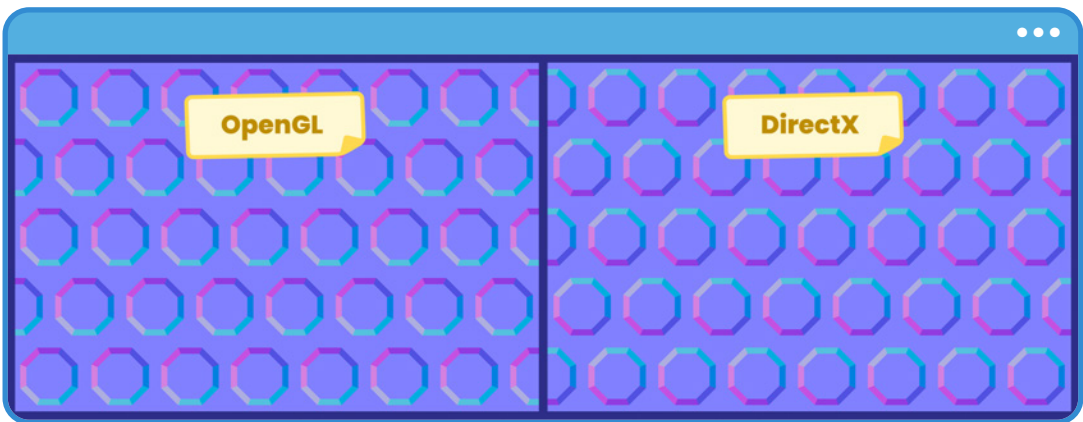
2.14 Implementación de mapa de normales.

En Godot, cuando creamos un `StandardMaterial3D`, encontramos entre sus propiedades una opción llamada **Normal Map**, que por defecto está deshabilitada. Al activarla, obtenemos acceso a dos parámetros principales:

- **Scale:** controla la intensidad del efecto.
- **Texture:** corresponde a la textura del mapa de normales.

El uso de un mapa de normales es una técnica ampliamente empleada para añadir detalle a la superficie de un objeto sin necesidad de incrementar su número de vértices. En combinación con la iluminación, este recurso permite simular relieves y variaciones en la superficie, logrando un aspecto mucho más rico y realista sin afectar de forma significativa el rendimiento.

Un ejemplo de cómo luce un mapa de normales es el siguiente:



(2.14.a Map de normales en OpenGL y DirectX)

Un mapa de normales se compone de distintos colores que representan direcciones específicas:

- Coordenada X: rojo cuando apunta en dirección positiva, cian cuando es negativa.
- Coordenada Y: verde cuando apunta en dirección positiva, púrpura cuando es negativa.
- Coordenada Z: representa la dirección perpendicular hacia fuera de la superficie.

Si observamos la Figura 2.14.a, notaremos que la coordenada Y difiere en ambos mapas de normales. Esto se debe a la manera en que cada API define la orientación de las coordenadas UV. Sin embargo, a pesar de esta diferencia, el resultado gráfico final es equivalente.

Para implementar mapas de normales en un shader debemos considerar tres aspectos fundamentales:

- Remapear las coordenadas.
- Reconstruir la coordenada Z.
- Aplicar la matriz TBN (Tangente, Binormal y Normal).

En la sección 1.8 del capítulo 1 exploramos el uso de tangentes, normales y binormales al crear un efecto de profundidad en un ojo de estilo anime. Ahora llevaremos este concepto un paso más allá y lo aplicaremos a través de una matriz completa que nos permitirá transformar las normales desde el mapa a espacio de vista para agregar detalles en nuestro objeto.

Es cierto que en la etapa de fragmento podemos utilizar la variable interna **NORMAL_MAP**, la cual realiza automáticamente el cálculo de las normales a partir de un mapa. Esta variable se emplea junto con **NORMAL_MAP_DEPTH** para aumentar o disminuir la intensidad del efecto. Sin embargo, si queremos comprender a fondo lo que ocurre en el proceso, es necesario revisar paso a paso cómo implementar manualmente un mapa de normales.

Para ello, haremos lo siguiente:

- 1 Dentro de nuestro proyecto, en **chapter_02**, crearemos una nueva carpeta llamada **normal_map**.
- 2 Dentro de esta carpeta, añadiremos las siguientes subcarpetas:
 - a **materials**.
 - b **shaders**.
 - c **meshes**.
 - d **textures**.

Con esta organización lista, incluiremos los archivos descargables correspondientes a esta sección.

Para demostrar la utilidad de los mapas de normales realizaremos un ejercicio en el que, mediante un **shader**, implementaremos todos los pasos necesarios para aplicarlos. Nos detendremos en cada función por separado para entender su lógica desde un punto de vista matemático. En este proceso utilizaremos tanto la variable **NORMAL** como **NORMAL_MAP** en distintas etapas, de modo que podamos apreciar sus diferencias técnicas.

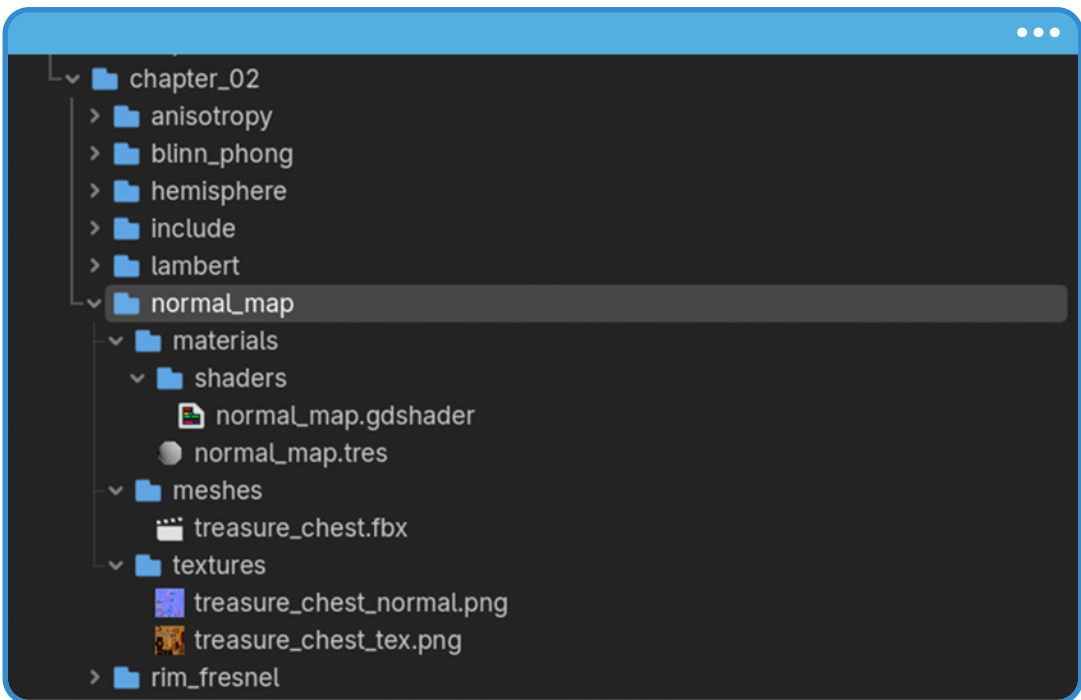
Comenzaremos creando un material. Para ello, sobre la carpeta **materials**, presionamos clic derecho y seleccionamos:

➤ Create New > Resource > ShaderMaterial.

Por razones prácticas, lo nombraremos `normal_map`. Luego crearemos un shader dentro de la carpeta `shaders`, repitiendo el mismo nombre para mantener consistencia:

➤ Create New > Resource > Shader.

Si hemos seguido los pasos correctamente, nuestro proyecto deberá lucir de la siguiente manera:



(2.14.b Se han agregado un nuevo material y un shader al proyecto)

Antes de comenzar con el desarrollo del shader, crearemos una nueva escena 3D y arrastraremos el objeto **treasure_chest.fbx** dentro de ella. Dado que se trata de un archivo con extensión **.fbx**, nos aseguraremos de marcarlo como **Make Local** en la ventana **Scene**, de modo que podamos acceder al **MeshInstance3D** del modelo. Además, añadiremos una luz direccional 3D, ya que la necesitaremos para apreciar los cambios que producirá el mapa de normales sobre los detalles del cofre.

A continuación, asignaremos el shader **normal_map** a su respectivo material y luego aplicaremos ese material al objeto en el **Viewport**. De esta manera podremos observar los cambios conforme avancemos en la implementación.

El primer paso en nuestro shader será declarar las propiedades que utilizaremos a lo largo del desarrollo:

- Un mapa de albedo al que llamaremos **_MainTex**,
- Un mapa de normales al que llamaremos **_NormalMap**,
- Una variable flotante que nos permitirá controlar la intensidad del efecto, a la que llamaremos **_Scale**.

El código inicial se verá de la siguiente manera:

```
shader_type spatial;

uniform sampler2D _MainTex : source_color;
uniform sampler2D _NormalMap : hint_normal;
uniform float _Scale : hint_range(-16.0, 16.0, 0.1);

void vertex() { ... }
```

Si observamos el código anterior, notaremos que **_NormalMap** posee el atributo **hint_normal**. Este hint es importante porque activa la reimportación de una textura como mapa de normales cuando la asignamos desde el Inspector. Por lo tanto, es fundamental incluirlo en el shader.

En cuanto a la variable `_Scale`, hemos definido un rango que va de -16.0 a 16.0. Desde un punto de vista práctico, un rango entre 0.0 y 1.0 sería más que suficiente. Sin embargo, en este caso utilizamos ese rango específico para mantener la coherencia con el valor por defecto que ofrece un **StandardMaterial3D** para los mapas de normales.

El siguiente paso es inicializar e implementar la variable **ALBEDO** para mostrar la textura sobre el cofre en la escena. Para visualizar el resultado de forma correcta, debemos asegurarnos de asignar tanto la textura **treasure_chest_tex** como **treasure_chest_normal** en sus respectivas propiedades.

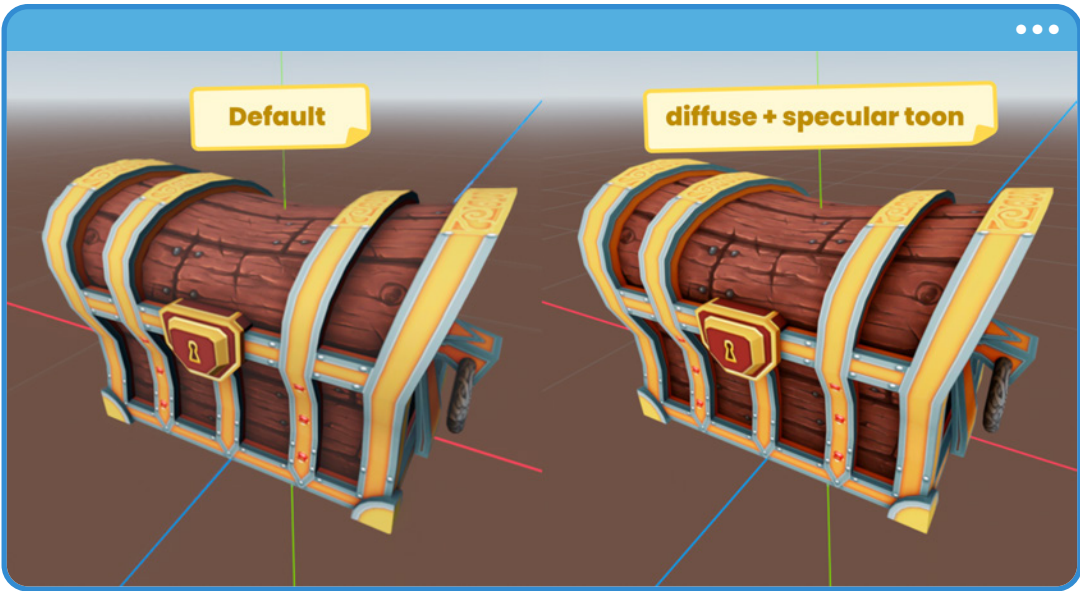
```
void fragment()
{
    vec3 albedo = texture(_MainTex, UV).rgb;
    ALBEDO = albedo;
}
```

En este punto, el cofre ya está siendo afectado por la iluminación direccional y ambiental. No obstante, realizaremos algunos cambios en la configuración de iluminación para resaltar mejor el relieve generado por el mapa de normales. Para ello, incluiremos los modos de renderizado **diffuse_toon** y **specular_toon**, como se muestra a continuación:

```
shader_type spatial;
render_mode diffuse_toon;
render_mode specular_toon;

uniform sampler2D _MainTex : source_color;
uniform sampler2D _NormalMap : hint_normal;
uniform float _Scale : hint_range(-16.0, 16.0, 0.1);
```

Estos modos de render nos permiten obtener un resultado gráfico estilizado, donde el sombreado y los brillos se simplifican, lo que a su vez resalta con mayor claridad los relieves producidos por el mapa de normales.



(2.14.c Se ha mejorado el contraste sobre el cofre)

Para iniciar el proceso de implementación de normales, lo primero que debemos hacer es muestrear la textura y guardar su resultado en un vector tridimensional. A diferencia de las coordenadas UV, los mapas de normales trabajan en un rango que va de -1.0 a 1.0. Por lo tanto, el siguiente paso será remapear los valores de la textura a este rango. Podemos hacerlo de manera sencilla con la siguiente operación:

```
void fragment()
{
    vec3 albedo = texture(_MainTex, UV).rgb;

    vec3 normal_map = texture(_NormalMap, UV).rgb;
    normal_map = normal_map * 2.0 - 1.0;

    ALBEDO = albedo;
}
```


Nota

si el canal Y de la textura viene invertido por diferencias de convención (OpenGL vs DirectX), podemos corregirlo con `normal_map.y *= -1.0`; antes de la reconstrucción. También podemos realizar este proceso directamente desde la ventana **Import**, seleccionando el mapa de normales en nuestro proyecto y habilitando **Normal Map Invert Y**. Esto asegura que las luces reaccionen en la dirección esperada.

¿Por qué la operación `normal_map * 2.0 - 1.0` realiza el remapeo que necesitamos? La explicación es simple:

Si tomamos el valor mínimo del rango original (0.0), obtenemos:

$$0.0 * 2.0 - 1.0 = -1.0$$

(2.14.d)

Si tomamos el valor máximo (1.0), obtenemos:

$$1.0 * 2.0 - 1.0 = 1.0$$

(2.14.e)

De esta manera, logramos transformar el rango $[0.0 : 1.0]$ propio de una textura al rango $[-1.0 : 1.0]$ que necesitamos para trabajar con normales.

El siguiente paso consiste en reconstruir la componente Z del mapa de normales. Esto es necesario porque las texturas almacenan únicamente las componentes X (canal rojo) e Y (canal verde), mientras que la componente Z, que representa la dirección perpendicular a la superficie, suele omitirse para ahorrar espacio.

Además, debemos recordar que los mapas de normales están expresados en espacio de tangente, donde las normales apuntan aproximadamente hacia la dirección positiva del eje Z. El espacio de tangente se define mediante tres vectores ortogonales: la normal, el tangente y el bitangente. Esta base nos permite transformar la normal extraída del mapa hacia un

espacio de iluminación común, que en este caso será el espacio de vista, garantizando que la interacción con la luz sea consistente y precisa.

Podemos obtener la componente Z con la siguiente operación:

$$z = \sqrt{1.0 - (x^2 + y^2)}$$

(2.14.f)

Si prestamos atención a la ecuación anterior, observaremos que se está empleando el teorema de Pitágoras en tres dimensiones.

$$x^2 + y^2 + z^2 = 1$$

(2.14.g)

Las normales deben ser vectores unitarios; como el mapa solo nos entrega X e Y, reconstruimos Z y nos aseguramos de mantener su longitud igual a 1. En código, conviene protegernos de errores numéricos usando `clamp` (o `max`) antes de la raíz:

```
void fragment()
{
    ...

    float n_z = clamp(dot(normal_map.xy, normal_map.xy), 0.0, 1.0);
    normal_map.z = sqrt(1.0 - n_z);

    ALBEDO = albedo;
}
```

A continuación transformamos la normal desde espacio de tangente a espacio de vista con la matriz TBN (Tangente, Binormal, Normal). Recordemos que en Godot contamos con las variables **TANGENT**, **BINORMAL** y **NORMAL**; dentro de la etapa de fragmento; al construir la matriz `mat3`, cada vector se usa como columna dentro de la misma:

```

void fragment()
{
    ...

    mat3 TBN = mat3(TANGENT.xyz, BINORMAL.xyz, NORMAL.xyz);
    vec3 normal_vs = normalize(TBN * normal_map);

    ALBEDO = albedo;
    NORMAL = normal_vs;
}

```

Al asignar el valor de `normal_vs` a la variable `NORMAL`, inmediatamente notaremos cambios en la cantidad de detalles que posee la textura sobre el cofre en el **Viewport**. Sin embargo, si deseamos modificar este valor, podemos multiplicar el vector `normal_map` por `_Scale`, como se muestra en el siguiente ejemplo:

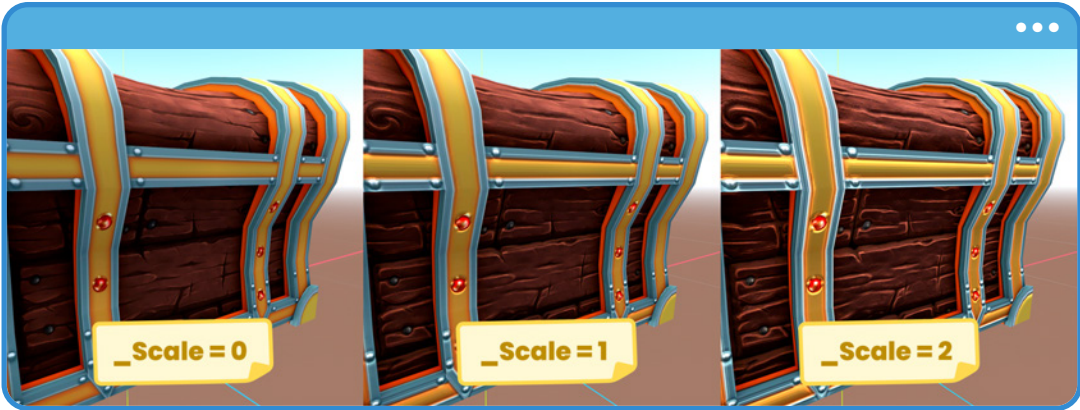
```

void fragment()
{
    vec3 albedo = texture(_MainTex, UV).rgb;

    vec3 normal_map = texture(_NormalMap, UV).rgb;
    normal_map = normal_map * 2.0 - 1.0;
    normal_map *= _Scale;

    ...
}

```



(2.14.h Distintas durezas para el mapa de normales)

Ahora, en Godot no es necesario realizar manualmente todos los pasos para calcular un mapa de normales, ya que contamos con una variable interna llamada **NORMAL_MAP** que lleva a cabo el proceso automáticamente. De esta manera, podemos obtener el mismo resultado con una implementación mucho más sencilla:

```
void fragment()
{
    vec3 albedo = texture(_MainTex, UV).rgb;
    vec3 normal_map = texture(_NormalMap, UV).rgb;

    NORMAL_MAP = normal_map;
    NORMAL_MAP_DEPTH = 1.0;
    ALBEDO = albedo;
}
```

Resumen del capítulo.

En este capítulo se adquirieron los conocimientos fundamentales sobre iluminación y renderizado, comprendiendo cómo los materiales y los modelos de iluminación influyen directamente en la apariencia final de los objetos en pantalla. Se estudiaron las propiedades de un material y su relación con los cálculos de iluminación realizados en el shader.

Se aprendieron los principios del modelo Lambertiano y su implementación práctica como base de la iluminación difusa, así como la diferencia entre vectores que representan puntos y direcciones, un concepto clave para el correcto cálculo de la iluminación en el espacio tridimensional. A partir de estos fundamentos, se profundizó en el modelo Blinn-Phong, incorporando la iluminación especular y entendiendo su comportamiento dentro del pipeline de renderizado.

El capítulo también permitió comprender la importancia de trabajar en espacio lineal y realizar la conversión adecuada a sRGB para obtener resultados visuales personalizados. Posteriormente, se exploraron efectos de sombreado más avanzados, como el efecto Rim (Fresnel) y la iluminación anisotrópica, culminando con la implementación del modelo Ashikhmin-Shirley para simular superficies con propiedades direccionales.

Finalmente, se aprendió a aplicar sombreado hemisférico como una aproximación simple de iluminación ambiental, a estructurar código reutilizable mediante ShaderInclude, y a implementar mapas de normales, consolidando así una base sólida para el desarrollo de shaders de iluminación más complejos en capítulos posteriores.



Capítulo 3

Figuras procedurales y animación por vértices.

Durante este capítulo nos adentraremos en varios tópicos esenciales para llevar nuestras habilidades con shaders a un nuevo nivel. Comenzaremos explorando cómo dibujar figuras procedimentales bidimensionales utilizando funciones matemáticas simples, desigualdades y coordenadas UV. Al tratarse de un espacio cartesiano, podremos definir áreas, contornos y transiciones directamente sobre la superficie de la textura, estableciendo así la base para construir formas complejas sin necesidad de recursos externos.

Avanzaremos luego hacia la animación y modificación procedural, entendiendo cómo transformar estas figuras en elementos dinámicos mediante operaciones temporales, interpolaciones y ajustes basados en parámetros. Estos conceptos nos permitirán desarrollar un personaje completamente procedural implementado en GDScript, donde cada parte visual surge matemáticamente y responde a variables configurables.

Además, estudiaremos técnicas de deformación de vértices orientadas a la interfaz de usuario, lo que nos permitirá animar elementos directamente desde el shader. Sobre esta misma línea, aprenderemos a integrar un efecto especular aplicado a la UI, útil para generar superficies brillantes, realzar detalles o añadir un acabado más estilizado a cualquier panel o figura.

En la segunda parte del capítulo, nos adentraremos en las rotaciones. Revisaremos su interpretación geométrica y aprenderemos a implementar matrices de rotación personalizadas, indispensables para manipular puntos en el espacio y animar geometrías de forma precisa. Luego pasaremos al estudio de los cuaterniones, entendiendo su estructura y las ventajas que ofrecen frente a las matrices tradicionales al evitar problemas como el Gimbal Lock. Implementaremos estas herramientas en GDScript para realizar rotaciones estables y eficientes dentro del shader.

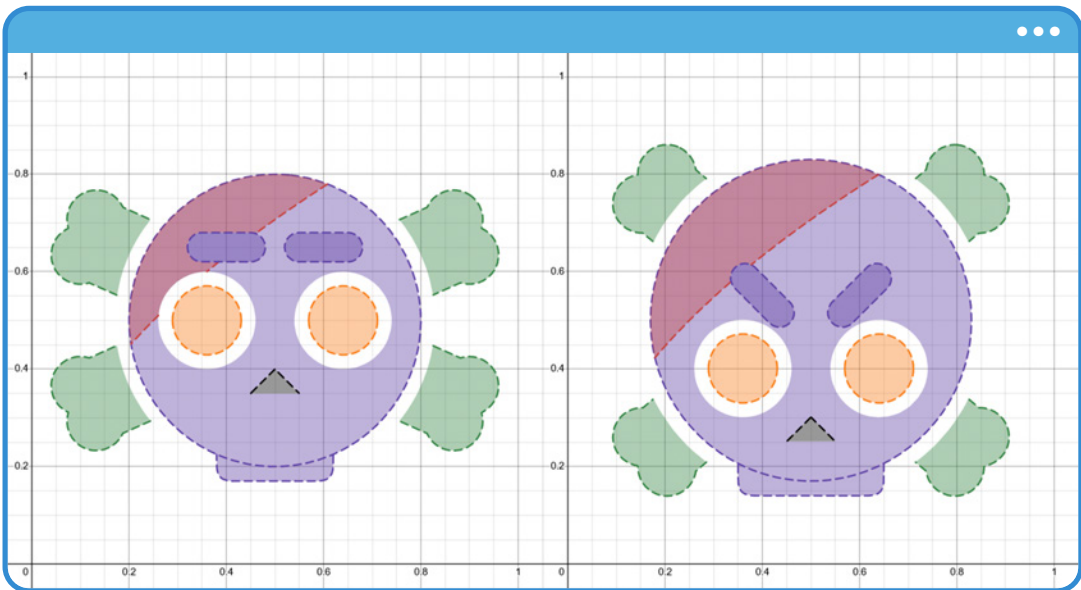
Cada uno de estos temas será desarrollado mediante ejemplos prácticos que nos permitirán visualizar paso a paso cómo se integran dentro de un flujo artístico y técnico. El objetivo final es ampliar nuestro repertorio de herramientas y técnicas para resolver problemas complejos mediante el uso inteligente de las matemáticas y shaders personalizados.

3.1 Dibujando con funciones e inecuaciones.

Cuando hablamos de funciones e inecuaciones, nos referimos a expresiones matemáticas que podemos aplicar directamente sobre coordenadas cartesianas. Algunas de las más comunes incluyen:

- Funciones lineales
- Funciones cuadráticas.
- Funciones trigonométricas.
- Funciones de valor absoluto.
- Funciones exponenciales.
- Etc.

Cada una de estas funciones posee propiedades únicas que, al ser combinadas, nos permiten construir formas complejas de manera procedimental. Si bien analizar su comportamiento en profundidad y su implementación detallada en el lenguaje de shaders de Godot podría requerir un libro completo, en este capítulo adoptaremos un enfoque más práctico: nos centraremos en traducir estas funciones al contexto de shaders mientras diseñamos una figura procedural paso a paso. Como ejemplo concreto, construiremos una calavera de estilo pirata utilizando únicamente funciones matemáticas e instrucciones condicionales, como se muestra en la siguiente referencia visual:



(3.1.a Calavera pirata en Desmos)

Nota

Hace un tiempo escribí un libro llamado *Shaders y Formas Procedurales en Unity 6*, donde se explora este tema en profundidad. Aunque el enfoque fue Unity, las funciones y conceptos descritos son perfectamente transferibles a Godot. Si te interesa conocer más sobre este proyecto, puedes visitarlo en el siguiente enlace: <https://jettelly.com/store/visualizing-equations-vol-2>

Si observamos con atención la Figura 3.1.a, notaremos que la calavera está construida principalmente a partir de ecuaciones matemáticas aplicadas sobre coordenadas bidimensionales. Estas ecuaciones, que definiremos más adelante, nos permiten delimitar regiones visibles en pantalla mediante comparaciones y operaciones sobre las coordenadas x e y . Este enfoque facilita el proceso de diseño procedural, ya que cada parte de la figura responde a una condición matemática precisa.

Ahora bien, ¿por dónde deberíamos empezar si quisiéramos crear esta figura desde cero? Aunque no existe un único punto de partida, sí podemos seguir un flujo lógico que nos ayude a comprender mejor el uso de coordenadas UV y, con el tiempo, dominar el diseño bidimensional en shaders.

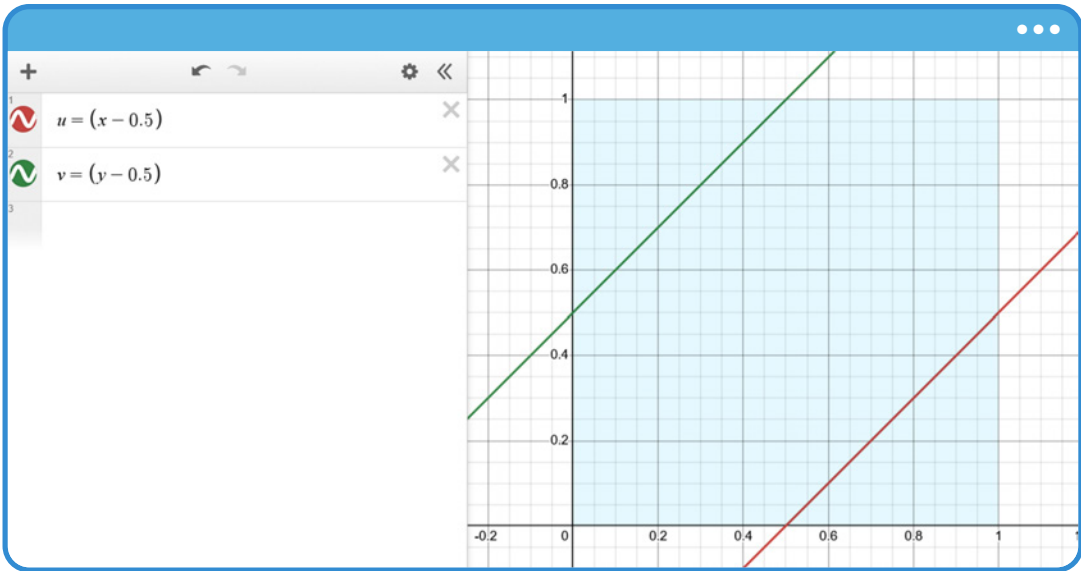
Como primer paso, exploraremos la forma utilizando Desmos, una herramienta gráfica que nos permitirá experimentar con ecuaciones y visualizar sus efectos en tiempo real. Recomendamos tener abierta la siguiente dirección para comenzar nuestro análisis:

➤ <https://www.desmos.com/calculator>

Nota

Este libro incluye un proyecto descargable que contiene una carpeta llamada **case_study**, donde podrás encontrar más ejemplos de formas procedurales creadas utilizando Desmos. Estos casos adicionales nos permitirán profundizar en la construcción de formas mediante ecuaciones y explorar distintas estrategias de modelado visual basado en coordenadas.

Asumiendo que ya estas dentro de la interfaz de la calculadora de Desmos, comenzaremos a desarrollar nuestra forma procedural paso a paso. El primero consiste en declarar e inicializar las coordenadas UV sobre el plano cartesiano, lo cual nos permitirá trabajar con funciones aplicadas directamente sobre estas variables, como se muestra a continuación:



(3.1.b <https://www.desmos.com/calculator/i5ad2cvmqh>)

Este primer paso resulta especialmente valioso, ya que nos permite entender cómo se comportan las coordenadas UV en un espacio normalizado. A partir de la visualización en Desmos, podemos extraer las siguientes observaciones:

- 1 La coordenada u corresponde directamente al eje x .
- 2 La coordenada v corresponde al eje y .
- 3 Ambas coordenadas están acotadas entre 0.0 y 1.0, lo que representa el espacio UV típico en un fragment shader.
- 4 Para centrar cualquier forma en el plano, es necesario restar 0.5 a cada coordenada, desplazando así el origen al centro del dominio.

Con esta base, estamos listos para comenzar el diseño de la forma general de nuestra calavera. De acuerdo con la Figura 3.1.a, la cabeza está compuesta por dos regiones definidas mediante ecuaciones implícitas:

La primera describe la parte superior del cráneo, representada mediante una ecuación circular.

$$u^2 + v^2 < r^2$$

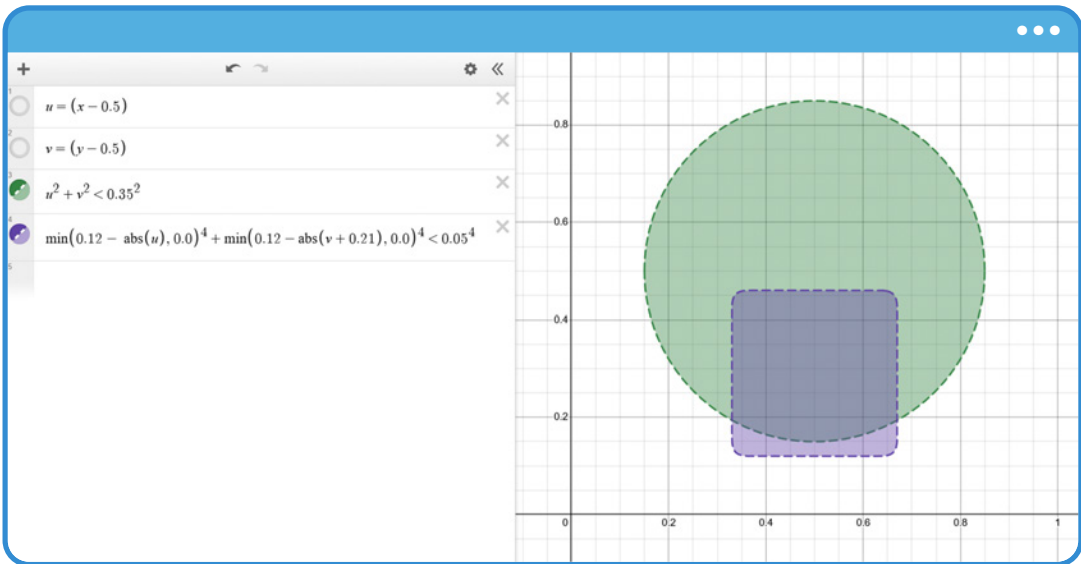
(3.1.c)

La segunda corresponde al mentón, modelado mediante una deformación rectangular de bordes suaves.

$$\min(n_1 - |u|, k_1)^4 + \min(n_2 - |v|, k_2)^4 < r^4$$

(3.1.d)

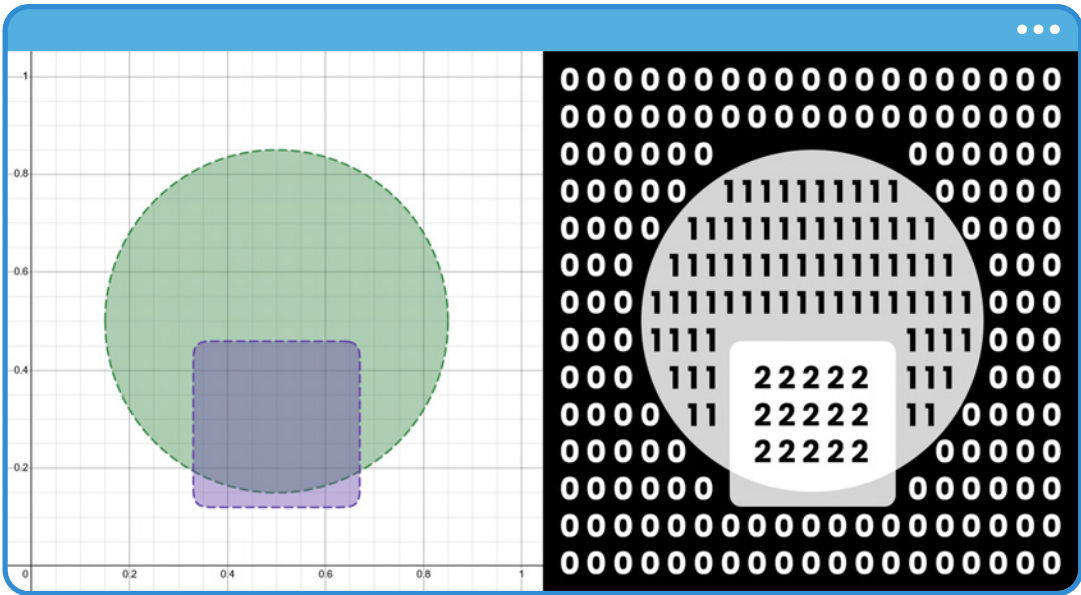
Para comenzar, utilizaremos valores constantes que nos permitirán fijar la posición y el tamaño de ambos elementos en el plano, lo que facilitará su ajuste posterior al momento de combinarlos en una sola forma cohesiva.



(3.1.e <https://www.desmos.com/calculator/9kgoilxgz2>)

Un aspecto relevante que podemos observar en la Figura 3.1.e es la intersección entre ambas figuras. Este fenómeno es especialmente interesante desde el punto de vista de los gráficos por computadora. Cuando más adelante traslademos estas operaciones matemáticas al lenguaje GDSL dentro de nuestro shader, las regiones visibles de cada figura serán evaluadas como 1.0, mientras que las áreas fuera de estas regiones tomarán el valor 0.0.

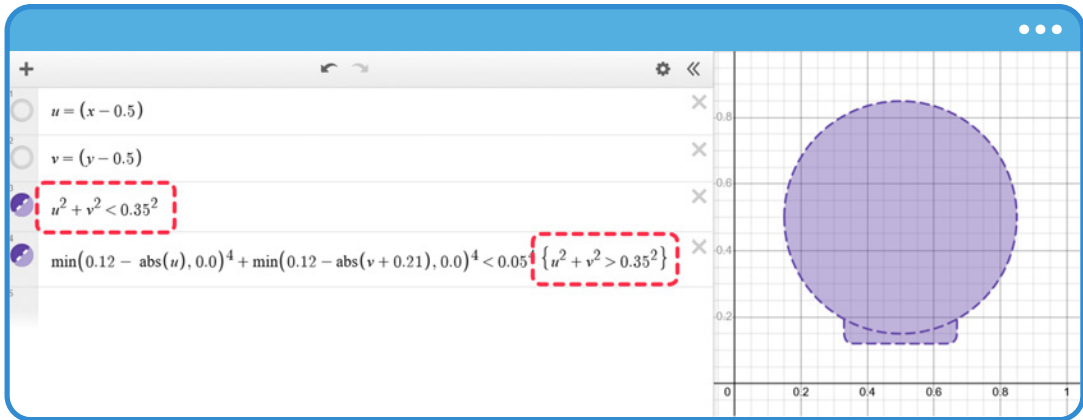
Esto significa que, si combinamos ambas figuras mediante una suma directa, el área de intersección pasará a tener un valor de 2.0. Este exceso puede derivar en artefactos visuales no deseados, como saturación de color o luminancia, especialmente si estamos utilizando estos valores para controlar color, opacidad o intensidad lumínica.



(3.1.f Se han modificado los contrastes para representar la diferencia de valores)

Por lo tanto, al momento de combinar formas, debemos considerar técnicas como la saturación (`clamp()`), el promedio, o incluso la operación `max()`, según el comportamiento visual que queramos lograr. Sin embargo, abordaremos estos métodos con más detalle más adelante.

Por ahora, evitaremos que ambas figuras se superpongan dentro de Desmos, restringiendo el área del rectángulo para que no interfiera con el círculo. Para lograr esto, podemos limitar el dominio de la función utilizando corchetes, de modo que la ecuación solo se aplique dentro de un rango específico. En este caso, acotaremos la función del rectángulo para que actúe únicamente fuera del área definida por el círculo, como se muestra a continuación:



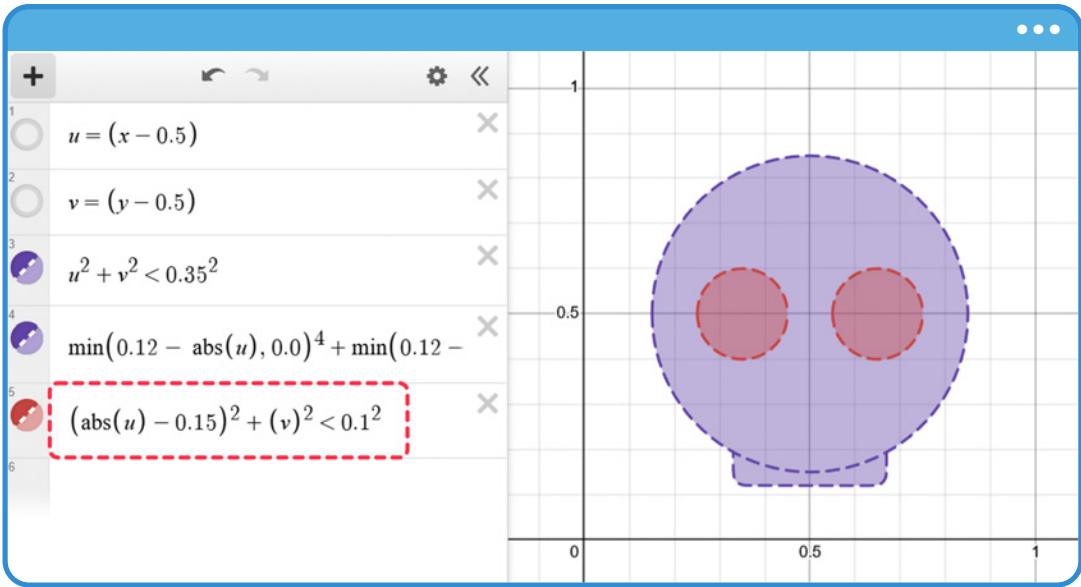
(3.1.g <https://www.desmos.com/calculator/2hytxw2ee6>)

Nota

Las formas procedurales suelen ser exigentes para la GPU, por lo que se recomienda utilizarlas con precaución, especialmente en entornos de producción para videojuegos. No obstante, comprender estas técnicas permite profundizar en el manejo de las coordenadas UV y nos brinda herramientas para optimizar el rendimiento visual combinando texturas y funciones matemáticas.

Cabe mencionar que también podemos establecer este tipo de límites condicionales directamente en GDSP. Sin embargo, su uso dependerá del resultado que deseemos obtener, ya que no siempre es necesario realizar recortes explícitos si el efecto visual no lo requiere.

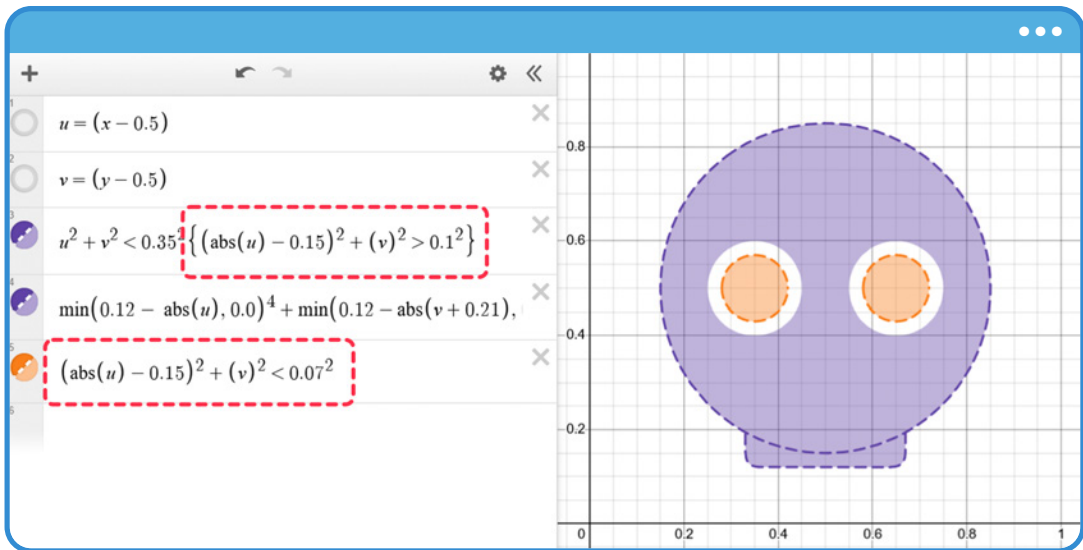
Continuemos ahora con el siguiente paso en el diseño: los ojos de la calavera. En este caso, también utilizaremos una ecuación circular, pero no necesitaremos replicarla dos veces. Gracias a la simetría horizontal, basta con aplicar la función $\text{abs}(u)$ para reflejar la forma automáticamente y obtener ambos ojos con una sola expresión matemática.



(3.1.h <https://www.desmos.com/calculator/6dbg8pf7vw>)

Como podemos observar, la ecuación que acabamos de incorporar corresponde a la misma forma definida anteriormente en la Figura 3.1.c. La diferencia principal es que, en esta ocasión, hemos aplicado el valor absoluto a la coordenada u . Este pequeño ajuste nos permite aprovechar la simetría horizontal del objeto, representando ambos ojos con una sola operación y optimizando así el proceso.

Además, si lo deseamos, podemos utilizar esta nueva región como condición de recorte para la forma general de la cabeza. Esto nos permitiría generar una perforación precisa sobre el área del cráneo, respetando la jerarquía de construcción y manteniendo un control sobre la superposición de formas.



(3.1.i <https://www.desmos.com/calculator/qqv6aqwmjp>)

Con este ajuste, reducimos el tamaño de los ojos para lograr el resultado visual que buscamos. Esta técnica nos permite afinar detalles visuales con precisión y sin necesidad de definir múltiples formas independientes.

A continuación, podemos definir la nariz. Una opción sería utilizar una función lineal de la forma:

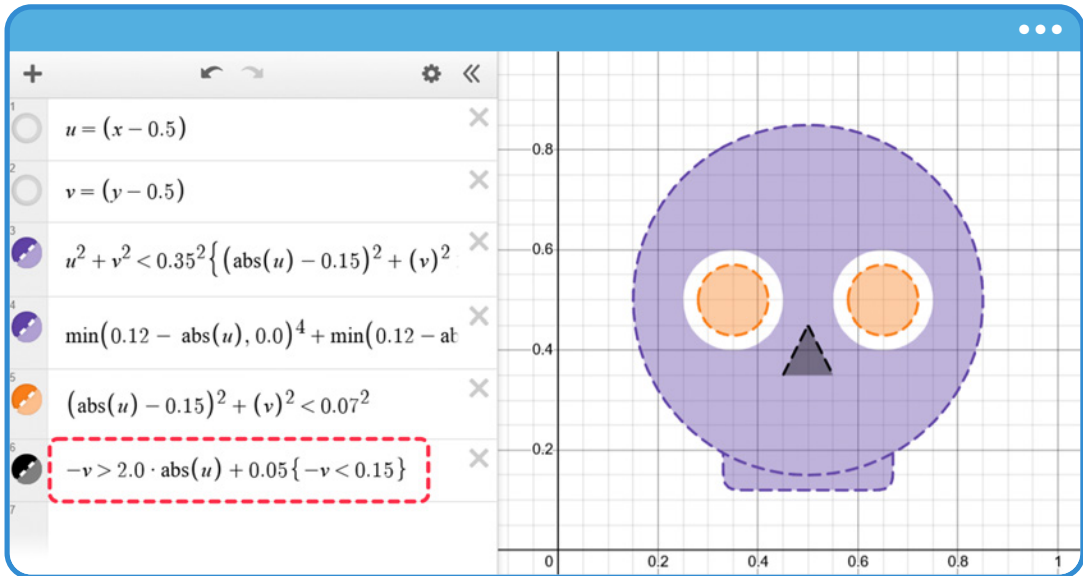
$$v > mu + b$$

(3.1.j)

Sin embargo, esta expresión solo describe una línea recta. Para modelar una nariz en forma de triángulo, necesitamos definir una región delimitada por dos rectas simétricas que converjan en un punto. Podemos lograr esto en Desmos mediante dos pasos:

- 1 Aplicar el valor absoluto sobre la coordenada u para obtener simetría horizontal.
- 2 Limitar la expresión a un rango vertical específico para cerrar la forma.

Por ejemplo,



(3.1.k <https://www.desmos.com/calculator/tkw4cieh3u>)

Esta condición genera una región triangular invertida, ubicada en la parte central inferior de la cabeza, ideal para representar la nariz. Al acotar el rango de v , evitamos que la figura se extienda más allá de lo necesario, manteniendo una forma compacta y controlada.

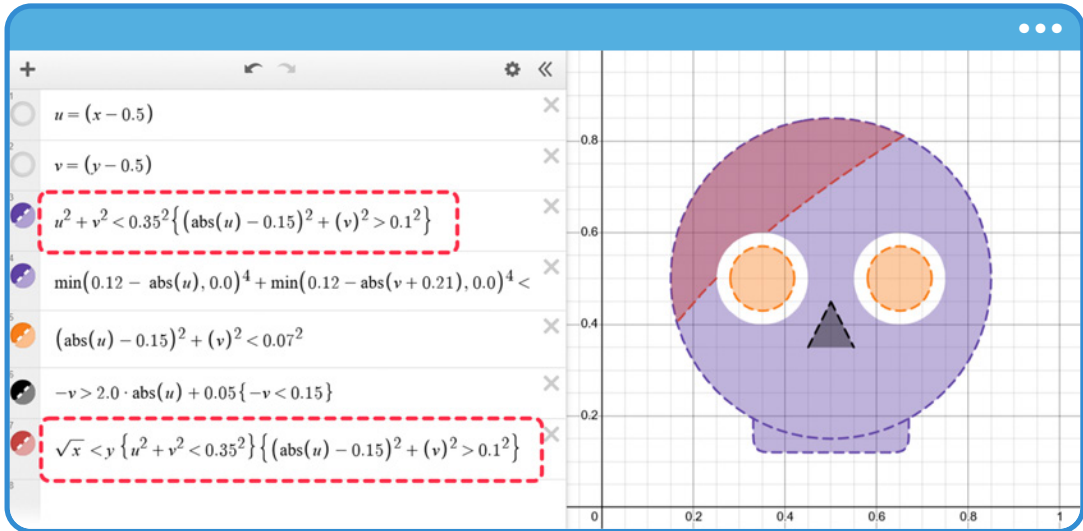
Si observamos la Figura 3.1.k, notaremos que en nuestra función los parámetros toman los valores $m = 2.0$ y $b = 0.05$. Además, hemos invertido la coordenada v para orientar la pendiente hacia abajo, y aplicamos el valor absoluto sobre u para generar la simetría que caracteriza la forma triangular de la nariz en una calavera. Al modificar estos valores, podemos controlar tanto el ancho como la posición vertical de la figura.

Una vez definida la nariz, podemos avanzar con la bandana pirata. Existen varias formas de abordarla, pero una opción simple y efectiva para este personaje consiste en utilizar una condición basada en una raíz cuadrada. En concreto, podríamos visualizar todos los puntos donde la raíz cuadrada de x sea menor que y , lo que puede expresarse como:

$$\sqrt{x} < y$$

(3.1.l)

Esta expresión genera una curva ascendente que atraviesa el rostro, simulando parte del contorno de la tela. Posteriormente, podemos recortar esta región utilizando las áreas previamente definidas para la cabeza y los ojos, asegurando que la bandana no interfiera con otras partes del diseño.



(3.1.m <https://www.desmos.com/calculator/nvkem2ifdl>)

Como se aprecia en la Figura 3.1.m, luego de aplicar la condición basada en la raíz cuadrada, hemos utilizado dos límites: el área de la cabeza y la región de los ojos. Esto nos permite asegurar que la bandana pirata no se superponga a elementos importantes del rostro ni se extienda más allá de los bordes del cráneo.

Es importante destacar que, en este caso, estamos utilizando directamente las coordenadas xy en lugar de uv . Esto se debe a que las primeras parten desde 0.0 en Desmos, lo que resulta ideal para representar expresiones como $\sqrt{x} < y$ de forma más directa, sin necesidad de recentrado.

Ya solo nos quedan algunos elementos para completar nuestro personaje: las cejas y los huesos cruzados detrás de la cabeza. Para ello, introduciremos un nuevo recurso fundamental en gráficos por computadora: la rotación bidimensional mediante matrices.

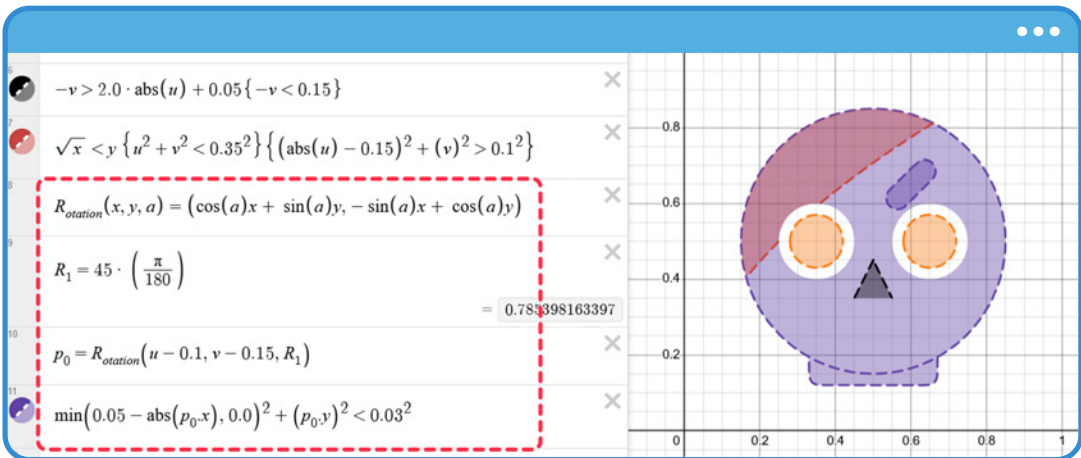
Crearemos una función en Desmos a la que llamaremos Rotation. Esta función recibirá tres argumentos: las coordenadas x e y del punto a transformar, y un ángulo a . La rotación se define como:

$$R_{otation}(x, y, a) = (\cos(a)x + \sin(a)y, -\sin(a)x + \cos(a)y)$$

(3.1.n)

Esta transformación nos permitirá rotar cualquier punto en el plano, lo que resulta especialmente útil para darle expresión a las cejas o ajustar la orientación de los huesos.

Para las cejas, utilizaremos cilindros suaves, definidos mediante la misma ecuación que usamos en la Figura 3.1.d, pero con exponentes menores para crear bordes menos rígidos. Como aplicaremos rotación, será necesario inicializar un nuevo punto que almacene las coordenadas uv rotadas y definir un ángulo de rotación específico para cada ceja. Esta separación de variables nos permitirá reutilizar la misma forma base en distintas posiciones y orientaciones.

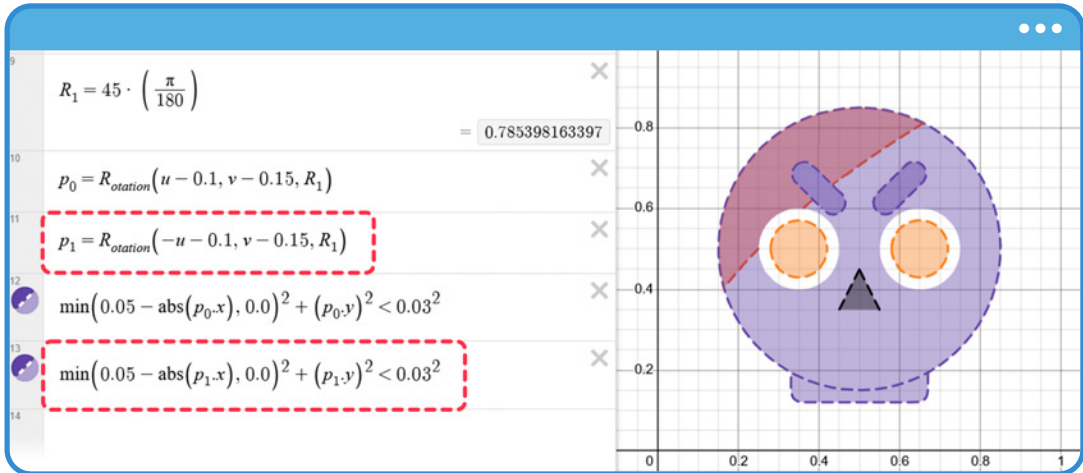


(3.1.ñ <https://www.desmos.com/calculator/juc94owkuo>)

En la Figura 3.1.ñ podemos ver que se ha definido un ángulo de rotación constante igual a 45° , representado por R_1 . Luego, declaramos e inicializamos un nuevo punto p_0 , que resulta de aplicar la rotación a las coordenadas uv previamente desplazadas. Este nuevo punto reemplaza a las coordenadas originales y nos permite determinar con precisión la posición y orientación de la ceja derecha del personaje.

Para crear la ceja izquierda, podemos optimizar el proceso mediante una reflexión horizontal. En lugar de rotar un segundo conjunto de coordenadas, basta con declarar un nuevo punto p_1 que sea el reflejo de p_0 respecto al eje vertical. Esto nos permite reutilizar la misma

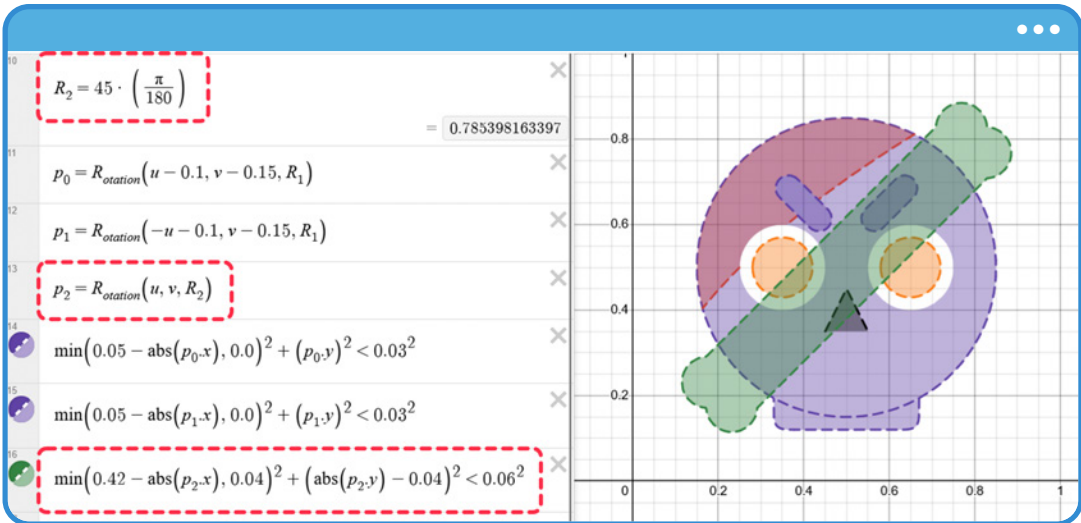
forma base y la misma lógica de sombreado, logrando simetría con mínima sobrecarga computacional.



(3.1.o <https://www.desmos.com/calculator/oillqby4fv>)

Con esta expresión definimos la ceja izquierda, utilizando el reflejo de las coordenadas en u para lograr simetría sin necesidad de duplicar cálculos complejos. El punto p_1 es el resultado de aplicar la misma rotación que usamos para p_0 , pero desplazando las coordenadas en dirección opuesta. De este modo, obtenemos ambas cejas a partir de una misma forma base rotada, mejorando la eficiencia y manteniendo coherencia visual.

Para completar la figura de referencia, solo nos resta agregar los huesos cruzados que se ubican detrás de la cabeza. Seguiremos el mismo procedimiento utilizado anteriormente: primero definiremos un nuevo punto rotado y luego lo aplicaremos sobre una ecuación que genere la forma deseada. En este caso, utilizamos una variación de la forma cilíndrica, ajustando los valores mínimos para esculpir los extremos y así obtener una silueta más orgánica y cercana a la de un hueso. La ecuación resultante es la siguiente:

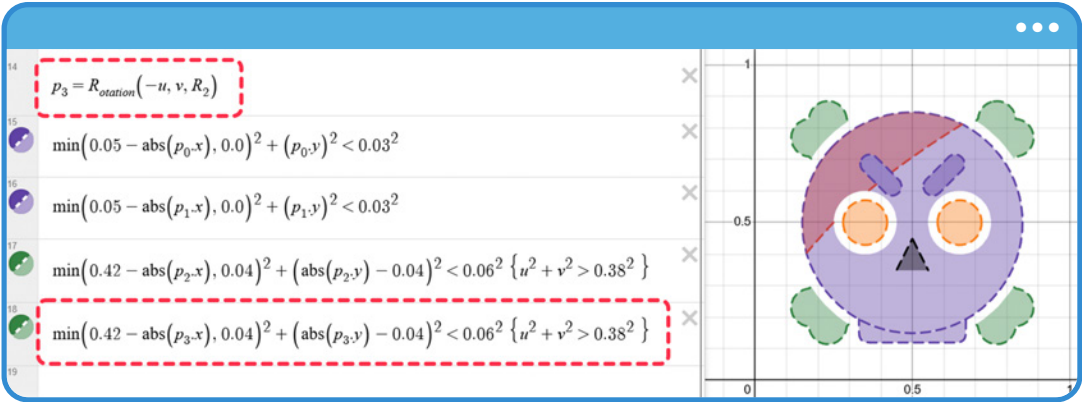


(3.1.p <https://www.desmos.com/calculator/g9u8f63rdi>)

Esta expresión genera una figura alargada con bordes redondeados y extremos más anchos, lo que da lugar a la forma característica de un hueso. A través del uso del valor 0.04 como mínimo en la función $\min()$, logramos suavizar el recorte lateral, evitando el borde recto del cilindro tradicional y dándole un aspecto más estilizado.

Es importante destacar que deberemos limitar la región del hueso utilizando la definición de la cabeza. Esto evitará que la figura se superponga visualmente al cráneo, manteniendo una composición limpia y jerárquica.

Siguiendo el mismo enfoque anterior, podemos duplicar el hueso aplicando una reflexión horizontal. Para ello, declaramos un nuevo punto p_3 que representa el reflejo en x del punto p_2 , y lo sometemos a la misma transformación de rotación:



(3.1.q <https://www.desmos.com/calculator/9m7xkdz2zf>)

Además, establecemos una condición adicional sobre el radio ($u^2 + v^2 > 0.38^2$) para asegurarnos de que estos huesos no invadan el espacio de la cabeza. Esta restricción mantiene separados los elementos y evita que el resultado final se vea visualmente saturado o confuso.

3.2 Animación y modificación procedural.

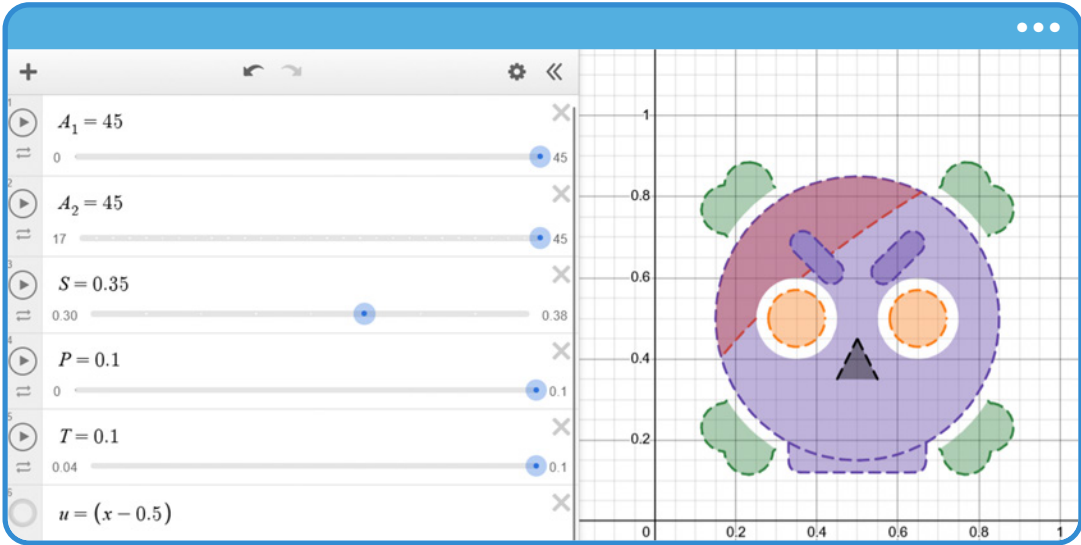
Hasta este punto, nuestra calavera pirata ha sido construida utilizando únicamente coordenadas fijas y valores constantes. Esta aproximación nos ha permitido interpretar y construir la figura de manera controlada. Sin embargo, si queremos introducir movimiento o variaciones visuales, será necesario incorporar valores variables que nos permitan manipular elementos clave del diseño.

Para lograrlo, definiremos una serie de propiedades en la interfaz de Desmos. Estas variables actuarán como parámetros de control y nos permitirán modificar la expresión del personaje en tiempo real. A continuación, describimos cada una de ellas:

- A_1 : Controlará el ángulo de rotación de las cejas. Su rango estará definido entre 0 y 45 grados.
- A_2 : Permitirá modificar el ángulo de los huesos traseros, con un rango comprendido entre 17 y 45 grados.
- S : Se utilizará para ajustar el tamaño de la cara de la calavera. Su valor podrá oscilar entre 0.30 y 0.38.
- P : Modificará la posición de la cara (ojos, cejas y nariz), desplazándola verticalmente. Tendrá un rango entre 0.0 y 0.1.

- T : Controlará el tamaño de los ojos del personaje, variando entre 0.04 y 0.1.

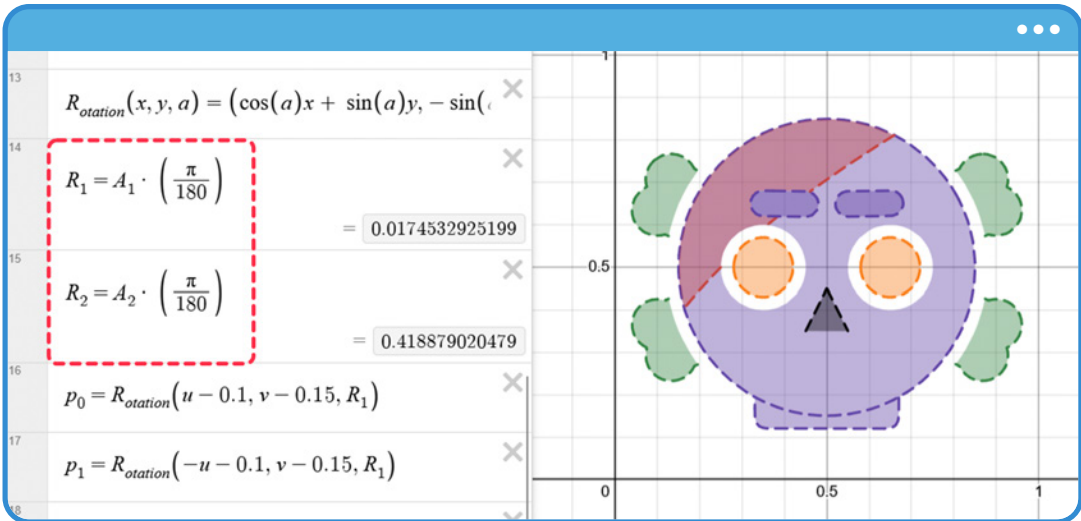
Con estas propiedades en mente, procederemos a implementarlas en la parte superior del panel izquierdo dentro de la interfaz de Desmos, como se muestra a continuación:



(3.2.a <https://www.desmos.com/calculator/yjhn9vpe3j>)

Implementar estas propiedades en nuestras ecuaciones es un proceso bastante sencillo. Solo necesitamos tener presente el propósito de cada variable y sustituir los valores constantes por ellas en las expresiones correspondientes.

Por ejemplo, si reemplazamos el valor fijo 45 por la variable A_1 en la constante R_1 , podremos modificar dinámicamente el ángulo de rotación de las cejas. Lo mismo ocurre con la constante R_2 : al sustituir su valor 45 por A_2 , podremos controlar el ángulo de rotación de los huesos traseros en tiempo real.



(3.2.b <https://www.desmos.com/calculator/v9krsdd1h2>)

Continuemos con la variable S , que utilizaremos para aumentar o disminuir el tamaño de la cabeza. Si aplicáramos esta variable únicamente al radio de la cabeza, los demás elementos como la bandana, la mandíbula y los huesos traseros no se ajustarían en proporción, generando desalineación visual. Para evitar esto, necesitaremos propagar la variable S a otras partes del diseño. A continuación se detalla cómo hacerlo:

- **Cabeza:** Reemplazamos el valor fijo 0.35 por S , de modo que el radio de la cabeza sea dinámico.
- **Bandana:** Aplicamos S al primer límite utilizado en la construcción de la bandana, asegurando que su forma se mantenga dentro de la nueva escala.
- **Mandíbula:** Sustituimos el valor 0.35 en el límite correspondiente por S . Además, deberemos ajustar sus dimensiones para que crezca de forma coherente junto con la cabeza.
- **Huesos traseros:** Reemplazamos el valor 0.38 por S en ambos límites. Luego, sumamos 0.03 para mantener su posición relativa respecto al borde externo del cráneo.

Si todos los pasos se han aplicado correctamente, las expresiones resultantes deberían verse de la siguiente manera:

$$\begin{aligned}
 &u^2 + v^2 < S^2 \{ (\text{abs}(u) - 0.15)^2 + (v)^2 > 0.1^2 \} \\
 &\min(0.12 - \text{abs}(u), 0.0)^4 + \min(0.12 - \text{abs}(v + 0.21), 0.0)^4 < 0.05^4 \{ u^2 + v^2 > S^2 \} \\
 &\sqrt{x} < y \{ u^2 + v^2 < S^2 \} \{ (\text{abs}(u) - 0.15)^2 + (v)^2 > 0.1^2 \} \{ \min(0.05 - \text{abs}(p_1.x), 0.0)^2 + (p_1.y)^2 > 0.03^2 \} \\
 &\min(0.42 - \text{abs}(p_2.x), 0.04)^2 + (\text{abs}(p_2.y) - 0.04)^2 < 0.06^2 \{ u^2 + v^2 > S^2 + 0.03 \} \\
 &\min(0.42 - \text{abs}(p_3.x), 0.04)^2 + (\text{abs}(p_3.y) - 0.04)^2 < 0.06^2 \{ u^2 + v^2 > S^2 + 0.03 \}
 \end{aligned}$$

(3.2.c <https://www.desmos.com/calculator/qcdrrd2jav>)

Para asegurarnos de que la mandíbula se expanda junto con el radio de la cabeza, también debemos ajustar sus dimensiones en función de la variable S . En particular, modificaremos tanto la coordenada u como la coordenada v dentro de la ecuación original, de modo que su tamaño y posición se adapten dinámicamente al crecimiento de la cabeza.

La expresión ajustada es la siguiente:

$$\min(S - 0.23 - \text{abs}(u), 0.0)^4 + \min(0.12 - \text{abs}(v + (S - 0.15)), 0.0)^4 < 0.05^4 \{ u^2 + v^2 > S^2 \}$$

(3.2.d <https://www.desmos.com/calculator/zay3r7ynov>)

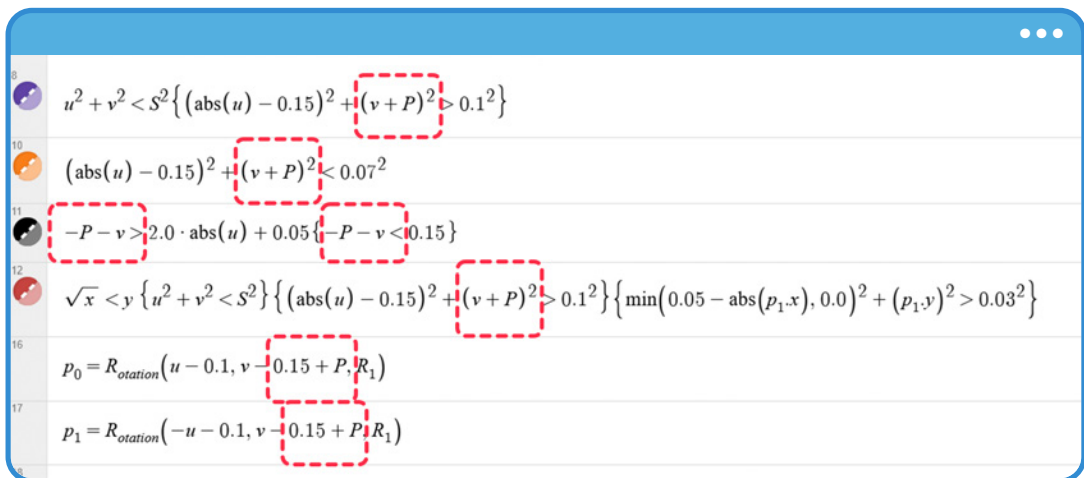
Aquí, la cantidad $S - 0.23$ define el nuevo ancho máximo permitido para la mandíbula, mientras que $S - 0.15$ ajusta su altura relativa dentro del rostro. Al aplicar estos valores, logramos que la mandíbula crezca proporcionalmente junto con la cabeza, manteniendo la forma original y evitando desalineaciones visuales.

La variable P nos permitirá modificar la posición vertical de los elementos del rostro, lo que resulta útil para generar variaciones expresivas o animaciones simples. Para que el desplazamiento sea coherente, deberemos aplicarla no solo sobre los ojos, la nariz y las

cejas, sino también sobre los límites que definen el contorno del rostro y la bandana. A continuación, se detallan los cambios necesarios:

- **Cabeza:** Sumamos P a la coordenada v para ajustar los límites visuales que la rodean.
- **Ojos:** Aplicamos P directamente sobre la variable v dentro de la ecuación.
- **Nariz:** Restamos P a v (es decir, usamos $-P - v$) tanto en la condición principal como en los límites para invertir la dirección del desplazamiento, ya que la nariz apunta hacia abajo.
- **Cejas:** Sumamos P a la coordenada y de los puntos rotados p_0 y p_1 para mantener su alineación con el resto del rostro.
- **Bandana:** Agregamos P a la variable v dentro del segundo límite utilizado para recortar correctamente según la rotación de las cejas.

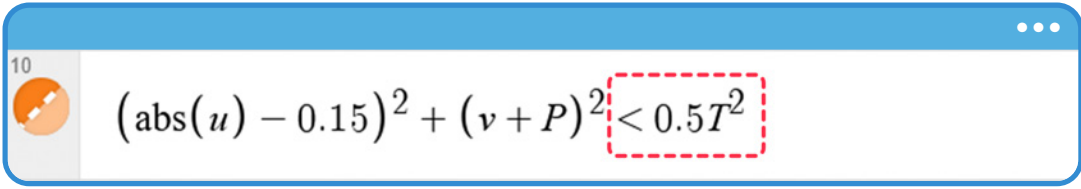
Si aplicamos todos estos ajustes correctamente, nuestras expresiones deberían verse de la siguiente manera:



(3.2.e <https://www.desmos.com/calculator/hbet5kdjgp>)

Por último, utilizaremos la variable T para aumentar o disminuir el tamaño de los ojos del personaje. Su implementación es sencilla: basta con integrarla al radio de la ecuación que define los ojos, multiplicando su valor por una constante que conserve la proporción y estética del diseño original.

En este caso, hemos elegido un valor base de 0.5, como se muestra a continuación:



$$(\text{abs}(u) - 0.15)^2 + (v + P)^2 < 0.5T^2$$

(3.2.f <https://www.desmos.com/calculator/bvccatrkgb>)

Este enfoque nos permite controlar de forma dinámica la apertura de los ojos, lo que puede ser útil tanto para ajustar la expresión del personaje como para experimentar con estilos más caricaturescos.

En la próxima sección implementaremos todas estas expresiones directamente en GDSDL. Para ello, será necesario regresar a Godot, crear un shader, asignarlo a un material, y preparar la estructura del proyecto una vez más.

3.3 Implementación del personaje procedural en GDSDL.

En esta sección implementaremos las ecuaciones matemáticas desarrolladas en las secciones anteriores para dibujar nuestro pirata utilizando GDSDL. Antes de comenzar con la codificación, organizaremos la estructura de carpetas del proyecto para mantener un flujo de trabajo ordenado y fácilmente escalable. Para ello:

- 1 Dentro de la carpeta **assets**, crearemos una nueva subcarpeta llamada **chapter_03**, donde almacenaremos todos los recursos correspondientes a este capítulo.
- 2 Dentro de **chapter_03**, añadiremos otra carpeta con el nombre **procedural_shape**.
- 3 Finalmente, dentro del **procedural_shape**, crearemos dos subcarpetas:
 - a **materials**.
 - b **shaders**.

Para este ejercicio utilizaremos únicamente un **QuadMesh**, que configuraremos en la escena utilizando un **Node3D** junto con un **MeshInstance3D**. Este objeto será suficiente para visualizar la forma de nuestro pirata con precisión, por lo que resulta ideal para este caso de estudio.

También necesitaremos un **shader** y su correspondiente **material** para proyectar la figura sobre el Quad. Por lo tanto, en la carpeta **materials**, haremos clic derecho y seleccionaremos:

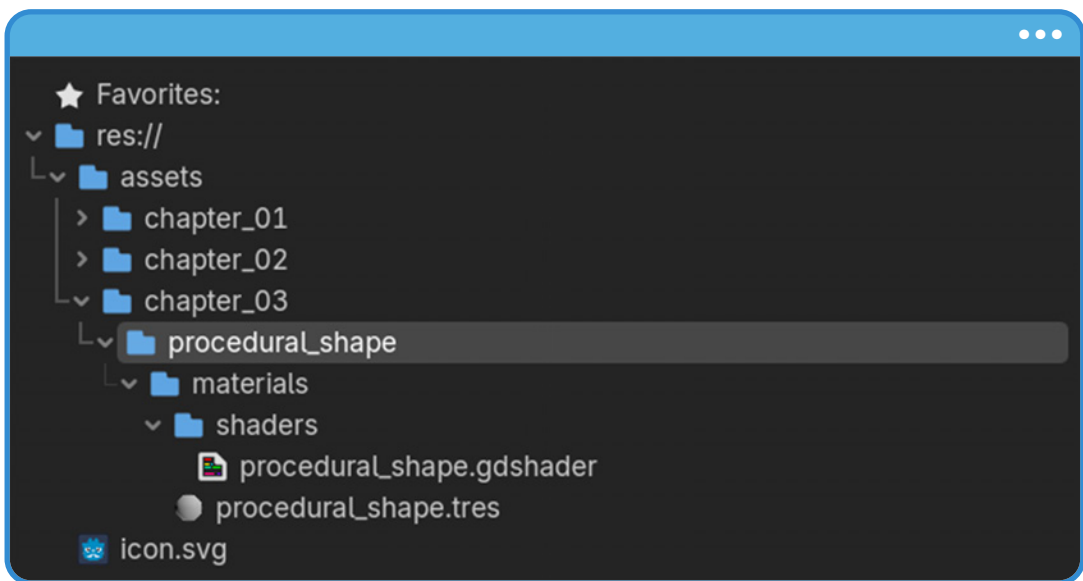
➤ Create New > Resource > ShaderMaterial.

Por razones prácticas, lo nombraremos **procedural_shape**. Luego, crearemos el **shader** asociado. En la carpeta **shaders**, haremos clic derecho y seleccionamos:

➤ Create New > Resource > Shader.

Le asignaremos el mismo nombre (**procedural_shape**) para mantener una relación clara y coherente entre ambos recursos.

Si hemos seguido todos los pasos correctamente, la estructura del proyecto debería verse de la siguiente manera:



(3.3.a Se ha incluido **procedural_shape** en el proyecto)

Nota

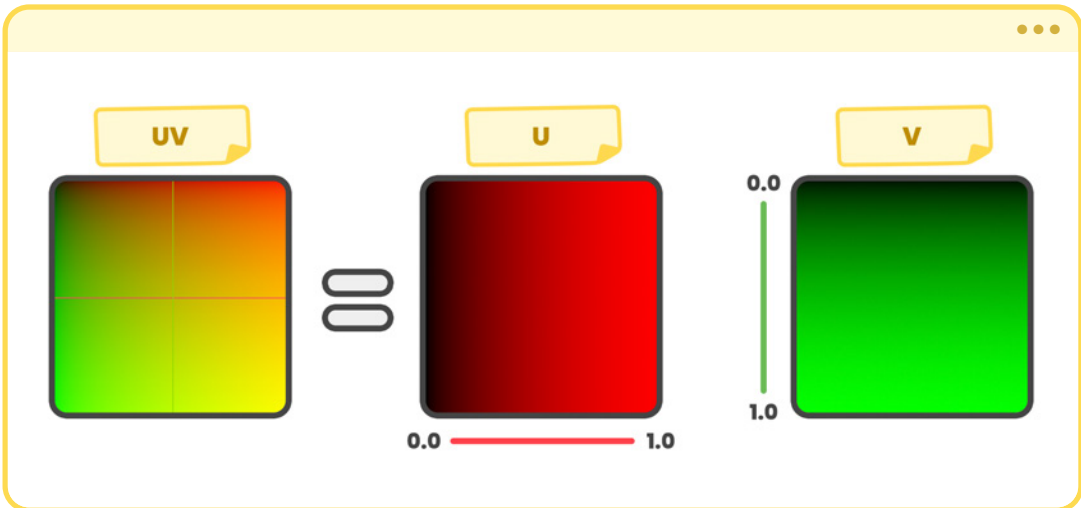
Antes de comenzar, debemos asegurarnos de asignar el shader **procedural_shape** a su correspondiente material y, posteriormente, aplicar este material al QuadMesh en nuestra escena. Esto nos permitirá visualizar en tiempo real los cambios que realicemos durante el desarrollo del ejercicio.

Como primer paso, realizaremos una breve prueba práctica para analizar el comportamiento de las coordenadas UV en Godot, ya que su orientación puede variar con respecto a otros lenguajes o entornos.

Si vamos a la etapa de fragmento en nuestro shader y escribimos el siguiente código:

```
void fragment()
{
    vec2 uv = vec2(UV.x, UV.y);
    ALBEDO = vec3(uv, 0.0);
}
```

Podemos observar gráficamente sobre el QuadMesh que la coordenada V (equivalente al eje *y* en el plano cartesiano) está invertida, con el origen (0.0) ubicado en la esquina superior izquierda, como se muestra a continuación:



(3.3.b La coordenada V apunta hacia abajo)

Este comportamiento, común en lenguajes como HLSL, representa un problema al trabajar con formas procedurales, ya que, si utilizamos las mismas ecuaciones junto con sus respectivas variables y constantes, el personaje se visualizará invertido verticalmente por razones evidentes.

Por lo tanto, el primer paso que debemos realizar es ajustar la coordenada V para que siga la misma orientación que el eje *y* del plano cartesiano. Esto nos permitirá mantener la coherencia visual entre lo que diseñamos matemáticamente y lo que renderizamos en pantalla.



(3.3.c Comparación de coordenadas UV entre lenguajes)

Nota

Si has trabajado previamente con Unity, observarás que las coordenadas UV se encuentra en la esquina inferior izquierda, y no en la superior izquierda como se muestra en la Figura 3.3.c (basada en HLSL). Esto se debe a que Unity es compatible con múltiples APIs de renderizado, como Direct3D, OpenGL, Metal y Vulkan. Para mantener la consistencia visual entre plataformas, Unity adopta un sistema de coordenadas unificado y voltea automáticamente la coordenada V, haciendo que su comportamiento se asemeje al de GLSL en lugar de HLSL.

Para replicar este comportamiento en Godot, simplemente debemos invertir la coordenada vertical realizando la siguiente operación:

```
void fragment()
{
    vec2 uv = vec2(UV.x, 1.0 - UV.y);
    ALBEDO = vec3(uv, 0.0);
}
```

Con esta corrección aplicada, ahora podemos comenzar a dibujar las distintas partes de la calavera pirata directamente en nuestro shader. El primer paso será declarar las propiedades que nos permitirán animar el personaje. Si bien podríamos utilizar los mismos nombres definidos en Desmos, para facilitar la lectura y claridad del código, optaremos por una nomenclatura más estructurada, como se muestra a continuación:

```

1  shader_type spatial;
2  render_mode unshaded;
3
4  // source : https://www.desmos.com/calculator/bvccatrkgb
5
6  uniform float _EyebrowRotation : hint_range(0.0, 45.0, 0.1); // A1
7  uniform float _BackBoneRotation : hint_range(17.0, 45.0, 0.1); // A2
8  uniform float _HeadSize : hint_range(0.30, 0.38, 0.01); // S
9  uniform float _FacePosition : hint_range(0.0, 0.1, 0.01); // P
10 uniform float _EyeSize : hint_range(0.04, 0.1, 0.01); // T
11
12 uniform vec3 _HeadColor : source_color;
13 uniform vec3 _EyeColor : source_color;
14 uniform vec3 _BackBoneColor : source_color;
15 uniform vec3 _BandanaColor : source_color;
16 uniform vec3 _EyebrowColor : source_color;

```

¿Qué está ocurriendo en nuestro código? Vamos a analizarlo:

- Línea 2: Hemos configurado el shader como **unshaded**, ya que no necesitamos que el QuadMesh reciba luz de la escena. Esto simplifica la visualización de formas y colores planos.
- Líneas 6 a 10: Se han declarado las mismas propiedades que utilizamos en Desmos (A_1 , A_2 , S , P , T). Sin embargo, en este caso, las hemos renombrado utilizando un estilo más descriptivo y compatible con convenciones comunes en shaders, lo que facilitará la lectura y mantenimiento del código.
- Líneas 12 a 16: Hemos añadido una serie de propiedades de tipo **vec3**, que nos permitirán definir colores personalizados para cada parte del personaje, como la cabeza, los ojos, la bandana y las cejas. Estas variables actuarán como parámetros de entrada que podrán ser modificados desde el Inspector.

Ahora que hemos declarado todas nuestras propiedades, estamos listos para comenzar a dibujar la cabeza del pirata. Empezaremos utilizando la primera de nuestras ecuaciones: un círculo centrado.

Sin embargo, antes de ello, debemos asegurarnos de centrar las coordenadas UV. Como hemos visto anteriormente, las coordenadas UV en Godot van de 0.0 a 1.0, con el origen en la esquina inferior izquierda (tras la inversión manual de V). Para que nuestra figura se dibuje correctamente en el centro del QuadMesh, será necesario restar 0.5 a ambas componentes:

```

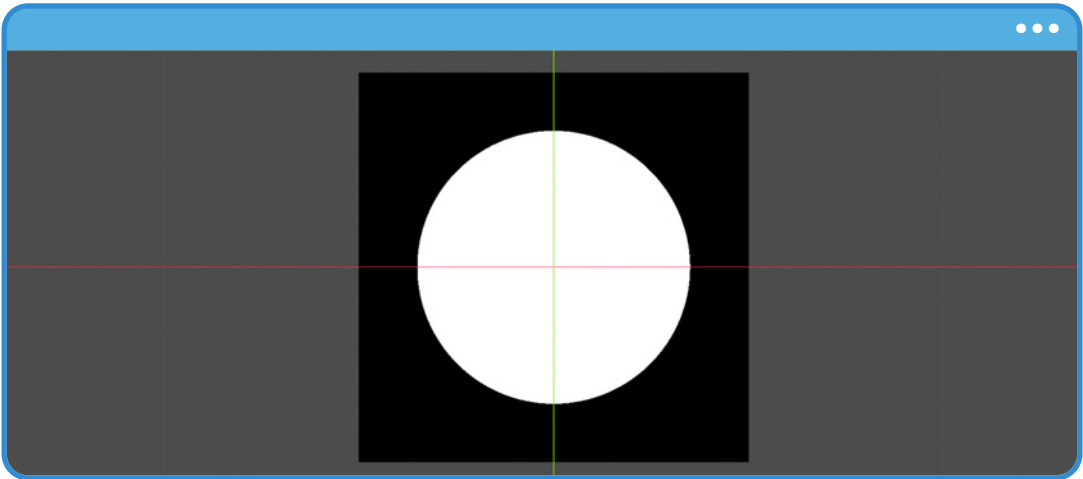
22 void fragment()
23 {
24     float u = UV.x - 0.5;
25     float v = (1.0 - UV.y) - 0.5;
26     vec2 uv = vec2(u, v);
27
28     float head = float(uv.x * uv.x + uv.y * uv.y < _HeadSize * _HeadSize);
29
30     ALBEDO = vec3(head);
31 }

```

Como podemos observar:

- Líneas 24 a 26: Hemos declarado un nuevo vector bidimensional llamado **uv**, el cual contiene las coordenadas centradas.
- Línea 28: Se declara un escalar llamado **head**, que representa el resultado de la ecuación de un círculo centrado. Esta operación verifica si el punto actual (**uv**) se encuentra dentro del radio definido por **_HeadSize**. Como la comparación retorna un valor booleano (true o false), lo convertimos explícitamente a float para poder utilizarlo como salida visual.
- Línea 30: Finalmente, asignamos el valor de **head** como color base (**ALBEDO**), lo que nos permite visualizar la forma sobre el Quad.

Este procedimiento genera la representación visual del **cráneo del pirata**, como se muestra a continuación:



(3.3.d Se ha dibujado la cabeza del pirata)

Cabe destacar que la inicialización de **head** puede optimizarse utilizando el producto punto **dot()** en lugar de realizar la suma manual de las componentes cuadradas, ya que ambas operaciones producen el mismo resultado:

```
float head = float(dot(uv, uv) < pow(_HeadSize, 2.0))
```

Con esta optimización aplicada, podemos continuar con la declaración de la mandíbula, la cual implementamos mediante una ecuación compuesta por dos funciones **min()** elevadas a la cuarta potencia. Esta forma crea una figura rectangular de bordes suavizados. La implementación se ve de la siguiente manera:


```

void fragment()
{
    ...

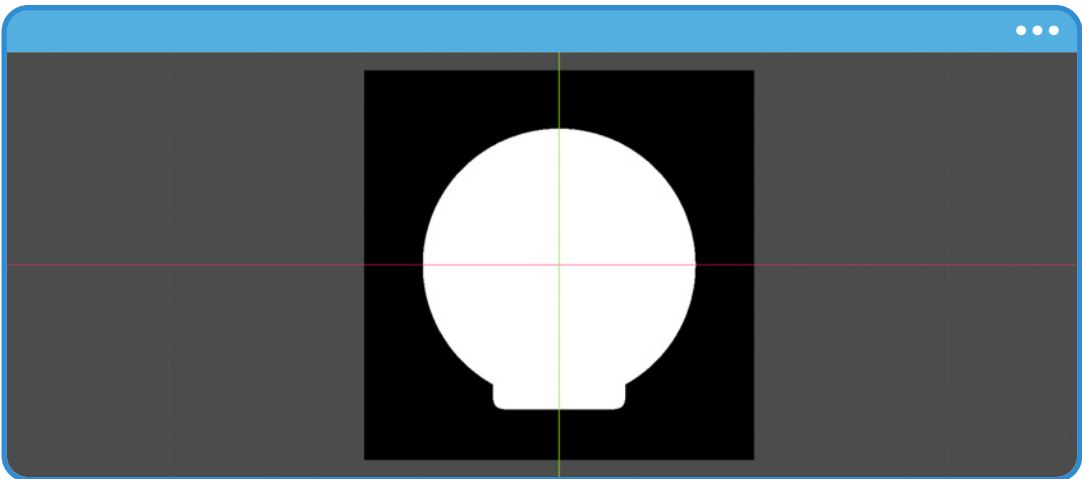
    float head = float(dot(uv, uv) < pow(_HeadSize, 2.0));
    float jaw = float(pow(min(_HeadSize - 0.23 - abs(u), 0.0), 4.0) +
        ↪ pow(min(0.12 - abs(v + (_HeadSize - 0.15)), 0.0), 4.0) < pow(0.05,
        ↪ 4.0));

    head = clamp(head + jaw, 0.0, 1.0);

    ALBEDO = vec3(head);
}

```

Finalmente, combinamos ambas formas sumando `jaw` a `head`, y utilizamos `clamp()` para limitar el valor resultante entre 0.0 y 1.0, evitando así posibles artefactos visuales por saturación. Esta operación nos permite visualizar tanto la parte superior del cráneo como la mandíbula inferior en una sola figura compuesta, manteniendo la estética definida en Desmos.



(3.3.e Se ha dibujado la mandíbula del pirata)

Podemos continuar ahora con la implementación de los ojos del personaje. Para lograr el efecto deseado, necesitaremos dos círculos: uno para el borde exterior del ojo y otro para su

interior. Para simplificar el código y mantenerlo modular, podemos encapsular esta lógica dentro de una función personalizada, como se muestra a continuación:

```
float eyes_shape (vec2 uv, float p, float r, float s)
{
    float u_coord = abs(uv.x) - 0.15;
    float v_coord = uv.y + p;
    return float(u_coord * u_coord + v_coord * v_coord > (r * r) * s);
}
```

Esta función representa una ecuación de círculo desplazado, basada en la forma descrita en la Figura 3.1.i, con la diferencia de que aquí estamos utilizando la propiedad P (definida anteriormente en la sección 3.2) para controlar dinámicamente la posición vertical de los ojos respecto a la cabeza. También se incluyen dos parámetros adicionales: r , que representa el radio base del ojo, y s , que actúa como factor de escala para controlar su tamaño.

```
void fragment()
{
    ...

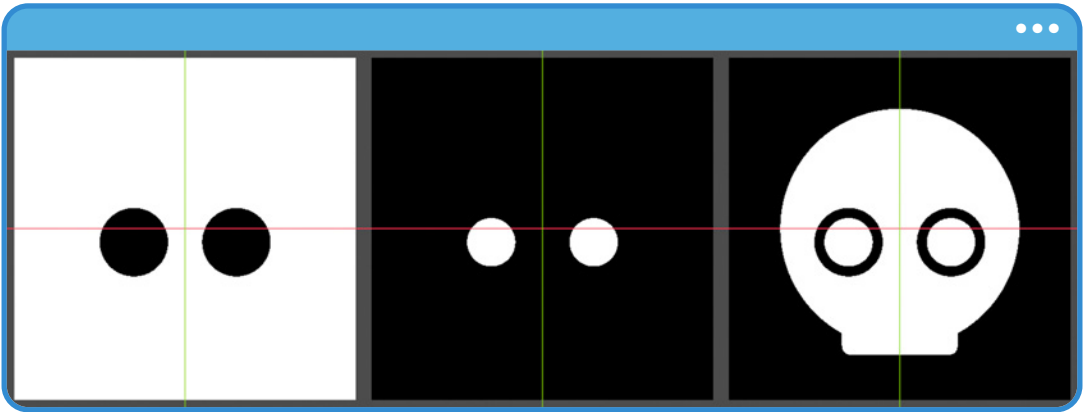
    head = clamp(head + jaw, 0.0, 1.0);
    head *= eyes_shape (uv, _FacePosition, 0.1, 1.0);
    head += 1.0 - eyes_shape (uv, _FacePosition, _EyeSize, 0.5);

    ALBEDO = vec3(head);
}
```

Primero, multiplicamos la salida de `eyes_shape()` por la cabeza, utilizando un radio base de 0.1 y un factor de escala de 1.0. Esto genera el agujero del ojo, asegurándonos de que quede recortado dentro del área de la cabeza.

Luego, utilizando nuevamente la función pero con la variable `_EyeSize` y un factor de escala de 0.5, agregamos el interior del ojo. Invertimos el resultado para que la forma resultante aparezca como una masa blanca circular dentro del hueco previamente definido.

Los valores utilizados aquí coinciden con los parámetros definidos previamente en Desmos, por lo tanto, el resultado visual debería ser prácticamente idéntico al de referencia.



(3.3.f Distintas capas de los ojos)

Además, dado que ya hemos integrado las propiedades `_FacePosition` y `_EyeSize`, si regresamos al Inspector en Godot, podremos ajustar dinámicamente la posición y el tamaño de los ojos, lo que nos ofrece mayor flexibilidad visual al momento de experimentar con distintas expresiones.

Antes de continuar con otras formas que componen el personaje, será necesario modificar nuestro código para incorporar las propiedades de color declaradas previamente. Hasta ahora, las formas renderizadas utilizan únicamente valores en blanco o negro (1.0 o 0.0), lo cual es útil para representar máscaras, pero insuficiente para lograr el render final del personaje.

Si queremos que cada componente visual tenga su propio color RGB personalizado, debemos realizar lo siguiente:

```

29 void fragment()
30 {
31     ...
32
33     float head = float(dot(uv, uv) < pow(_HeadSize, 2.0));
34     float jaw = float(pow(min(_HeadSize - 0.23 - abs(u), 0.0), 4.0)
35         + pow(min(0.12 - abs(v + (_HeadSize - 0.15)), 0.0), 4.0) < pow(0.05,
36             4.0));
37     float eyes = 1.0 - eyes_shape(uv, _FacePosition, _EyeSize, 0.5);
38     float nose = 1.0 - float(-_FacePosition - v > 2.0 * abs(u) + 0.05)
39         * float(-_FacePosition - v < 0.15);
40
41     head = clamp(head + jaw, 0.0, 1.0);
42     head *= eyes_shape(uv, _FacePosition, 0.1, 1.0);
43     head *= nose;
44
45     vec3 render_rgb = vec3(0.0);
46     float alpha = 0.0;
47
48     vec3 head_color = _HeadColor * head;
49     vec3 eyes_color = _EyeColor * eyes;
50
51     render_rgb += head_color;
52     render_rgb += eyes_color;
53
54     ALBEDO = render_rgb;
55 }

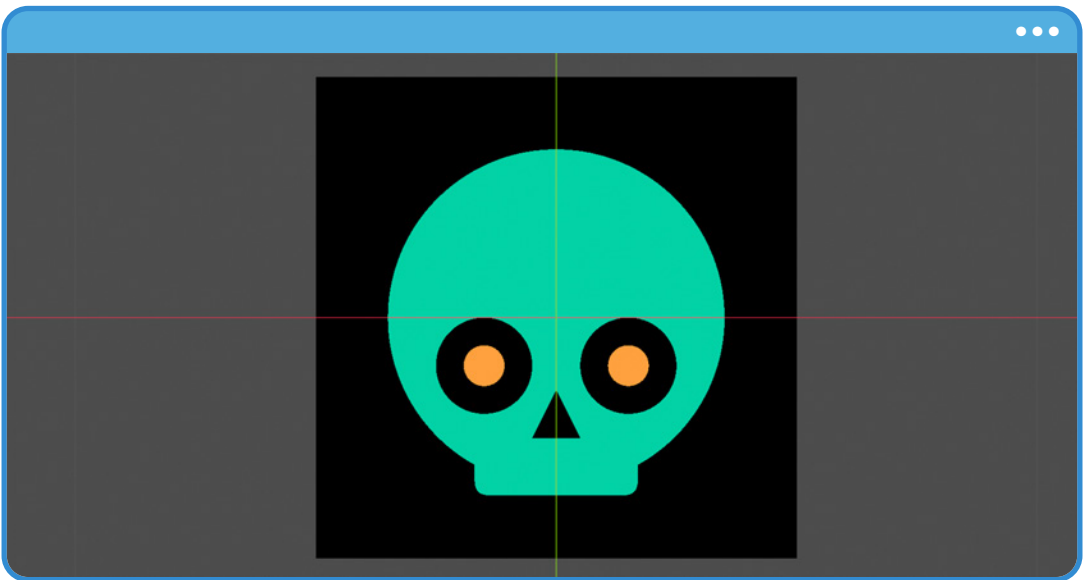
```

¿Qué está ocurriendo en nuestro código? Analicemos cada parte:

- Línea 38: Declaramos un nuevo escalar llamado **eyes**, que almacena el valor del ojo generado previamente, el cual antes habíamos sumado directamente a **head**. Ahora, lo separamos para poder enmascararlo con su propio color.
- Líneas 39 y 40: Definimos la variable **nose**, la cual representa la forma triangular de la nariz, basada en la ecuación descrita en la Figura 3.1.j. Incluye las propiedades necesarias para controlar su posición dinámicamente.
- Línea 44: Multiplicamos **head** por **nose** para restar visualmente la nariz de la cabeza, generando así un agujero en su lugar.

- Líneas 46 y 47: Declaramos **render_rgb**, un vector RGB que acumulará el color final del fragmento, y **alpha**, un escalar para controlar la transparencia del shader.
- Línea 49: Creamos **head_color**, un vector RGB que contiene el color definido para la cabeza, y lo multiplicamos más adelante por la máscara **head**.
- Línea 50: Creamos **eyes_color**, que contiene el color de los ojos, y lo enmascaramos con la variable **eyes**.
- Líneas 52 y 53: Sumamos **head_color** y **eyes_color** al acumulador **render_rgb**.
- Línea 55: Asignamos el resultado final a la salida del shader mediante **ALBEDO**.

Con esta implementación, si asignamos colores personalizados desde el **Inspector** para las propiedades **_HeadColor** y **_EyeColor**, obtendremos el siguiente resultado visual:



(3.3.g Color del cráneo: 00d2a6; color de los ojos: ffa13e)

A continuación, implementaremos la bandana del pirata. Según lo desarrollado previamente en Desmos, esta forma está compuesta por una curva que se recorta en tres regiones específicas:

- Alrededor de la cabeza,
- por debajo de las cejas,
- y en los bordes superiores de los ojos.

Para simplificar el proceso en esta primera aproximación, nos centraremos en la máscara general de la cabeza, ya que esta ya contiene las restricciones necesarias para evitar superposición con los ojos y la nariz. El código queda de la siguiente forma:

```

29 void fragment()
30 {
31     ...
40
41     float bandana = float(sqrt(UV.x) < 1.0 - UV.y);
42
43     head = clamp(head + jaw, 0.0, 1.0);
44     head *= eyes_shape(uv, _FacePosition, 0.1, 1.0);
45     head *= nose;
46     bandana *= head;
47
48     vec3 render_rgb = vec3(0.0);
49     float alpha = 0.0;
50
51     vec3 head_color = _HeadColor * head;
52     vec3 eyes_color = _EyeColor * eyes;
53     vec3 bandana_color = _BandanaColor * bandana;
54
55     render_rgb += head_color;
56     render_rgb += eyes_color;
57     render_rgb = mix(render_rgb, bandana_color, bandana);
58
59     ALBEDO = render_rgb;
60 }

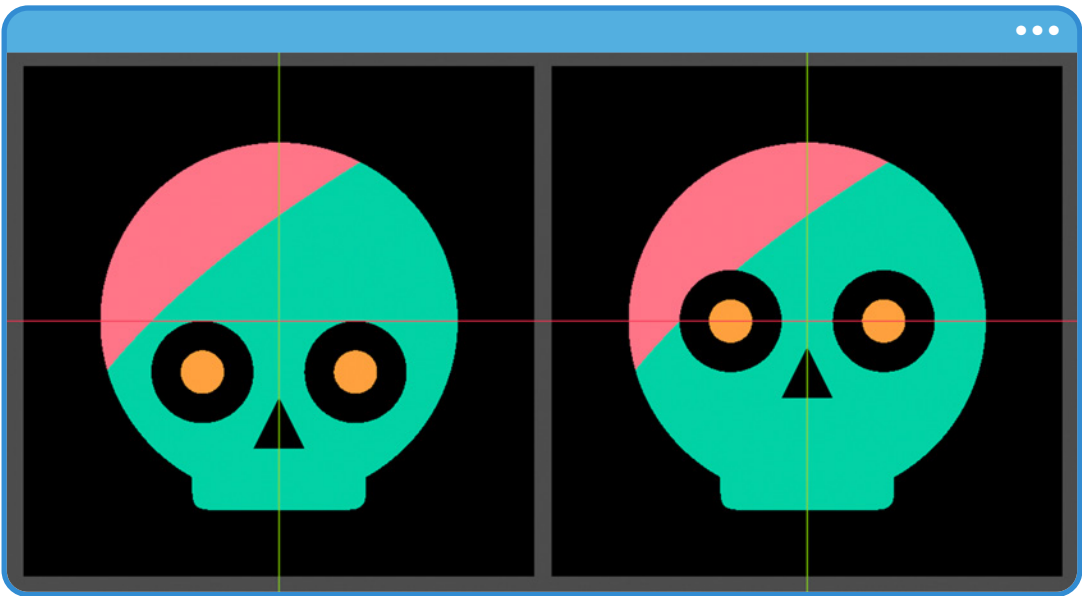
```

Analicemos lo que está ocurriendo:

- Línea 41: Declaramos la variable **bandana**, la cual define su forma mediante una comparación basada en **$\text{sqrt}(\text{UV.x}) < 1.0 - \text{UV.y}$** . Esta curva atraviesa el rostro, imitando el diseño mostrado en Desmos.
- Línea 46: Enmascaramos **bandana** con **head**, asegurándonos de que solo se renderice dentro de los límites del cráneo. Como la cabeza ya excluye ojos y nariz, no es necesario aplicar mascarado adicional.

- Línea 53: Definimos **bandana_color**, un vector RGB que contiene el color de la bandana multiplicado por su máscara.
- Línea 57: Realizamos una interpolación lineal entre el color acumulado en **render_rgb** y **bandana_color**, usando **bandana** como peso. Esto asegura una mezcla visual suave cuando las formas se solapan parcialmente.

Si vamos una vez más al **Inspector** y seleccionamos un color para la bandana, obtendremos el siguiente resultado:



(3.3.h Color de la bandana: ff7587)

Únicamente nos faltan por implementar las cejas y los huesos de la espalda. Ya sabemos que estos elementos requieren ser rotados, por lo que necesitaremos definir una función que nos permita aplicar rotaciones a puntos específicos en el espacio:

```
vec2 rotation(float x, float y, float a)
{
    float ru = cos(a) * x + sin(a) * y;
    float rv = -sin(a) * x + cos(a) * y;
    return vec2(ru, rv);
}
```

Esta función, que introdujimos inicialmente en la sección 3.1, nos permite rotar un punto bidimensional mediante un ángulo a expresado en radianes. Aunque podríamos optimizar los parámetros x e y utilizando directamente un vector **vec2**, los hemos mantenido separados para conservar una correspondencia más directa con la función original mostrada en la Figura 3.1.n, facilitando así su análisis matemático.

Una vez implementada, ya podemos utilizar esta función dentro del fragmento para rotar puntos clave del personaje. En este caso, comenzaremos con la ceja derecha. Sin embargo, como necesitamos representar ambas cejas, izquierda y derecha, definiremos un nuevo método que nos permita determinar su forma de forma reutilizable. Para ello, procederemos de la siguiente manera:

```
float eyebrow_shape(vec2 p)
{
    float r = 0.03;
    float u = min(0.05 - abs(p.x), 0.0);
    float v = p.y;
    vec2 uv = vec2(u, v);
    return float(dot(uv, uv) < r * r);
}
```

La función **eyebrow_shape()** representa una traducción directa al lenguaje GDSDL de la forma mostrada en la Figura 3.1.ñ. En este caso, utilizamos la expresión **dot(uv, uv)** para calcular la magnitud al cuadrado del vector **uv**, lo cual es una forma optimizada de verificar si un punto se encuentra dentro de un radio r .

Nota

Recordemos que existen distintas maneras de expresar esta operación:

1) $x * x + y * y$, 2) $\text{pow}(x, 2.0) + \text{pow}(y, 2.0)$, 3) **dot(uv, uv)**. De las tres, **dot()** es la más eficiente y legible, y por eso la utilizamos aquí para definir la forma curva de la ceja.

En este punto, comenzaremos con la ceja derecha, la cual será evaluada mediante el método **eyebrow_shape()**, como se muestra a continuación:


```

45 void fragment()
46 {
47     ...
48
49     float r1 = _EyebrowRotation * (PI / 180.0);
50     vec2 p0 = rotation(u - 0.1, v - 0.15 + _FacePosition, r1);
51     float eyebrow_r = eyebrow_shape(p0);
52
53     head = clamp(head + jaw, 0.0, 1.0);
54     head *= eyes_shape(uv, _FacePosition, 0.1, 1.0);
55     head *= nose;
56     bandana *= head;
57
58     vec3 render_rgb = vec3(0.0);
59     float alpha = 0.0;
60
61     vec3 head_color = _HeadColor * head;
62     vec3 eyes_color = _EyeColor * eyes;
63     vec3 bandana_color = _BandanaColor * bandana;
64     vec3 eyebrows_color = _EyebrowColor * eyebrow_r;
65
66     render_rgb += head_color;
67     render_rgb += eyes_color;
68     render_rgb = mix(render_rgb, bandana_color, bandana);
69     render_rgb = mix(render_rgb, eyebrows_color, eyebrow_r);
70
71     ALBEDO = render_rgb;
72 }

```

Analizamos lo que está ocurriendo:

- Línea 59: Se transforma el valor de **_EyebrowRotation** desde grados a radianes, y el resultado se almacena en la variable **r1**.
- Línea 60: Se aplica una traslación previa para ubicar el origen local de la ceja con un desplazamiento en **u** y **v**, más el ajuste dinámico de posición vertical mediante **_FacePosition**. Luego se aplica la función **rotation()** utilizando el ángulo **r1**. El resultado se almacena en el vector **p0**.

- Línea 61: Se declara e inicializa un nuevo escalar llamado **eyebrow_r**, el cual corresponde a la ceja derecha del pirata.
- Línea 74: Se declara e inicializa un nuevo vector RGB llamado **eyebrows_color**, el cual multiplica el color de la ceja por la máscara **eyebrow_r**, de manera que solo se aplica color a los píxeles donde la forma de la ceja esté presente.
- Línea 79: Se realiza una interpolación lineal entre el color actual acumulado en **render_rgb** y el color de la ceja **eyebrows_color**.

Una vez guardados los cambios, al seleccionar un color desde el **Inspector** para la propiedad **_EyebrowColor**, podremos visualizar claramente la forma de la ceja derecha renderizada sobre el QuadMesh.

Ahora continuaremos agregando la ceja izquierda mediante una reflexión horizontal, siguiendo el mismo principio aplicado previamente en Desmos y retomado en la Figura 3.1.o. Para ello, regresaremos al código y realizaremos una operación simétrica a la que ya aplicamos para la ceja derecha.

```
void fragment()
{
    ...

    float r1 = _EyebrowRotation * (PI / 180.0);
    vec2 p0 = rotation(u - 0.1, v - 0.15 + _FacePosition, r1);
    vec2 p1 = rotation(-u - 0.1, v - 0.15 + _FacePosition, r1);
    float eyebrow_r = eyebrow_shape(p0);
    float eyebrow_l = eyebrow_shape(p1);
    float eyebrows = eyebrow_l + eyebrow_r;

    ...

    vec3 eyebrows_color = _EyebrowColor * eyebrows;

    ...

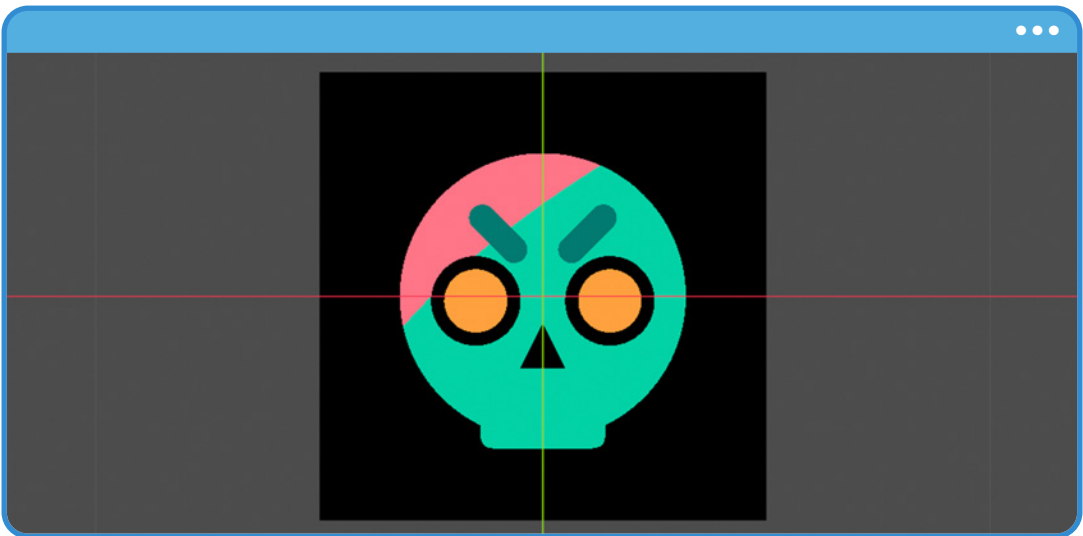
    render_rgb = mix(render_rgb, eyebrows_color, eyebrows);

    ALBEDO = render_rgb;
}
```

Como podemos observar, se ha declarado un nuevo punto llamado **p1**, el cual corresponde a la reflexión horizontal de **p0** para generar la ceja izquierda. Posteriormente, se evalúa la forma de cada ceja utilizando una vez más la función **eyebrow_shape()**, y sus respectivos resultados se almacenan en **eyebrow_r** (derecha) y **eyebrow_l** (izquierda). Ambos valores se suman y se almacenan en la variable **eyebrows**, que representa la máscara total de ambas cejas.

Finalmente, esta máscara se utiliza tanto para generar el color correspondiente (**eyebrows_color**) como para realizar una interpolación lineal sobre el color acumulado (**render_rgb**), logrando integrar visualmente las cejas con el resto del personaje.

Estas operaciones producen el siguiente resultado visual:



(3.3.i Color Turquesa oscuro: 007a71)

Por último, agregaremos los huesos de la espalda. Para ello, partiremos de la ecuación presentada en la Figura 3.1.p, que definía esta forma en Desmos. Su traducción directa a GDScript es la siguiente:

```
float back_bone_shape(vec2 p)
{
    float r = 0.06;
    float u = min(0.42 - abs(p.x), 0.04);
    float v = abs(p.y) - 0.04;
    vec2 uv = vec2(u, v);
    return float(dot(uv, uv) < r * r);
}
```

A continuación, necesitaremos rotar ambos huesos de la espalda. Para ello, aplicaremos la función **rotation()** a dos nuevos puntos: p_2 y p_3 , que corresponden a los huesos derecho e izquierdo, respectivamente. Este procedimiento se integra en la etapa de fragmento como sigue:

```
void fragment()
{
    ...

    float to_radians = PI / 180.0;
    float r1 = _EyebrowRotation * to_radians;
    float r2 = _BackBoneRotation * to_radians;

    vec2 p0 = rotation(u - 0.1, v - 0.15 + _FacePosition, r1);
    vec2 p1 = rotation(-u - 0.1, v - 0.15 + _FacePosition, r1);
    vec2 p2 = rotation(u, v, r2);
    vec2 p3 = rotation(-u, v, r2);

    ...
}
```

Si prestamos atención al fragmento de código anterior, notaremos que hemos declarado una nueva variable llamada **to_radians**, la cual convierte valores en grados a radianes. Esta conversión es fundamental para utilizar correctamente funciones trigonométricas en GDSDL, ya que estas esperan ángulos en radianes. Gracias a esta constante, podemos ingresar

valores en grados desde el Inspector, manteniendo así una interfaz más intuitiva para ajustar las rotaciones.

Los valores rotados de **_EyebrowRotation** y **_BackBoneRotation** se almacenan en **r1** y **r2**, respectivamente. Luego, se declaran dos nuevos vectores bidimensionales: **p2** y **p3**. El primero representa el punto base para uno de los huesos de la espalda, mientras que el segundo es su reflejo en el eje vertical, generando así la simetría necesaria para ambos lados del personaje.

Una vez definidos estos puntos, podemos utilizar la función **back_bone_shape()** para dibujar los huesos en el shader. Sin embargo, antes de hacerlo, debemos recordar que los huesos de la espalda se superponen parcialmente con la cabeza, y que su área es ligeramente mayor, como observamos en la Figura 3.1.q.

Por esta razón, conviene reutilizar la máscara de la cabeza como una forma de limitar visualmente los huesos. Para ello, encapsularemos la operación que calcula la cabeza en una nueva función, como se muestra a continuación:

```
float head_shape(vec2 uv, float s)
{
    return float(dot(uv, uv) < pow(s, 2.0));
}
```

El método **head_shape()** devuelve la misma máscara circular que utilizamos para representar la cabeza. Gracias a esta función reutilizable, ahora podemos reemplazar directamente la operación anterior con una llamada más clara y concisa:

```
float head = head_shape(uv, _HeadSize);
```

Esta línea mejora la legibilidad del código y nos permitirá utilizar la misma lógica en otros elementos que dependan de la forma de la cabeza, como es el caso de los huesos de la espalda.

Para generar estos huesos, comenzamos evaluando su forma utilizando los puntos rotados **p2** y **p3**, previamente definidos. Estos puntos representan los extremos simétricos de la espalda, y son evaluados con la función **back_bone_shape()** como se muestra a continuación:

```
void fragment()
{
    ...
    float eyebrows = eyebrow_l + eyebrow_r;

    float back_bone_l = back_bone_shape(p2);
    float back_bone_r = back_bone_shape(p3);
    float back_bones = clamp(back_bone_l + back_bone_r, 0.0, 1.0);

    ...
    bandana *= head;
    back_bones *= 1.0 - head_shape(uv, _HeadSize + 0.03);

    ...
    vec3 eyebrows_color = _EyebrowColor * eyebrows;
    vec3 back_bone_color = _BackBoneColor * back_bones;

    ...
    render_rgb += back_bone_color;

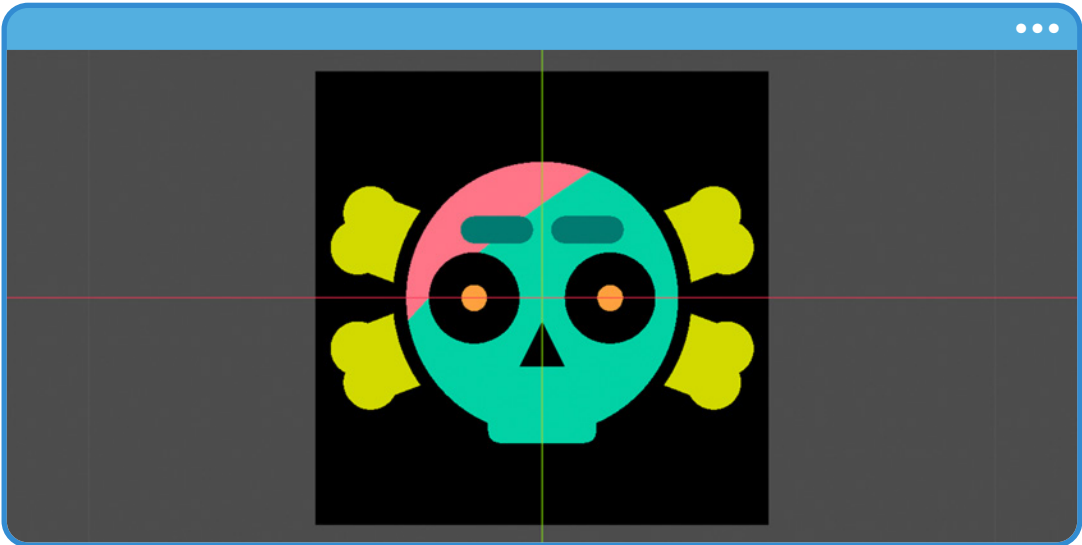
    ALBEDO = render_rgb;
}
```

La suma de ambas evaluaciones genera la máscara **back_bones**, la cual representa los dos huesos. Aplicamos una función **clamp()** para asegurarnos de que el valor final esté acotado entre 0.0 y 1.0, previniendo problemas de interpolación o saturación.

Posteriormente, utilizamos la forma de la cabeza como una máscara inversa sobre los huesos, de modo que estos no interfieran visualmente con el rostro del personaje. Para ello, simplemente ampliamos ligeramente el radio y restamos la máscara, como se muestra en esta línea de código:

```
back_bones *= 1.0 - head_shape(uv, _HeadSize + 0.03);
```

Finalmente, calculamos el color correspondiente a los huesos y lo integramos en la composición general de color. El color se aplica únicamente donde la máscara **back_bones** tiene valor, lo cual asegura que los huesos se integren de manera armónica en la composición visual. Al observar el resultado final en pantalla, deberíamos ver al pirata completo, con todos los elementos correctamente enmascarados, rotados y coloreados.



(3.3.j Color de los huesos de la espalda: d2da00)

Hasta este punto, podríamos considerar que nuestro pirata está completo en términos de forma y color. Sin embargo, aún nos falta implementar un aspecto fundamental para integrarlo visualmente de manera correcta en cualquier escena: la transparencia. Incorporar transparencia en GDSL es un proceso bastante simple.

Para lograrlo, retomaremos la variable **alpha**, que declaramos previamente como acumulador de opacidad. Luego, sumaremos las máscaras correspondientes a los elementos visibles del personaje: la cabeza (**head**), los ojos (**eyes**) y los huesos traseros (**back_bones**). Este valor compuesto será utilizado como canal alfa en nuestro fragmento.

El código resultante se ve de la siguiente manera:

```
void fragment()
{
    ...
    render_rgb += back_bone_color;

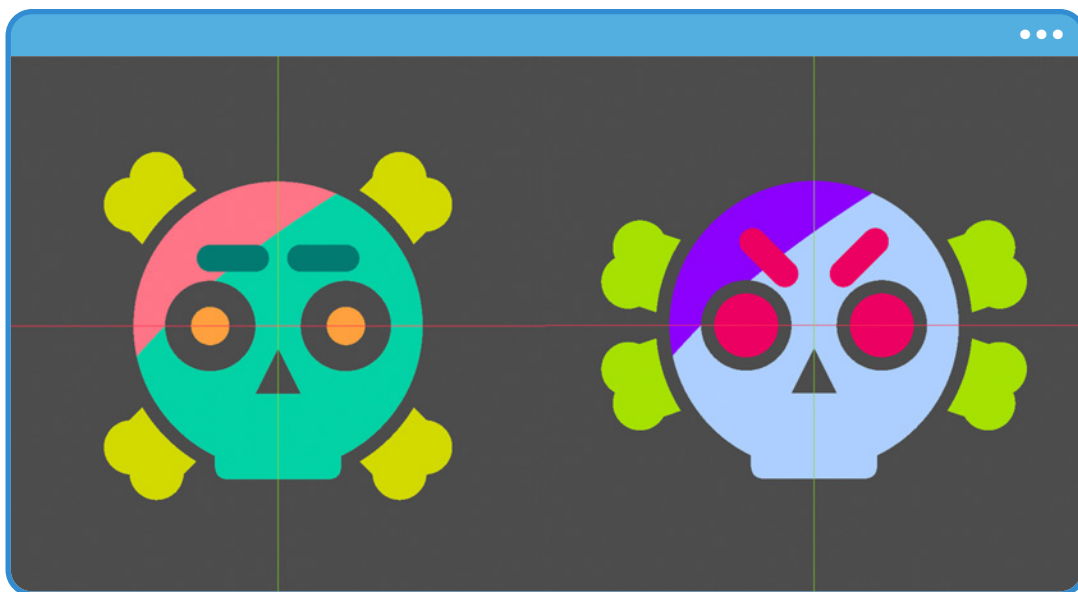
    alpha += head;
    alpha += eyes;
    alpha += back_bones;

    float back = back_bone_shape(p2);

    ALBEDO = render_rgb;
    ALPHA = alpha;
}
```

Gracias a esta suma acumulativa, la transparencia se aplica únicamente en aquellas zonas donde las máscaras tienen presencia. Esto permite que el resto del quad permanezca completamente transparente, eliminando cualquier fondo no deseado alrededor del personaje.

Al visualizar el resultado, notaremos que el pirata se encuentra ahora completamente integrado con un canal alfa que delimita únicamente su silueta. Esto es particularmente útil si queremos proyectarlo sobre diferentes escenarios o combinarlo con otros efectos visuales.



(3.3.k Configuración de color distinta para la calavera pirata procedural)

3.4 Deformación de vértices y animación.

Cuando hablamos de vértices, inmediatamente pensamos en esos pequeños cubos de color magenta (o negros en el caso de Blender) que marcan los bordes de una malla 3D. Sin embargo, si dejamos de lado la representación visual, los vértices no son más que valores numéricos (vectores) que almacenan información clave, como su posición en el espacio relativa al modelo.

Comprender esta naturaleza es fundamental, especialmente cuando buscamos crear efectos a través de deformación y animación en shaders. Al hablar de deformación y animación nos referimos a cualquier operación que modifique la posición de un vértice en el espacio. Para lograrlo necesitamos operaciones matemáticas básicas: adición, multiplicación, división y, sobre todo, una variable fundamental: el tiempo. ¡Todo ocurre en función del tiempo!

En Godot disponemos de una variable interna llamada **TIME**, que según la documentación oficial:

“

Tiempo global desde el arranque del motor, en segundos. Se repite cada 3600 segundos (que se puede modificar con la configuración de rollover). Se ve afectado por `time_scale`, pero no por la pausa. Si necesita una variable **TIME** que no se vea afectada por la escala de tiempo, añada su propio shader uniforme global y actualícelo en cada fotograma.

”

Durante esta sección experimentaremos con la deformación de vértices y la animación aplicada a una composición de imágenes, y qué mejor que hacerlo directamente sobre elementos de UI. ¿Por qué trabajar con imágenes de interfaz de usuario? Porque a nivel técnico presentan un desafío mayor: tanto en la etapa de vértices como en la de fragmento contamos con menos variables internas disponibles. En la práctica esto significa que debemos construir gran parte de la lógica por nuestra cuenta, lo que nos obliga a diseñar soluciones más creativas para alcanzar el resultado deseado.

Para poner en práctica estos conceptos desarrollaremos un shader que realizará dos tareas principales:

- 1 Animar la escala de los vértices de la UI.
- 2 Añadir un efecto especular animado, adaptado automáticamente a la imagen que estemos utilizando en la composición.

El segundo punto resulta especialmente interesante porque trabajaremos con **un solo material** aplicado a toda la composición, y aun así los resultados se ajustarán de forma dinámica a cada imagen, tal como se observa en la siguiente referencia:



(3.4.a Animación sobre vértices y especularidad)

Iniciaremos organizando nuestro proyecto de la siguiente manera:

- Dentro de la carpeta **chapter_03**, crearemos una nueva carpeta llamada **vertex_animation**.
- Dentro de esta última, crearemos tres nuevas carpetas, las que organizaremos de la siguiente manera:
 - **materials**.
 - **shaders**.
 - **textures**.

Para el ejercicio necesitaremos tanto un shader como un material. Para ello, nos ubicaremos en la carpeta **materials**, haremos clic derecho y seleccionaremos:

- Create New > Resources > ShaderMaterial.

Nombraremos a este material como **ui_animation**.

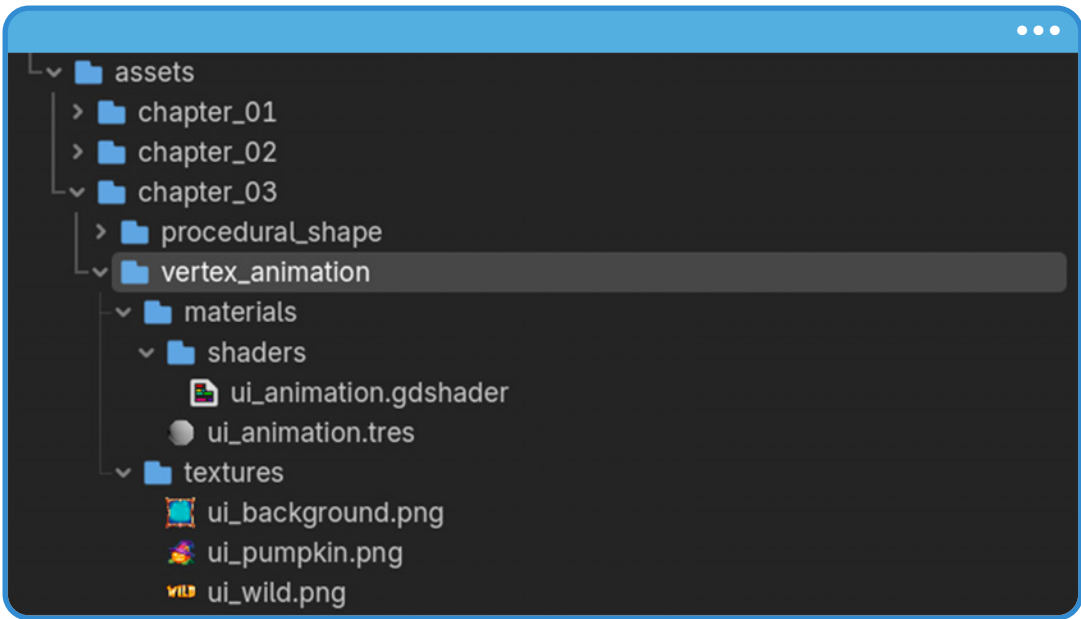
A continuación, crearemos el shader correspondiente dentro de la carpeta **shaders**, también mediante clic derecho y seleccionando:

- Create New > Resources > Shader.

Tal como es nuestra costumbre, lo llamaremos **ui_animation** para mantener la coherencia con el nombre de la sección.

Es importante asegurarnos de descargar las imágenes que utilizaremos a lo largo de esta sección. Estas se encuentran disponibles en el paquete adjunto al libro y corresponden a: **ui_background**, **ui_pumpkin** y **ui_wild**.

Si hemos configurado todo correctamente, la estructura de carpetas de nuestro proyecto debería verse de la siguiente manera:

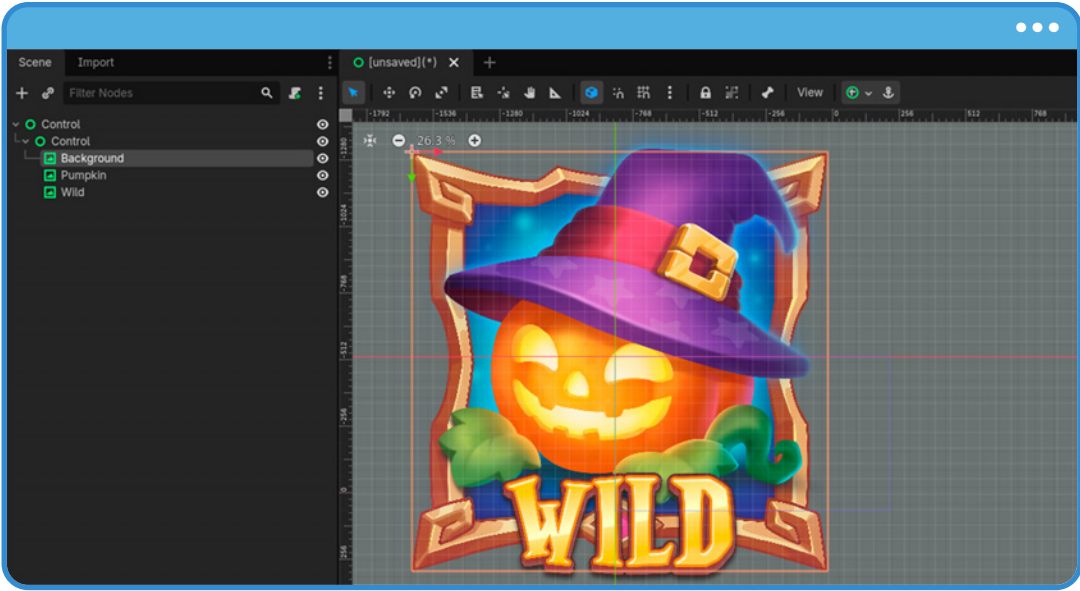


(3.4.b Estructura del proyecto)

Antes de comenzar con el shader, configuraremos la escena y los elementos que utilizaremos en esta sección. También será necesario asignar el material **ui_animation** a cada imagen en la UI, dentro de su propiedad **Material**.

Para ello, crearemos una nueva escena vacía con un nodo **User Interface** como raíz. Luego añadiremos un nodo **Control** como padre y, dentro de él, tres nodos **TextureRect** que nombraremos de la siguiente manera: **Background**, **Pumpkin** y **Wild**. A continuación, asignaremos cada textura a su respectivo nodo y nos aseguraremos de que todos los elementos queden centrados en la pantalla. Para lograrlo, seleccionaremos la opción **Anchor Preset** y marcaremos **Center**.

Si ajustamos correctamente la posición de los elementos, nuestra escena deberá lucir de la siguiente manera:

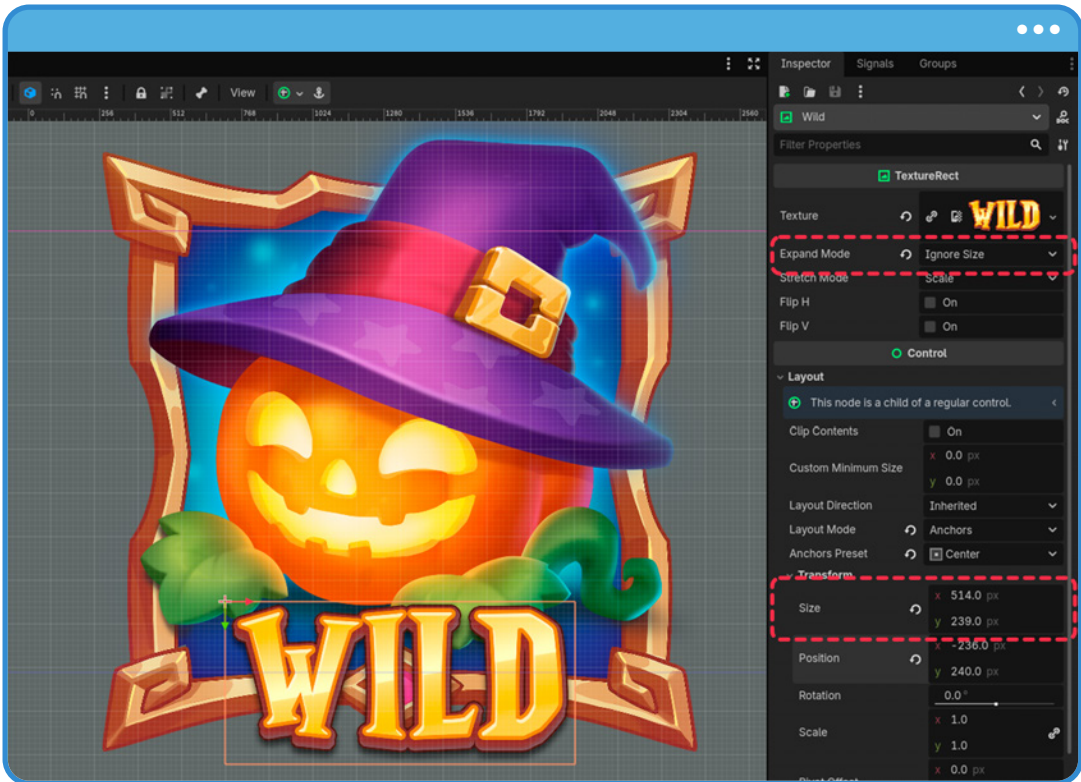


(3.4.c Elementos de la UI)

Las imágenes que estamos utilizando son bastante grandes; de hecho, fueron creadas en un lienzo de 2048x2048 píxeles. Para evitar que la composición exceda el área visible en el render final al compilar el juego, reduciremos su tamaño a la mitad. Para hacerlo, seguiremos estos pasos:

- Seleccionaremos una de las imágenes.
- En el Inspector, dentro de **TextureRect**, cambiamos la propiedad **ExpandMode** a **IgnoreSize**. Esto nos permitirá modificar el tamaño de la imagen.
- Luego vamos a **Layout > Transform** y multiplicamos tanto la componente **X** como la **Y** por 0.5. Este valor lo utilizaremos más adelante en el shader para asegurar que las proporciones en la animación y la transformación de los vértices se mantengan correctas.

Repetiremos este mismo proceso para cada imagen en la UI.



(3.4.d Todas las imágenes se han reducido a la mitad de su tamaño original)

En Godot podemos trabajar con distintos tipos de shaders. A lo largo de este libro hemos utilizado principalmente el tipo **spatial**, ideal para elementos 3D. Sin embargo, en este caso necesitamos un shader distinto, ya que estaremos trabajando con imágenes de interfaz. Por lo tanto, reemplazaremos el tipo de shader **spatial** por **canvas_item**, y además, añadiremos algunas propiedades que usaremos para recrear los efectos que desarrollaremos en esta sección.

```

shader_type canvas_item;

uniform bool _VertexAnimation;
uniform float _VertexScale : hint_range(0.0, 0.5, 0.01);
uniform float _VertexSpeed : hint_range(1.0, 3.0, 0.1);
uniform float _SpecularSpeed : hint_range(0.5, 2.0, 0.1);
uniform vec3 _SpecularColor : source_color;

void vertex() { ... }

```

Siempre que trabajemos con imágenes de UI en Godot será necesario utilizar shaders de tipo **canvas_item**, ya que este tipo está diseñado específicamente para dibujar y manipular elementos 2D.

Dentro del código podemos identificar varias propiedades, cada una con un propósito específico:

- **_VertexAnimation** es de tipo bool (verdadero o falso). La utilizaremos para activar o desactivar el efecto de animación sobre los vértices.
- **_VertexScale** nos permitirá definir el tamaño máximo de la escala en la animación de la imagen de UI.
- **_VertexSpeed**, como indica su nombre, controlará la velocidad con la que la animación se ejecuta, medida en segundos.
- **_SpecularSpeed** determinará la velocidad a la que se anima el efecto especular sobre la imagen de UI.
- Finalmente, **_SpecularColor** servirá para ajustar el color del efecto especular.

Como aplicaremos una transformación personalizada sobre los vértices durante la animación, será necesario incluir el modo de renderizado **skip_vertex_transform**. Tal como vimos en la sección 1.11 del capítulo 1, esta propiedad desactiva la transformación automática que Godot realiza en el método **vertex()**. En consecuencia, al agregarla al código notaremos que las imágenes cambian de posición en el Viewport, ya que ahora seremos nosotros quienes controlaremos completamente su transformación.

```
shader_type canvas_item;
render_mode skip_vertex_transform;
```

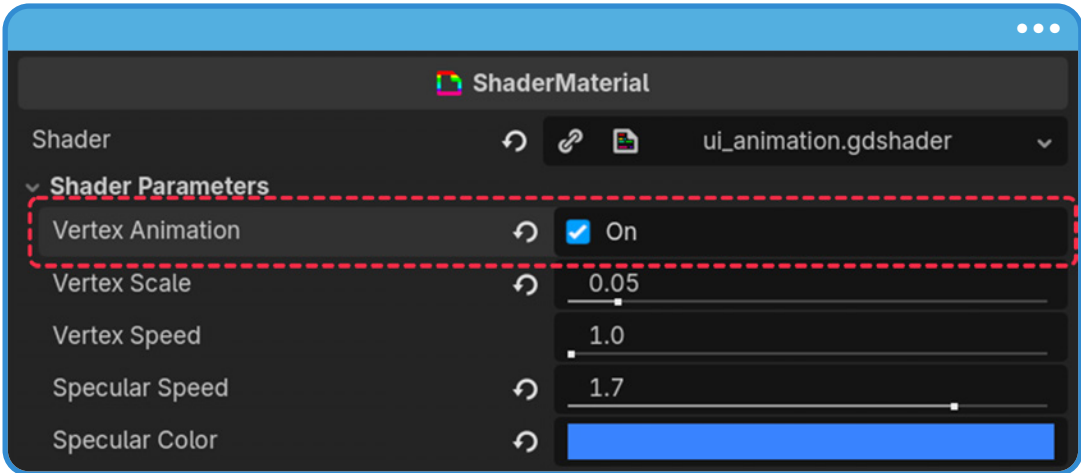
Para solucionar el problema de posición, multiplicaremos la matriz **MODEL_MATRIX** por los vértices en espacio local y guardaremos el resultado en **VERTEX**, como se muestra a continuación:

```
void vertex()
{
    if (!_VertexAnimation)
    {
        VERTEX = (MODEL_MATRIX * vec4(VERTEX, 0.0, 1.0)).xy;
    }
}
```

En este ejercicio hemos añadido una condición: cuando **_VertexAnimation** sea **false** (valor por defecto), los vértices conservarán su valor original. En cambio, cuando sea **true**, aplicaremos una animación escalando los vértices en el tiempo. Para visualizar este proceso comenzaremos con un ejemplo simple:

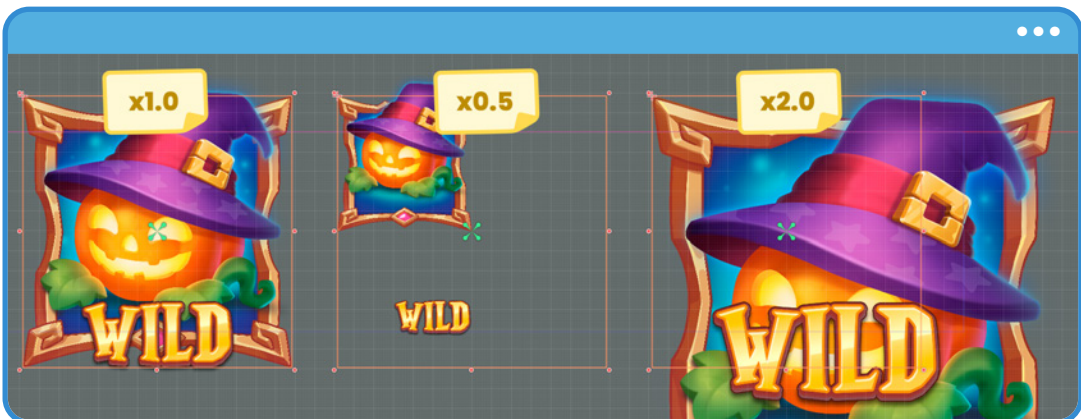
```
void vertex()
{
    if (!_VertexAnimation)
    {
        VERTEX = (MODEL_MATRIX * vec4(VERTEX, 0.0, 1.0)).xy;
    }
    else
    {
        vec2 scaled_vertex = VERTEX * vec2(1.0);
        VERTEX = (MODEL_MATRIX * vec4(scaled_vertex, 0.0, 1.0)).xy;
    }
}
```


Como podemos observar, ambas transformaciones producen el mismo resultado, con la diferencia de que en el segundo caso (**el1se**) hemos introducido una operación de escala sobre los vértices de la UI. Para visualizar los cambios que realizaremos a continuación, activaremos la propiedad **Vertex Animation** desde el **Inspector** en el material de nuestro proyecto.



(3.4.e Se ha activado la propiedad **Vertex Animation**)

Si regresamos a nuestro código y modificamos el valor de multiplicación de los vértices por un número mayor o menor que uno, veremos que los vértices de la UI efectivamente cambian de escala. Sin embargo, el punto de referencia de esa escala se encuentra en la esquina superior izquierda de la UI, lo cual no es óptimo, ya que necesitamos que la transformación ocurra desde el centro del objeto.



(3.4.f Distintas escalas para los vértices)

Existe una solución rápida a este problema: restar la mitad del tamaño actual de la imagen. Para comprenderlo, supongamos que tenemos una imagen de 512x512 píxeles. Si la escala es igual a 1, no es necesario realizar ninguna corrección. Sin embargo, si la escala es igual a 2, debemos sustraer de los vértices la mitad del tamaño de la imagen, es decir $512 / 2$.

¿Qué ocurriría si necesitamos escalar por n valores? Para responder a esta pregunta, definamos algunas variables:

- a representa el valor de la escala.
- b representa el valor que necesitamos sustraer.
- x corresponde a la resolución actual de la imagen de UI.

Con estas variables podemos deducir los siguientes casos:

- Si $a = 1$, entonces $b = 0$.
- Si $a = 2$, entonces $b = x/2$.
- Si $a = 3$, entonces $b = x$.
- Si $a = 4$, entonces $b = x + (x/2)$.
- Si $a = 5$, entonces $b = x + x$.
- Si $a = 6$, entonces $b = x + x + (x/2)$.
- Si $a = 7$, entonces $b = x + x + x$.

Al observar el patrón, obtenemos la siguiente fórmula general:

$$b = x * \frac{(a - 1)}{2}$$

(3.4.g)

Podemos implementarla en nuestro shader a través de la siguiente función:

```
float scale_from_center(float a, float x)
{
    float b = x * (a - 1.0) * 0.5;
    return b;
}

void vertex() { ... }
```

Para acceder a la resolución actual de la imagen, en Godot podemos utilizar la variable interna `TEXTURE_PIXEL_SIZE`, que según la documentación oficial se define así:

“

Tamaño de píxel normalizado de la textura 2D predeterminada. Para un `Sprite2D` con una textura de 64x32 píxeles,
`TEXTURE_PIXEL_SIZE = vec2(1/64, 1/32).`

”

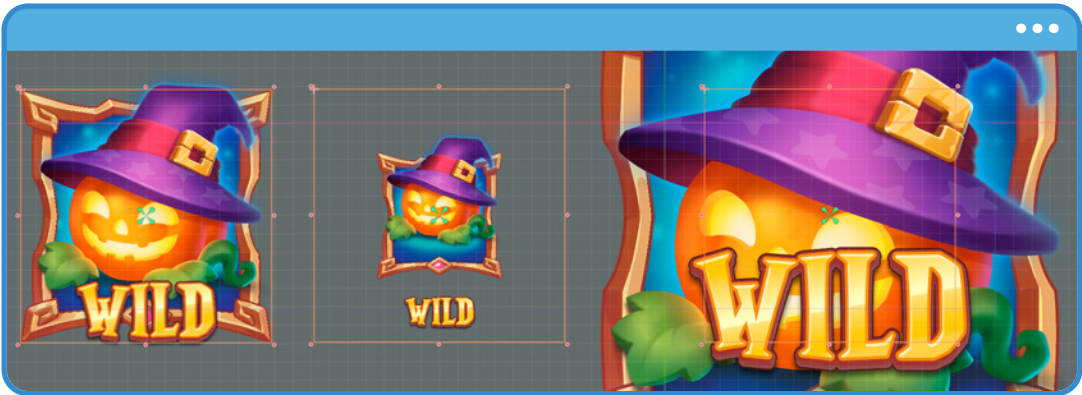
Sabiendo esto, podemos implementar la corrección de escala desde el centro con el siguiente bloque:

```
22 else
23 {
24     float scale_ratio = 1.0;
25     const float pixel_scaler = 0.5;
26
27     vec2 resolution = pixel_scaler / TEXTURE_PIXEL_SIZE;
28     float px = scale_from_center(scale_ratio, resolution.x);
29     float py = scale_from_center(scale_ratio, resolution.y);
30
31     vec2 scaled_vertex = VERTEX * vec2(scale_ratio);
32     scaled_vertex -= vec2(px, py);
33
34     VERTEX = (MODEL_MATRIX * vec4(scaled_vertex, 0.0, 1.0)).xy;
35 }
```

¿Qué ocurre línea a línea?:

- Línea 24, definimos **scale_ratio**, el factor de escala que aplicaremos a los vértices de la UI.
- Línea 25, **pixel_scaler = 0.5** refleja que previamente redujimos las imágenes al 50 % desde el Inspector; si no las hubiéramos reducido, este valor sería 1.0.
- Línea 27, calculamos **resolution** en píxeles efectivos. Recordemos que **TEXTURE_PIXEL_SIZE = (1/width, 1/height)**; al dividir **pixel_scaler** por ese valor obtenemos **(pixel_scaler * width, pixel_scaler * height)**, es decir, la resolución tras la reducción al 50 %.
- Líneas 28 y 29, obtenemos el desplazamiento necesario desde el centro con **scale_from_center()** (Figura 3.4.g) para cada eje.
- Línea 31, escalamos los vértices con **scale_ratio**.
- Línea 32, restamos **(px, py)** para recentrar y lograr que la escala suceda desde el centro geométrico de la imagen.

Al modificar **scale_ratio** veremos que las imágenes de UI se escalan desde su centro y no desde la esquina superior izquierda.



(3.4.h Las imágenes de UI se escalan desde el centro)

En este punto podemos experimentar con distintas operaciones matemáticas para animar la escala. Por ejemplo, usar **sin()** nos permite aumentar o disminuir automáticamente el tamaño de las imágenes:

```

else
{
    float time = (TIME * _VertexSpeed) * PI;
    float scale_ratio = sin(time) * _VertexScale + (1.0 +
        ↪ _VertexScale);
    const float pixel_scaler = 0.5;

    ...
}

```

Aquí `time` multiplica la constante global `TIME` por `_VertexSpeed` y por `PI`. La función `sin()` produce un valor en `[-1 : 1]`; luego escalamos su amplitud con `_VertexScale` y sumamos un término base para garantizar que la escala mínima sea 1.0. El resultado se guarda en `scale_ratio`, que determina cómo cambia el tamaño en pantalla. Si ajustamos `_VertexSpeed` y `_VertexScale` desde el Inspector, veremos la UI animarse dinámicamente.

El problema es que todas las imágenes comparten el mismo material, por lo que se escalan al mismo ritmo. Para introducir variación por imagen aprovecharemos la variable interna `COLOR`, un vector de cuatro componentes que, según la documentación, corresponde a:

“

Color de la primitiva de vértices, multiplicado por el `modulate` del **CanvasItem**, multiplicado por `self_modulate` del **CanvasItem**.

”

En pocas palabras, se trata del color por defecto de los vértices (blanco), multiplicado por el color que asignemos desde el material y desde las propiedades **Modulate** y **Self Modulate**, en la pestaña **Visibility**.

Con esta propiedad podemos crear un ID de color, como se muestra a continuación:

```

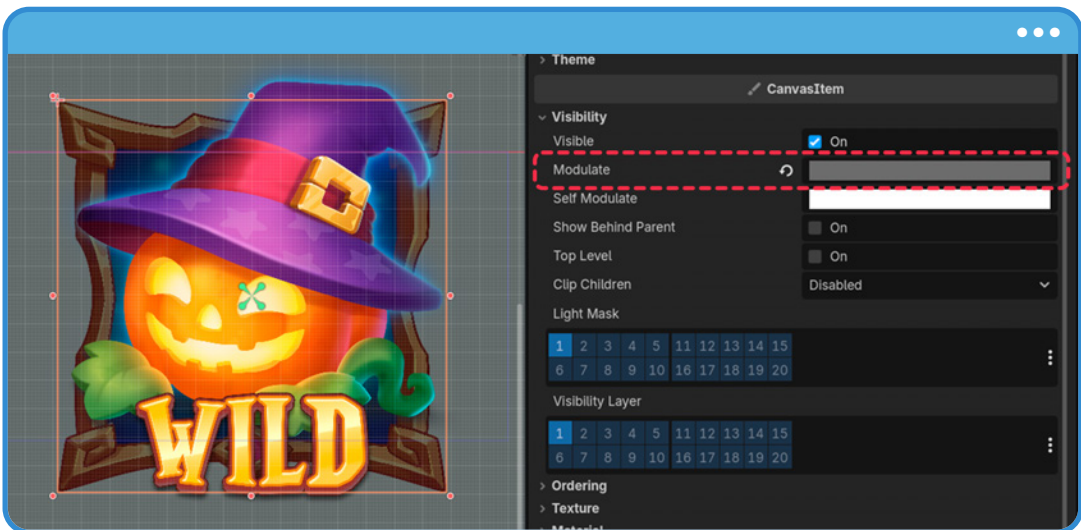
else
{
    float color_id = dot(COLOR.rgb, COLOR.rgb);
    float time = (color_id + TIME * _VertexSpeed) * PI;

    ...
}

```

Como podemos observar, se ha declarado una nueva variable llamada **color_id**, que obtiene el resultado de calcular el producto punto de la variable **COLOR** en RGB. Esto nos genera un valor en escala de grises, ideal para utilizar como ID.

Si vamos a la pestaña **Visibility** del material y cambiamos la escala de grises de la propiedad **Modulate** en alguna de nuestras imágenes de UI, veremos que en efecto se produce un desfase en la animación. Sin embargo, esto genera un nuevo problema: la imagen ahora luce opaca según el valor de color de la propiedad.



(3.4.i El color de los vértices está siendo afectado por la propiedad **Modulate**)

Podemos solucionar este problema con facilidad pasando el color de la textura a la variable **COLOR** en la etapa de fragmento.

```
void fragment()
{
    vec4 color = texture(TEXTURE, UV);
    COLOR = color;
}
```

Como es de esperar, la variable **TEXTURE** se refiere a la textura 2D por defecto que asignamos a la imagen de UI, la cual varía en cada caso.



(3.4.j Se ha corregido el color en las imágenes de UI)

En la próxima sección agregaremos un efecto especular personalizado para enfatizar aún más la animación en nuestras imágenes de UI.

3.5 Agregando un efecto especular sobre la UI.

Es cierto que el efecto especular en una imagen, ya sea un Sprite o un elemento de UI, podría crearse fácilmente utilizando una simple textura, como un gradiente vertical. Hacerlo de esta forma nos ahorraría tiempo y recursos, ya que las GPU están altamente optimizadas para este tipo de tareas. Sin embargo, desde un punto de vista técnico, resulta mucho más interesante generar esta textura de manera procedural, ya que esto nos permitirá

introducimos en otras áreas más avanzadas, como las funciones de distancia implícita (SDF).

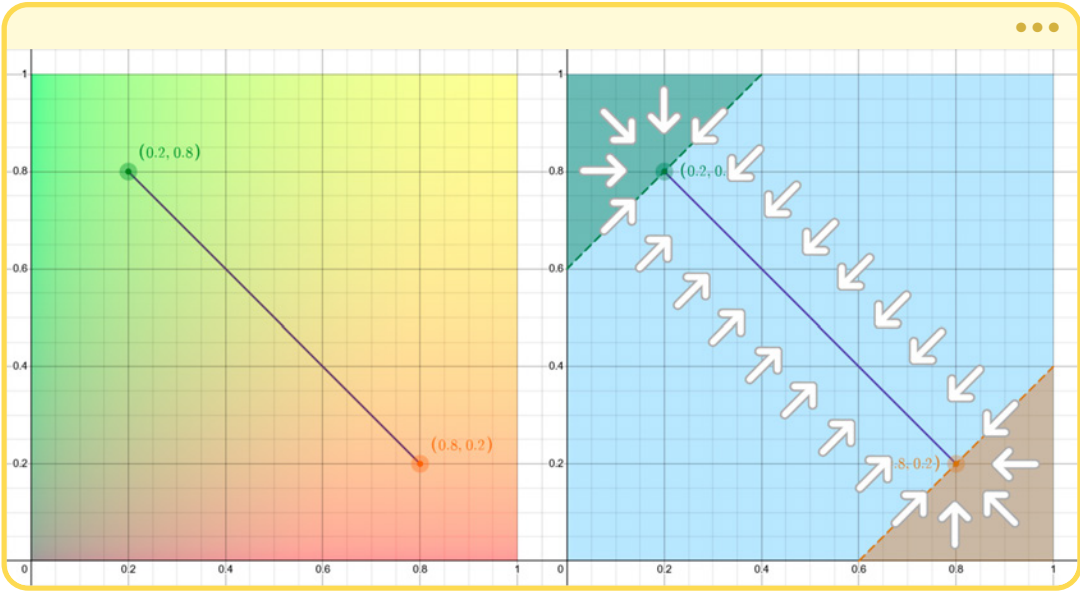
Por esta razón, durante esta sección desarrollaremos un segmento empleando la técnica de distancia implícita, para lo cual será necesario aplicar nuestros conocimientos en trigonometría.

Un segmento SDF luce de la siguiente manera:

```
float segment_sd(vec2 uv, vec2 p0, vec2 p1)
{
    float h = projection(uv, p0, p1);
    h = clamp(h, 0.0, 1.0);
    vec2 p2 = p0 + h * (p1 - p0);
    return distance(p2, uv);
}
```

Si te estas introduciendo en GDSL por primera vez, es posible que esta función te parezca compleja; sin embargo, resulta bastante simple si recordamos la definición de un segmento en el plano cartesiano.

Iniciemos dibujando dos puntos en el plano cartesiano:

(3.5.a <https://www.desmos.com/calculator/uvknv9m18n>)

De la Figura 3.5.a hay algunos conceptos a los que debemos prestar atención:

- 1 Los puntos fueron posicionados dentro del rango $[0,0 : 1,1]$, en concordancia con el rango por defecto de las coordenadas UV.
- 2 Al colocar los puntos sobre el plano cartesiano, se generan naturalmente tres áreas, denotadas en color verde, celeste y naranja.

Las flechas de la imagen de la derecha, representan la distancia entre un píxel (o posición) y el segmento que se forma al unir esos puntos. Ahora bien, ¿cómo unimos los puntos para generar dicho segmento? Mediante una proyección, que matemáticamente luce de la siguiente manera:

$$Proj = \frac{(a - b) \cdot (c - b)}{(\sqrt{c - b \cdot c - b})^2}$$

(3.5.b)

Podríamos traducir esta función a GDSL de la siguiente forma:

```
float projection(vec2 a, vec2 b, vec2 c)
{
    vec2 cb = c - b;
    vec2 ab = a - b;
    return dot(ab, cb) / dot(cb, cb);
}
```

Solo nos falta calcular el segmento como tal. Para ello, determinaremos la distancia entre los píxeles que bordean el segmento y el propio segmento:

```
float segment_sd(vec2 uv, vec2 p0, vec2 p1)
{
    // 1
    float h = projection(uv, p0, p1);
    // 2
    h = clamp(h, 0.0, 1.0);
    // 3
    vec2 p2 = p0 + h * (p1 - p0);
    // 4
    return distance(p2, uv);
}
```

¿Qué ocurre en esta función? Analicemos cada línea:

- ❶ Definimos la proyección de cada píxel entre los puntos **p0** y **p1**.
- ❷ Limitamos la proyección para que solo abarque el tramo entre **p0** y **p1**; de lo contrario, el segmento tendería al infinito.
- ❸ Definimos la posición sobre el segmento usando una combinación lineal (forma $mx + b$).
- ❹ Calculamos la distancia entre cada punto (**uv**) y el punto más cercano **p2** del segmento.

Como utilizaremos estas funciones en la etapa de fragmento, las incluiremos entre los métodos **vertex()** y **fragment()**, tal como se muestra a continuación:

```
...

float projection(vec2 a, vec2 b, vec2 c)
{
    vec2 cb = c - b;
    vec2 ab = a - b;
    return dot(ab, cb) / dot(cb, cb);
}

float segment_sd(vec2 uv, vec2 p0, vec2 p1)
{
    float h = projection(uv, p0, p1);
    h = clamp(h, 0.0, 1.0);
    vec2 p2 = p0 + h * (p1 - p0);
    return distance(p2, uv);
}

void fragment()
{
    vec4 color = texture(TEXTURE, UV);
    COLOR = color;
}
```

Si prestamos atención a los argumentos de **segment_sd()**, notaremos que recibe las coordenadas **UV** y los puntos **p0** y **p1**. Por lo tanto, podemos comenzar declarando e inicializando estos valores en el método **fragment()**. No obstante, debemos considerar que el segmento ilustrado en la Figura 3.5.a fue calculado en coordenadas cartesianas, donde el origen se ubica abajo a la izquierda, mientras que en GDSL el origen se encuentra arriba a la izquierda. Por esta razón, será necesario invertir la coordenada **V** de los **UV** al inicializar el segmento.

```

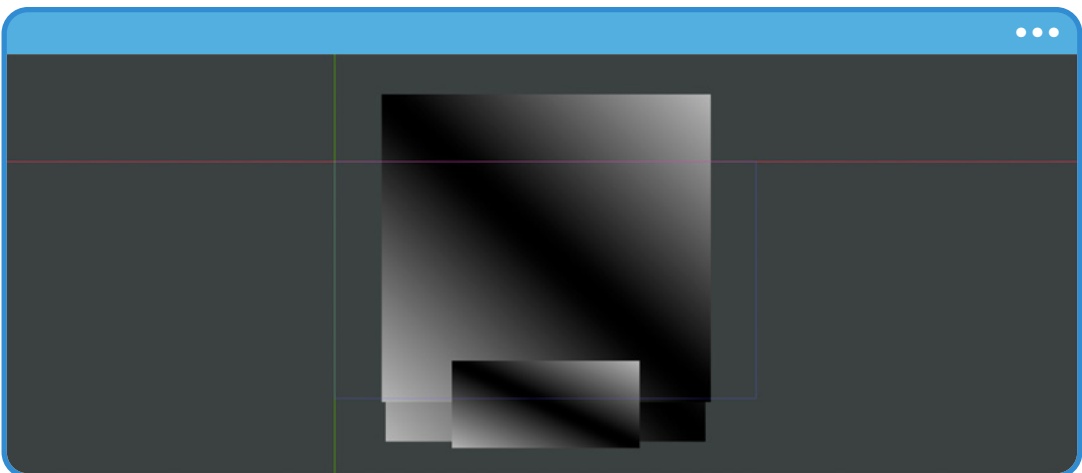
void fragment()
{
    vec4 color = texture(TEXTURE, UV);
    // 1
    vec2 p0 = vec2(1.0, 0.0);
    vec2 p1 = vec2(0.0, 1.0);
    // 2
    vec2 uv = vec2(UV.x, 1.0 - UV.y);
    // 3
    float specular = segment_sd(uv, p0, p1);
    // 4
    COLOR = vec4(vec3(specular), 1.0);
}

```

Como podemos observar:

- ❶ Declaramos los puntos **p0** y **p1** con valores constantes para su visualización.
- ❷ Invertimos la coordenada **v** de **UV** para utilizarla en el segmento.
- ❸ Calculamos el efecto especular a partir del segmento.
- ❹ Asignamos el resultado al color principal para visualizarlo en el Viewport.

Este bloque de código produce el siguiente resultado visual:



(3.5.c Se ha generado un gradiente en base al segmento)

La razón por la que el gradiente se ve negro en el centro se debe a la distancia entre el segmento y los píxeles que lo rodean. Dado que los píxeles más cercanos tienden a un valor cercano a cero, el color resultante es más oscuro. Por el contrario, cuando la distancia aumenta, el color se vuelve más claro.

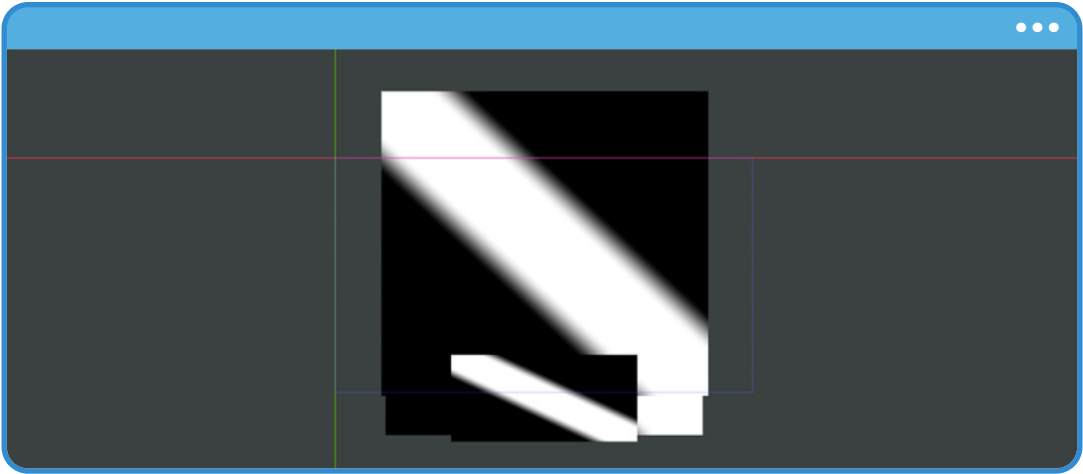
Ahora bien, necesitamos que el segmento sea blanco en el centro y, además, que su forma esté bien definida. Para lograrlo, podemos usar la función `smoothstep()`, la cual nos permite generar una transición suave entre dos valores dentro de un rango específico. Esta función evalúa un valor de entrada y devuelve un resultado interpolado entre 0 y 1, donde 0 representa el inicio del rango y 1 el final. Gracias a este comportamiento, podemos controlar con precisión en qué zona comienza y termina la transición del color, lo que nos permite definir con claridad el contorno del segmento y concentrar el blanco en el centro.

```
void fragment()
{
    ...
    float specular = segment_sd(uv, p0, p1);
    specular = smoothstep(0.2, 0.1, specular);

    COLOR = vec4(vec3(specular), 1.0);
}
```

Nota

Cabe destacar que los valores constantes utilizados en las funciones han sido previamente testeados para obtener el resultado mostrado en la Figura 3.5.d. Sin embargo, siempre es recomendable experimentar con distintos valores, tanto para comprender mejor el comportamiento del efecto como para lograr resultados visuales diferentes.



(3.5.d Se ha definido el cuerpo del efecto especular)

Ahora bien, podríamos mover los puntos de manera manual y aplicar el efecto sobre las imágenes de UI. Sin embargo, para representar adecuadamente el efecto especular necesitamos que el brillo se mueva de forma automática, es decir, incorporando la variable **TIME**.

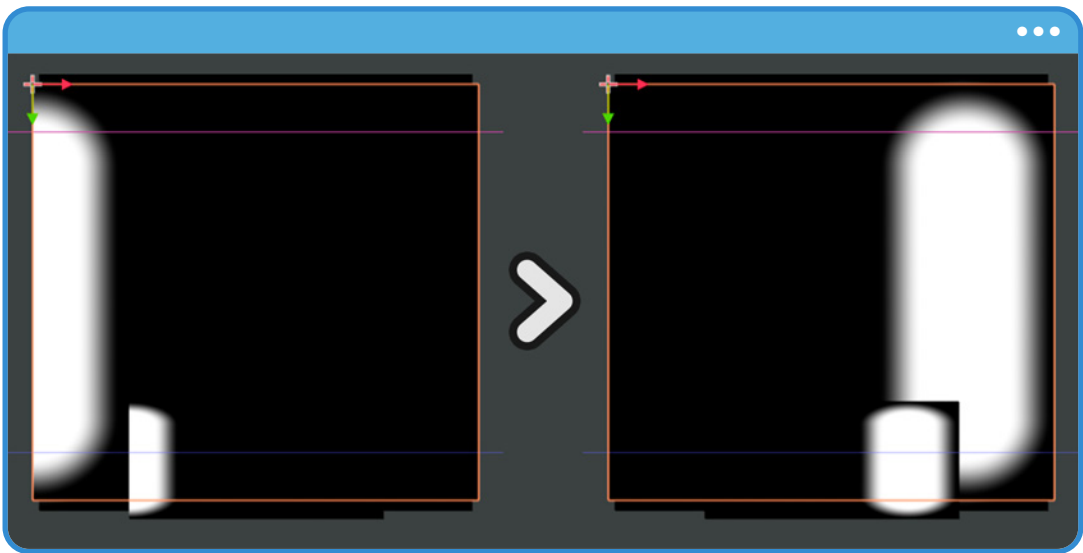
Una manera de animar nuestro brillo especular es hacer que el efecto se repita cada cierto intervalo: que aparezca sobre la imagen de UI, luego desaparezca y reinicie el ciclo de manera constante. Este movimiento puede realizarse horizontalmente, de izquierda a derecha.

```
void fragment()
{
    ...
    // 1
    float time = TIME / 2.0;
    // 2
    float offset = fract(time) * 3.0 - 1.5;
    // 3
    vec2 p0 = vec2(0.5 + offset, 0.8);
    vec2 p1 = vec2(0.5 + offset, 0.2);

    ...
}
```

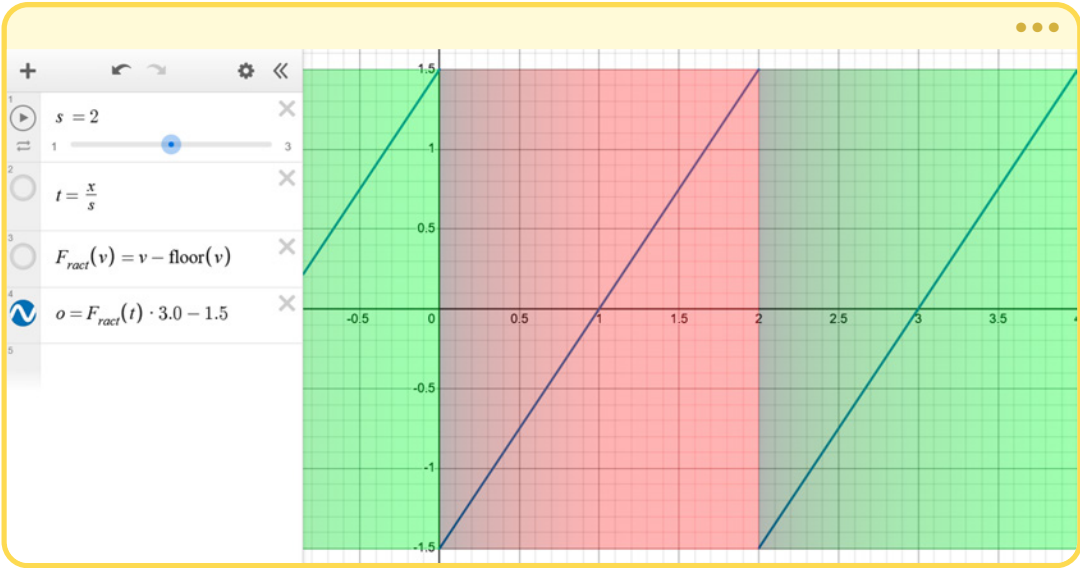
Si prestamos atención al código, observaremos:

- 1 La variable **time** resulta de dividir **TIME** por 2.0, lo que implica que el efecto se repetirá cada dos segundos. Dado que este valor es solo de ejemplo, eventualmente reemplazaremos 2.0 por la propiedad **_SpecularSpeed**.
- 2 Calculamos la parte fraccionaria de **time**; cuando **time** llega a 0.99, vuelve a 0.0. Luego la multiplicamos por 3.0 y restamos 1.5, generando para **offset** un rango que va de -1.5 a 1.5.
- 3 Finalmente, aplicamos **offset** a la componente X de cada punto y ajustamos los valores de la componente Y para visualizar mejor el segmento, ahora centrado en cada imagen. Con ello el segmento entra y sale de la imagen de UI.

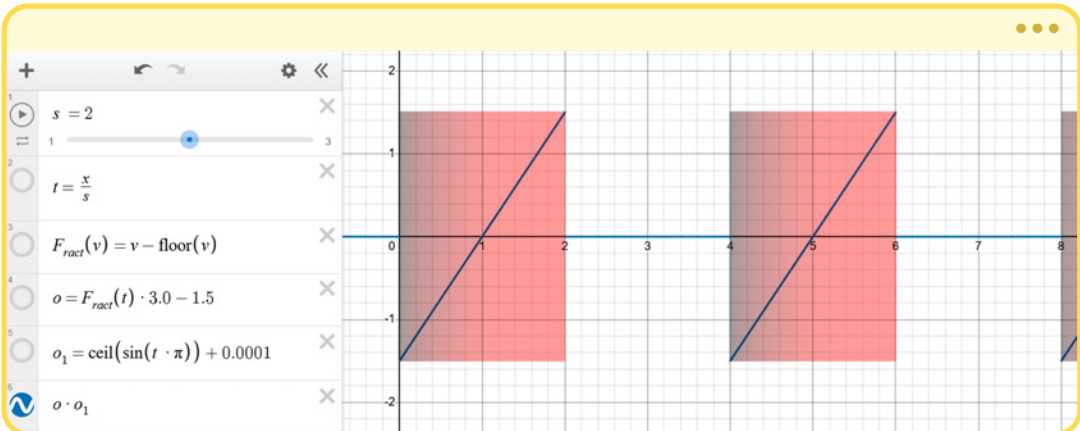


(3.5.e El segmento se mueve desde izquierda a derecha)

Es posible que la frecuencia de aparición del efecto resulte muy repetitiva si lo reproducimos durante mucho tiempo. Podemos limitar su frecuencia; para ello, primero necesitamos visualizar la función que estamos aplicando. Utilizando la interfaz gráfica de Desmos, podemos reproducir, por ejemplo, la función $\text{fract}(\text{time}) * 3.0 - 1.5$ sobre el plano cartesiano.

(3.5.f <https://www.desmos.com/calculator/4o5dww2dxf>)

Como se observa en el gráfico anterior (Figura 3.5.f), usamos dos colores para analizar el comportamiento de la animación del segmento, que se mantiene constante a lo largo del tiempo. Los ciclos se repiten una y otra vez, sin pausas. Para limitarlo, podemos multiplicar el valor de offset por la función $\text{ceil}(\sin(\text{time} \cdot \text{PI})) + 0.0001$, como se muestra a continuación:

(3.5.g <https://www.desmos.com/calculator/a3konqs6nt>)

Esto provoca que un ciclo por medio sea igual a cero, lo que limita la frecuencia del efecto.


```

void fragment()
{
    ...
    float time = TIME / _SpecularSpeed;
    float offset = fract(time) * 3.0 - 1.5;
    // 1
    float offset_mask = ceil(sin(time * PI)) + 0.00001;
    // 2
    offset *= offset_mask;

    vec2 p0 = vec2(0.5 + offset, 0.8);
    vec2 p1 = vec2(0.5 + offset, 0.2);

    ...
    specular = smoothstep(0.2, 0.1, specular);
    // 3
    specular *= offset_mask;

    ...
}

```

Analicemos lo que ocurre en el código:

- ❶ Declaramos una nueva variable llamada **offset_mask**, que obtiene el resultado de **ceil(sin(time * PI)) + 0.0001**.
- ❷ Multiplicamos **offset** por **offset_mask** para asegurar que el segmento (o efecto especular) deje de moverse cada ciclo por medio
- ❸ Multiplicamos el valor de **specular** por **offset_mask** para asegurar que el efecto no sea visible cuando no se está moviendo.

Solo nos queda distorsionar el cuerpo del segmento para adaptarlo a la imagen de UI y que se perciba como un reflejo especular, además de sumarlo a las imágenes. Para ello, podemos seguir estos pasos:

- Distorsionar las coordenadas UV del segmento utilizando uno de los componentes del vector de color.

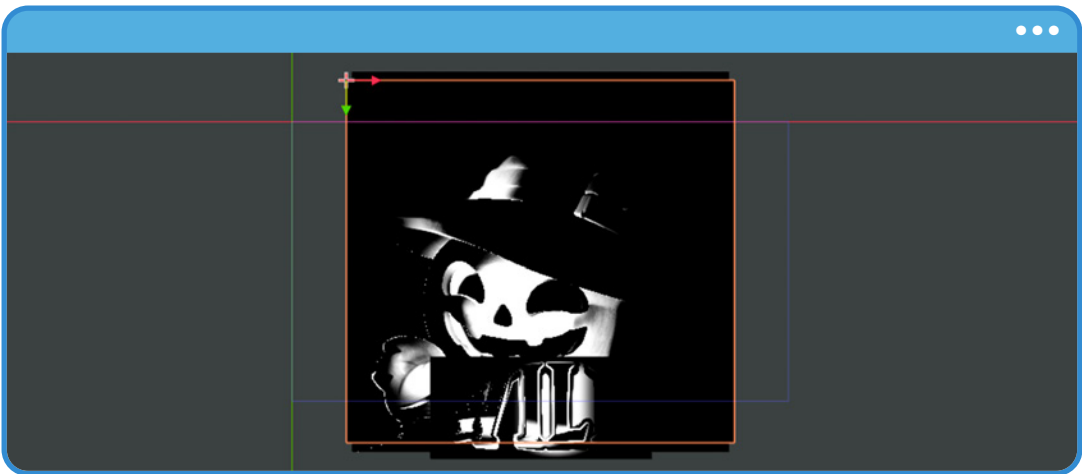
- Transformar el efecto especular a un vector de tres dimensiones (RGB) para poder configurar un color distinto a blanco.
- Agregar el efecto a los componentes RGB del vector de color.

Si, por ejemplo, utilizamos el componente B del color sobre las coordenadas UV del segmento, obtendremos el siguiente resultado:

```
void fragment()
{
    ...

    vec2 uv = vec2(UV.x, 1.0 - UV.y);
    uv += color.b;

    ...
}
```



(3.5.h El efecto especular adquiere la forma de la imagen de UI)

Recordemos que las coordenadas UV funcionan de manera lineal. Cuando aplicamos un color sobre ellas, este alineamiento se altera, generando una nueva forma que se adapta a los valores que estamos introduciendo; en este caso, al componente B del color.

El efecto especular, por su parte, es un valor escalar. Si deseamos aplicar color sobre él, será necesario declarar un nuevo vector RGB que multiplique el efecto por el color que queramos utilizar.

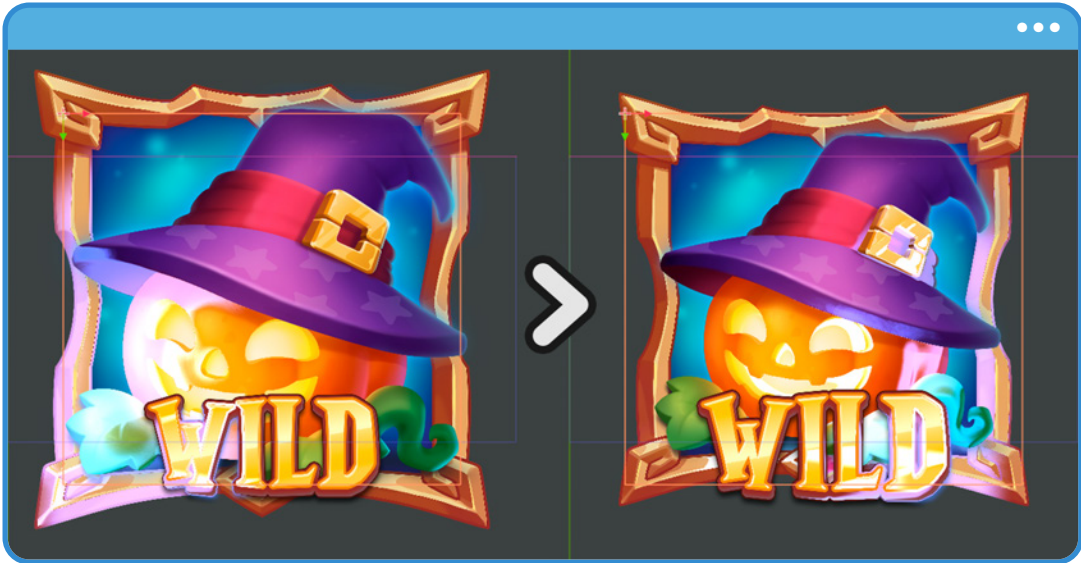
```
void fragment()
{
    ...

    float specular = segment_sd(uv, p0, p1);
    specular = smoothstep(0.2, 0.1, specular);
    specular *= offset_mask;
    // 1
    vec3 specular_color = vec3(specular) * _SpecularColor;
    // 2
    color.rgb += specular_color;
    // 3
    color.rgb = clamp(color.rgb, 0.0, 1.0);
    // 4
    COLOR = color;
}
```

Si prestamos atención al código anterior, observaremos lo siguiente:

- 1 Se declara un nuevo vector llamado **specular_color**, que almacena el resultado de multiplicar el valor de especularidad por la propiedad **_SpecularColor**.
- 2 Sumamos el valor de **specular_color** a los componentes RGB del color base.
- 3 Nos aseguramos de liminar los valores del color para que no excedan 1.0.
- 4 Asignamos el color final a la propiedad **COLOR**.

Esta operación genera el siguiente resultado visual:



(3.5.i Se ha aplicado el efecto especular sobre la imagen)

3.6 Introducción a las rotaciones.

Cuando hablamos de rotaciones en gráficos computacionales, nos referimos principalmente a dos representaciones matemáticas fundamentales: matrices y cuaterniones. En la sección 1.10 introdujimos el concepto de matrices y las utilizamos para implementar un efecto Billboard en nuestro shader. Sin embargo, hasta ahora no hemos explorado cómo estas estructuras pueden emplearse directamente para generar rotaciones en el espacio tridimensional.

Recordemos que las matrices no son más que arreglos organizados de valores que transforman vectores cuando se multiplican por ellos. En el caso de las rotaciones, estos valores incluyen las funciones seno y coseno, ubicadas en posiciones específicas de la matriz. La disposición de estos términos determina el eje en torno al cual se realiza la rotación.

Por ejemplo, una rotación alrededor del eje x modifica las coordenadas yz , mientras que una rotación alrededor del eje y altera xz , y una rotación alrededor de z afecta xy . Esta estructura nos permite construir matrices de rotación para cada eje individualmente o combinarlas para obtener rotaciones más complejas.

Matemáticamente, las matrices de rotación en el espacio euclídeo tridimensional se expresan de la siguiente forma:

Para el eje x (Phi),

$$R_x(\phi) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos \theta & -\sin \theta \\ 0 & \sin \theta & \cos \theta \end{bmatrix}$$

(3.6.a)

Para el eje y (Theta),

$$R_y(\theta) = \begin{bmatrix} \cos \theta & 0 & \sin \theta \\ 0 & 1 & 0 \\ -\sin \theta & 0 & \cos \theta \end{bmatrix}$$

(3.6.b)

Para el eje z (Psi),

$$R_z(\psi) = \begin{bmatrix} \cos \theta & -\sin \theta & 0 \\ \sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

(3.6.c)

Estas matrices producen rotaciones en sentido antihorario cuando se observan siguiendo la convención de la mano derecha, es decir, con el pulgar apuntando en la dirección positiva del eje de rotación.

A pesar de que estas expresiones pueden parecer complejas al principio, su implementación en GDSL es bastante intuitiva:

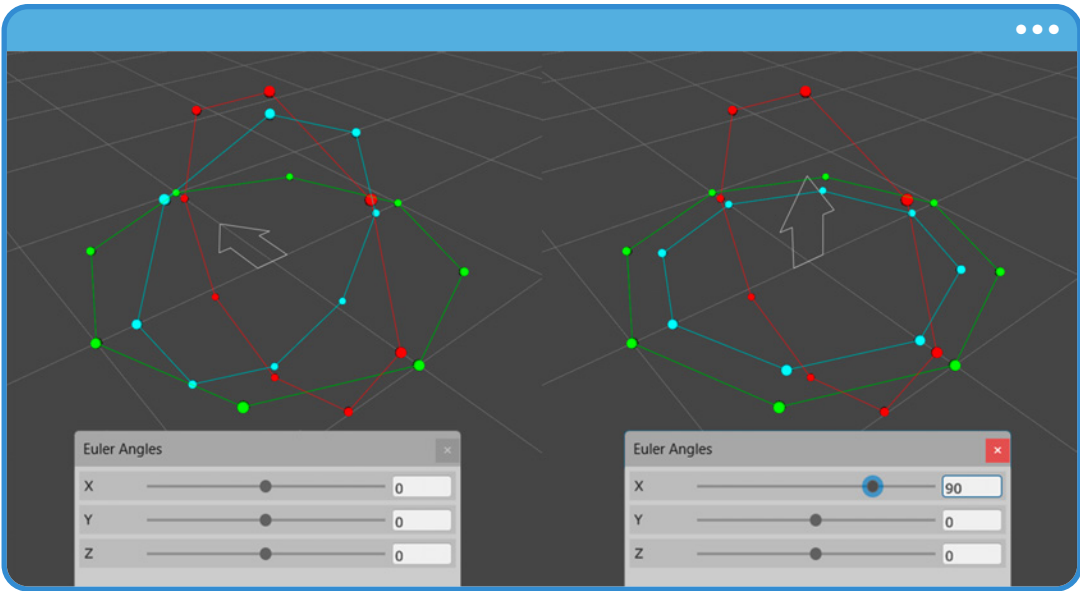
```
// x-axis rotation
#define RX(a) mat3(          \
    vec3(1, 0, 0),          \
    vec3(0, cos(a), -sin(a)), \
    vec3(0, sin(a), cos(a)))

// y-axis rotation
#define RY(a) mat3(          \
    vec3(cos(a), 0, sin(a)), \
    vec3(0, 1, 0),           \
    vec3(-sin(a), 0, cos(a)))

// z-axis rotation
#define RZ(a) mat3(          \
    vec3(cos(a), -sin(a), 0), \
    vec3(sin(a), cos(a), 0),  \
    vec3(0, 0, 1))
```

Ahora bien, surge una pregunta importante: ¿por qué existen dos formas de representar rotaciones (matrices y cuaterniones) si ambas cumplen el mismo propósito? La respuesta se encuentra en un fenómeno conocido como efecto Gimbal (Gimbal Lock).

Este efecto ocurre cuando utilizamos rotaciones en ángulos de **Euler**, es decir, cuando aplicamos rotaciones secuenciales alrededor de los ejes *xyz* mediante matrices. El problema radica en que las rotaciones no conmutan: el orden en que se multiplican las matrices **sí importa**. En ciertas configuraciones, dos de los tres ejes de rotación pueden alinearse, reduciendo los tres grados de libertad originales a solo dos. Este fenómeno, conocido como singularidad, provoca que el objeto pierda la libertad de rotar en todas sus direcciones, generando comportamientos inesperados.



(3.6.d A la izquierda, tres anillos concéntricos; a la derecha, el efecto Gimbal)

Aquí es donde entran en juego los cuaterniones. A diferencia de las matrices de rotación tradicionales, los cuaterniones representan rotaciones en un espacio de cuatro dimensiones, lo que les permite evitar completamente la singularidad del efecto Gimbal y mantener un comportamiento estable sin importar la orientación.

Un cuaternión puede representarse en la forma:

$$Q = w + xi + yj + zk$$

(3.6.e)

donde $wxyz$ son números reales, y ijk son unidades imaginarias que cumplen las relaciones:

$$i^2 = j^2 = k^2 = ijk = -1$$

(3.6.f)

En el contexto de las rotaciones, un cuaternión de un eje de rotación unitario puede expresarse como $u = (u_x, u_y, u_z)$:

$$Q = \left(\cos \left(\frac{\theta}{2} \right), \sin \left(\frac{\theta}{2} \right) u \right)$$

(3.6.g)

Donde u es un vector unitario que representa el eje de rotación, y θ el ángulo en radianes. Esta forma permite describir cualquier rotación tridimensional sin depender de los tres ángulos de Euler, evitando así el alineamiento de ejes.

En las siguientes secciones implementaremos rotaciones tanto con matrices como con cuaterniones, aplicándolas sobre primitivas para crear diferentes efectos visuales y comprender en profundidad cómo funcionan en la práctica.

3.7 Implementando una matriz de rotación.

En esta sección realizaremos dos ejercicios principales. Primero, nos introduciremos en los ángulos de Euler para implementar una rotación tridimensional sobre un cubo. Luego, utilizando las tres matrices que estudiamos en la sección anterior, extenderemos el shader **world_space** presentado en la sección 1.6 del Capítulo 1, de modo que el árbol no solo cambie de color según su posición, sino que también pueda rotar en sus distintos ejes.

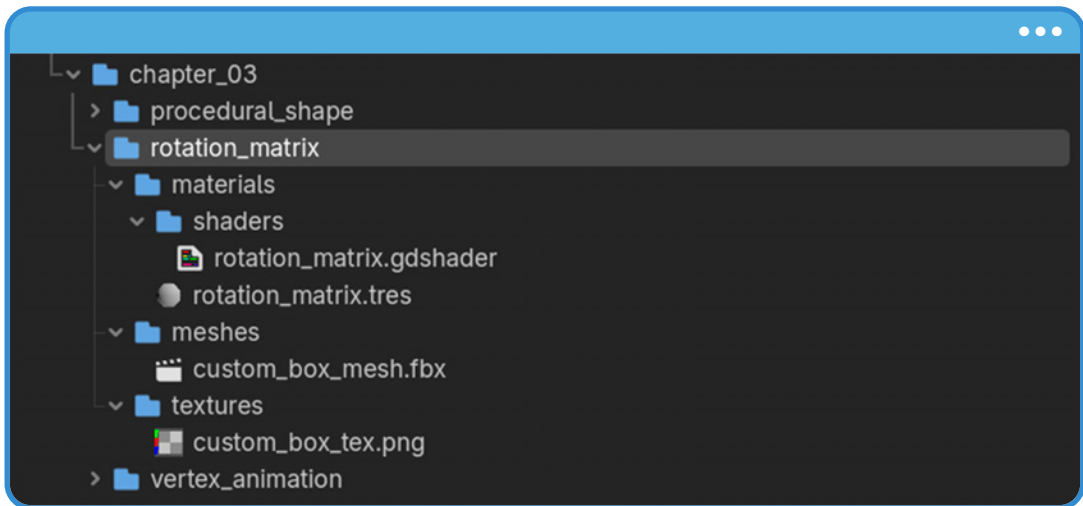
Para comenzar, iremos a nuestro proyecto y, dentro del **Capítulo 3**, crearemos una nueva carpeta llamada **rotation_matrix**.

En este punto del libro, ya debemos estar familiarizados con la estructura del proyecto que venimos utilizando, incluyendo la creación de materiales y shaders. A partir de ahora omitiremos la explicación detallada de estos pasos y nos centraremos únicamente en su implementación.

Como primer ejercicio, crearemos un nuevo **ShaderMaterial** y su correspondiente **Shader**, ambos con el nombre **rotation_matrix**. Siguiendo la misma organización utilizada en capítulos anteriores, añadiremos las carpetas necesarias dentro del proyecto antes de comenzar con el desarrollo del efecto.

En esta ocasión trabajaremos con un modelo 3D personalizado, un **BoxMesh** llamado **custom_box_mesh**. Este archivo **.fbx**, junto con su textura, se encuentra disponible en la carpeta descargable adjunta al libro, dentro de la sección **rotation_matrix**.

Si hemos seguido los pasos correctamente, nuestro proyecto debería verse de la siguiente manera:



(3.7.a Se ha agregado una nueva carpeta al proyecto)

Nota

Recuerda que para realizar este ejercicio, debemos asignar el shader al material, y luego aplicar ese material al **custom_box_mesh** mediante la propiedad **Material Override**.

El primer paso dentro de nuestro shader **rotation_matrix** será implementar un **mapa base** para visualizar el color de albedo sobre el cubo personalizado que tenemos en la ventana Scene. Esto nos permitirá comprobar tanto que el material está funcionando correctamente antes de añadir las rotaciones como visualizar los distintos ejes 3D del modelo. El código inicial es el siguiente:

```

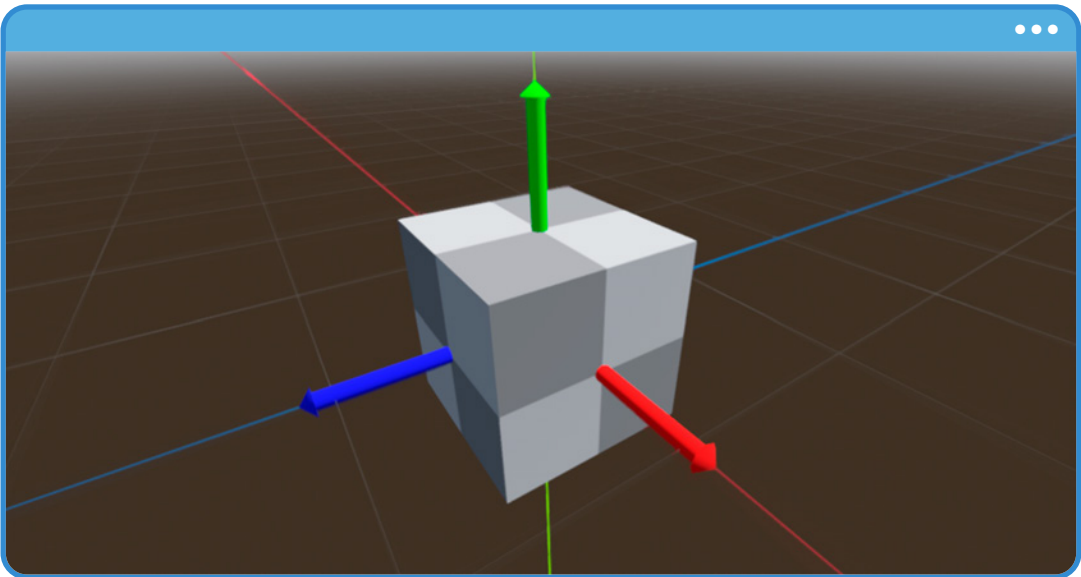
shader_type spatial;

uniform sampler2D _BaseMap : source_color;

void vertex() { ... }

void fragment()
{
    vec3 albedo = texture(_BaseMap, UV).rgb;
    ALBEDO = albedo;
}

```



(3.7.b Se ha asignado la textura al cubo personalizado)

A continuación, implementaremos las matrices necesarias para rotar el cubo en cada uno de sus ejes. Para ello, seguiremos las guías presentadas en las Figuras 3.6.a, 3.6.b y 3.6.c de la sección anterior, que describen la estructura de cada matriz de rotación.

```

uniform sampler2D _BaseMap : source_color;
// 1
uniform vec3 _Angles;

// 2
#define RX(a) mat3(           \
    vec3(1, 0, 0),           \
    vec3(0, cos(a), -sin(a)), \
    vec3(0, sin(a), cos(a)))

// 3
#define RY(a) mat3(           \
    vec3(cos(a), 0, sin(a)),  \
    vec3(0, 1, 0),           \
    vec3(-sin(a), 0, cos(a)))

// 4
#define RZ(a) mat3(           \
    vec3(cos(a), -sin(a), 0),  \
    vec3(sin(a), cos(a), 0),   \
    vec3(0, 0, 1))

void vertex() { ... }

```

Analizamos lo que ocurre en el código:

- ❶ Declaramos un nuevo vector tridimensional llamado **_Angles**, encargado de definir el ángulo de rotación para cada eje. Este valor podrá modificarse dinámicamente desde el **Inspector**, lo que nos permitirá controlar las rotaciones en tiempo real.
- ❷ Definimos la matriz **RX**, responsable de rotar los vértices en el eje **x**, siguiendo la estructura mostrada en la Figura 3.6.a.
- ❸ Definimos la matriz **RY**, responsable de la rotación sobre el eje **y**, según la Figura 3.6.b.
- ❹ Finalmente, declaramos la matriz **RZ**, que controla la rotación sobre el eje **z**, conforme a la Figura 3.6.c.

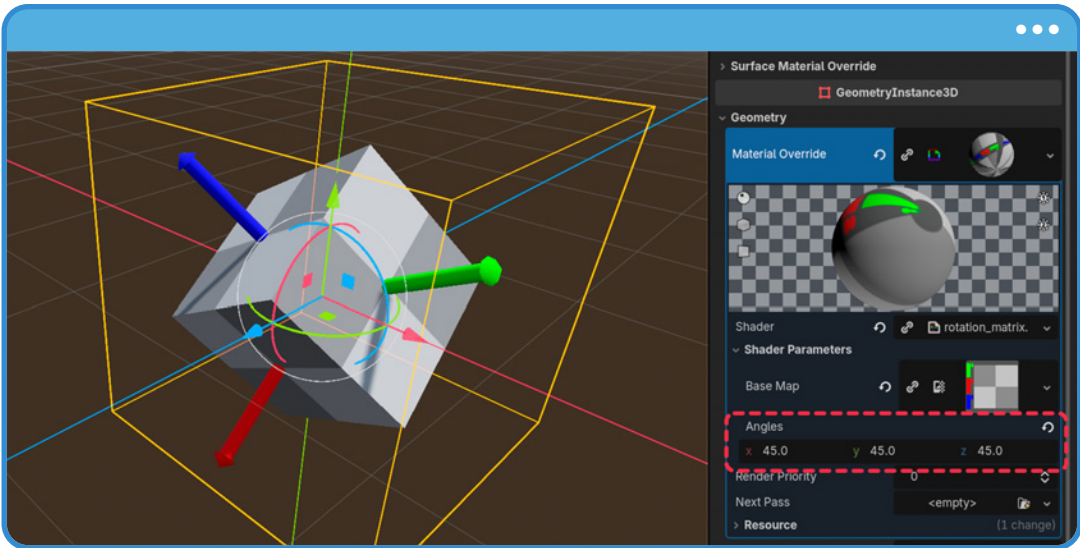
Una vez declaradas las tres matrices, el siguiente paso será utilizarlas para transformar los vértices del mesh. Para ello, iremos al método `vertex()` y aplicaremos la rotación compuesta mediante la multiplicación de las matrices de Euler:

```
void vertex()
{
    // 1
    vec3 angles = _Angles * (PI / 180.0);
    // 2
    mat3 EULER_ZXY = RY(angles.y) * RX(angles.x) * RZ(angles.z);
    // 3
    VERTEX = EULER_ZXY * VERTEX;
    // 4
    NORMAL = normalize(EULER_ZXY * NORMAL);
}
```

Veamos qué está ocurriendo en cada paso:

- 1 Creamos un nuevo vector tridimensional llamado **angles**, que convierte los valores de **_Angles** desde grados a radianes, ya que las funciones trigonométricas del shader trabajan en esta unidad.
- 2 Declaramos la matriz **EULER_ZXY**, que aplica las rotaciones en el orden **Z → X → Y**. Este orden no es arbitrario: en rotaciones de Euler, el resultado depende directamente del orden de multiplicación de las matrices. Aquí, comenzamos rotando sobre el eje **z**, luego el **x** y finalmente el **y**, siguiendo la convención **ZXY**.
- 3 Aplicamos la matriz resultante a los vértices del mesh en espacio local, de modo que toda la geometría se oriente según los nuevos ángulos definidos.
- 4 Rotamos las normales del modelo utilizando la misma matriz y normalizamos su dirección. Esto es fundamental, ya que si no actualizamos las normales, la iluminación del objeto no coincidirá con su nueva orientación en el espacio.

Si ahora regresamos al Inspector y modificamos los valores del parámetro **_Angles**, veremos que nuestro cubo personalizado comienza a rotar en tiempo real, siguiendo las transformaciones definidas por las matrices de Euler.



(3.7.c Se ha rotado el cubo en 45° en cada eje)

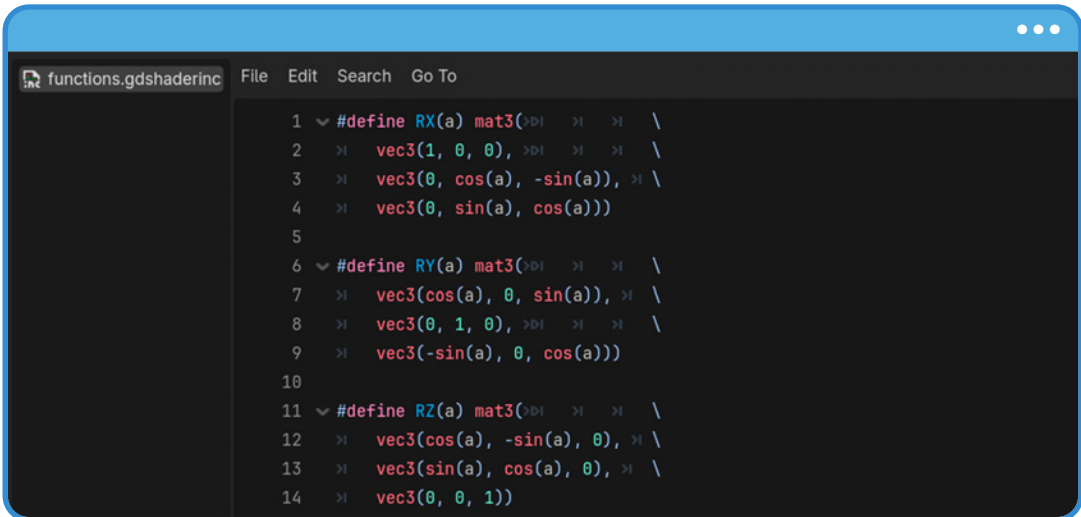
Si rotamos el cubo 45° en cada eje, observaremos que el sistema de rotación funciona correctamente en la mayoría de los casos. Sin embargo, esta implementación presenta una limitación importante. Si rotamos el cubo 90° sobre el eje x y luego intentamos modificar los ángulos yz , notaremos que el objeto pierde un grado de libertad. Ambos ejes parecen rotar el cubo en la misma dirección. Este fenómeno se conoce como efecto Gimbal.

Nota

En Godot, podemos configurar el modo de rotación directamente desde la propiedad **Transform > Rotation Edit Mode** de cada nodo. Por defecto, el editor utiliza el modo **Euler**, pero también es posible cambiarlo a **Quaternion**, lo que evita el problema del Gimbal Lock al representar rotaciones acumulativas mediante una estructura matemática diferente.

Aun con esta limitación, las matrices de rotación que implementamos siguen siendo extremadamente útiles. Podemos aplicarlas de forma individual o combinada para crear efectos dinámicos dentro del shader. De hecho, a continuación las reutilizaremos para extender el efecto desarrollado previamente en el shader **world_space**. Como emplearemos las mismas definiciones de matrices, el primer paso será centralizarlas dentro de nuestro proyecto.

Para lograrlo, abriremos el archivo **functions.gdshaderinc**, creado en el Capítulo 2, y copiaremos allí las tres matrices (**RX**, **RY** y **RZ**). Esto nos permitirá acceder a ellas desde cualquier shader del proyecto sin necesidad de volver a definir las.



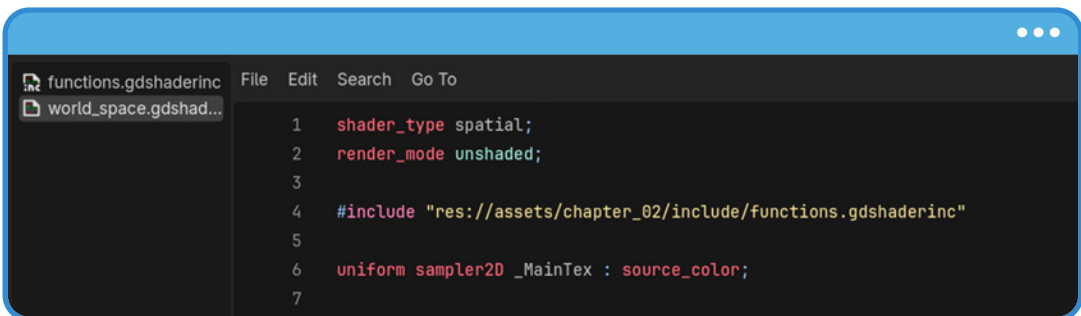
```

1  #define RX(a) mat3( >Dl  >I  >I  \
2  >I  vec3(1, 0, 0), >Dl  >I  >I  \
3  >I  vec3(0, cos(a), -sin(a)), >I  \
4  >I  vec3(0, sin(a), cos(a)))
5
6  #define RY(a) mat3( >Dl  >I  >I  \
7  >I  vec3(cos(a), 0, sin(a)), >I  \
8  >I  vec3(0, 1, 0), >Dl  >I  >I  \
9  >I  vec3(-sin(a), 0, cos(a)))
10
11 #define RZ(a) mat3( >Dl  >I  >I  \
12 >I  vec3(cos(a), -sin(a), 0), >I  \
13 >I  vec3(sin(a), cos(a), 0), >I  \
14 >I  vec3(0, 0, 1))

```

(3.7.d Se han incluido las matrices en **functions.gdshaderinc**)

Luego, iremos a nuestro shader **world_space** e incluiremos una referencia a este archivo compartido, como se muestra en la siguiente figura:



```

1  shader_type spatial;
2  render_mode unshaded;
3
4  #include "res://assets/chapter_02/include/functions.gdshaderinc"
5
6  uniform sampler2D _MainTex : source_color;
7

```

(3.7.e Se ha incluido **functions** en **world_space**)

El último paso consistirá en rotar dinámicamente los árboles de la escena para que cada instancia presente una orientación única. Para lograrlo, podemos utilizar la función **hash33()** en conjunto con la variable **NODE_POSITION_WORLD**. Esta combinación nos permitirá generar un valor pseudoaleatorio a partir de la posición del objeto en el mundo y convertirlo en un ángulo de rotación en radianes. De esta manera, cada repetición del árbol adoptará

una rotación distinta según su ubicación en el Viewport, evitando patrones repetitivos y añadiendo una apariencia más natural al entorno.

```
void vertex()
{
    // 1
    vec3 vertex_os = VERTEX;
    // 2
    vec3 vertex_ws = (MODEL_MATRIX * vec4(vertex_os, 1.0)).xyz;
    // 3
    float gradient_ws = clamp(0.0, 1.0, vertex_ws.y - NODE_POSITION_WORLD.y);
    // 4
    float angle_map = hash33(floor(NODE_POSITION_WORLD)).y;
    angle_map *= gradient_ws;
    // 5
    float st = (sin(TIME + angle_map * 2.0) * 0.05 + 0.05);
    // 6
    float ct = (cos(TIME + angle_map * 2.0) * 0.05 + 0.05);
    // 7
    mat3 EULER_ZXY =
        RY(angle_map * TAU) *
        RX((angle_map * 0.1345) - st) *
        RZ((angle_map * 0.1567) - ct);
    // 8
    VERTEX = EULER_ZXY * vertex_os * clamp(angle_map, 0.4, 1.0);
}
```

Analicemos lo que ocurre paso a paso:

- ❶ Inicializamos la variable **vertex_os** con los vértices en espacio local. Este será nuestro punto de partida antes de aplicar cualquier transformación.
- ❷ Transformamos los vértices a espacio global utilizando la matriz **MODEL_MATRIX**, lo que nos permite relacionar su posición con las coordenadas absolutas del mundo.
- ❸ Creamos un gradiente vertical, restando la posición Y del nodo **NODE_POSITION_WORLD** a la posición Y del vértice **vertex_ws**, y limitando el resultado entre 0 y 1. De esta forma, los valores cercanos al suelo se aproximan a 0, mientras que los más altos se acercan a 1.

- 4 Generamos un valor pseudoaleatorio llamado `angle_map` a partir de la posición del objeto en el mundo mediante `hash33()`. Este valor se multiplica por `gradient_ws` para asegurar que la base de los árboles permanezca estable, evitando deformaciones en las raíces.
- 5 Agregamos una oscilación senoidal utilizando `sin(TIME)`, que introduce un movimiento suave y periódico en la rotación.
- 6 Añadimos una oscilación cosenoidal complementaria, que genera un desfase temporal y otorga un comportamiento más orgánico al movimiento.
- 7 Construimos la matriz de rotación `EULER_ZXY`, combinando rotaciones sobre los tres ejes con los valores aleatorios y ondulatorios. Esto produce una leve variación continua en la orientación de cada árbol, simulando el efecto del viento.
- 8 Aplicamos la matriz de rotación a los vértices locales y los escalamos mediante `clamp(angle_map, 0.4, 1.0)`, lo que añade variaciones sutiles en tamaño entre cada árbol, reforzando la sensación de naturalidad en la escena.

Si regresamos al Viewport, veremos que los árboles no solo cambian de color, sino que también se rotan y escalan dinámicamente según su posición en el mundo, generando un paisaje más orgánico.



(3.7.f) Los árboles rotan y se escalan según su posición en el mundo)

Nota

Aunque este ejercicio ilustra de manera práctica cómo combinar coordenadas, matrices y funciones matemáticas para crear rotaciones y escalas, no es recomendable aplicar estas operaciones directamente en un shader para producción.

Estas transformaciones se ejecutan por vértice en cada fotograma, lo que puede afectar el rendimiento en modelos con alta densidad de polígonos. En casos reales, es preferible implementar este mismo comportamiento desde un script que se ejecute una sola vez al iniciar el juego, calculando las rotaciones y escalas en la CPU, y almacenándolas en las transformaciones del objeto.

3.8 Implementando cuaterniones.

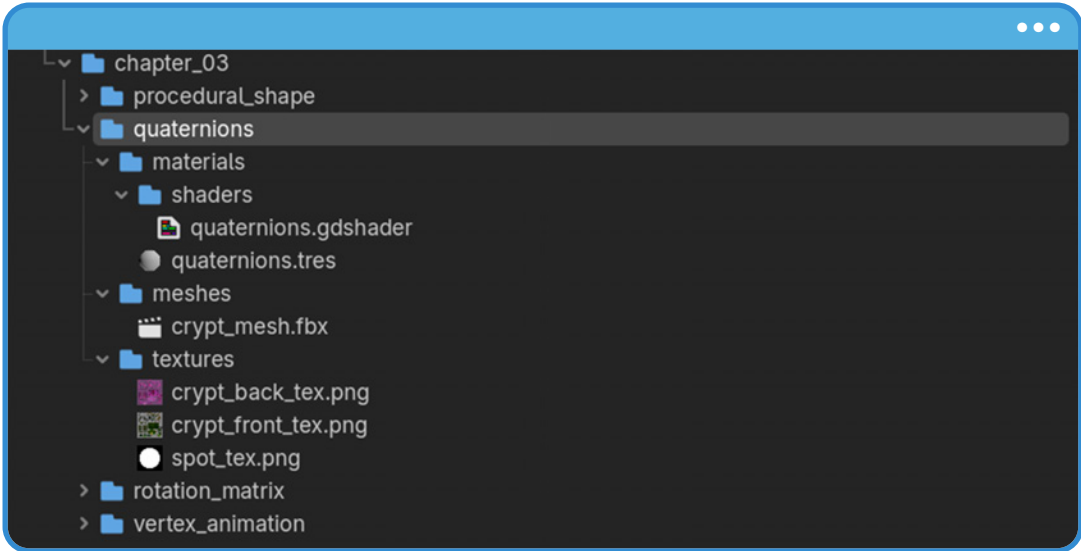
Continuando con nuestro análisis de rotaciones, en esta sección implementaremos cuaterniones en el shader para proyectar una textura sobre un modelo 3D que siga la dirección de la iluminación de la escena.

Para comenzar, iremos al proyecto y, dentro de la carpeta **chapter_03**, creamos una nueva carpeta llamada **quaternions**. Para mantener todo organizado, añadimos las siguientes subcarpetas dentro de esta última:

- **materials**
- **shaders.**
- **textures.**
- **meshes.**

Para este ejercicio, será necesario descargar los archivos adjuntos a este libro y guardarlos en sus respectivas carpetas dentro del proyecto, además de crear un shader y un material asociado a la sección.

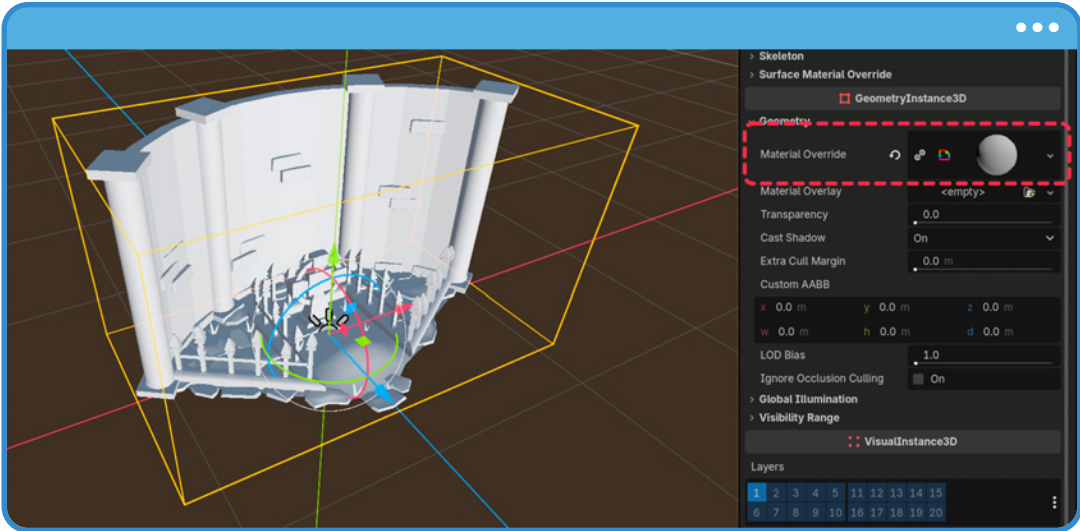
Si todo ha marchado correctamente, el proyecto debería lucir de la siguiente manera:



(3.8.a Se han agregado los archivos al proyecto)

A continuación configuramos la nueva escena. Iniciaremos arrastrando el archivo **crypt_mesh.fbx** al **Viewport**, repitiendo el proceso realizada en la sección 1.4: donde marcamos el modelo como **Make Unique**. Nos aseguramos de guardar el recurso en la carpeta meshes de la sección. Para identificarlo con facilidad, lo nombraremos **Crypt.tres**. Este paso asegura que el **MeshInstance3D** asociado quede como hijo del **Node3D** en la escena, evitando referencias frágiles a nodos hijos desde el script que crearemos más adelante.

Con el objeto **Crypt** en la escena, asignamos el shader **quaternions** a su material y, finalmente, asignamos ese material al objeto en la propiedad **Material Override**, como se muestra a continuación.



(3.8.b Se ha asignado el material **quaternion** al modelo 3D)

Será necesario añadir una **DirectionalLight3D** a la escena y verificar que su posición y rotación estén en (0, 0, 0). Así podremos vincular correctamente el efecto con este objeto más adelante.

Iniciaremos el desarrollo declarando las siguientes propiedades globales en el shader:

```
shader_type spatial;
// 1
render_mode unshaded;
// 2
uniform sampler2D _FrontMap : source_color, repeat_disable;
// 3
uniform sampler2D _BackMap : source_color, repeat_disable;
// 4
uniform sampler2D _SpotMap : source_color, repeat_disable;
// 5
uniform vec3 _LightRotation;
// 6
uniform vec3 _LightPosition;
```

Como podemos observar:

- 1 Desactivamos la iluminación sobre el modelo con **unshaded**.
- 2 Usaremos dos texturas: **_FrontMap** es la textura principal y cubre la mayor parte del modelo.
- 3 **_BackMap** es la textura secundaria; la proyectaremos dinámicamente “por detrás” de la principal.
- 4 **_SpotMap** servirá como máscara para interpolar entre las dos texturas anteriores.
- 5 **_LightRotation** será la rotación actual de la luz direccional (la vincularemos desde un script).
- 6 **_LightPosition** será la posición actual de la luz direccional (también la pasaremos desde un script).

A modo de visualización, inicializaremos las texturas en la etapa de fragmento y, de manera momentánea, asignaremos la textura **_FrontMap** como **ALBEDO**.

```
void fragment()
{
    vec3 front_map = texture(_FrontMap, UV).rgb;
    vec3 back_map = texture(_BackMap, UV).rgb;
    float spot_map = texture(_SpotMap, UV).r;

    ALBEDO = front_map;
}
```

Nuestro siguiente objetivo es proyectar **_SpotMap** en espacio global usando las componentes XY de los vértices, de modo que:

- 1 La textura se proyecte en el mundo.
- 2 Siga la dirección por defecto de la iluminación.

Ahora bien, queremos una proyección global por dos razones:

- 1 Necesitamos que la textura siga la posición y rotación de la luz direccional.
- 2 Además, la textura debe conservar sus proporciones (si usamos UV directamente, el círculo podría deformarse por el mapeo UV).

Para ello, haremos lo siguiente:

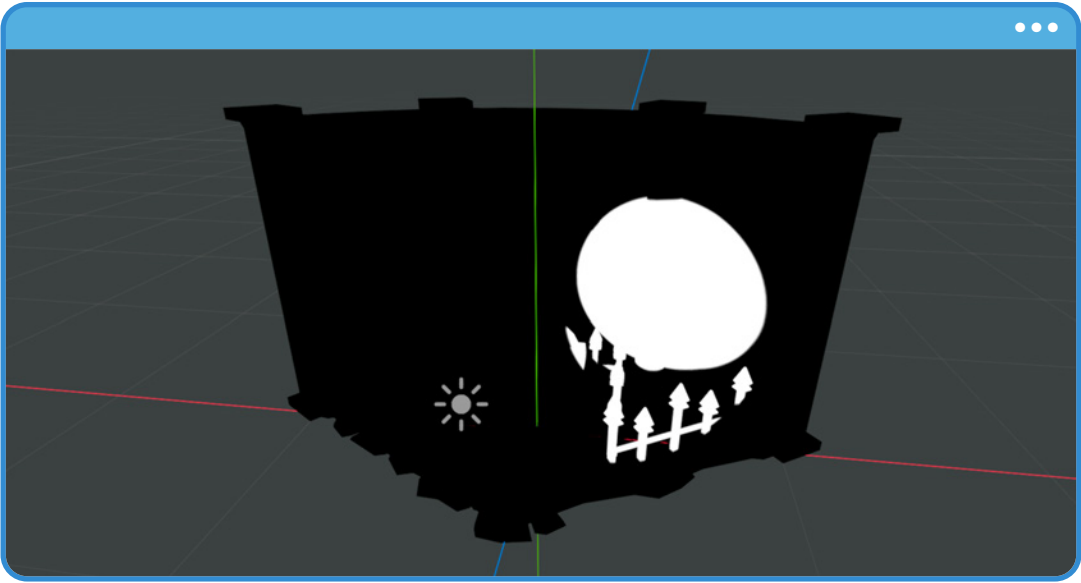
```
void fragment()
{
    // 1
    vec3 vertex_ws = (INV_VIEW_MATRIX * vec4(VERTEX, 1.0)).rgb;
    // 2
    vec3 position_ws = vertex_ws - _LightPosition;

    vec3 front_map = texture(_FrontMap, UV).rgb;
    vec3 back_map = texture(_BackMap, UV).rgb;
    //3
    float spot_map = texture(_SpotMap, position_ws.xy).r;
    // 4
    ALBEDO = vec3(spot_map);
}
```

¿Qué está ocurriendo en el código?

- 1 Transformamos los vértices desde espacio de vista a espacio global.
- 2 Restamos la posición de la luz direccional y almacenamos el resultado en **position_ws**.
- 3 Calculamos **spot_map** usando **position_ws.xy** en lugar de UV.
- 4 Proyectamos **spot_map** como **ALBEDO** para visualizar la máscara.

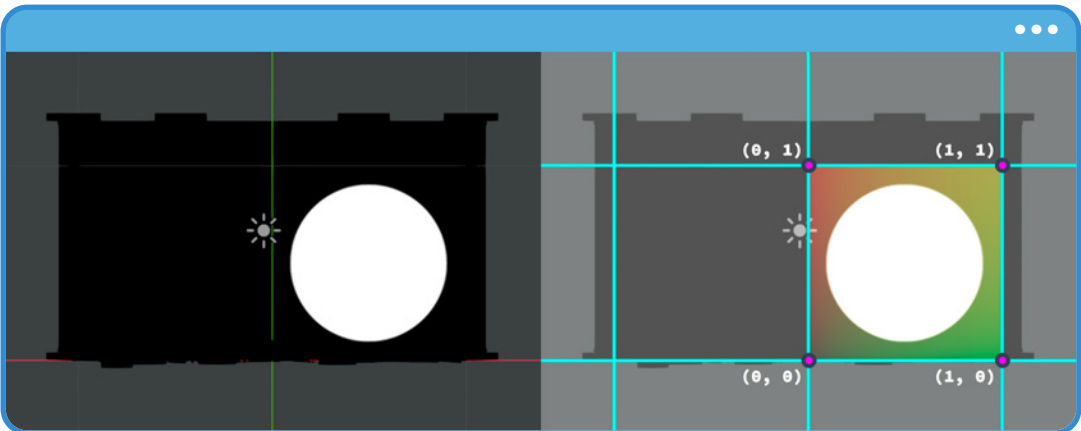
Asumiendo que **spot_tex.png** está asignada en el material, al volver al Viewport veremos el círculo proyectado sobre el modelo siguiendo la dirección de la luz.



(3.8.c El círculo se proyecta en espacio global)

No obstante, la posición actual del círculo no coincide aún con la posición de la luz que se encuentra actualmente en $(0, 0, 0)$. Esto se debe a que `spot_map`, está utilizando las coordenadas $[0,0 : 1,1]$ del mundo, análogo las coordenadas UV.

Prestemos atención a la siguiente referencia para entender el concepto.



(3.8.d Proyección de la textura en espacio global)

Para centrar la proyección, basta con restar 0.5 a las componentes XY de **position_ws**. Sin embargo, este proceso lo haremos más adelante; primero vincularemos la luz direccional con el shader. Para ello, dentro de la carpeta **shaders**, presionamos clic derecho:

➤ Create New > Script.

Nombraremos el script como **light_to_shader.gd**, y agregaremos las siguientes líneas de código:

```
// 1
@tool
extends Node
// 2
@onready var light = $DirectionalLight3D
@onready var mesh = $Crypt
@onready var shader_mat := mesh.material_override as ShaderMaterial

func _process(_delta):
    // 3
    if Engine.is_editor_hint():
        // 4
        shader_mat.set_shader_parameter("_LightPosition", light.position)
        shader_mat.set_shader_parameter("_LightRotation", light.rotation)
```

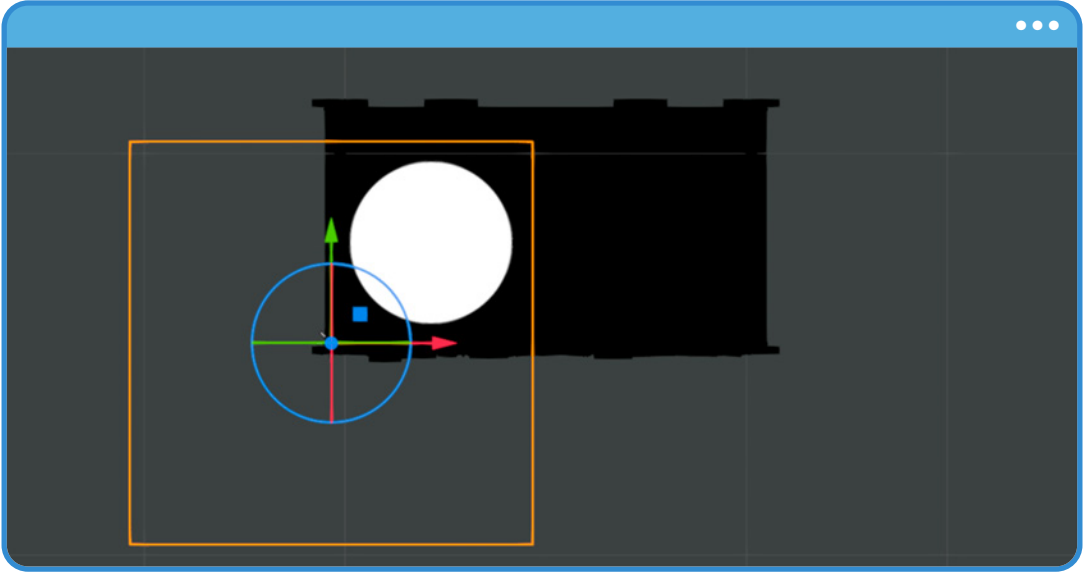
¿Qué estamos haciendo?

- ➊ Ejecutamos el código en el **Editor** para ver cambios en tiempo real.
- ➋ Referenciamos los nodos de la ventana **Scene** que necesitamos.
- ➌ Ejecutamos el código solo si nos encontramos en el Editor.
- ➍ Enviamos la posición y rotación de la luz a las propiedades **_LightPosition** y **_LightRotation** del shader.

Nota

Cabe destacar que el código de ejemplo no está optimizado. Este se ejecuta por cada fotografía, independientemente de si la posición o rotación de la luz cambia. No obstante, funciona perfectamente para el ejercicio.

Si regresamos al Viewport, al mover la luz la máscara se moverá con ella. Aún se ve desfasada (no hemos corregido el centrado en el shader), pero funciona. Si rotamos la luz, notaremos que la máscara no rota: aquí es donde entra la implementación con cuaterniones.



(3.8.e La máscara sigue la posición de la luz)

Nota

Es posible que el script no funcione inmediatamente cuando cambiamos la posición de la luz. En ese caso, se recomienda reiniciar Godot.

Cuando trabajamos con cuaterniones, es recomendable seguir este flujo para obtener una rotación:

- Declararlos.
- Crearlos.
- Conjugarlos.
- Multiplicarlos.
- Aplicar la rotación (transformar).

Con esto en mente, declaramos nuestro cuaternión mediante una **struct**:

```
struct quaternion
{
    float x;
    float y;
    float z;
    float w;
};
```

Las variables o miembros que observamos dentro del **quaternion**, corresponden a las componentes intrínsecas de la definición de este tipo de objeto, como pudimos ver en la Figura 3.6.e. Para crearlos, podemos utilizar la siguiente función:

```
struct quaternion { ... }

quaternion create(float angle, vec3 axis)
{
    float s = sin(angle/2.0);
    float c = cos(angle/2.0);
    vec3 v = normalize(axis) * s;

    quaternion q = quaternion(v.x, v.y, v.z, c);
    return q;
}
```

Del código anterior, hay tres puntos claves que debemos prestar atención:

- 1 El argumento **axis** corresponde al vector de rotación; a los ejes *ijk*.
- 2 El vector **v** genera las tres componentes vectoriales del cuaternión a partir de un eje unitario.
- 3 El cuaternión **q** se construye en formato estándar *xyzw*.

Nuestro siguiente paso correspondería al conjugado, es decir,

$$\bar{q} = s - v$$

(3.8.f)

```

quaternion create() { ... }

quaternion conjugate(quaternion q)
{
    float s = q.w;
    vec3 v = vec3(-q.x, -q.y, -q.z);

    quaternion cq = quaternion(v.x, v.y, v.z, s);
    return cq;
}

```

Luego, la multiplicación, la cual se define cómo,

$$q_1 q_2 = (s_1 s_2 - v_1 \cdot v_2, s_1 v_2 + s_2 v_1 + v_1 \times v_2)$$

(3.8.g)

```

quaternion conjugate() { ... }

quaternion multiplication(quaternion q1, quaternion q2)
{
    float s1 = q1.w;
    float s2 = q2.w;

    // El código continúa en la página siguiente.
}

```

```

vec3 v1 = vec3(q1.x, q1.y, q1.z);
vec3 v2 = vec3(q2.x, q2.y, q2.z);

float s = s1 * s2 - dot(v1, v2);
vec3 v = s1 * v2 + s2 * v1 + cross(v1, v2);

quaternion q = quaternion(v.x, v.y, v.z, s);
return q;
}

```

Finalmente, queda aplicar la transformación. Para ello, iremos al método **fragment()** e inicializamos los ángulos XYZ a partir de la rotación de la luz y los ejes unitarios del mundo IJK:

```

void fragment()
{
    vec3 vertex_ws = (INV_VIEW_MATRIX * vec4(VERTEX, 1.0)).rgb;
    vec3 position_ws = vertex_ws - _LightPosition;

    // 1
    float x = -_LightRotation.x;
    float y = -_LightRotation.y;
    float z = -_LightRotation.z;

    // 2
    vec3 axis_i = vec3(1, 0, 0); // x
    vec3 axis_j = vec3(0, 1, 0); // y
    vec3 axis_k = vec3(0, 0, 1); // z

    // 3
    quaternion qx = create(x, axis_i);
    quaternion qy = create(y, axis_j);
    quaternion qz = create(z, axis_k);
    ...
}

```

En este caso, se han separado las variables para comprender de mejor manera la creación de los cuaterniones para cada eje, de hecho:

- 1 Inicializamos los ángulos de rotación siguiendo la orientación de la luz direccional.
- 2 Definimos los ejes unitarios del mundo.
- 3 Creamos los cuaterniones por cada eje.

Al igual que con las matrices de rotación, debemos respetar un orden específico al multiplicar cuaterniones, ya que el resultado depende directamente de la secuencia en la que aplicamos cada rotación. En este caso, utilizaremos el orden YXZ para asegurar que la orientación final coincida con la rotación observada al manipular el gizmo de la luz direccional en la escena.

Asimismo, es fundamental conjugar el cuaternión resultante, ya que la conjugación invierte el sentido de la rotación. Este paso es crucial cuando deseamos transformar un vector entre distintos espacios de referencia, por ejemplo, del espacio del objeto al espacio global o viceversa, garantizando que la orientación y el sentido de la rotación se mantengan coherentes.

```
void fragment()
{
    ...

    quaternion qx = create(x, axis_i);
    quaternion qy = create(y, axis_j);
    quaternion qz = create(z, axis_k);

    quaternion q_YXZ = multiplication(qz, multiplication(qx, qy));
    quaternion _q = conjugate(q_YXZ);

    ...
}
```

Con el cuaternión de orientación listo, lo multiplicamos por los puntos que deseamos rotar; en este caso, la posición global de los vértices:

```

void fragment()
{
    ...
    quaternion _q = conjugate(q_YXZ);
    // 1
    quaternion p = quaternion(position_ws.x, position_ws.y, position_ws.z,
        ↪ 0.0);
    quaternion pr = multiplication(q_YXZ, p);
    // 2
    pr = multiplication(pr, _q);

    ...
}

```

Como podemos observar:

- 1 Creamos un nuevo cuaternión **p** que contiene la posición de los vértices en espacio global con $W = 0$.
- 2 Aplicamos la rotación mediante la regla $p' = q \cdot p \cdot \bar{q}$ usando **q_YXZ** y su conjugado **_q**, para obtener la orientación final en espacio global.

En este punto, el cuaternión **pr** contiene los componentes rotados en XYZ. Por lo tanto, podemos extraer un **vec3** para representar la posición rotada:

```

void fragment()
{
    ...
    pr = multiplication(pr, _q);

    // 1
    vec3 position_ws_rot = vec3(pr.x, pr.y, pr.z);

    // El código continúa en la página siguiente.
}

```

```
// 2
vec2 uv_ws = position_ws_rot.xy + vec2(0.5);

vec3 front_map = texture(_FrontMap, UV).rgb;
vec3 back_map = texture(_BackMap, UV).rgb;
// 3
float spot_map = texture(_SpotMap, uv_ws.xy).r;

ALBEDO = vec3(spot_map);
}
```

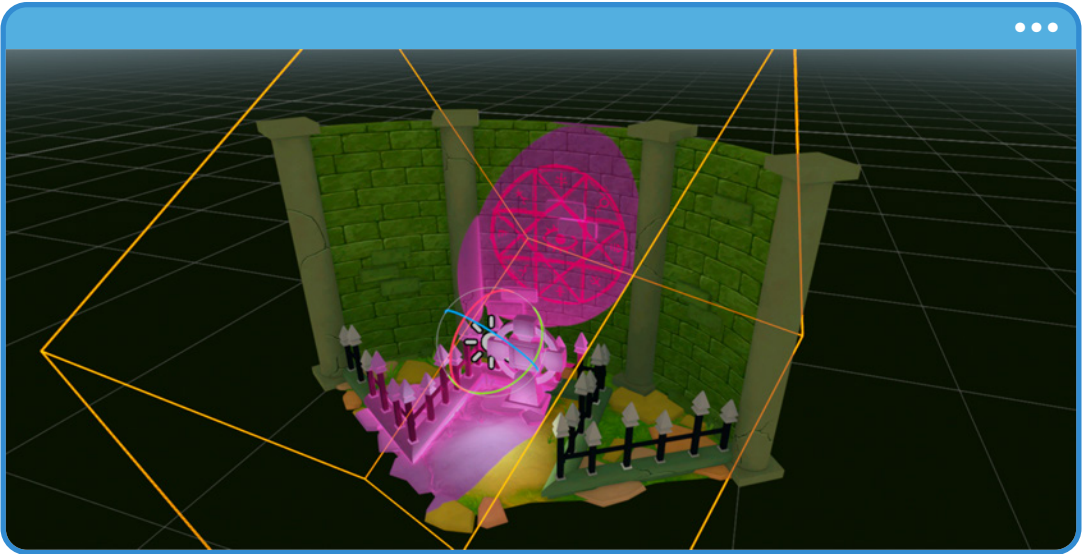
Si prestamos atención al código anterior, observaremos:

- ❶ Definimos **position_ws_rot** como un vector tridimensional que contiene la posición rotada de los vértices en espacio global.
- ❷ Construimos **uv_ws** con las componentes XY de esa posición rotada y sumamos 0.5 para centrar la proyección en el rango $[0 : 1]$ lo que alinea el círculo con el origen de la escena.
- ❸ Usamos **uv_ws** como coordenadas de muestreo para **_SpotMap**, mientras que **front_map** y **back_map** se siguen leyendo en **UV** del modelo (esto preserva el mapeo propio de cada material y evita distorsiones en las texturas base).

Si regresamos al Viewport y rotamos la iluminación, la proyección ahora rota correctamente y coincide con la orientación del gizmo. Para finalizar la composición, interpolamos linealmente entre **front_map** y **back_map** usando **spot_map** como factor:

```
void fragment()
{
    ...
    vec3 color = mix(front_map, back_map, spot_map);

    ALBEDO = color;
}
```



(3.8.h Ambas texturas sobre el modelo)

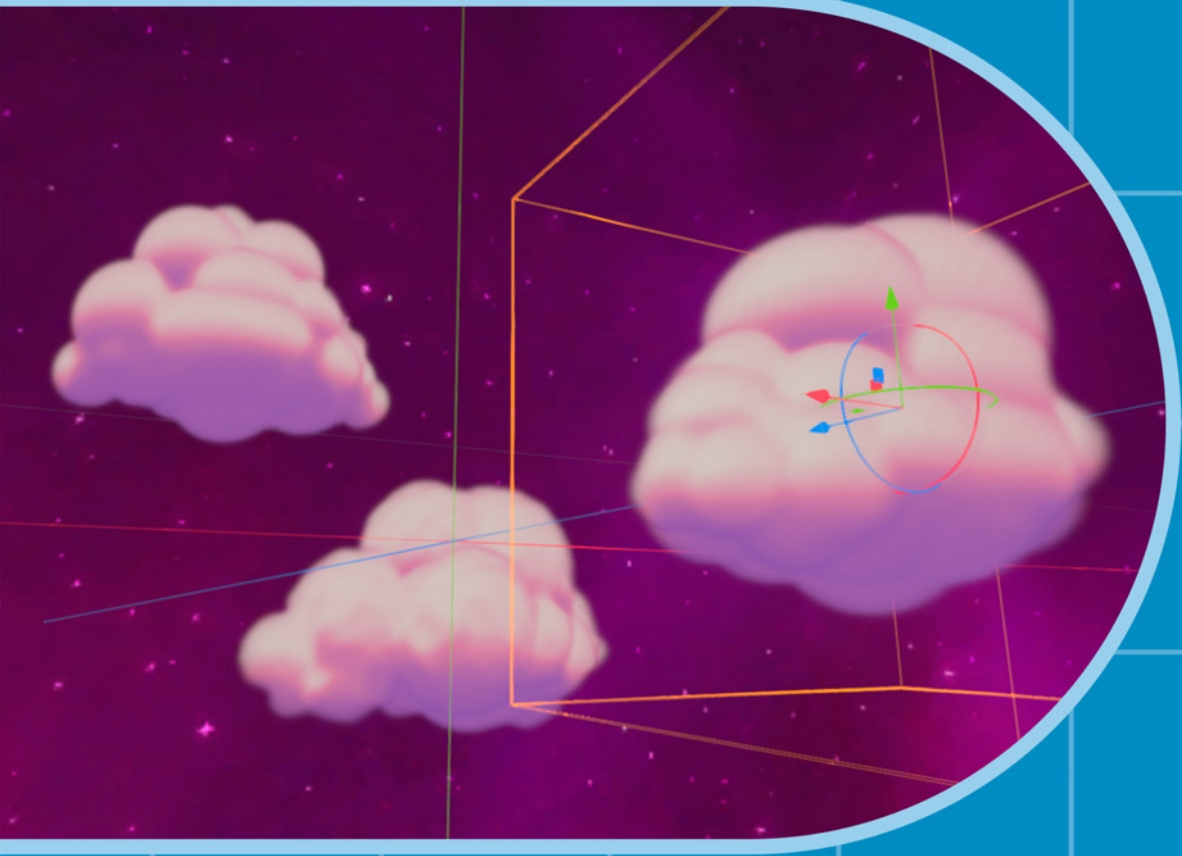
Resumen del capítulo.

En este capítulo se adquirieron los conocimientos necesarios para crear figuras procedurales y aplicar animación por vértices, comprendiendo cómo las funciones matemáticas pueden utilizarse para generar y modificar geometría de manera dinámica dentro del shader.

Se aprendió a dibujar formas mediante funciones e inecuaciones, utilizando expresiones matemáticas como base para la construcción de figuras procedurales. A partir de estos fundamentos, se abordaron técnicas de animación y modificación procedural, permitiendo alterar tanto la forma como el comportamiento de los objetos sin depender de animaciones cuadro por cuadro.

El capítulo también abordó la implementación de un personaje procedural utilizando GDScript, integrando los conceptos matemáticos con el lenguaje de sombreado del motor. Posteriormente, se estudiaron técnicas de deformación de vértices y animación aplicadas a la interfaz de usuario, ampliando el uso de shaders más allá de la geometría 3D tradicional, e incorporando un efecto especular en la UI para enriquecer el resultado visual.

Finalmente, se adquirieron los fundamentos de las rotaciones en el espacio tridimensional, comprendiendo su formulación matemática e implementación práctica mediante matrices de rotación y cuaterniones. Estos conceptos permitieron entender y aplicar transformaciones espaciales complejas de forma estable y eficiente, consolidando así un conjunto de herramientas esenciales para la animación y manipulación procedural de vértices.



Capítulo 4

VFX avanzados y post-processing.

Hasta este punto hemos aprendido varios conceptos clave sobre el desarrollo de shaders en Godot. Sin embargo, llega un momento en el que es necesario poner en práctica todo ese conocimiento para consolidarlo realmente. Por esta razón, durante este capítulo, aplicaremos lo aprendido desarrollando efectos comunes en videojuegos modernos, entendiendo tanto su funcionamiento visual como su implementación técnica.

Comenzaremos revisando cómo integrar shaders dentro del `canvas_item` para realizar post-processing. Utilizaremos este enfoque para crear efectos sobre la pantalla, incluyendo un ejercicio práctico en el que transformaremos nuestro render en un estilo inspirado en la Game Boy mediante gradientes de color y texturas predefinidas. Esto nos permitirá comprender mejor cómo manipular el resultado final de una escena sin modificar sus materiales individuales.

Avanzaremos luego hacia efectos basados en la distancia y la cámara, implementando un fade progresivo que se activa cuando la cámara se encuentra cercana a un objeto, mediante una distancia predefinida. Este tipo de translucencia es ideal para juegos de exploración o aventuras isométricas, donde mantener la visibilidad del personaje es esencial.

Posteriormente, nos adentraremos en el mundo de Ray Marching. Estudiaremos su estructura, su algoritmo paso a paso y la analogía geométrica que lo impulsa. Con este conocimiento construiremos una figura tridimensional desde cero; una nube, lo que nos permitirá comprender cómo definir volúmenes sin mallas mediante texturas 3D y cómo trasladar estas ideas a GDScript.

Luego, introduciremos el uso del **Stencil Buffer**, una herramienta poderosa para controlar qué partes de la escena se dibujan o descartan. Revisaremos sus propiedades, configuraciones y casos típicos de uso, y construiremos un shader personalizado que nos permitirá entender su comportamiento directamente desde código.

Finalmente, realizaremos un ejercicio de porting desde Shadertoy. Tomaremos un shader sencillo y lo adaptaremos a la estructura de Godot, aprendiendo a reinterpretar funciones, coordenadas y convenciones. Este proceso ampliará nuestra comprensión del lenguaje orientado a gráficos y abrirá la puerta a un abanico mucho mayor de posibilidades artísticas y técnicas.

4.1 Post-Procesado y efecto Game Boy.

Si has desarrollado videojuegos alguna vez, probablemente conoces la Game Boy o incluso tuviste la oportunidad de jugar con una. Esta consola portátil apareció a finales de los años ochenta y alcanzó su mayor popularidad durante los noventa. A pesar de sus limitaciones técnicas, tenía un encanto único, y uno de los rasgos más reconocibles era su característico color: una pantalla verdosa, con una gama extremadamente reducida de tonos

Recrear esa paleta con las tecnologías actuales es sencillo, aunque no siempre práctico en un proyecto real. Aun así, resulta muy interesante estudiar cómo construir filtros de post-procesado que nos permitan simular efectos clásicos. En esta sección, desarrollaremos un shader que imita el estilo de color de la Game Boy. Si bien el resultado no será una reproducción cien por ciento fiel, será un excelente ejercicio para profundizar en técnicas de post-procesado dentro de Godot.

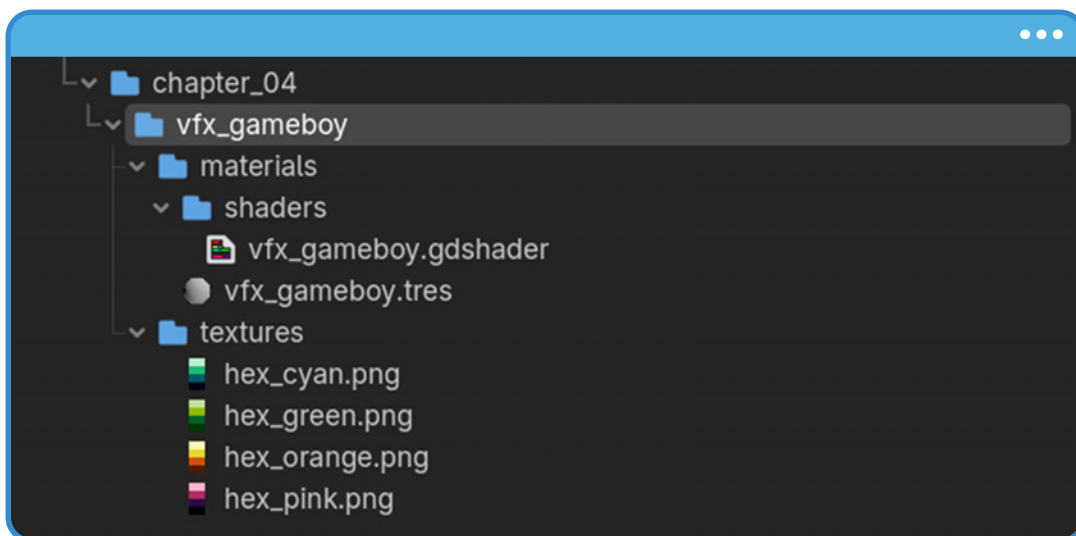
Comenzaremos organizando nuestro proyecto. Mantener una estructura clara facilita seguir el flujo del capítulo y nos permitirá reutilizar los recursos más adelante. Para ello:

- ❶ Dentro de la carpeta **assets**, crearemos una subcarpeta llamada **chapter_04**, donde almacenaremos los recursos utilizados en esta sección.
- ❷ Dentro de **chapter_04**, agregaremos una carpeta llamada **vfx_gameboy**.
- ❸ Finalmente, dentro de **vfx_gameboy**, crearemos tres subcarpetas:
 - a **materials**.
 - b **shaders**.
 - c **textures**.

Para este ejercicio, reutilizaremos algunos de los recursos que ya empleamos en secciones anteriores para construir nuestra escena base. Sin embargo, necesitaremos descargar un conjunto adicional de texturas específicas para este capítulo, ya que las utilizaremos para definir la paleta y el comportamiento del color en el efecto final.

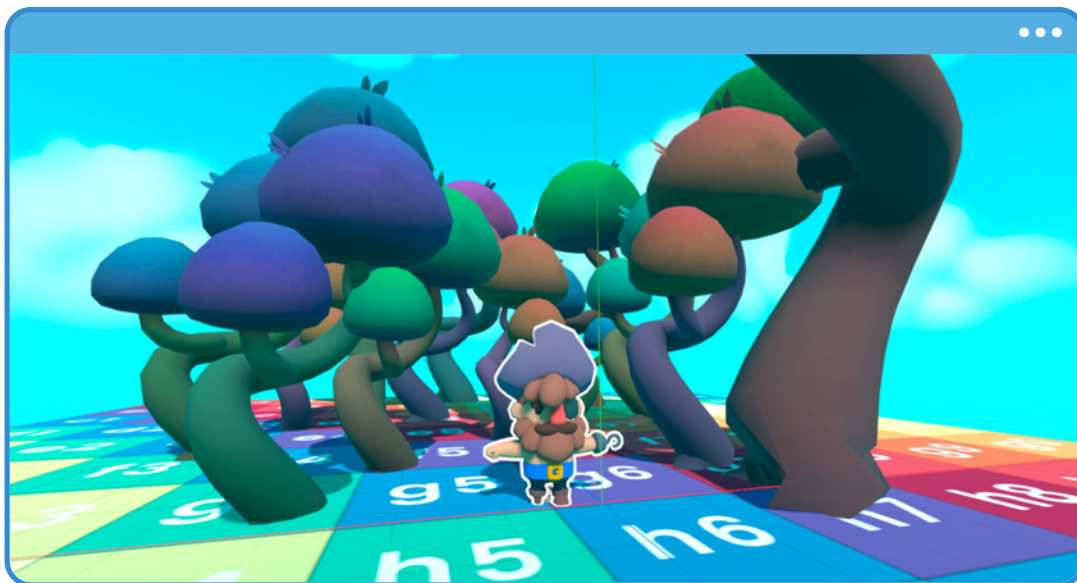
Como de costumbre, crearemos tanto un **Shader** como su **ShaderMaterial** correspondiente. Ambos serán esenciales para integrar el efecto dentro de la interfaz y aplicarlo como parte del proceso de post-procesado. Para mantener una nomenclatura clara y consistente, nombraremos estos recursos igual que la sección: **vfx_gameboy**.

Si hemos seguido todos los pasos correctamente, la estructura de nuestro proyecto debería verse de la siguiente manera:



(4.1.a Se ha incluido **vfx_gameboy** en el proyecto)

Para este ejercicio, trabajaremos con una escena previamente preparada. Esta escena contiene algunos árboles, nuestro personaje **Rebey Vane**, un plano que utilizaremos como suelo y un nodo **WorldEnvironment** encargado del cielo y la iluminación global.



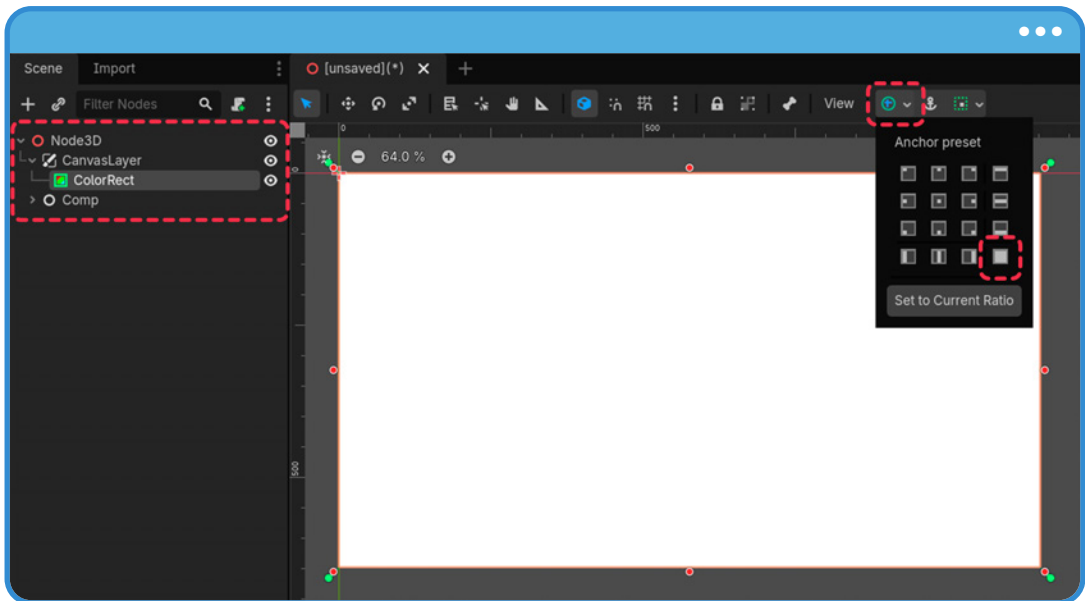
(4.1.b Se ha configurado una escena)

Aunque la composición tiene su propio estilo, el color en esta etapa no es especialmente relevante. Vamos a reemplazar por completo el tinte, la gamma y buena parte de la información cromática mediante un efecto de post-procesado. Incluso si nuestra escena estuviera en escala de grises, el resultado del efecto sería prácticamente el mismo, ya que el filtro sobrescribirá la paleta de la composición.

Para comenzar con la configuración del efecto, realizaremos los siguientes pasos:

- 1 Iremos a la ventana **Scene**.
- 2 Agregaremos un nuevo nodo **CanvasLayer**.
- 3 Como hijo del **CanvasLayer**, agregaremos un nodo **ColorRect**.

Debemos asegurarnos de que el **Anchor** del ColorRect esté configurado como **Full Rect**. De esta manera, el rectángulo cubrirá la pantalla completa, lo cual es fundamental para aplicar el shader como un filtro global sobre toda la imagen renderizada.



(4.1.c Se ha configurado el ColorRect como Full Rect)

Nota

Para asegurarnos de visualizar correctamente los cambios que aplicaremos en esta sección, será necesario ejecutar el proyecto. Esto implica establecer nuestra escena actual como **main** y, posteriormente, presionar el botón **Play**. Los efectos de post-procesado no se muestran en el editor; solo se aplican durante la ejecución.

En este punto, debemos asignar nuestro material **vfx_gameboy** a la propiedad **Material** del nodo **ColorRect**. Ahora bien, si corremos el proyecto en este momento, no veremos ningún cambio aún. Por defecto, los shaders nuevos se crean con el tipo **spatial**, y para que nuestro efecto funcione como filtro de pantalla, debemos cambiarlo explícitamente a **canvas_item** (como vimos en la sección 3.4 del capítulo 3).

Cuando trabajamos con post-processing en Godot, existe un detalle técnico importante: para aplicar un efecto sobre la imagen final, necesitamos acceder tanto a las coordenadas de pantalla (**SCREEN_UV**) como a la textura completa del fotograma renderizado. Esto significa que, al final del Render Pipeline, inmediatamente después de la etapa de fragmentos (como explicamos en la sección 1.5 del capítulo 1), debemos tomar una “captura” del fotograma y usarla como entrada para nuestro shader, pixel por pixel.

Godot nos facilita esta operación a través del **hint_screen_texture**, el cual nos permite acceder a esta textura de pantalla dentro del shader. Su uso básico se ve así:

```
shader_type canvas_item;

uniform sampler2D _ScreenTexture : hint_screen_texture, repeat_disable,
    filter_nearest;

void vertex()
{
    // Called for every vertex the material is visible on.
}
```

Podemos deducir que **repeat_disable** evita que la textura se repita fuera del rango UV, mientras que **filter_nearest** obliga a Godot a muestrear la textura utilizando nearest-

neighbor en lugar de interpolación lineal. Esto significa que cada píxel se toma tal cual, sin suavizados, lo cual es ideal para efectos retro como el estilo Game Boy.

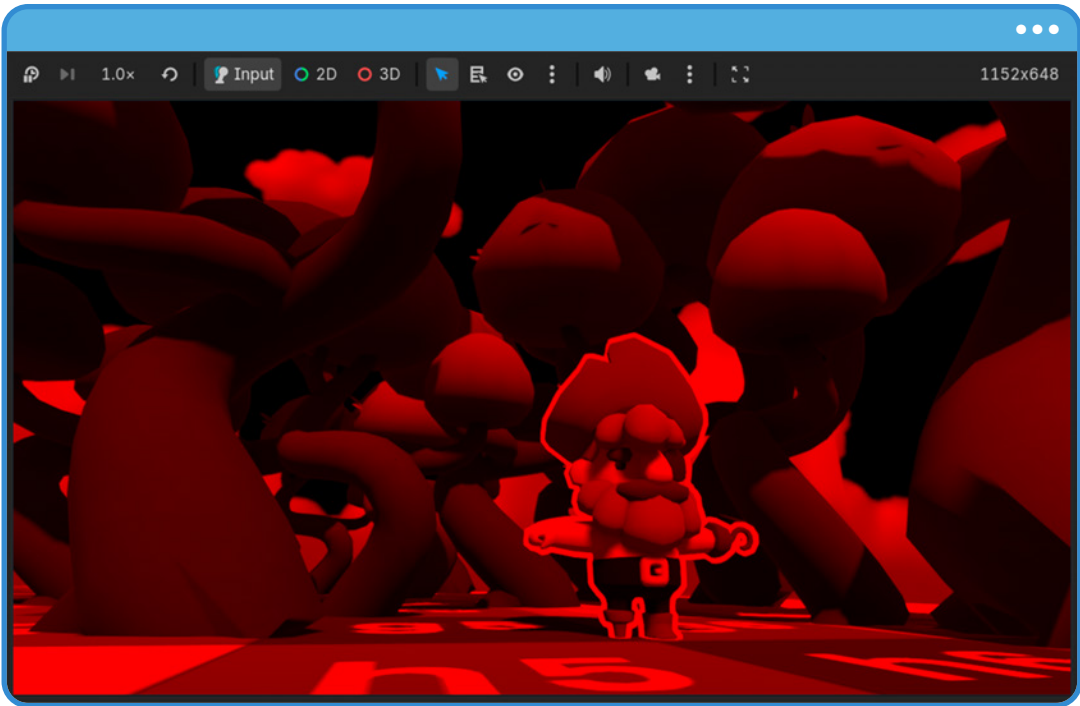
Con nuestro sampler declarado, podemos avanzar al método `fragment()` y realizar las siguientes operaciones:

```
void fragment()
{
    // 1
    vec2 screen_uv = SCREEN_UV;
    // 2
    vec4 screen_texture = texture(_ScreenTexture, screen_uv);
    // 3
    screen_texture.rgb *= vec3(1.0, 0.0, 0.0);
    // 4
    COLOR = screen_texture;
}
```

Podemos analizar el flujo del código de la siguiente manera:

- 1 Accedemos a las coordenadas UV de la pantalla utilizando **SCREEN_UV**.
- 2 Muestreamos la textura del fotograma actual con esas mismas coordenadas y almacenamos el resultado en **screen_texture**.
- 3 Aplicamos un cambio temporal al color, multiplicando el valor RGB por un vector rojo. Este paso nos permite verificar si el pipeline de post-procesado está funcionando correctamente.
- 4 Asignamos el resultado final a **COLOR**, lo que hace que el **ColorRect** dibuje la textura procesada sobre la pantalla.

En el editor no deberíamos observar ningún cambio, ya que los efectos de post-procesado solo se aplican durante la ejecución. Sin embargo, si ejecutamos la escena principal presionando **Play**, nuestro render debería teñirse de rojo.



(4.1.d Se ha aplicado un color rojo sobre el render final)

Con el pipeline funcionando y la UI correctamente configurada, estamos listos para comenzar a desarrollar el efecto Game Boy dentro de nuestro shader. El primer paso consiste en transformar la composición a escala de grises. Como mencionamos anteriormente, el color original de los objetos no es relevante para nuestro efecto final, ya que aplicaremos una paleta completamente nueva basada en niveles de luminancia.

Para lograr esta conversión, agregaremos la siguiente función justo antes del método `fragment()`:

```
float luma(vec3 c)
{
    return max(0.0, dot(c, vec3(0.2126, 0.7152, 0.0722)));
}

void fragment() { ... }
```


¿Qué hace exactamente la función `luma`? En esencia:

- Calcula el producto punto entre el color RGB de entrada y un conjunto de coeficientes sRGB estándar. Estos coeficientes representan la contribución relativa de cada canal a la luminancia percibida por el ojo humano: R aporta un 21.26 %, G aporta un 71.52 %, B aporta un 7.22 %.

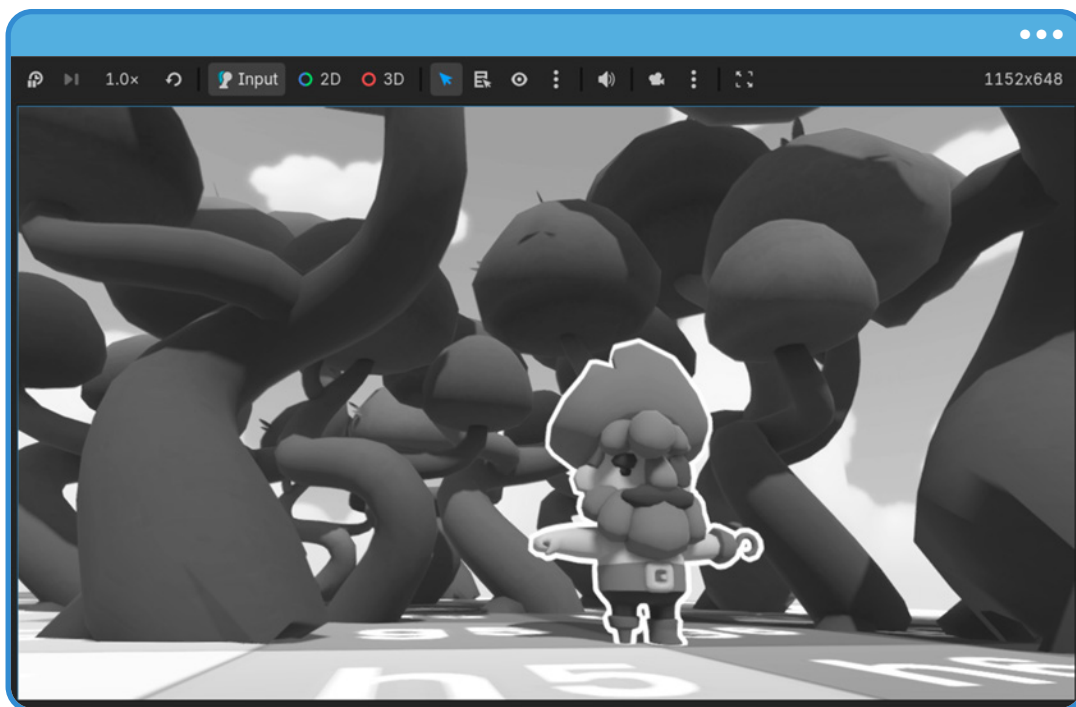
Esta función retorna un valor escalar que describe qué tan luminosa es una combinación RGB. Con ese valor, podemos reemplazar los tres canales del color y así generar un tono uniforme, logrando la conversión a escala de grises.

Si actualizamos nuestro método `fragment()` como se muestra a continuación:

```
void fragment()
{
    vec2 screen_uv = SCREEN_UV;
    vec4 screen_texture = texture(_ScreenTexture, screen_uv);
    screen_texture.rgb = vec3(luma(screen_texture.rgb));

    COLOR = screen_texture;
}
```

y luego ejecutamos el proyecto presionando **Play**, deberíamos ver nuestra escena completamente en escala de grises, lo cual funciona perfectamente como punto de partida para aplicar la paleta verde característica de la Game Boy.



(4.1.e Se ha transformado la composición a escala de grises)

Una vez que hemos convertido la composición a escala de grises, podemos utilizar esa luminancia para mapear el color de cada píxel hacia una gama predefinida. En otras palabras, tomaremos el valor de gris, que representa cuánta luz aporta cada píxel, y lo reemplazaremos por uno de los colores de nuestra paleta Game Boy. Por ejemplo, podemos asignar un color para los valores altos (casi blancos), otro para los tonos medios, y otro para las zonas más oscuras.

Si exploramos la carpeta **textures** correspondiente a esta sección, encontraremos cuatro texturas que representan distintas variaciones del tono Game Boy. Estas paletas imitan el estilo cromático clásico de la consola, especialmente su versión más retro. Cada textura contiene cuatro colores ordenados de mayor a menor luminancia: un tono alto, dos tonos medios y un tono oscuro. Esto nos permitirá clasificar cada píxel según su luminancia y asignarle el color apropiado.

Por ejemplo, la textura **hex_green** define los siguientes valores RGB (de arriba hacia abajo):

- 202, 220, 160 (tono alto).
- 155, 187, 14 (tono medio alto).
- 48, 98, 47 (tono medio bajo).
- 15, 56, 14 (tono bajo).

Podemos separar los colores de nuestra paleta en índices, comenzando en **0** para el tono más claro y terminando en **3** para el más oscuro. La pregunta ahora es: ¿cómo accedemos a cada color de manera independiente dentro del shader? Para lograrlo, comenzaremos declarando una nueva propiedad **sampler2D** que representará nuestra paleta Game Boy:

```
shader_type canvas_item;

uniform sampler2D _Palette : source_color;
uniform sampler2D _ScreenTexture : hint_screen_texture, repeat_disable,
    ↪ filter_nearest;

void vertex() { ... }
```

Con esta textura declarada, podemos crear un método que nos permita obtener cada color utilizando un índice entero:

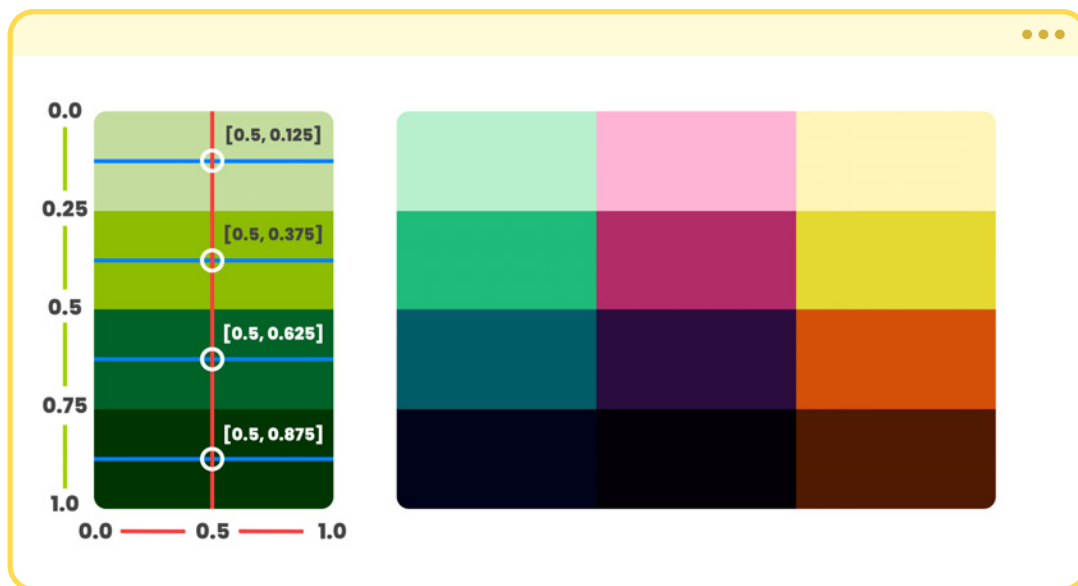
```
// 1
vec3 palette_fetch(int i)
{
    // 2
    i = clamp(i, 0, 3);
    // 3
    float v = 0.125 + 0.25 * float(i);
    // 4
    return texture(_Palette, vec2(0.5, 1.0 - v)).rgb;
}
```

Podemos analizar este método paso a paso:

- 1 Declaramos el método `palette_fetch()`, que recibe un índice como argumento. Este índice representará el nivel de luminancia que queremos mapear (alto, medios o bajo).
- 2 Aseguramos que el índice esté entre 0 y 3, que corresponde al número de colores disponibles en la paleta.
- 3 Calculamos la posición vertical (v) dentro de la textura. Cada color ocupa una franja de 0.25 en el eje vertical, ya que hay cuatro colores. Sumamos 0.125 para apuntar al centro de cada franja (evitando bordes). De esta forma, obtenemos estas posiciones UV:
 - a Cuando $i = 0$: $v = 0.125$.
 - b Cuando $i = 1$: $v = 0.375$.
 - c Cuando $i = 2$: $v = 0.625$.
 - d Cuando $i = 3$: $v = 0.875$.
- 4 Retornamos el color usando `texture()`, muestreando la paleta en `vec2(0.5, 1.0 - v)`. Usamos 0.5 en U para ubicarnos en el centro horizontal. Restamos v a 1.0 porque Godot almacena las texturas comenzando desde la esquina superior izquierda, por lo que esta inversión nos permite leer los colores en el orden correcto (de arriba hacia abajo).

En otras palabras, este método toma un índice entero, calcula la posición exacta donde se encuentra el color correspondiente dentro de la textura y lo devuelve como un valor RGB. De esta manera, podremos transformar nuestra composición en escala de grises hacia una paleta Game Boy auténtica, asignando el color adecuado para cada píxel según su luminancia.

A continuación, podemos revisar la referencia visual para comprender mejor cómo se distribuyen estos texels y por qué apuntamos a sus posiciones centrales.



(4.1.f Coordenadas de los índices de los texels)

Como podemos observar en la Figura 4.1.f, los texels que utilizamos para cada índice se encuentran exactamente en el centro de cada bloque vertical de la paleta. Esto sucede gracias a las coordenadas que pasamos en `vec2(0.5, 1.0 - v)`, donde seleccionamos el punto medio horizontal (0.5) y la posición vertical precisa calculada con `v`.

En teoría, podríamos reducir aún más el tamaño de la textura, ya que solo estamos utilizando cuatro muestras puntuales. Esto disminuiría su peso y optimizaría ligeramente el proceso de muestreo. Sin embargo, como este detalle no es el foco de la sección, continuaremos utilizando las texturas tal como fueron diseñadas.

Hasta este punto, ya contamos con una función que nos permite extraer colores de la paleta usando índices. Lo que necesitamos ahora es una función que determine qué color asignar a cada píxel, basándose en la luminancia de la composición. Para lograrlo, implementaremos lo siguiente:

```
// 1
vec3 gameboy_color(vec3 color)
{
    // 2
    float t = luma(color);
    // 3
    t = smoothstep(0.1, 0.9, t);
    // 4
    int idx = int(floor(t * 4.0));
    // 5
    if(idx == 4) idx = 3;
    // 6
    return palette_fetch(idx);
}
```

Como podemos observar:

- 1 Declaramos el método **gameboy_color()**, que recibe un color RGB como argumento. Este será el color original del píxel antes de aplicar el efecto.
- 2 Convertimos el color a luminancia usando **luma()**. Esto nos da un valor entre 0.0 y 1.0, que representa cuánta luz aporta ese píxel.
- 3 Aplicamos **smoothstep(0.1, 0.9, t)** para remapear la luminancia. Esto comprime ligeramente los valores oscuros y los claros. El resultado es una imagen con un contraste más marcado, similar al comportamiento visual de pantallas retro.
- 4 Calculamos un índice entre 0 y 4, multiplicando la luminancia por el número de intervalos posibles y aplicando **floor()**.
- 5 Corregimos el caso límite: si el índice resulta ser 4, lo reducimos a 3, ya que la paleta solo tiene cuatro colores.
- 6 Retornamos el color de la paleta utilizando **palette_fetch(idx)**.

De esta forma, cada rango de luminancia se mapea a uno de los cuatro colores de la paleta, emulando el estilo de la Game Boy. Finalmente, regresaremos al método **fragment()** y actualizaremos el color RGB de la textura de pantalla utilizando el resultado de **gameboy_color()**:

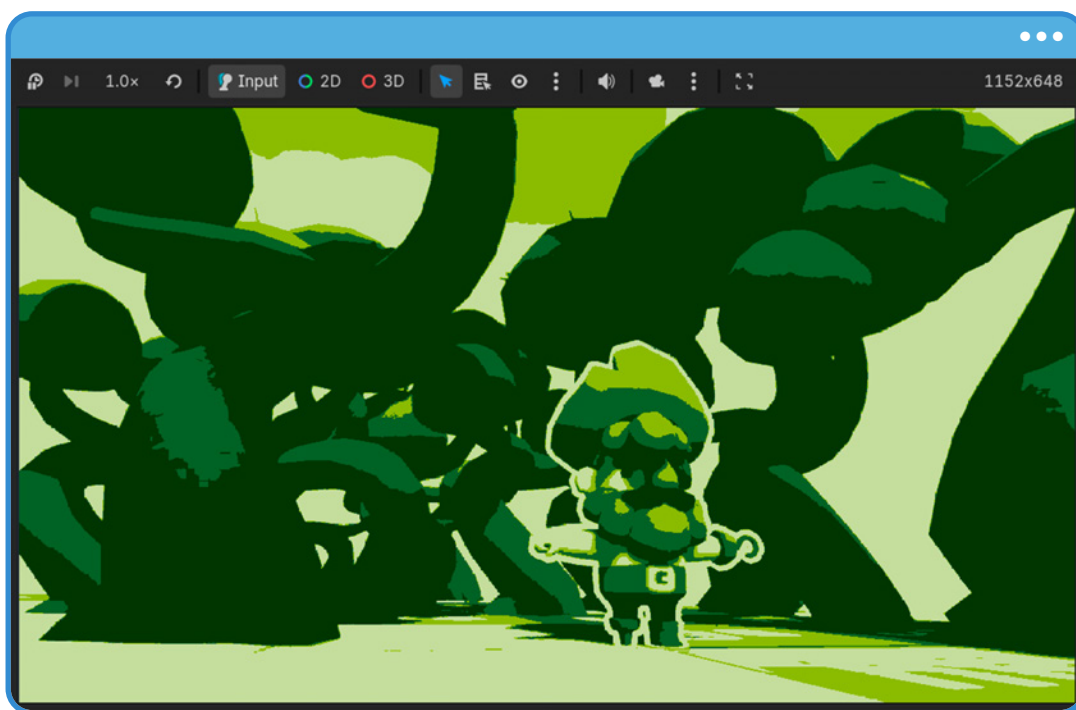
```

void fragment()
{
    vec2 screen_uv = SCREEN_UV;
    vec4 screen_texture = texture(_ScreenTexture, screen_uv);
    screen_texture.rgb = gameboy_color(screen_texture.rgb);

    COLOR = screen_texture;
}

```

Al ejecutar nuestro proyecto, deberíamos ver la paleta seleccionada aplicada sobre la gama de luminancias de la composición.



(4.1.g Se ha aplicado la paleta hex_green a la composición)

Para que el efecto se sienta aún más cercano a la estética de la Game Boy, podemos añadir un ligero “pixelado” sobre el render. Implementar este efecto es relativamente sencillo. Para ello, podemos definir la siguiente función auxiliar:

```
// 1
vec2 to_pixel(vec2 screen_uv, vec2 screen_pixel, float pixel_size)
{
    // 2
    vec2 px = screen_uv * screen_pixel;
    // 3
    px = floor(px / pixel_size) * pixel_size + pixel_size;
    // 4
    return px / screen_pixel;
}
```

¿Qué está ocurriendo en este método?

- 1 Declaramos un nuevo método llamado **to_pixel()**, el cual recibe como argumentos las coordenadas UV de la pantalla, el tamaño de la pantalla en píxeles y el tamaño del píxel al que queremos “redondear”.
- 2 Convertimos las coordenadas UV a coordenadas de píxel multiplicando por el tamaño de la pantalla. De esta forma trabajamos en unidades de píxel en lugar de valores normalizados entre 0 y 1.
- 3 Dividimos esas coordenadas de píxel por el tamaño de bloque (**pixel_size**), aplicamos **floor()** y luego volvemos a multiplicar por **pixel_size**. Con esto agrupamos varios píxeles en un mismo bloque y hacemos que todos tomen la misma muestra de color, lo que genera el efecto de pixelado. El desplazamiento + **pixel_size** nos ayuda a muestrear hacia el centro del bloque en lugar de en el borde.
- 4 Finalmente, normalizamos nuevamente dividiendo por el tamaño de la pantalla, de manera que obtenemos coordenadas UV válidas entre 0 y 1, independientes de la resolución.

Antes de volver al método **fragment()**, necesitamos declarar una nueva propiedad que nos permita modificar de manera dinámica el tamaño de los píxeles. Para ello, regresamos a las variables globales y agregamos la siguiente propiedad:


```

shader_type canvas_item;

uniform sampler2D _Palette : source_color;
uniform int _PixelSize : hint_range(1, 10, 1);
uniform sampler2D _ScreenTexture : hint_screen_texture, repeat_disable,
    ↪ filter_nearest;

```

A través de la propiedad **_PixelSize**, controlaremos el tamaño de los bloques de píxeles en nuestra composición: valores pequeños producen un pixelado suave, mientras que valores grandes generan un efecto mucho más notorio.

A continuación, regresamos al método **fragment()** y actualizamos el vector **screen_uv** para que utilice el nuevo método:

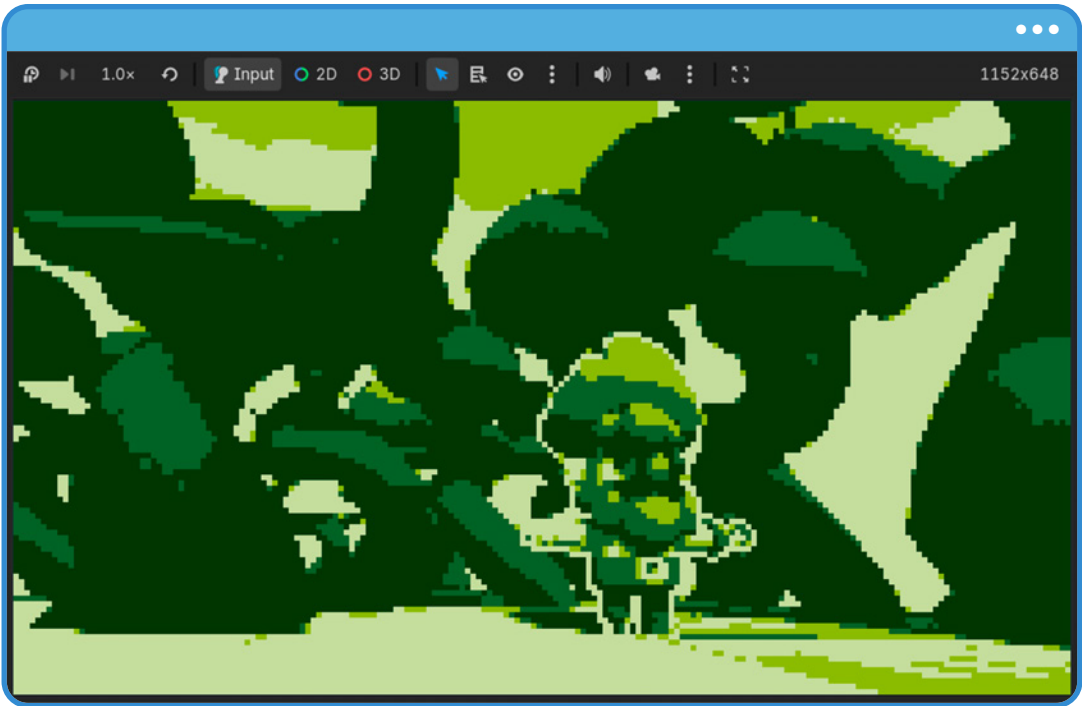
```

void fragment()
{
    vec2 screen_uv = to_pixel(SCREEN_UV, 1.0 / SCREEN_PIXEL_SIZE,
    ↪ float(_PixelSize));
    vec4 screen_texture = texture(_ScreenTexture, screen_uv);
    screen_texture.rgb = gameboy_color(screen_texture.rgb);

    COLOR = screen_texture;
}

```

Para observar los cambios, necesitamos ajustar el valor de **Pixel Size** desde el Inspector a un valor distinto de 1. En el ejemplo de referencia, hemos utilizado **Pixel Size = 6**, lo que produce un efecto de pixelado similar al de las pantallas de baja resolución de la Game Boy clásica.



(4.1.h Se ha pixelado la composición)

Podríamos dar por finalizada esta sección en este punto, ya que el resultado se aproxima bastante al clásico efecto Game Boy: disponemos de una paleta limitada, una composición en escala de grises mapeada a dicha paleta y un pixelado que refuerza la estética retro. Sin embargo, aún podemos dar un paso adicional: portar un efecto desde **Shadertoy** para enriquecer todavía más nuestra composición y explorar cómo adaptar shaders externos a Godot. Este proceso lo desarrollaremos en la siguiente sección.

4.2 Portando un efecto desde Shadertoy.

Shadertoy es una plataforma ampliamente utilizada por quienes disfrutamos trabajar con coordenadas, funciones matemáticas y efectos visuales en tiempo real. Si has leído más de uno de nuestros libros, sabrás que frecuentemente mencionamos a **Inigo Quilez**, uno de los artistas y programadores más influyentes en gráficos por computadora, además de ser el fundador de Shadertoy.

El efecto que portaremos en esta sección corresponde a un patrón **checkerboard** (damero) desarrollado por **Alundra**, el cual luce de la siguiente manera:

(4.2.a Patrón **checkerboard**)

Su implementación original en GLSL es la siguiente:

```
void mainImage(out vec4 fragColor, in vec2 fragCoord)
{
    float Size = 10.0;
    vec2 Pos = floor(fragCoord / Size);
    float PatternMask = mod(Pos.x + mod(Pos.y, 2.0), 2.0);
    fragColor = PatternMask * vec4(1.0, 1.0, 1.0, 1.0);
}
```

Nota

Puedes acceder al efecto en el siguiente enlace: <https://www.shadertoy.com/view/lt2XWK>

Analicemos brevemente la implementación:

- **mainImage()** es el punto de entrada que genera la imagen de salida. En Godot, este rol lo cumple el método **fragment()**.
- **fragColor** representa el color final del fragmento. En un shader de post-processing podemos asumir que corresponde a la variable **COLOR**. En otro tipo de materiales podría equivaler a **ALBEDO**.
- **fragCoord** representa las coordenadas de pantalla. Podríamos pensar que se corresponde directamente con **SCREEN_UV**, y aunque la relación es cercana, no es exactamente lo mismo. **fragCoord** trabaja en coordenadas absolutas de píxel, mientras que **SCREEN_UV** está normalizado entre 0 y 1. Para obtener un equivalente funcional, utilizamos la siguiente operación:

$$\text{fragCoord} \approx \frac{\text{SCREEN_UV}}{\text{SCREEN_PIXEL_SIZE}}$$

(4.2.b)

Esto mantiene el patrón estable incluso si cambiamos la escala del Viewport. demás operaciones son relativamente sencillas. Por ejemplo, la función **mod()** devuelve el residuo de dividir X entre Y, lo que permite alternar colores para generar el patrón.

Ahora bien, ¿cómo podemos implementar esta función dentro de nuestro efecto? El proceso es bastante simple. A continuación podemos visualizar su adaptación a Godot, manteniendo los mismos nombres utilizados en Shadertoy para facilitar la comparación:

```
void fragment()
{
    // 1
    float size = float(_PixelSize);
    // 2
    vec2 screen_uv = to_pixel(SCREEN_UV, 1.0 / SCREEN_PIXEL_SIZE, size);
    vec4 screen_texture = texture(_ScreenTexture, screen_uv);

    // El código continúa en la página siguiente.
```

```

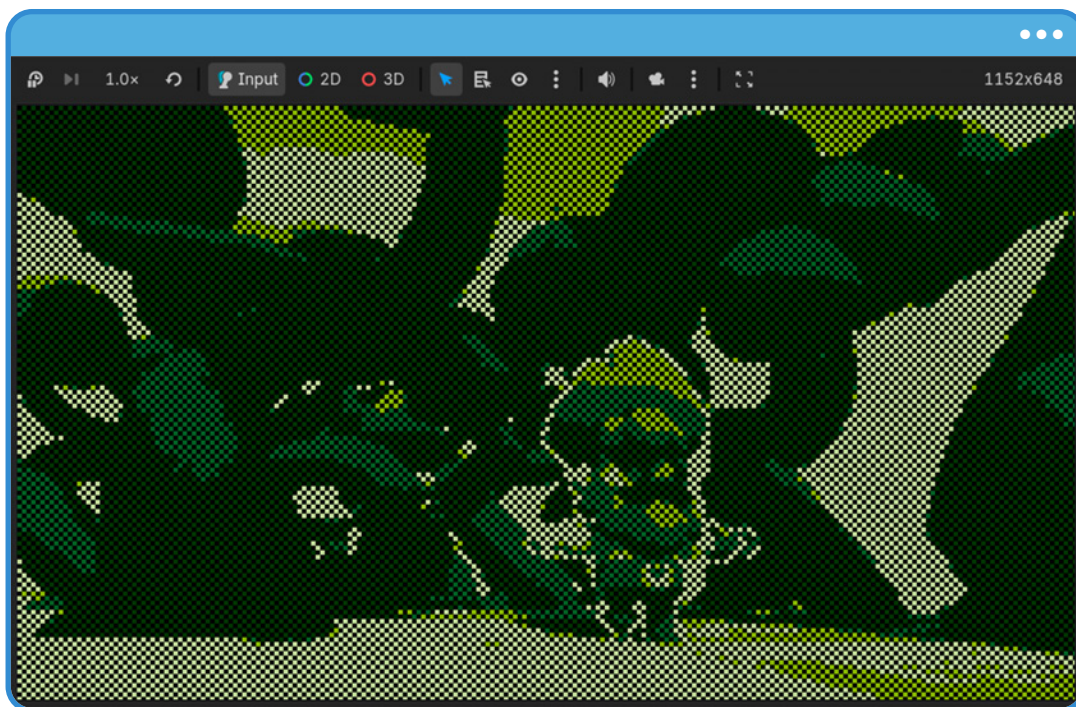
screen_texture.rgb = gameboy_color(screen_texture.rgb);
//3
vec2 pos = floor((SCREEN_UV / SCREEN_PIXEL_SIZE) / size);
float pattern_mask = mod(pos.x + mod(pos.y, 2.0), 2.0);
//4
screen_texture.rgb *= pattern_mask;
COLOR = screen_texture;
}

```

¿Qué está ocurriendo aquí?

- 1 Declaramos una nueva variable llamada **size**, que almacena **_PixelSize** convertido a flotante. Esto nos permite reutilizar el valor tanto en **to_pixel()** como en el cálculo del patrón.
- 2 Agregamos **size** como tercer argumento del método **to_pixel()**, asegurándonos de que el pixelado y el patrón utilicen la misma escala.
- 3 Utilizamos **size** para dividir las coordenadas de pantalla normalizadas (**SCREEN_UV / SCREEN_PIXEL_SIZE**) antes de aplicar **floor()**. Esto determina el tamaño de cada celda del checkerboard.
- 4 Multiplicamos las componentes RGB de **screen_texture** por **pattern_mask**, aplicando así el patrón damero sobre la composición ya procesada con la paleta Game Boy.

El resultado visual es una combinación del pixelado, la paleta Game Boy y el patrón checkerboard, lo que genera un acabado retro interesante.



(4.2.c Se ha aplicado el patrón sobre el render final)

4.3 Transparencia y distancia.

Cuando hablamos de transparencia, esencialmente nos referimos al canal **ALPHA**, el cual determina el nivel de opacidad que posee cada fragmento durante el proceso de renderizado. En un shader, este canal adquiere un rol particular, ya que no solo define la visibilidad del color final, sino que también puede emplearse como una herramienta para generar efectos basados en distancia, profundidad o intensidad luminosa dentro de la composición.

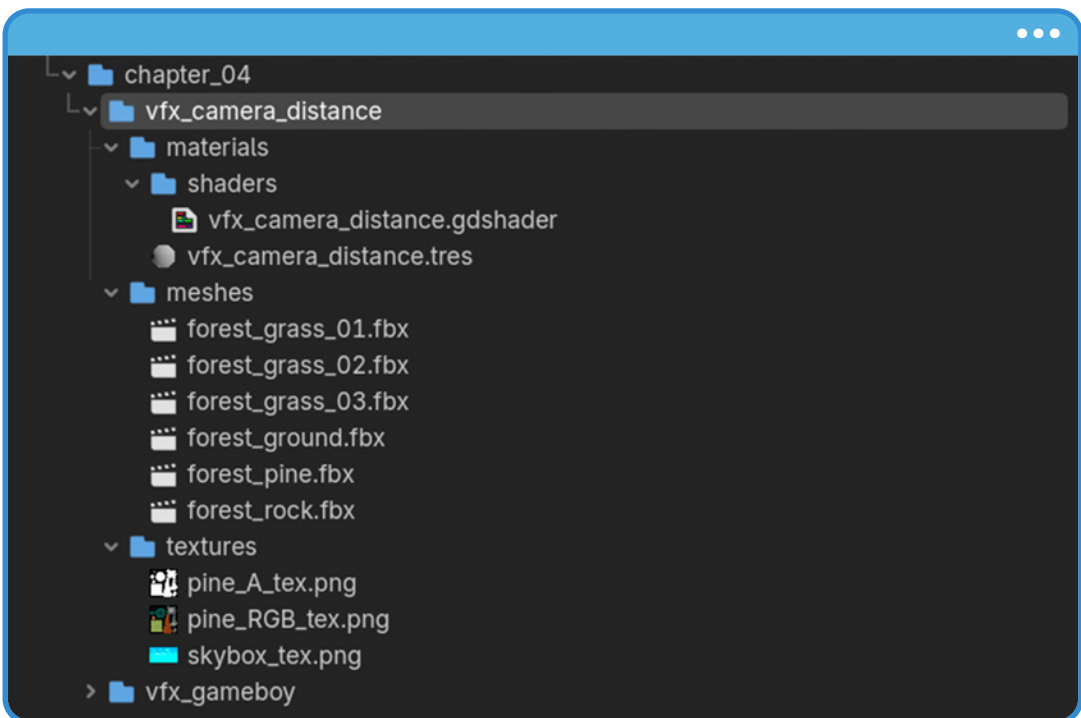
En esta sección exploraremos cómo manipular el canal alfa utilizando la distancia entre un objeto 3D y la cámara, con el fin de crear un efecto de transparencia que se active cuando ciertos elementos interfieren con la visibilidad del personaje principal. Este recurso es ampliamente utilizado en videojuegos de tercera persona: si un árbol, roca o muro se interpone entre la cámara y el jugador, el objeto se vuelve parcialmente transparente, evitando ocultar información relevante para el gameplay.

Para comenzar, organizaremos nuestro proyecto con la misma estructura modular que hemos utilizado durante los capítulos anteriores:

- 1 Dentro de la carpeta **chapter_04**, crearemos una subcarpeta llamada **vfx_camera_distance**, donde almacenaremos los recursos necesarios para esta sección.
- 2 Dentro de **vfx_camera_distance**, crearemos cuatro subcarpetas:
 - a **materials**.
 - b **shaders**.
 - c **meshes**.
 - d **textures**.

Antes de continuar, debemos asegurarnos de colocar en estas carpetas los archivos asociados a la sección, incluyendo modelos de rocas, pinos, texturas y cualquier otro recurso entregado. Luego crearemos un **Shader** y su respectivo **ShaderMaterial**, los cuales nombraremos siguiendo nuestra convención de proyecto.

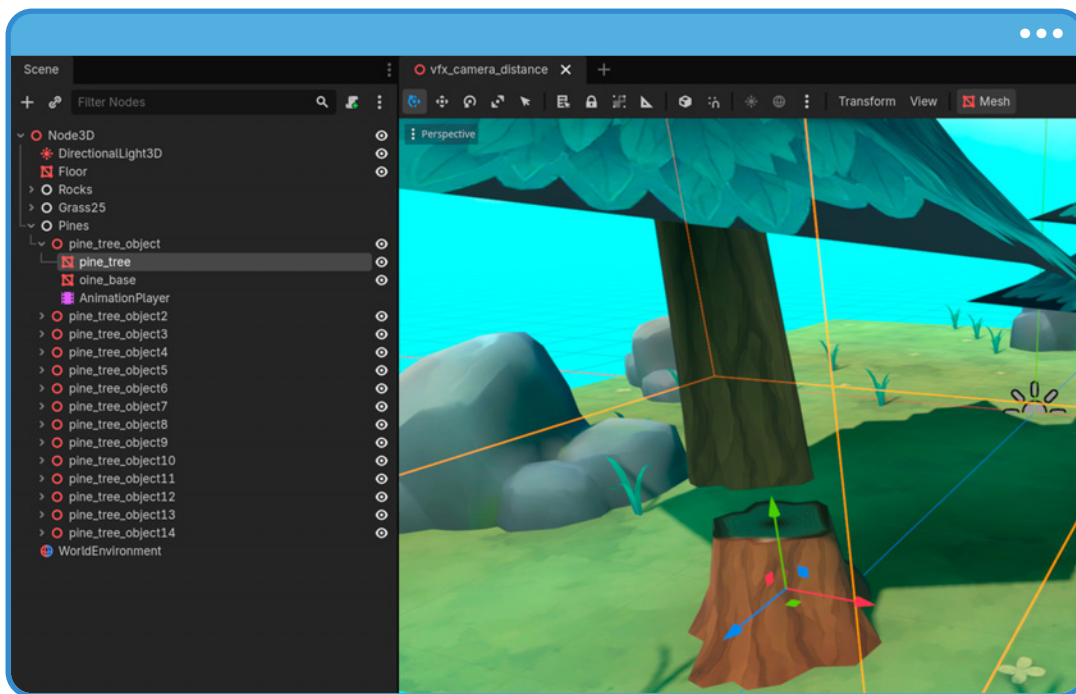
Si hemos realizado cada paso correctamente, nuestro proyecto debería verse de la siguiente manera:



(4.3.a Estructura del proyecto)

Si observamos con atención la Figura 4.3.a, notaremos que en la carpeta meshes se incluyen varios objetos 3D que podemos emplear para construir nuestra composición. No obstante, el más relevante para este ejercicio es **forest_pine**. Como su nombre lo indica, este recurso corresponde a un pino, y lo particular es que se encuentra dividido en dos mallas independientes: **pine_tree** y **pine_base**.

El shader que creamos previamente lo aplicaremos exclusivamente a **pine_tree**, mientras que **pine_base** conservará su material original. Gracias a esta separación, podremos reproducir un efecto de transparencia progresiva similar al utilizado en los pinos del juego **DOGWALK**, desarrollado por el equipo de Blender. Esta técnica resulta especialmente útil porque nos permite atenuar únicamente la parte superior del pino cuando el personaje queda cubierto por la vegetación, manteniendo la base sólida y visible para aportar coherencia visual a la escena.



(4.3.b El objeto **forest_pine** está dividido en dos partes)

Antes de avanzar con la lógica del shader, prepararemos una pequeña composición dentro de nuestra escena. Esto nos permitirá evaluar con claridad los cambios que aplicaremos al canal alfa en los siguientes pasos. Para ello, debemos asegurarnos de asignar correctamente el material que acabamos de crear:

- Asignamos el **Shader** al **ShaderMaterial**, verificando que ambos recursos estén ubicados en las carpetas indicadas.
- Aplicamos el **ShaderMaterial** exclusivamente al nodo **pine_tree** de cada uno de los modelos **forest_pine** presentes en la escena.



(4.3.c Se ha aplicado el ShaderMaterial a cada pino en la escena)

El primer paso que llevaremos a cabo en nuestro shader será incorporar las propiedades de textura, de modo que podamos apreciar mejor la silueta del pino. Si abrimos la carpeta **textures** de nuestro proyecto, veremos que el canal alfa de la textura principal se encuentra en una imagen independiente. En un proyecto real podríamos empaquetar esta información en la misma textura, pero aquí la separamos con fines educativos, para visualizar con mayor claridad qué aporta cada mapa.

```

shader_type spatial;
render_mode diffuse_toon;
render_mode cull_disabled;

uniform sampler2D _MainTexRGB : source_color;
uniform sampler2D _MainTexA : source_color;

void vertex() { ... }

```

De este fragmento de código ya conocemos varias cosas, pero aparecen dos modos de render que aún no hemos revisado en el libro: **diffuse_toon** y **cull_disabled**. El primero aplica un sombreado difuso estilizado, más cercano a un look ilustrado, mientras que el segundo desactiva el descarte de caras y nos permite ver tanto la parte interior como la exterior del modelo. Ambos modos de render son opcionales, pero añaden un toque extra de estilo al resultado final.

Luego, iremos al método **fragment()** e inicializaremos las texturas, como se muestra a continuación:

```

void fragment()
{
    vec3 albedo = texture(_MainTexRGB, UV).rgb;
    float alpha = texture(_MainTexA, UV).r;

    ALBEDO = albedo;
    ALPHA = alpha;
}

```

Si regresamos al Viewport, lo primero que vamos a notar es un problema en cómo el render pipeline interpreta las transparencias y la profundidad entre los objetos. Este comportamiento, conocido como **depth sorting issue**, aparece porque los materiales transparentes no pueden escribir directamente en el Depth Buffer: necesitan mezclar su color con todo lo que ya se ha dibujado por detrás.

Para resolverlo, el motor ordena los objetos transparentes según su distancia a la cámara y los dibuja de atrás hacia adelante. Cuando esa ordenación falla (por geometrías complejas, objetos que se cruzan o cambios bruscos de cámara), obtenemos resultados visuales incorrectos: elementos que deberían quedar detrás se ven por delante, o ciertas partes parecen “saltarse” de orden al mover la cámara.



(4.3.d Problemas de profundidad en la transparencia)

Para solucionar este problema, necesitamos realizar primero un pre-pase de profundidad sobre la parte opaca de los objetos y utilizar ese resultado al momento de dibujar las transparencias. En Godot, contamos con un modo de render llamado **depth_prepass_alpha**, que precisamente habilita este pre-pase y hace que el motor escriba la información de profundidad antes de mezclar los fragmentos transparentes.

```
shader_type spatial;
render_mode diffuse_toon;
render_mode cull_disabled;
render_mode depth_prepass_alpha;
```

Al activar **depth_prepass_alpha**, indicamos a Godot que el objeto se renderice en dos etapas: primero se registra su profundidad (como si fuera opaco) y luego se aplica la transparencia respetando correctamente qué fragmentos deberían quedar delante o detrás. Con esto mitigamos los problemas de *depth sorting* más evidentes en nuestros pinos.

Sin embargo, todavía nos enfrentamos a otro inconveniente relacionado con las transparencias: el borde de translucidez (o “halo”) que aparece alrededor de las hojas, donde se percibe una transición poco definida entre el contorno opaco de la hoja y las zonas transparentes de la textura, como podemos observar en la siguiente referencia.



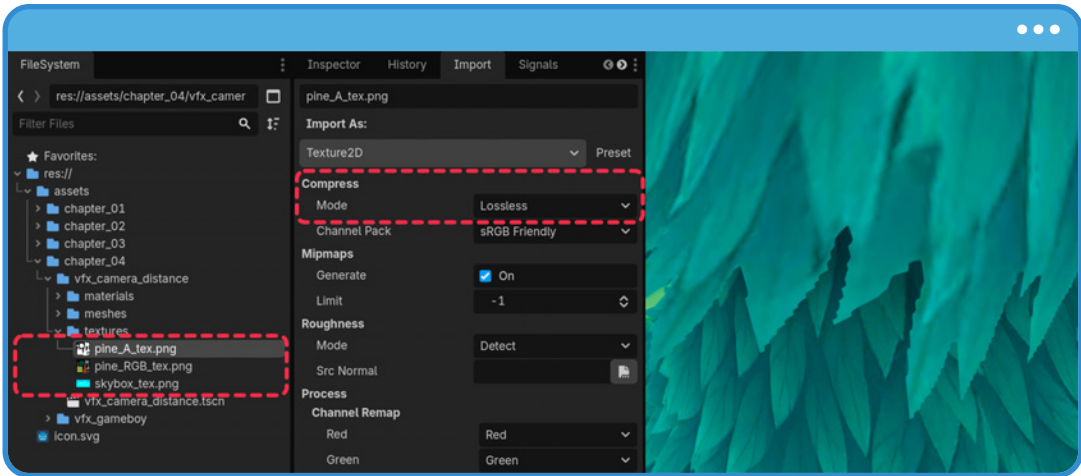
(4.3.e Se han generado bordes duros en la transparencia)

Este problema aparece principalmente por dos razones:

- 1 Por el tamaño y la compresión aplicada a la textura.
- 2 Porque aún no hemos aplicado ningún tipo de antialiasing sobre los bordes recortados.

Si revisamos nuestro proyecto y seleccionamos la textura desde la pestaña **Import**, veremos que su compresión está configurada como **VRAM Compressed**. Este formato es adecuado para optimizar memoria, pero introduce artefactos visibles en áreas donde el canal alfa tiene transiciones bruscas, como ocurre en las hojas de un árbol.

Si modificamos esta opción a **Lossless**, evitamos la pérdida generada por la compresión y los bordes del recorte aumentan su definición, lo que produce un acabado más limpio y suave alrededor de la silueta del modelo. Esta mejora resulta especialmente útil cuando trabajamos con texturas que dependen fuertemente del canal alfa para definir su forma, como es el caso de vegetación, cabello o elementos dibujados a mano.



(4.3.f Se ha cambiado la configuración de compresión de la textura)

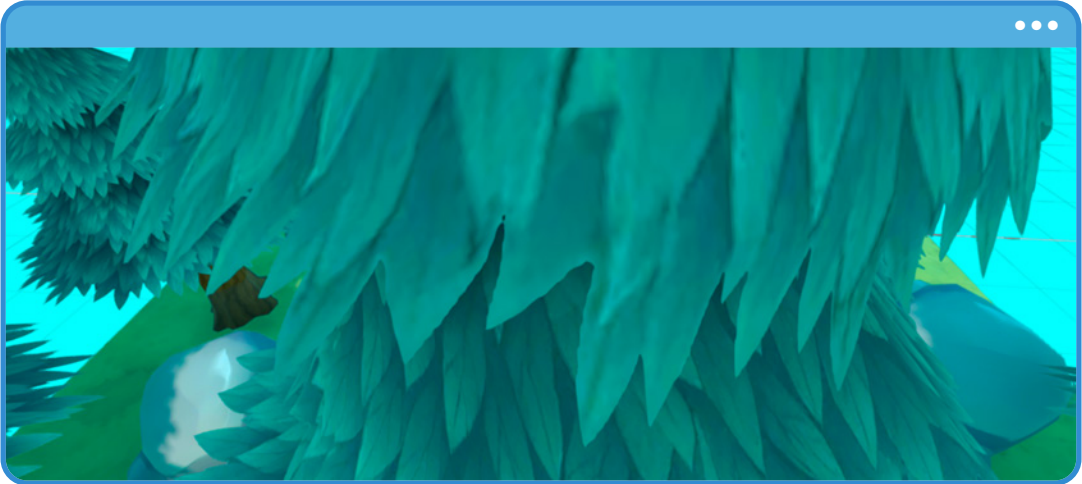
Como podemos observar en la Figura 4.3.f, la definición del borde transparente mejora aproximadamente un 40 %. Sin embargo, aún podemos optimizarlo sin incrementar el peso ni la compresión de la textura. Para ello, Godot ofrece una variable interna llamada **ALPHA_ANTIALIASING_EDGE**, la cual suaviza los contornos calculando un borde transicional entre las zonas opacas y transparentes del fragmento. Esta variable actúa directamente en el proceso de mezcla (Blending), evitando recortes bruscos y reduciendo de manera notable el serrucho visual típico de las texturas con alpha recortado.

Para utilizarla, simplemente la asignamos dentro del método **fragment()**, justo después de definir el valor de **ALPHA**:

```
void fragment()
{
    vec3 albedo = texture(_MainTexRGB, UV).rgb;
    float alpha = texture(_MainTexA, UV).r;

    ALBEDO = albedo;
    ALPHA = alpha;
    ALPHA_ANTIALIASING_EDGE = 0.5;
}
```

Si prestamos atención a la escena nuevamente, veremos que los bordes presentan una transición más suave y natural, lo que mejora de forma significativa la integración visual del follaje en nuestra composición.



(4.3.g Se ha mejorado la resolución del borde de la transparencia)

Habiendo solucionado los problemas asociados a la transparencia, es momento de comenzar a implementar un gradiente basado en la distancia. Para lograrlo, necesitamos acceder tanto a la posición de la cámara como a la posición de cada vértice en espacio global. Con esta información podremos calcular qué tan lejos se encuentran los fragmentos del árbol respecto del observador y, en consecuencia, modificar su visibilidad o color.

Para comenzar, agregaremos nuevas propiedades y variables globales en nuestro shader, tal como se muestra a continuación:

```
uniform sampler2D _MainTexRGB : source_color;
uniform sampler2D _MainTexA : source_color;
// 1
uniform float _Distance : hint_range(0.0, 10.0, 0.1);
// 2
varying vec3 vertex_ws;

// El código continúa en la página siguiente.
```

```
void vertex()
{
    // 3
    vertex_ws = (MODEL_MATRIX * vec4(VERTEX, 1.0)).rgb;
}
```

¿Qué estamos haciendo aquí?

- 1 Declaramos una nueva propiedad llamada **_Distance**, la cual utilizaremos para controlar la intensidad del gradiente según la distancia entre la cámara y los vértices del objeto.
- 2 Creamos una variable global **vertex_ws**, donde almacenaremos la posición de cada vértice en espacio global. Esto nos permitirá evaluar, fragmento a fragmento, qué tan lejos se encuentra la geometría del árbol.
- 3 Transformamos los vértices desde espacio local a espacio global mediante **MODEL_MATRIX**.

Podemos emplear la variable **vertex_ws** dentro del cálculo de distancia para obtener el efecto que buscamos. Para ello, iremos al método **fragment()** y actualizaremos su contenido de la siguiente manera:

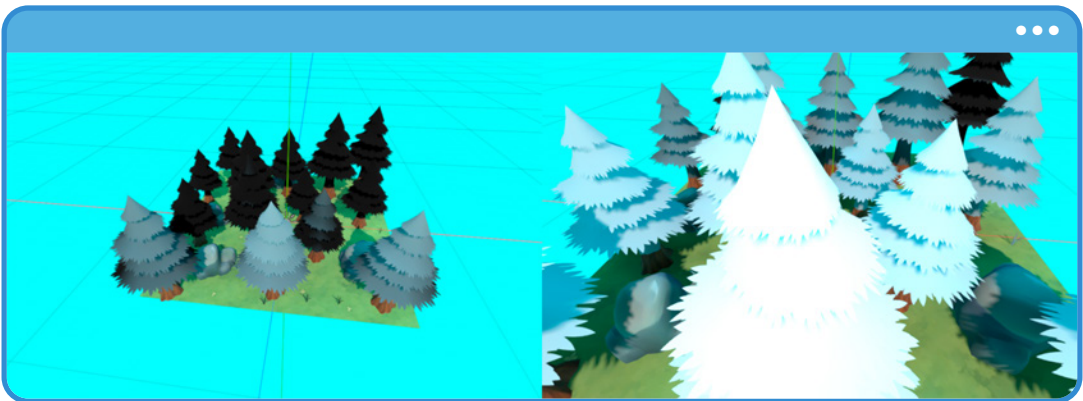
```
void fragment()
{
    vec3 albedo = texture(_MainTexRGB, UV).rgb;
    float alpha = texture(_MainTexA, UV).r;
    // 1
    float distance_color = clamp(distance(vertex_ws, CAMERA_POSITION_WORLD) /
    ↪ _Distance, 0.0, 1.0);
    // 2
    ALBEDO = vec3(1.0 - distance_color);

    ALPHA = alpha;
    ALPHA_ANTIALIASING_EDGE = 0.5;
}
```


Como podemos observar,

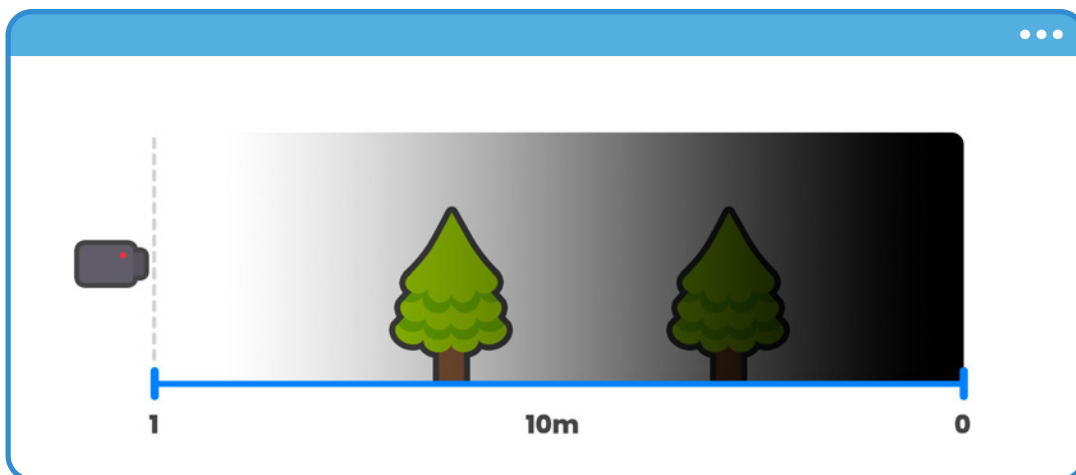
- 1 Declaramos una nueva variable llamada **distance_color**, que almacena la distancia normalizada entre cada vértice del objeto y la cámara. Utilizamos la función **distance()** en espacio global y limitamos el resultado al rango $[0 : 1]$ mediante **clamp()**. Esto nos entrega un valor preparado para emplearse como factor de mezcla o como color en un gradiente.
- 2 De manera temporal, asignamos **distance_color** al canal **ALBEDO** para visualizar el comportamiento del gradiente directamente sobre el objeto. Invertimos el valor con **1.0 - distance_color** para resaltar mejor las zonas cercanas a la cámara mientras realizamos pruebas.

Si regresamos al Viewport, veremos que el color del pino varía dinámicamente según su cercanía a la cámara. Podemos ajustar la intensidad y alcance del gradiente modificando el valor de la propiedad **Distance** en el **Inspector**.



(4.3.h Configuración con **Distance** igual a 10)

Ahora bien, ¿qué está ocurriendo en el cálculo de distancia en términos gráficos? Para entender el concepto, prestemos atención a la siguiente referencia:



(4.3.i El color producido por la distancia se ha invertido)

Dado que el resultado de la operación corresponde a un valor normalizado entre 0 y 1, podemos emplearlo directamente para modular la transparencia del objeto. Sin embargo, si aplicamos esta transición tal cual, la variación del gradiente será demasiado amplia y afectará zonas que no deberían volverse translúcidas. Para lograr un comportamiento más controlado, necesitamos “cerrar” la curva del gradiente, de modo que el cambio de transparencia ocurra únicamente en una franja específica del objeto que se encuentra frente a la cámara.

Para conseguir este efecto más localizado, podemos aplicar una función de suavizado o curva de transición sobre el valor resultante antes de asignarlo al canal alfa. Esto nos permitirá definir con mayor precisión qué áreas se desvanecen y en qué rango de distancia sucede el efecto.

```
void fragment()
{
    vec3 albedo = texture(_MainTexRGB, UV).rgb;
    float alpha = texture(_MainTexA, UV).r;

    float distance_color = clamp(distance(vertex_ws, CAMERA_POSITION_WORLD) /
        ↪ _Distance, 0.0, 1.0);

    // El código continúa en la página siguiente.
```

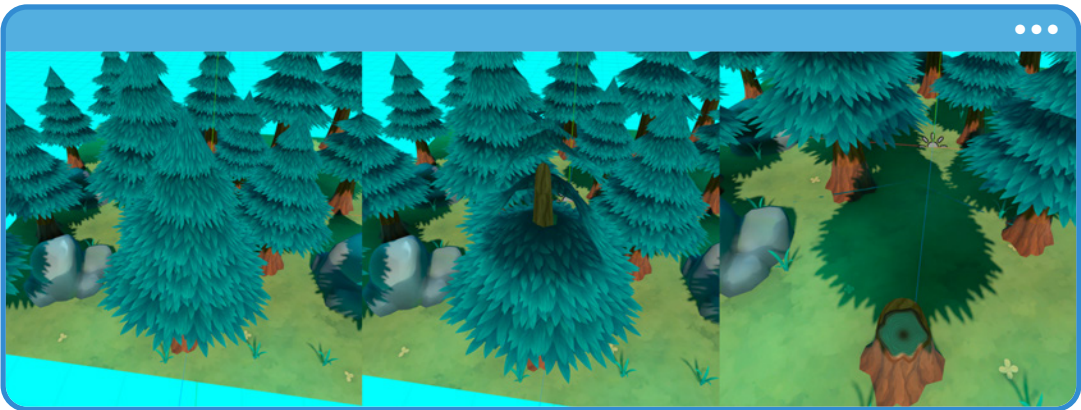
```
// 1
float focus = pow(distance_color, 2.0);

ALBEDO = albedo;
// 2
ALPHA = alpha * focus;
ALPHA_ANTIALIASING_EDGE = 0.5;
}
```

¿Qué está ocurriendo aquí?

- 1 El valor **distance_color** representa la distancia normalizada entre los vértices del objeto y la cámara. Este valor es lineal por naturaleza, por lo que lo elevamos al cuadrado para producir una transición más suave y progresiva en las zonas cercanas, y más agresiva en las lejanas.
- 2 Multiplicamos el canal alfa original por **focus** para controlar la visibilidad en función de la distancia. De esta manera, las zonas más cercanas a la cámara se desvanecen progresivamente, mientras que las partes más alejadas conservan su opacidad.

Esta implementación genera el siguiente resultado visual:



(4.3.j) Transparencia según distancia entre los vértices y la cámara)

Como podemos observar en la Figura 4.3.i, cuando acercamos la cámara a los objetos, la transparencia funciona correctamente. Sin embargo, desde el punto de vista visual el borde de transición es demasiado abrupto, lo que produce cortes duros y resta naturalidad al efecto.

Una forma habitual de suavizar este comportamiento es introducir “ruido” en la composición, de modo que la transición entre opacidad y transparencia ocurra de manera más orgánica. Este tipo de técnica se utiliza con frecuencia en efectos de dithering, fade-outs atmosféricos o transiciones estilizadas, ya que permite romper la linealidad del gradiente y evitar que se perciban patrones demasiado rígidos.

Para implementar este comportamiento, agregaremos la siguiente función sobre el método `fragment()`:

```
float hash21(vec2 uv)
{
    return fract(sin(dot(uv, vec2(12.9898, 78.233))) * 43758.5453);
}
```

Esta función genera un valor pseudoaleatorio entre 0 y 1 a partir de un par de coordenadas UV. Internamente evalúa un producto punto, lo pasa por una función seno y posteriormente extrae su parte fraccionaria. Aunque no corresponde a un generador de ruido físicamente preciso, es más que suficiente para producir variaciones aleatorias de alta frecuencia, ideales para suavizar bordes o generar patrones de dithering sobre transparencias.

A continuación, podríamos utilizar las coordenadas de pantalla para generar el ruido:

```
void fragment()
{
    vec3 albedo = texture(_MainTexRGB, UV).rgb;
    float alpha = texture(_MainTexA, UV).r;

    float distance_color = clamp(distance(vertex_ws, CAMERA_POSITION_WORLD) /
        ↪ _Distance, 0.0, 1.0);
    float focus = pow(distance_color, 2.0);
    // 1
    float noise = hash21(SCREEN_UV);

    // El código continúa en la página siguiente.
```

```
// 2
noise = mix(noise, 1.0, focus) * focus;

ALBEDO = albedo;
// 3
ALPHA = alpha * noise;
ALPHA_ANTIALIASING_EDGE = 0.5;
}
```

Si prestamos atención al fragmento anterior, podemos notar:

- 1 Declaramos una nueva variable llamada **noise**, cuyo valor corresponde al resultado de **hash21()** aplicado sobre **SCREEN_UV**. Esto genera un ruido estático distribuido de manera uniforme por toda la pantalla, lo que nos permitirá romper los bordes duros del gradiente.
- 2 Ajustamos el ruido mediante **mix(noise, 1.0, focus)** y luego lo multiplicamos por **focus**. Con ello logramos que el ruido sea más notorio en las zonas donde el gradiente es más fuerte, mientras que desaparece de manera natural en las áreas donde el objeto aún es opaco.
- 3 Finalmente, multiplicamos el canal alfa original por el valor resultante de **noise**, generando un patrón irregular que suaviza la transición entre zonas opacas y transparentes. Esto evita el corte abrupto que observábamos anteriormente y aporta un acabado más natural al efecto.

Si regresamos al Viewport, notaremos que los bordes que antes se veían demasiado rígidos ahora presentan una dispersión de ruido, lo que mejora considerablemente la estética y suavidad del desvanecimiento.



(4.3.k Los bordes de transparencia ahora poseen ruido)

En este punto, podríamos dar nuestro efecto por finalizado. Sin embargo, si deseamos acentuar aún más el comportamiento visual, podemos incorporar color a las zonas afectadas por la transparencia. Para ello, declaramos una nueva propiedad de tipo color, como se muestra a continuación:

```
uniform sampler2D _MainTexRGB : source_color;
uniform sampler2D _MainTexA : source_color;
uniform float _Distance : hint_range(0.0, 10.0, 0.1);
uniform vec3 _EdgeColor : source_color;
```

A continuación, emplearemos esta propiedad dentro del método **fragment()** para generar una interpolación lineal que combine el color original del objeto con un color de borde:

```
void fragment()
{
    ...

    albedo = mix(_EdgeColor, albedo, focus);

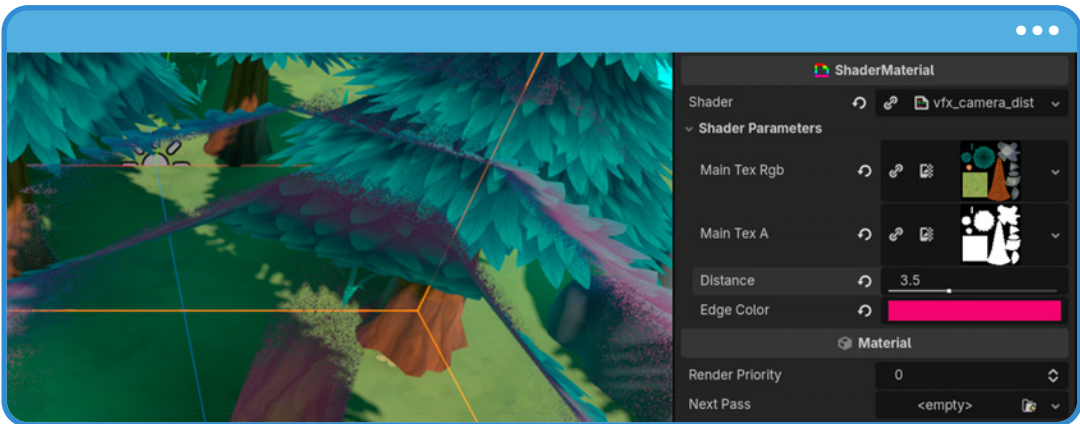
    // El código continúa en la página siguiente.
```

```

ALBEDO = albedo;
ALPHA = alpha * noise;
ALPHA_ANTIALIASING_EDGE = 0.5;
}

```

Gracias a esta interpolación, los fragmentos cercanos a la cámara adoptan gradualmente el color definido en **_EdgeColor**, mientras que las zonas más alejadas conservan el color original de la textura. Si regresamos al Inspector y modificamos el valor de **_EdgeColor**, veremos cómo dicho color aparece progresivamente a medida que la cámara se acerca al objeto 3D, reforzando el efecto de desvanecimiento con un tinte visual más expresivo.



(4.3.I. Se ha empleado un color violeta sobre el borde de la transparencia)

4.4 Ray Marching y texturas 3D.

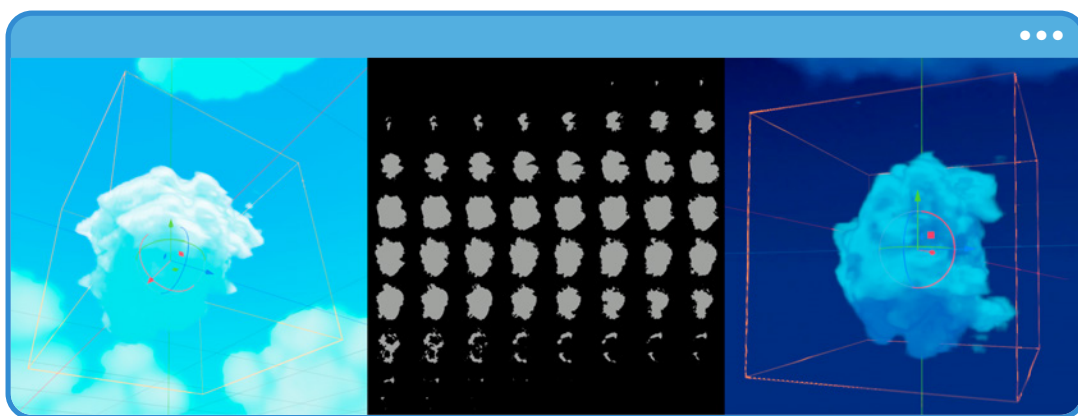
Generalmente, cuando hablamos de ray marching, también se suele asociar esta técnica con funciones de distancia implícitas (SDF). Sin embargo, el ray marching no está limitado a este enfoque. También puede emplearse junto con texturas tridimensionales, lo que resulta especialmente útil para representar fenómenos volumétricos como nubes, humo o niebla, otorgándonos un mayor control artístico sobre el resultado final.

El ray marching es un método que permite generar volúmenes tridimensionales lanzando un rayo desde cada píxel de la pantalla y avanzando en una dirección determinada mediante pasos sucesivos. En cada uno de estos pasos, se evalúa una función que describe el volumen.

Dependiendo del enfoque, esta función puede representar una distancia (como ocurre con las SDF) o un valor de densidad obtenido desde una textura 3D.

En esta sección nos centraremos en el uso de texturas tridimensionales, donde cada muestreo del rayo devuelve un valor escalar de densidad. Dichos valores se integran a lo largo del recorrido del rayo para simular fenómenos físicos como la absorción de luz, la transmisión y el auto-sombreado volumétrico, características fundamentales en la representación de nubes y otros medios participativos.

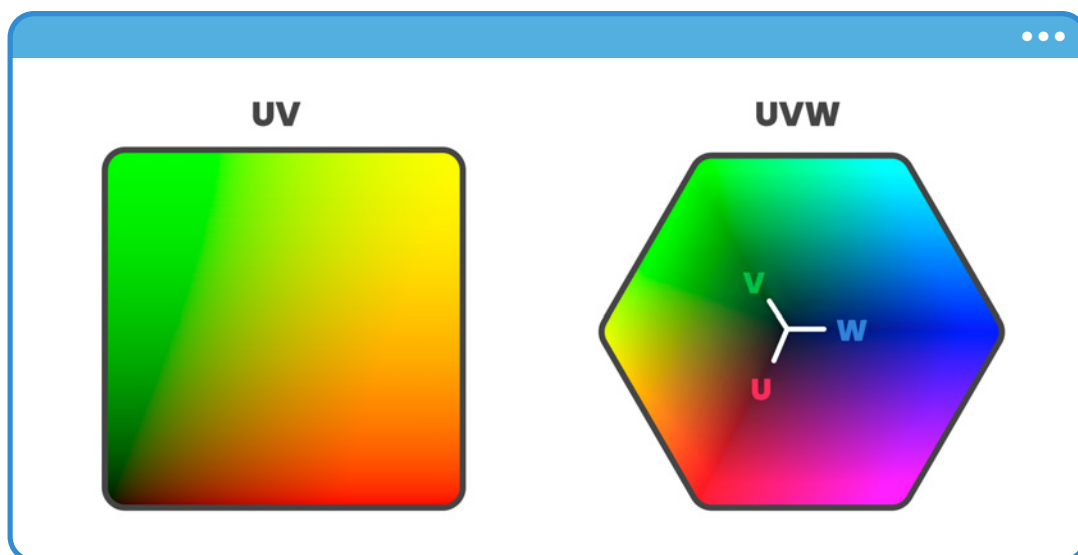
Antes de implementar el algoritmo, comenzaremos analizando las propiedades de una textura 3D. Para ello, prestemos atención a la siguiente figura.



(4.4.a Textura tridimensional de una nube)

Este volumen puede representarse mediante una textura 3D, es decir, un bloque de datos con ancho, alto y profundidad. En la práctica, esta información suele generarse a partir de múltiples “rebanadas” 2D (slices) apiladas en el eje de profundidad, o bien a partir de un recurso que internamente organiza dichas capas para construir el volumen que observamos en la Figura 4.4.a.

Para comprender este concepto, conviene estar familiarizados con las coordenadas UV. En el caso de las texturas 3D, añadimos una tercera componente y pasamos a trabajar con coordenadas UVW (o, de forma equivalente, XYZ), donde W representa la profundidad dentro del volumen.



(4.4.b Comparación entre coordenadas UV y UVW en GLSL)

Como ya sabemos, al trabajar con un **sampler2D**, cada muestra se determina mediante dos coordenadas UV que describen una posición dentro de un plano bidimensional. En cambio, un **sampler3D** requiere tres coordenadas: U, V y W. Esta tercera componente indica qué capa interna del volumen estamos muestreando, es decir, la profundidad a la que se encuentra el valor consultado.

Debemos considerar que añadir una dimensión extra incrementa tanto el costo computacional como el costo de memoria. Además, a diferencia de una textura 2D, que podemos visualizar directamente, una textura 3D representa un volumen de datos que no puede mostrarse en pantalla como una imagen única sin antes elegir una rebanada, proyectarla o recorrerla mediante un método de muestreo.

Nota

En esta sección nos centraremos en utilizar una textura 3D dentro del shader, por lo que no profundizaremos en su creación. Si deseas entender cómo se construyen estas texturas a partir de rebanadas 2D, puedes revisar un tutorial que muestre el flujo de trabajo y su organización de capas: https://www.youtube.com/shorts/IGx_I4nzfxQ

Las texturas 3D suelen almacenarse en formatos que preservan bien la información volumétrica. Por ejemplo, **.exr** es una opción común cuando necesitamos alta precisión o valores

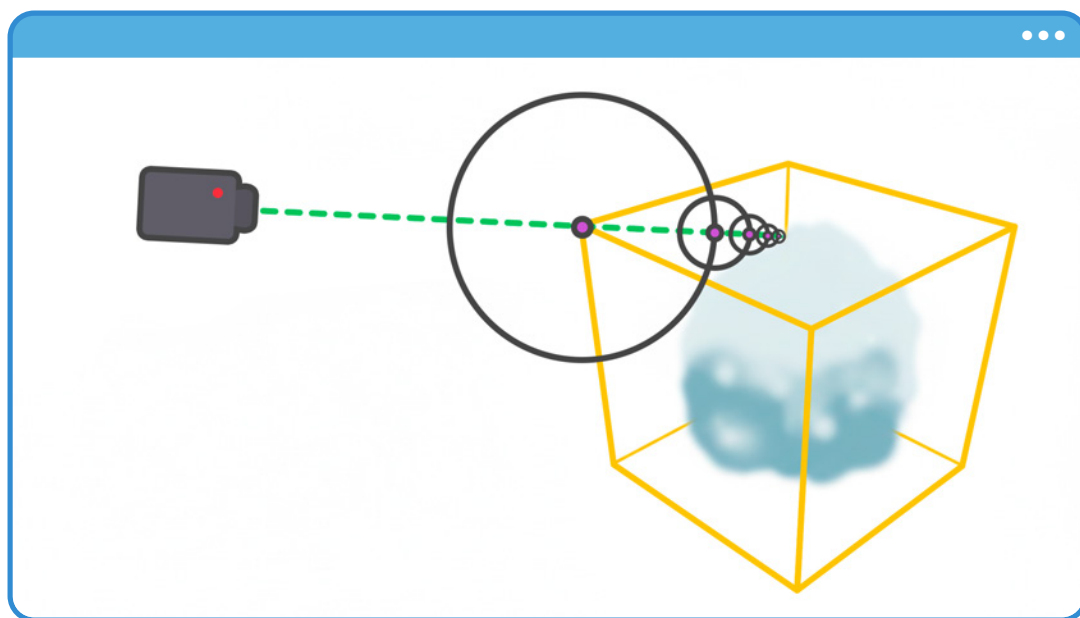
en rango HDR, lo cual resulta útil si la textura almacena densidad, ruido o datos intermedios con buena fidelidad.

En Godot también podemos utilizar imágenes en **.png**, aunque debemos recordar que, en la mayoría de los casos, **.png** es un formato LDR (a menudo 8 bits por canal), por lo que puede introducir cuantización y banding si pretendemos representar transiciones suaves de densidad. Por esta razón, cuando buscamos resultados más estables y sin artefactos, suele ser preferible trabajar con formatos de mayor precisión.

Ahora bien, ¿cómo renderizamos un volumen definido por una textura 3D dentro de un shader? Aquí es donde entra la técnica de ray marching.

El ray marching nos permite recorrer el volumen paso a paso a lo largo de un rayo lanzado desde cada píxel. En cada paso muestreamos la textura 3D para obtener un valor de densidad y, con esa información, integramos el comportamiento del medio (absorción, transmisión y auto-sombreado). De este modo reconstruimos la apariencia visual del objeto volumétrico que deseamos renderizar.

Para entender este proceso, prestemos atención a la siguiente figura.



(4.4.c Ray Marching en acción)

Para implementar esta técnica en nuestro shader, debemos tener en cuenta una serie de factores clave. Entre ellos se encuentran la dirección del rayo, su longitud de recorrido y la forma en que este interactúa con el volumen para determinar la densidad acumulada en función de la distancia con respecto a la cámara. En nuestro caso, la nube estará contenida dentro de un cubo genérico, por lo que también será necesario conocer la posición global del objeto que actúa como contenedor del volumen.

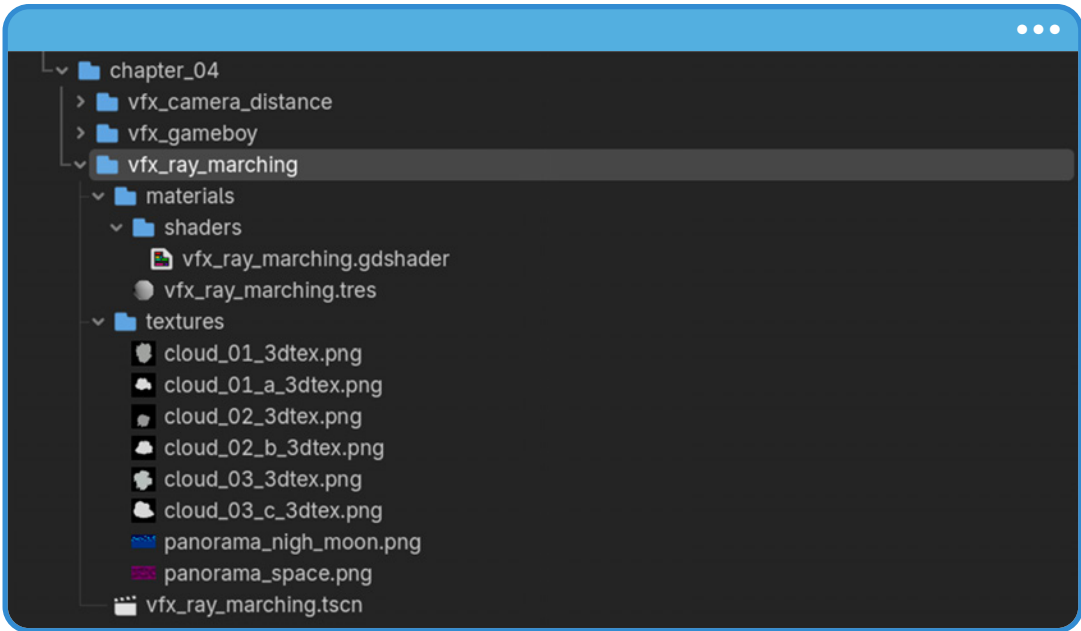
Antes de entrar en la implementación del shader, organizaremos el proyecto de manera que podamos trabajar de forma clara y ordenada con los recursos involucrados.

Para ello, seguiremos los siguiente pasos:

- 1 Dentro de la carpeta **chapter_04**, crearemos una subcarpeta llamada **vfx_ray_marching**, donde almacenaremos los recursos utilizados en esta sección.
- 2 Dentro de esta carpeta, crearemos a su vez tres subcarpetas:
 - a **materials**.
 - b **shaders**.
 - c **textures**.

Siguiendo la estructura habitual del libro, crearemos tanto un **Shader** como un **ShaderMaterial** asociados a esta sección. Por motivos prácticos y de consistencia, los llamaremos **vfx_ray_marching**. Asimismo, incluiremos las texturas 3D necesarias para el efecto, las cuales pueden descargarse desde el proyecto adjunto a este libro.

Si todos los pasos se han realizado correctamente, la estructura del proyecto debería verse como se muestra en la siguiente figura.



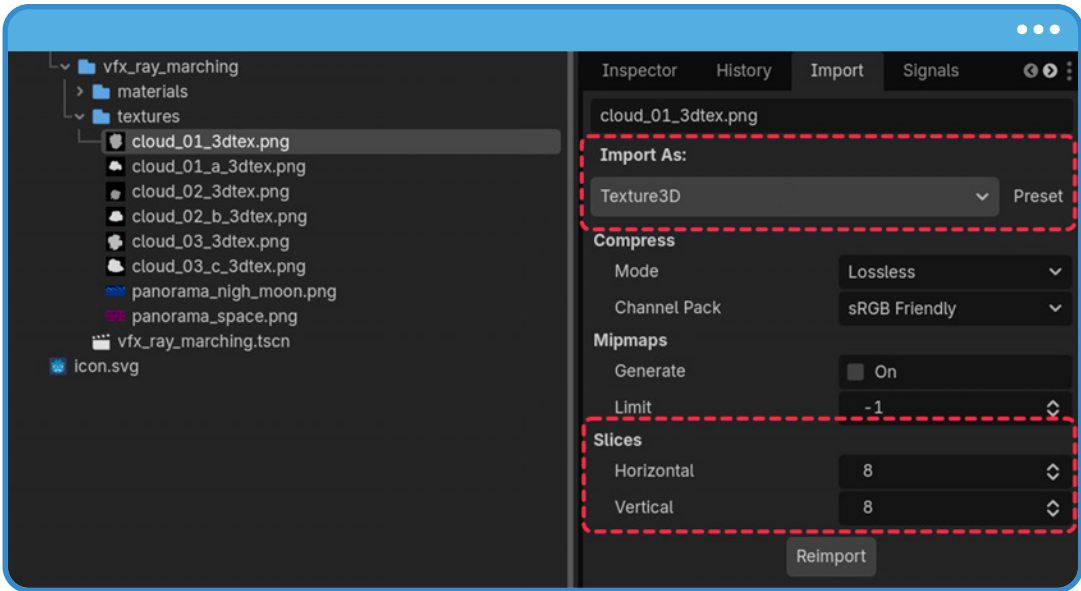
(4.4.d Estructura del proyecto)

El primer paso consiste en configurar la escena donde renderizaremos el volumen. Para ello, añadiremos un **BoxMesh** a la escena, el cual actuará como contenedor del volumen. Tal como mencionamos anteriormente, la nube se renderizará dentro de este cubo, por lo que su geometría define los límites espaciales del efecto volumétrico.

El siguiente paso corresponde a la configuración de las texturas 3D. Cuando importamos imágenes en Godot, estas se configuran por defecto como **Texture2D**, lo que provoca que, al tratarse de un conjunto de imágenes, se muestren organizadas en forma de cuadrícula. Sin embargo, esta no es la configuración que necesitamos para trabajar con un volumen tridimensional.

Para convertir estas imágenes en una textura 3D, realizaremos el siguiente procedimiento:

- 1 Seleccionamos la imagen que compone la textura volumétrica.
- 2 Nos dirigimos a la ventana **Import**.
- 3 Desde **Import As**, seleccionaremos **Texture3D**.
- 4 Configuraremos el número de **Slices** con un valor de **8 en Horizontal** y **8 en Vertical**, de acuerdo con la disposición de las imágenes.
- 5 Finalmente, presionaremos **Reimport** para aplicar los cambios.

(4.4.e La textura ha sido configurada como **Texture3D**)

Una vez configurada correctamente la textura como **Texture3D**, podemos comenzar con el desarrollo del shader. Iniciaremos definiendo un conjunto de propiedades globales que utilizaremos posteriormente tanto para el cálculo del color como para la evaluación del volumen durante el proceso de ray marching.

```
shader_type spatial;
render_mode unshaded;

uniform sampler3D _BaseTex : source_color, repeat_disable;
uniform vec3 _ColorLight : source_color;
uniform vec3 _ColorShadow : source_color;

void vertex() { ... }
```

El **sampler3D _BaseTex** representa la textura volumétrica que almacena la densidad del medio y será muestreada a lo largo del recorrido del rayo. Por su parte, las propiedades **_ColorLight** y **_ColorShadow** nos permitirán definir una variación cromática en función de la densidad e iluminación del volumen, lo que resulta especialmente útil para resaltar su forma y profundidad visual.

Es importante destacar que nuestro shader se declara como **unshaded**. Esto implica que no se utilizarán los cálculos de iluminación estándar del pipeline de Godot. Sin embargo, esto no significa que el volumen carezca completamente de iluminación. A lo largo de esta sección implementaremos un modelo de iluminación simplificado, calculado manualmente dentro del shader, con el objetivo de mejorar la lectura visual del volumen y enfatizar su estructura tridimensional.

Nota

La implementación de ray marching que desarrollaremos a continuación está basada en una solución creada por **DMeville**, la cual ha sido adaptada y portada a Godot por razones de simplicidad y claridad didáctica. La versión original puede consultarse en el siguiente repositorio: <https://github.com/DMeville/DMVolumeRendering?tab=readme-ov-file>

Continuaremos definiendo un conjunto de constantes globales que utilizaremos más adelante durante el proceso de renderización de nuestra nube 3D. Estas constantes controlan tanto la calidad visual como el costo computacional del algoritmo de ray marching, por lo que su correcta configuración es fundamental.

```
render_mode unshaded;
// 1
#define NUM_STEPS 128
// 2
#define STEP_SIZE 0.01
// 3
#define NUM_LIGHT_STEPS 6
// 4
#define LIGHT_STEP_SIZE 0.05
```

¿Qué función cumplen estas constantes?

- 1 **NUM_STEPS:** Define la cantidad de muestras que el rayo de vista tomará a lo largo de su recorrido dentro del volumen. Cada paso representa una evaluación de la densidad de la textura 3D. Un valor mayor incrementa la precisión y suavidad del volumen, pero también eleva el costo de procesamiento.

- 2 **STEP_SIZE**: Determina la distancia que el rayo de vista avanza en cada iteración. Junto con **NUM_STEPS**, establece la longitud total del recorrido del rayo dentro del volumen. Pasos más pequeños permiten un muestreo más fino, mientras que pasos más grandes reducen el número efectivo de evaluaciones.
- 3 **NUM_LIGHT_STEPS**: Indica cuántos pasos se utilizan para evaluar la iluminación volumétrica. En este caso, se emplea un rayo secundario que avanza en dirección a la luz para estimar la cantidad de densidad que bloquea su paso, lo que nos permite simular auto-sombreado dentro del volumen.
- 4 **LIGHT_STEP_SIZE**: Define la distancia que el rayo de luz avanza en cada uno de estos pasos. Este valor controla el alcance y la suavidad de las sombras volumétricas. Un tamaño de paso mayor genera sombras más suaves pero menos precisas, mientras que uno menor produce resultados más definidos a costa de mayor costo computacional.

Comencemos definiendo el método que utilizaremos para renderizar nuestra nube volumétrica. Para ello, nos posicionaremos sobre el método **fragment()** y declararemos una nueva función a la que llamaremos **ray_marching()**, tal como se muestra a continuación:

```
vec3 ray_marching()
{
    return vec3(0.0);
}

void fragment() { ... }
```

Como podemos observar, este nuevo método aún no realiza ningún cálculo. Por el momento, su única función es devolver un color negro por defecto, de este modo, establecemos primero la estructura del algoritmo antes de incorporar la lógica del ray marching.

A continuación, añadiremos las variables fundamentales que utilizaremos para describir la interacción de la luz con el volumen de la nube:

```
vec3 ray_marching()
{
    float density = 0.0;
    float transmission = 0.0;
    float light_accumulation = 0.0;
    float final_light = 0.0;

    return vec3(0.0);
}
```

Estas variables representan magnitudes clave en el renderizado volumétrico, por ejemplo, **density** almacena la densidad acumulada del volumen a lo largo del recorrido del rayo. La variable **transmission** describe cuánta luz logra atravesar el medio tras interactuar con dicha densidad. A su vez, **light_accumulation** se utiliza para estimar la cantidad de densidad entre un punto del volumen y la fuente de luz, lo que nos permitirá simular auto-sombreado. Finalmente, **final_light** representa la iluminación integrada final que contribuirá al color del fragmento (píxel).

Estas cantidad propias del efecto, son fundamentales para modelar cómo la luz se absorbe, se transmite y atenúa dentro de un medio participativo como una nube.

Nota

Estas variables forman parte de la base física del renderizado volumétrico. Si deseas profundizar en los principios que rigen la interacción de la luz con volúmenes, recomendamos revisar el video de Sebastián Lague, donde se explica este proceso de forma clara y visual: <https://youtu.be/4QOcCGI6xOU?si=WdTzlr6LEcw8SgKV>

Como comentamos al comienzo de esta sección, para determinar el volumen de la nube es necesario avanzar en una dirección determinada mediante pasos sucesivos, evaluando la densidad almacenada en la textura 3D. Cuando hablamos de “pasos sucesivos”, nos referimos directamente al uso de un bucle **for()**, que nos permite recorrer el volumen de manera incremental a lo largo del rayo de vista.

En esta etapa, nuestro objetivo principal es muestrear y acumular la densidad del volumen. Esto lo podemos lograr de la siguiente manera:

```

vec3 ray_marching(vec3 ray_origin, vec3 ray_direction, vec3 offset)
{
    float density = 0.0;
    float transmission = 0.0;
    float light_accumulation = 0.0;
    float final_light = 0.0;
    // 1
    for (int i = 0; i < NUM_STEPS; i++)
    {
        // 2
        ray_origin += (ray_direction * STEP_SIZE);
        // 3
        vec3 sampled_position = ray_origin + offset;
        // 4
        float sample_density = texture(_BaseTex, sampled_position).r;
        // 5
        density += sample_density;
    }

    return vec3(0.0);
}

```

¿Qué está ocurriendo en nuestro código?

- ❶ El bucle **for()** controla la cantidad de muestras que el rayo tomará a lo largo de su recorrido dentro del volumen. Cada iteración representa un paso del proceso de ray marching.
- ❷ En cada iteración, desplazamos el origen del rayo en la dirección indicada por **ray_direction**, avanzando una distancia constante definida por **STEP_SIZE**. Esto determina la resolución espacial del muestreo.
- ❸ Calculamos la posición desde la cual se realizará el muestreo de la textura 3D. El valor **offset** se utiliza para alinear correctamente el volumen en el espacio y asegurar que la nube quede centrada dentro del **BoxMesh**.
- ❹ Consultamos la textura tridimensional **_BaseTex** utilizando **sampled_position** como coordenadas UVW. El valor obtenido representa la densidad del volumen en ese punto específico del espacio.

- 5 Sumamos la densidad muestreada al valor acumulado `density`. Esta suma discreta aproxima la cantidad total de materia que el rayo ha atravesado hasta el momento, y será utilizada posteriormente para calcular transmisión, absorción y sombreado volumétrico.

En definitiva, este fragmento implementa un recorrido incremental del rayo de vista a través del volumen. En cada paso se muestrea la densidad almacenada en la textura 3D y se acumula su contribución. Además, hemos incorporado nuevos argumentos que nos permitirán controlar el comportamiento del efecto más adelante desde el método `fragment()`.

El siguiente paso consiste en calcular la iluminación volumétrica, así como la transmisión y el sombreado interno del volumen. Para lograrlo, realizaremos una segunda iteración, esta vez en dirección a la luz, como se muestra a continuación:

```
vec3 ray_marching(vec3 ray_origin, vec3 ray_direction, vec3 offset, vec3
↳ light_direction)
{
    ...

    for (int i = 0; i < NUM_STEPS; i++)
    {
        ...

        // 1
        vec3 light_ray_origin = sampled_position;
        // 2
        for(int j = 0; j < NUM_LIGHT_STEPS; j++)
        {
            // 3
            light_ray_origin += (light_direction * LIGHT_STEP_SIZE);
            // 4
            float light_density = texture(_BaseTex, light_ray_origin).r;
            // 5
            light_accumulation += light_density;
        }

        // El código continúa en la página siguiente.
```

```

    }

    return vec3(0.0);
}

```

Como podemos observar:

- 1 Inicializamos el rayo hacia la luz. Tomamos la posición actual del rayo de vista **sampled_position** y la utilizamos como punto de partida para un rayo secundario que se dirige hacia la fuente de luz.
- 2 Este segundo bucle define cuántas muestras tomamos en dirección hacia la luz. Mientras más pasos, las sombras son más suaves o precisas, pero más es el costo de cómputo.
- 3 Desplazamos el punto de muestreo en la dirección de la luz, con un tamaño de paso fijo mediante **LIGHT_STEP_SIZE**.
- 4 Consultamos nuevamente la textura 3D para obtener la densidad del volumen en el punto actual del rayo de luz.
- 5 Finalmente, acumulamos la densidad muestreada en **light_accumulation**. A mayor densidad acumulada, menor cantidad de luz alcanzará el punto original, lo que se traduce en un mayor nivel de sombreado volumétrico.

Dado que **light_accumulation** puede acumularse a través de múltiples pasos del rayo principal, una optimización opcional consiste en reinicializarla dentro del primer bucle **for()**. De este modo, la oclusión hacia la luz se calcula de forma local para cada punto muestreado, lo que estabiliza el sombreado y suele producir un resultado más limpio.

A continuación, utilizamos la densidad acumulada a lo largo del rayo de luz para estimar cuánta iluminación alcanza el punto actual dentro del volumen. Con este valor construimos un factor de sombra que modulamos por un mínimo de oscuridad, lo incorporamos a la iluminación acumulada y, al mismo tiempo, actualizamos la transmitancia del rayo de vista, es decir, cuánta energía luminosa aún “sobrevive” luego de atravesar el medio.

```

vec3 ray_marching(vec3 ray_origin, vec3 ray_direction, vec3 offset, vec3
↳ light_direction, float darkness, float transmittance, float light_absorb)
{
    ...

    for (int i = 0; i < NUM_STEPS; i++)
    {
        ...
        // 1
        float light_accumulation = 0.0;

        for(int j = 0; j < NUM_LIGHT_STEPS; j++)
        {
            ...
        }

        // 2
        float light_transmission = exp(-light_accumulation);
        // 3
        float shadow = darkness + light_transmission * (1.0 - darkness);
        // 4
        final_light += density * transmittance * shadow;
        // 5
        transmittance *= exp(-density * light_absorb);
    }

    // 6
    transmission = exp(-density); // exp(-density * LIGHT_STEP_SIZE)
    // 7
    return vec3(final_light, transmission, transmittance);
}

```

¿Qué está ocurriendo aquí? Vamos a revisarlo:

- 1 Reiniciamos **light_accumulation** por cada paso del rayo de vista. Con esto, la sombra calculada representa únicamente la oclusión local entre el punto actual y la fuente de luz, evitando que el sombreado se vea afectado por acumulaciones provenientes de pasos anteriores.

- 2 Transformamos la densidad acumulada a lo largo del rayo de luz en un valor de transmisión utilizando una función exponencial. Si **light_accumulation** es alto, entonces **exp(-x)** se vuelve pequeño, lo que implica que llega menos luz y, por lo tanto, el punto se sombrea con mayor intensidad.
- 3 Calculamos **shadow** mezclando la transmisión con el argumento **darkness**. En este contexto, **darkness** actúa como un mínimo de iluminación en sombra: incluso si la transmisión hacia la luz es casi cero, el volumen conservará un nivel base de luz, lo que ayuda a evitar regiones completamente negras y mantiene legibilidad visual.
- 4 Sumamos la contribución de luz del paso actual a **final_light**. Esta contribución se modula por las variables **density**, **transmittance** y **shadow**.
- 5 Actualizamos la transmitancia del rayo principal, aplicando absorción. El argumento **light_absorb** controla qué tan rápido el volumen “consume” la luz. A mayor densidad, más disminuye la transmitancia.
- 6 Calculamos un valor global de transmisión a partir de la densidad acumulada total. En este paso, podríamos multiplicar a la densidad por **LIGHT_STEP_SIZE** para obtener transparencia en la densidad de la nube.
- 7 Finalmente, retornamos tres valores: la iluminación acumulada del volumen **final_light**, la transmisión global estimada **transmission**, y la transmitancia progresiva del rayo principal **transmittance**.

Con esto, completamos la implementación del método de **ray_marching()** y ya contamos con una función capaz de muestrear densidad y estimar iluminación dentro del volumen.

A continuación, utilizaremos este método dentro de la etapa de fragmento para calcular el volumen por cada píxel en pantalla. Para ello, nos dirigiremos al método **fragment()** e inicializaremos las siguientes variables:

```
void fragment()
{
    // 1
    vec3 ray_origin_ws = (INV_VIEW_MATRIX * vec4(VERTEX.xyz, 1.0)).xyz;

    // El código continúa en la página siguiente.
```

```

// 2
vec3 ray_direction = normalize(ray_origin_ws - CAMERA_POSITION_WORLD);
// 3
vec4 transform = (MODEL_MATRIX * vec4(0.0, 0.0, 0.0, 1.0));
// 4
vec3 offset = (vec4(0.5, 0.5, 0.5, 0.0) - transform).xyz;

ALBEDO = vec3(0.0);
ALPHA = 1.0;
}

```

Si prestamos atención, observaremos lo siguiente:

- ❶ Calculamos **ray_origin_ws** transformando la posición del fragmento mediante **INV_VIEW_MATRIX**. En este contexto, obtenemos una posición en espacio global a partir de la información disponible en el shader.
- ❷ Calculamos la dirección del rayo desde la cámara hacia el punto actual del mundo y la normalizamos. Al normalizar, garantizamos que el vector tenga longitud 1, de modo que **STEP_SIZE** represente una distancia constante en cada iteración del ray marching.
- ❸ Dado que estamos renderizando la nube dentro de un **BoxMesh**, necesitamos conocer la posición mundial del origen del objeto (su pivote). Para ello, multiplicamos el origen local (0, 0, 0, 1) por **MODEL_MATRIX**. Con esto, el volumen queda correctamente anclado al **BoxMesh**, de manera que si movemos el cubo en la escena, la nube se desplaza junto a él.
- ❹ Calculamos **offset** para centrar el muestreo del volumen dentro del **BoxMesh**. El vector (0.5, 0.5, 0.5) nos permite compensar el centro del volumen, y al restar la posición global del objeto **transform** alineamos el sistema de coordenadas de muestreo con el contenedor.

Además, como parte del proceso, hemos inicializado temporalmente **ALBEDO** y **ALPHA**. Esta asignación funciona como un punto de partida, ya que en los siguientes pasos reemplazaremos ambos valores con el resultado final calculado mediante ray marching.

Para continuar con la implementación, declararemos e inicializaremos los argumentos restantes que requiere **ray_marching()**. En particular, definiremos la dirección de la luz, un valor mínimo de iluminación para las zonas en sombra, la transmitancia inicial del rayo y

el coeficiente de absorción del volumen. Finalmente, ejecutaremos el método para el píxel actual y utilizaremos su salida para definir el color del fragmento.

```
void fragment()
{
    ...

    // 1
    const vec3 light_direction = vec3(0.0, 1.0, 0.0);
    // 2
    const float darkness = 0.19;
    // 3
    const float transmittance = 1.0;
    // 4
    const float light_absorb = 1.5;
    // 5
    vec3 render = ray_marching(
        ray_origin_ws,
        ray_direction,
        offset,
        light_direction,
        darkness,
        transmittance,
        light_absorb
    );

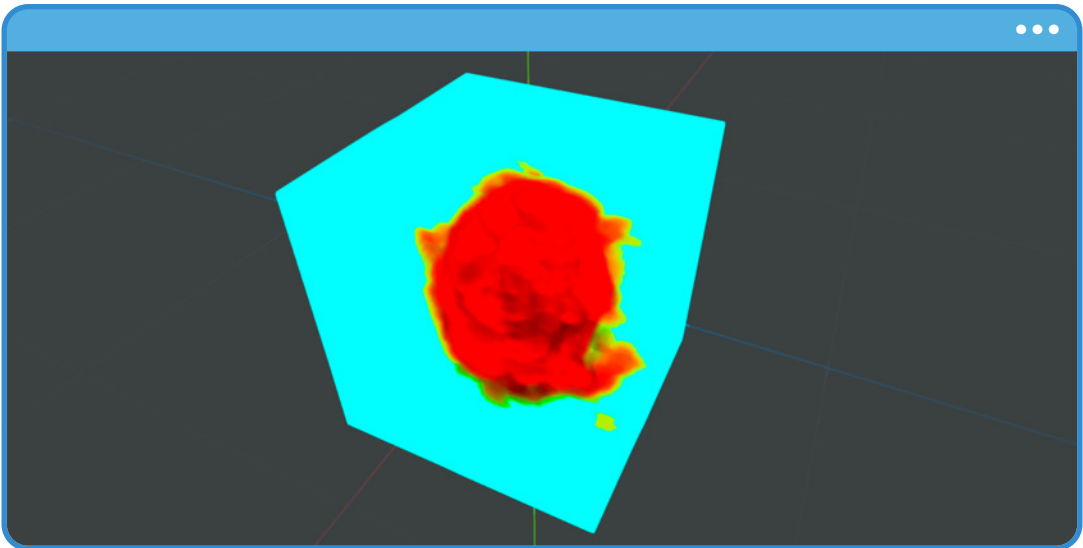
    // 6
    ALBEDO = render;
    ALPHA = 1.0;
}
```

Como podemos observar:

- 1 Definimos la dirección desde la cual incide la luz sobre el volumen. En este caso, utilizamos el eje +y, por lo que la luz proviene desde arriba. En una implementación más completa, podríamos obtener esta dirección desde la variable **LIGHT**.
- 2 Este valor define el mínimo de iluminación en las zonas sombreadas. Con **darkness = 0.0**, el volumen puede presentar sombras mucho más profundas; con

darkness = 1.0, eliminamos completamente el sombreado, ya que el término de sombra se mantiene constante.

- 3 Este valor representa la transmitancia inicial del rayo, es decir, cuánta luz “sobrevive” antes de comenzar a atravesar el volumen. Un valor igual a 0.0 genera una nube completamente oscura.
- 4 Este parámetro controla la tasa de absorción del medio. A valores más altos, el volumen absorbe más luz rápidamente, haciendo que el interior se oscurezca con mayor facilidad.
- 5 Ejecutamos el método **ray_marching()** para el píxel actual.
- 6 Asignamos el resultado a **ALBEDO**, que determina el color final del fragmento. En esta etapa mantenemos **ALPHA** fijo en 1.0 para visualizar el resultado de forma directa sobre el **BoxMesh**, tal como se aprecia en la Figura 4.4.f.



(4.4.f Render de la textura 3D, considerando **transmission** = **exp(-density)**)

Para completar el proceso de renderizado, solo resta calcular el color final del fragmento a partir de las dos propiedades de color declaradas de forma global, así como determinar el valor de transparencia del volumen. Para ello, utilizamos los valores devueltos por el método **ray_marching()**, los cuales nos permiten controlar tanto la variación cromática como el nivel de opacidad de cada píxel en función de la densidad integrada.

```

void fragment()
{
    ...

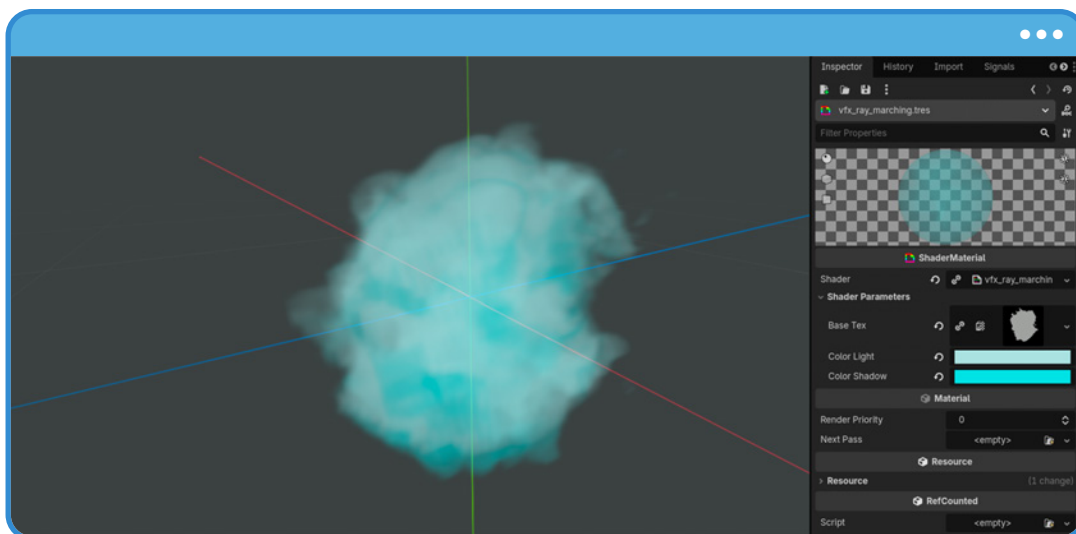
    // 1
    float gradient = clamp(render.x, 0.0, 1.0);
    // 2
    vec3 albedo = mix(_ColorShadow, _ColorLight, gradient);
    // 3
    float alpha = (1.0 - render.y);
    // 4
    ALBEDO = albedo;
    ALPHA = alpha;
}

```

¿Qué estamos realizando aquí?

- ❶ Extraemos la componente X del vector **render**, la cual representa la iluminación o densidad acumulada a lo largo del rayo. Además, limitamos su valor al rango [0 : 1] para asegurar que pueda utilizarse como factor de interpolación.
- ❷ Realizamos una interpolación lineal entre ambos colores, utilizando el valor **gradient**. De esta manera, las zonas más claras de la nube obtendrán el color de **_ColorLight**, mientras que las zonas más oscuras serán influenciadas por **_ColorShadow**.
- ❸ Interpretamos la componente Y del vector **render** como la transmisión acumulada del rayo a través del volumen. Al invertir este valor, obtenemos un coeficiente de opacidad coherente con el comportamiento físico del medio.
- ❹ Finalmente, asignamos el color calculado a **ALBEDO** y el valor de opacidad a **ALPHA**, completando así el renderizado volumétrico del fragmento en el Viewport.

Con esto, el efecto queda completamente parametrizado y funcional. Desde el Inspector, podemos modificar fácilmente los colores asociados a las zonas de mayor y menor densidad utilizando las propiedades **Color Light** y **Color Shadow**, lo que nos permite iterar visualmente sobre el resultado sin necesidad de realizar cambios adicionales en el código del shader.



(4.4.g Render final, $\text{transmission} = \exp(-\text{density} * \text{LIGHT_STEP_SIZE})$)

4.5 Introducción a Stencil Buffer.

Cuando se trata de efectos especiales, el **Stencil Buffer** es uno de los mecanismos más versátiles y potentes que podemos emplear en nuestros proyectos. Su principal función es permitir el enmascaramiento de regiones específicas de la pantalla, lo que abre la puerta a una amplia variedad de efectos visuales, como portales, efectos dimensionales, ocultamiento selectivo, outlines, máscaras personalizadas y más.

A nivel conceptual, el Stencil Buffer funciona como un Buffer auxiliar por píxel, independiente del Color y del Depth Buffer. Cada píxel de la pantalla posee un valor entero sin signo (unsigned integer **uint**) asociado a este Buffer, el cual puede ser leído o modificado durante el proceso de renderizado.

En la mayoría de las implementaciones, el Stencil Buffer utiliza 8 bits por píxel, lo que nos permite trabajar con valores que van desde 0 a 255. Por defecto, todos los píxeles comienzan con el valor 0, y este valor se mantiene hasta que un shader o una etapa del pipeline lo modifica explícitamente.

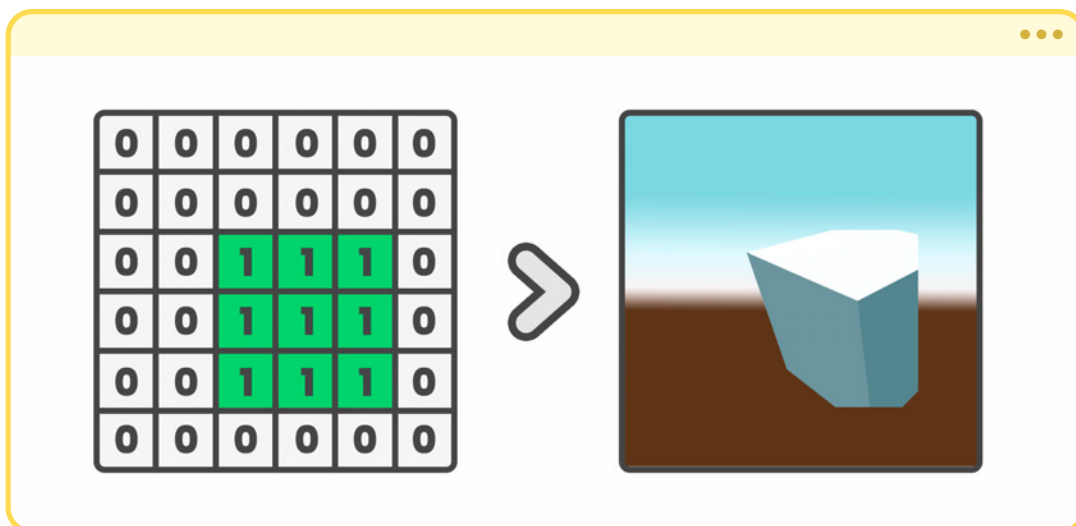
Para trabajar con Stencil correctamente, es importante tener en cuenta varias consideraciones:

- Qué tipo de comparación se realizará entre el valor almacenado en el Stencil Buffer y el valor definido por el shader.
- Si los píxeles serán descartados o no, lo cual impacta directamente en transparencias y recortes.
- El estado del **Depth Testing** y su interacción con el Stencil, especialmente en escenas con múltiples capas de geometría.
- El modo de Blending, en caso de combinar el resultado con otros objetos ya renderizados.
- El orden de renderizado (sorting), ya que el Stencil depende fuertemente de qué se dibuja primero y qué se evalúa después.

En Godot, mediante la propiedad `stencil_mode` en un shader, es posible escribir o leer valores del Stencil Buffer. Un flujo de trabajo común consiste en dividir el proceso en dos etapas:

- Escritura (**write**): Se renderiza un objeto o una geometría que actúa como máscara, asignando un valor específico al Stencil Buffer, por ejemplo, el valor 1 (o cualquiera distinto de 0).
- Lectura (**read**): Posteriormente, otro objeto se renderiza únicamente si el valor del Stencil Buffer en ese píxel coincide con el valor esperado (en este caso, 1).

Sí, por ejemplo, empleamos una comparación de tipo “equal”, el resultado de este proceso es que el segundo objeto solo será visible dentro del área previamente enmascarada, generando un efecto de recorte preciso controlado por la geometría de la máscara.



(4.5.a Aplicando una máscara sobre un BoxMesh)

Como pudimos observar en la Figura 1.5.b del Capítulo 1, el Stencil Test ocurre después del procesamiento del fragmento en la etapa del fragment shader y antes del depth test. Comprender este orden resulta fundamental, ya que nos permitirá analizar y resolver posteriormente problemas visuales relacionados con la profundidad de los objetos con respecto a la cámara.

Desde un punto de vista conceptual, tanto el **Stencil Test** como el **Depth Test** forman parte de la etapa conocida como pruebas por fragmento (per-fragment tests), la cual se evalúa una vez ejecutado el fragment shader. En este modelo lógico, el flujo de renderizado sigue siempre el mismo orden: primero se calcula el color del fragmento y luego se determina si dicho fragmento es válido o debe descartarse.

No obstante, en la práctica, es importante tener en cuenta que algunas GPUs modernas pueden adelantar estas pruebas mediante optimizaciones como early Depth Testing o early Stencil Testing. Estas optimizaciones permiten evaluar el stencil o la profundidad antes de ejecutar el fragment shader, siempre que el estado del shader y del pipeline lo permitan. Sin embargo, este comportamiento no está garantizado y depende de múltiples factores, como el uso de **discard**, escritura de profundidad o configuraciones específicas de Blending.

Por esta razón, desde el punto de vista del desarrollador, el flujo debe asumirse siempre de la siguiente manera: el fragment shader se ejecuta primero, luego se evalúa si el fragmento pasa o no el Stencil Test y, finalmente, se determina si supera o no el depth test. Si el fragmento

falla cualquiera de estas pruebas (o es descartado explícitamente), no se escribe ningún valor de color en el **FrameBuffer**, lo que provoca que dicho fragmento no sea visible en la imagen final.

Nota

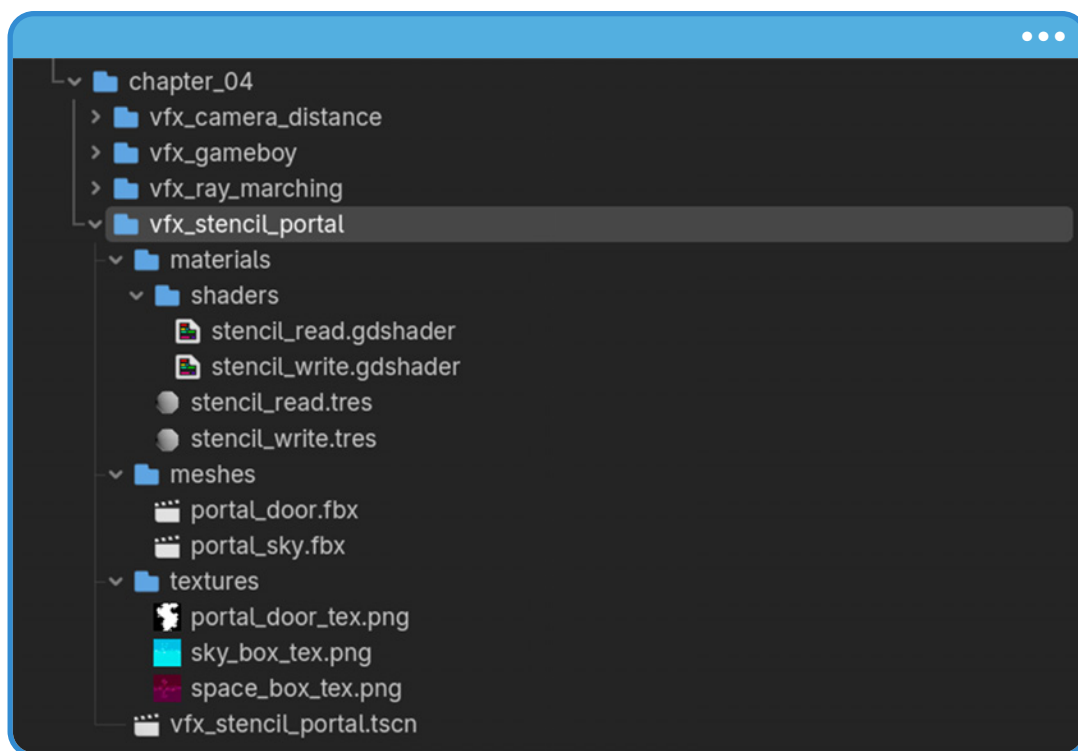
el **FrameBuffer** se refiere a la región de memoria donde la GPU almacena el resultado final del proceso de renderizado antes de que la imagen sea presentada en pantalla. Este Buffer contiene, entre otros, los valores del Color Buffer, Depth Buffer y el Stencil Buffer.

Para ver estos conceptos en práctica haremos lo siguiente:

- 1 Dentro de la carpeta **chapter_04**, crearemos una subcarpeta llamada **vfx_stencil_portal**, donde almacenaremos los recursos utilizados en esta sección.
- 2 Dentro de esta carpeta, crearemos a su vez tres subcarpetas:
 - a **materials**.
 - b **shaders**.
 - c **meshes**.
 - d **textures**.

Para esta sección, incluiremos tanto dos shaders como dos **ShaderMaterial** a los cuales llamaremos **stencil_write** y **stencil_read**, respectivamente. Además, incluiremos las texturas y objetos 3D necesarios para el efecto que desarrollaremos durante esta sección. Todos estos recursos pueden descargarse desde el proyecto adjunto a este libro.

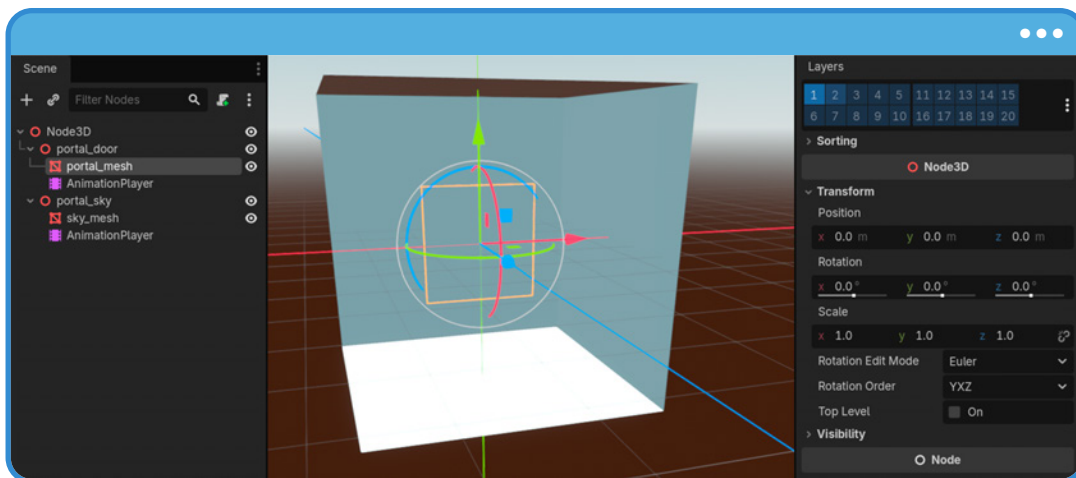
Si todos los pasos se han realizado correctamente, la estructura del proyecto debería verse como se muestra en la siguiente figura.



(4.5.b Estructura del proyecto)

El primer paso que llevaremos a cabo será organizar la escena. Para ello, crearemos una nueva escena 3D y arrastraremos los objetos **portal_door** y **portal_sky** a la misma, asegurándonos de que tanto su posición como su rotación se encuentren en $(0, 0, 0)$. Además, nos aseguraremos de marcar ambos objetos como **Make Local**, lo que nos permitirá acceder directamente a la propiedad **Mesh** de cada uno.

En este flujo de trabajo, utilizaremos el objeto **portal_door** como máscara, de modo que **portal_sky** será visible únicamente dentro del área delimitada por la geometría del portal.



(4.5.c Ambos objetos se encuentran centrados en la escena)

A continuación, asignaremos el material **stencil_write** y su shader correspondiente a la propiedad **Material Override** del objeto **portal_mesh**, tal como se muestra en la Figura 4.5.c. Luego, repetiremos el proceso utilizando el material **stencil_read** y su shader asociado sobre el objeto **sky_mesh**, el cual es hijo de **portal_sky**.

El objetivo de este flujo es dividir el proceso en dos etapas claramente diferenciadas: primero, definir el área de la máscara mediante la escritura en el Stencil Buffer y, posteriormente, restringir el renderizado de un objeto a dicha área.

Comenzaremos trabajando sobre el shader **stencil_write**, el cual se encargará de escribir un valor específico en el Stencil Buffer, definiendo así la región que actuará como máscara para el objeto **portal_sky**. Para ello, agregaremos las siguientes líneas de código:

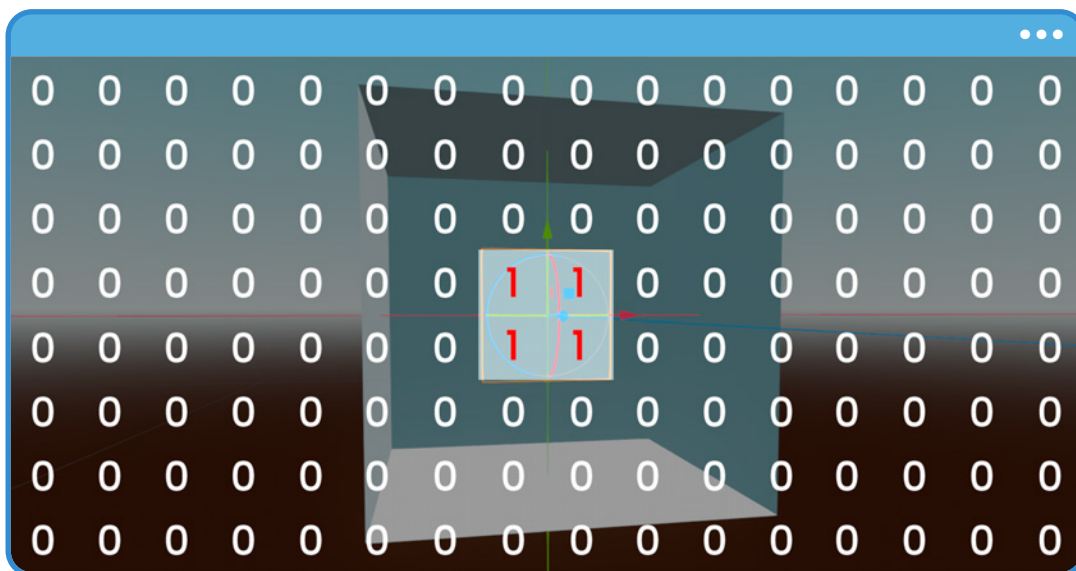
```
1 shader_type spatial;
2 render_mode unshaded;
3 stencil_mode write, 1;
```

Si prestamos atención a la tercera línea, **stencil_mode write, 1**, observaremos que esta instrucción define el comportamiento del shader con respecto al Stencil Buffer mediante dos parámetros bien definidos:

- 1 El primer parámetro, **write**, indica que este shader escribirá un valor en el Stencil Buffer en lugar de leerlo.
- 2 El segundo parámetro, **1**, especifica el valor entero **uint** que se almacenará en el Stencil Buffer para cada fragmento renderizado por este material.

En términos prácticos, cada fragmento generado por **portal_mesh** escribirá el valor 1 en el Stencil Buffer, delimitando de forma precisa el área que será utilizada posteriormente como máscara. En la siguiente etapa, este valor será evaluado por el shader **stencil_read**, permitiendo que el objeto **sky_mesh** se renderice únicamente en aquellos fragmentos donde el Stencil Buffer contenga dicho valor.

Desde una perspectiva visual, este proceso puede entenderse conceptualmente como se muestra a continuación:



(4.5.d Se ha asignado el valor 1 a cada fragmento del **portal_mesh**)

Podemos deducir que la imagen final contiene muchos más píxeles (o fragmentos) que los que se muestran en la Figura 4.5.d. No obstante, esta representación nos permite conceptualizar claramente el resultado del proceso. Por defecto, el Stencil Buffer asigna el valor

0 a todos los píxeles de la pantalla, mientras que el valor 1 ha sido escrito únicamente en aquellos fragmentos que corresponden al área cubierta por **portal_mesh**.

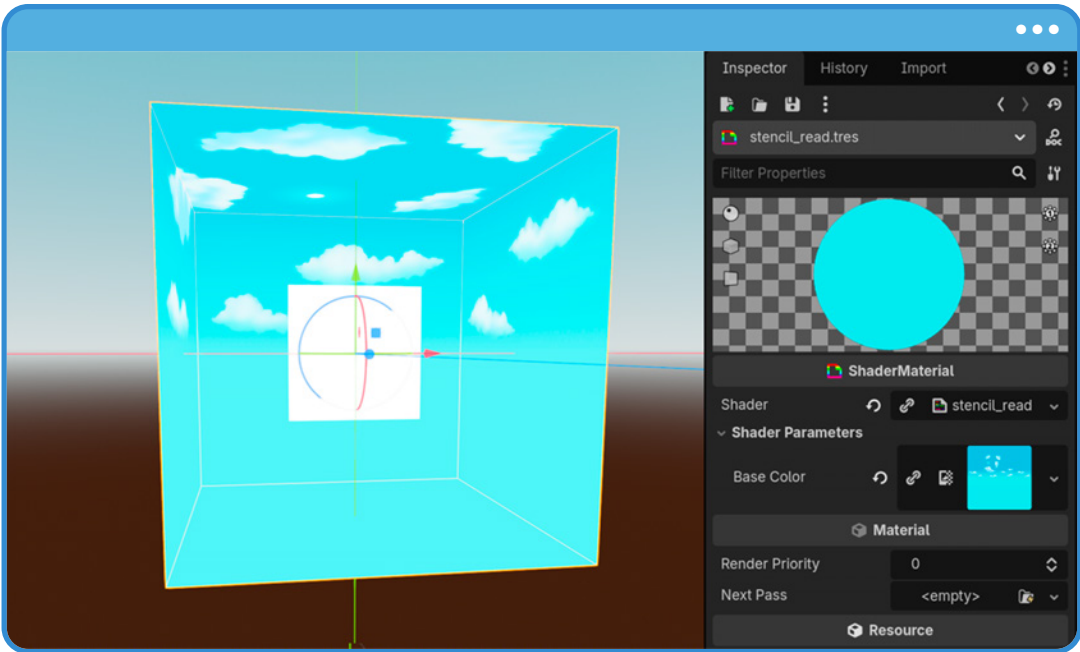
En este punto, si configuramos un shader para comparar el valor del Stencil Buffer en los píxeles que cubren el área del objeto **sky_mesh**, este último será visible únicamente dentro de los límites definidos por **portal_mesh**. Para visualizar este comportamiento de forma más clara, realizaremos el siguiente paso: abriremos el shader **stencil_read** y agregaremos las siguientes líneas de código. Inicialmente, implementaremos una textura para facilitar la observación del resultado.

```

1  shader_type spatial;
2  render_mode unshaded;
3
4  uniform sampler2D _BaseColor : source_color, repeat_disable;
5
6  void fragment()
7  {
8      vec3 albedo = texture(_BaseColor, UV).rgb;
9      ALBEDO = albedo;
10 }
```

Como podemos observar, se ha definido una textura denominada **_BaseColor**, la cual se utiliza para asignar el color **ALBEDO** en la etapa de fragmento. Esta implementación ya ha sido abordada en secciones anteriores, por lo que en este caso simplemente nos aseguraremos de asignar la textura **sky_box_tex** a su propiedad correspondiente dentro del material **stencil_read**.

Si todos los pasos se han realizado correctamente, obtendremos el siguiente resultado visual:



(4.5.e Se ha asignado una textura al material **stencil_read**)

En este punto, ya sabemos que los fragmentos que cubren el objeto **portal_mesh** poseen el valor 1 en el Stencil Buffer. Por lo tanto, si configuramos nuestro shader para comparar dicho valor, el cielo renderizado por el objeto **sky_mesh** será visible únicamente dentro del área delimitada por el portal. Para comprobar este comportamiento en la práctica, realizaremos la siguiente configuración en el shader **stencil_read**:

```

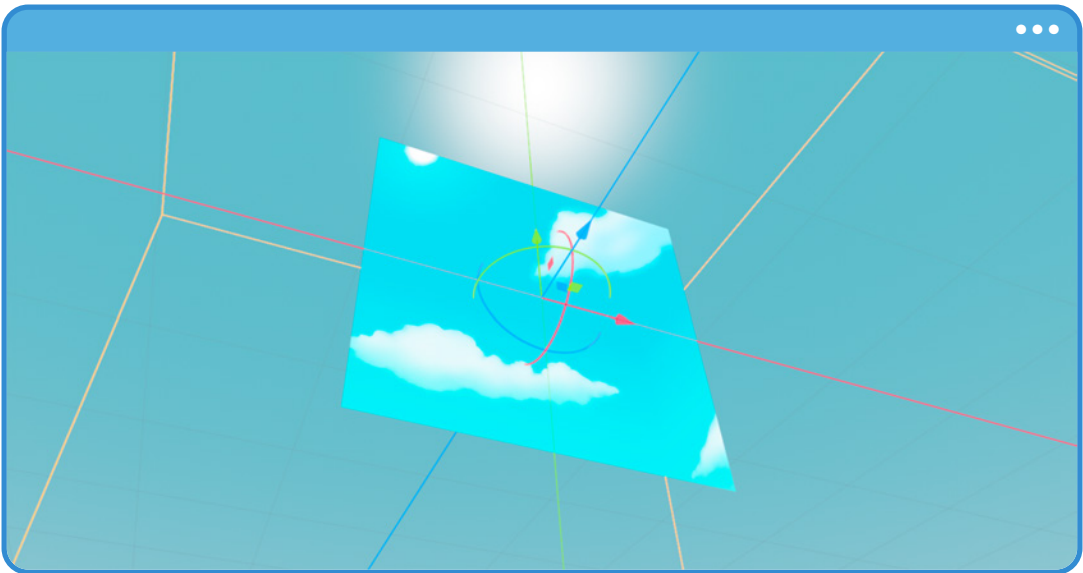
1  shader_type spatial;
2  render_mode unshaded, depth_draw_never, depth_test_disabled;
3  stencil_mode read, compare_equal, 1;
4
5  uniform sampler2D _BaseColor : source_color, repeat_disable;
6
7  void fragment()
8  {
9      vec3 albedo = texture(_BaseColor, UV).rgb;
10     ALBEDO = albedo;
11 }

```

¿Qué está ocurriendo aquí? Prestemos atención a las siguientes líneas de código:

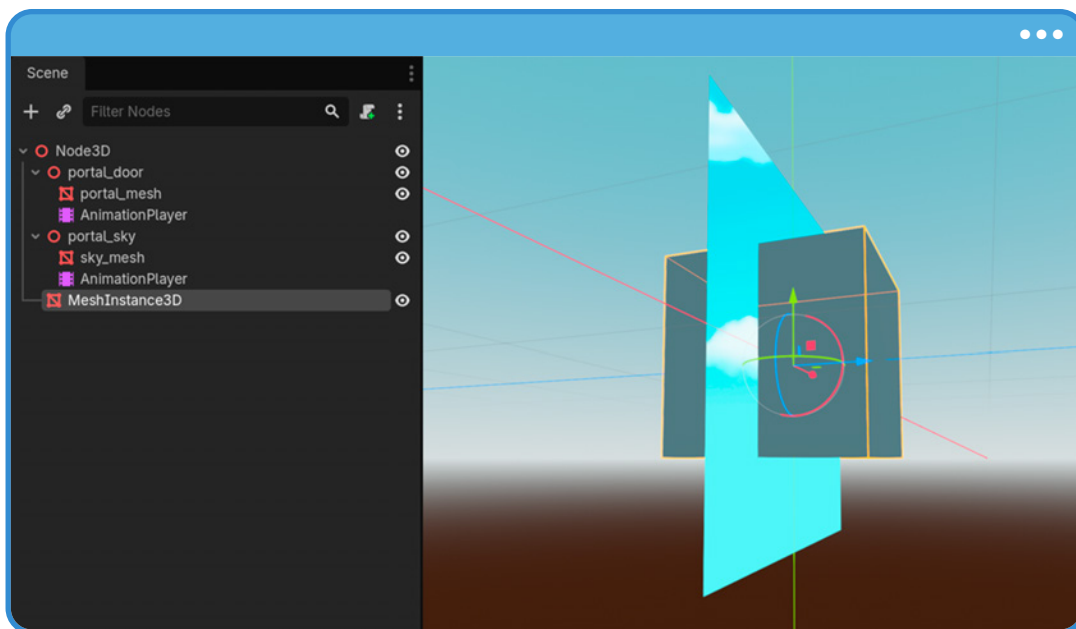
- Línea 2: Incorpora dos propiedades adicionales al modo de renderizado: Primero tenemos a **depth_draw_never**, el cual evita que este objeto escriba valores en el Depth Buffer (proceso siguiente del render pipeline). De esta manera, el cielo no afecta la profundidad de otros objetos renderizados posteriormente. Luego encontramos **depth_test_disabled**, el cual desactiva la comparación de profundidad, permitiendo que el cielo se dibuje independientemente de su distancia con respecto a la cámara.
- Línea 3: Esta instrucción define el comportamiento del shader con respecto al Stencil Buffer: La palabra **read** indica que el shader leerá el Stencil Buffer, sin modificarlo. Asimismo, **compare_equal** especifica que el fragmento solo será aceptado si el valor leído del Stencil Buffer es igual al valor de referencia, en este caso, 1.

Como resultado, únicamente los fragmentos cuyo valor de Stencil sea igual a 1 pasarán la prueba. Dado que este valor fue escrito previamente por **portal_mesh**, el cielo se renderizará exclusivamente dentro del área del portal, generando el efecto que estamos buscando.



(4.5.f el objeto **sky_mesh** se visualiza únicamente dentro del **portal_mesh**)

Para comprender con mayor claridad la relación entre las propiedades de profundidad y los mecanismos de comparación, podemos realizar el siguiente ejercicio práctico. Para ello, agregaremos un nuevo **MeshInstance3D** a la escena y lo centraremos junto a los objetos previamente configurados.

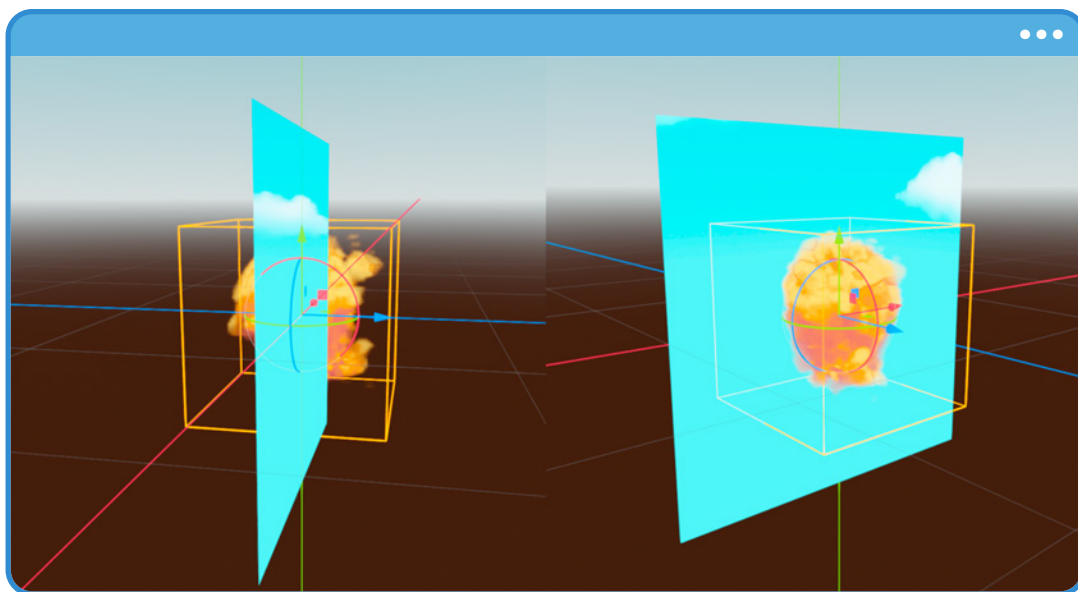


(4.5.g Se ha agregado un nuevo BoxMesh a la escena)

Como podemos observar, el nuevo BoxMesh se visualiza fuera del área delimitada por **portal_mesh**. Esto ocurre porque el Depth Buffer continúa escribiendo valores de profundidad para cada fragmento renderizado, y el depth test determina su visibilidad con respecto a la cámara y al resto de la geometría de la escena.

Si desactivamos el depth test, evitamos la escritura en el Depth Buffer o introducimos una comparación adicional mediante el Stencil Buffer, el comportamiento visual de este BoxMesh cambiará.

Para analizar este efecto en detalle, realizaremos el siguiente ejercicio: asignaremos al BoxMesh recién creado el material **vfx_ray_marching** utilizado en la sección anterior.

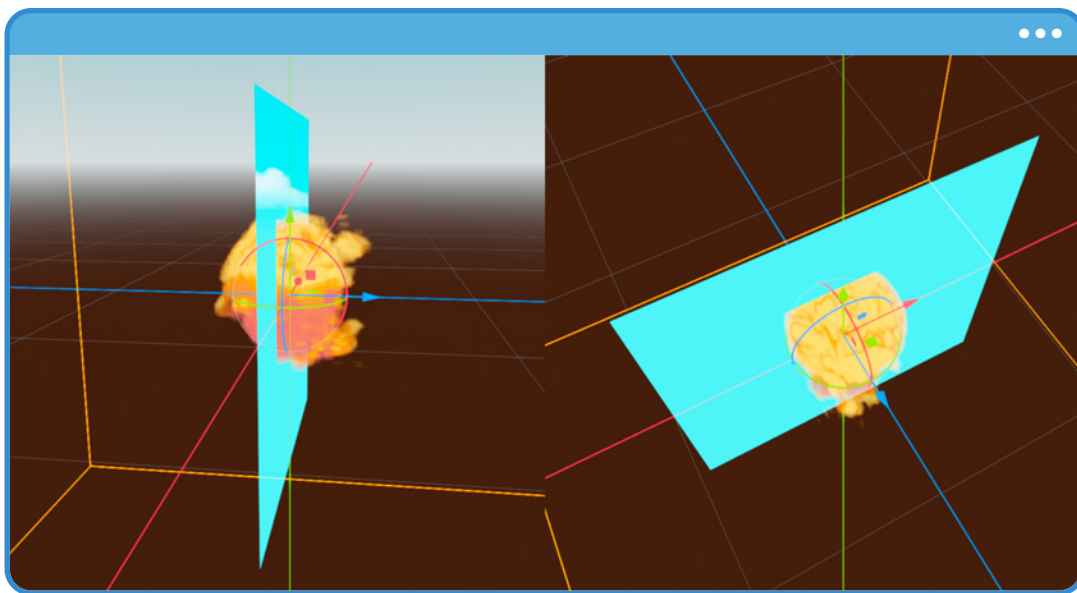


(4.5.h No se percibe la profundidad en la nube)

Si movemos la cámara alrededor de la nube, observaremos un efecto visual causado por el Depth Testing. Esto ocurre porque el BoxMesh, como geometría tradicional, realiza pruebas de profundidad para cada fragmento que lo compone, mientras que la nube renderizada mediante ray marching no escribe valores en el Depth Buffer (debemos calcularlo manualmente). Como resultado, se produce un parpadeo visual al rotar la cámara, debido a que la GPU no puede resolver correctamente el orden de visibilidad entre ambos elementos.

Este comportamiento puede corregirse parcialmente ajustando el orden de renderizado, concepto conocido como sorting. Actualmente, ambos materiales poseen la misma prioridad de procesamiento en la GPU. Sin embargo, si seleccionamos el objeto **sky_mesh** y modificamos el valor de **Offset** en su propiedad **Sorting** a un valor de -10.0 o menor, observaremos que la nube comienza a renderizarse de forma más estable con respecto a la profundidad.

No obstante, este ajuste introduce una limitación importante: aunque el orden de renderizado mejora, la GPU aún no puede determinar si la nube se encuentra dentro o fuera del área delimitada por **portal_mesh**. En otras palabras, hemos corregido el orden visual, pero no hemos resuelto completamente la relación espacial entre la nube, el portal y el resto de la escena.



(4.5.i **sky_mesh** se renderiza primero, luego la nube)

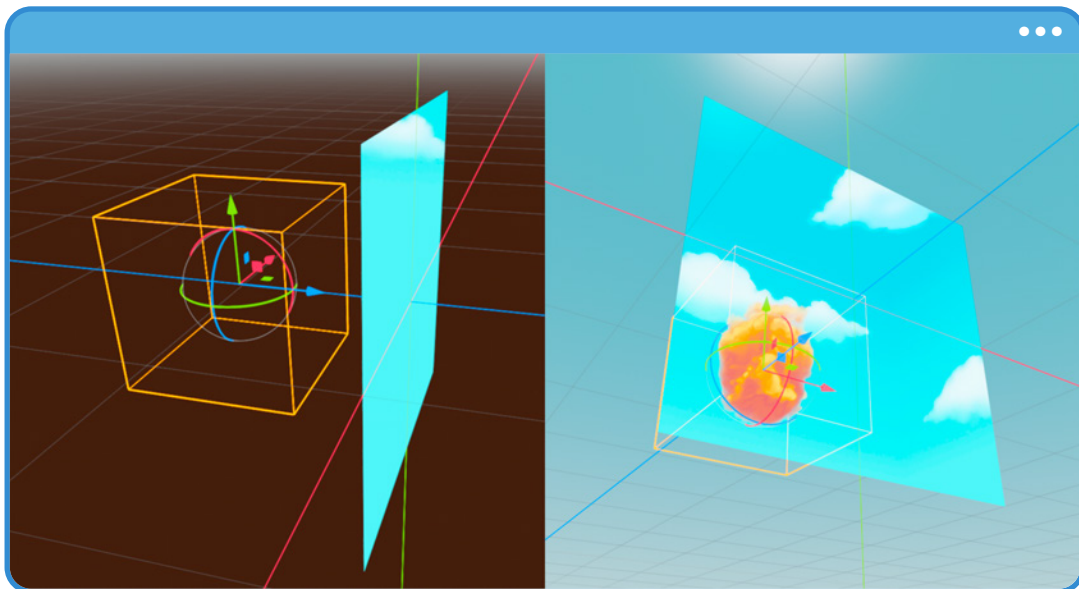
Para corregir este comportamiento, podemos aplicar el mismo principio que utilizamos anteriormente con el objeto **sky_mesh**. En este caso, simplemente incluiremos la configuración de Stencil directamente en el shader **vfx_ray_marching**, de modo que la nube también quede restringida al área definida por el portal.

Para ello, agregaremos las siguientes líneas al shader:

```
1 shader_type spatial;
2 // add stencil if needed
3 render_mode unshaded, depth_draw_never, depth_test_disabled;
4 stencil_mode read, compare_equal, 1;
```

Como podemos observar, estamos realizando la misma comparación de Stencil que aplicamos previamente al material **stencil_read**. Esto implica que los fragmentos generados por el shader de ray marching solo serán aceptados si el valor almacenado en el Stencil Buffer es igual a 1.

En consecuencia, la nube se renderizará exclusivamente dentro del área delimitada por **portal_mesh**, resolviendo tanto el problema de orden de renderizado como la ambigüedad espacial respecto al interior y exterior del portal.



(4.5.j La nube se renderiza dentro del **portal_mesh**)

Hasta este punto, hemos trabajado únicamente con un tipo de comparación del Stencil, específicamente **compare_equal**. Sin embargo, el Stencil Buffer admite múltiples modos de comparación que nos permiten controlar con mayor precisión qué fragmentos son aceptados o rechazados durante el proceso de renderizado. Entre los más comunes encontramos:

- > **compare_equal**,
- > **compare_always**,
- > **compare_not_equal**,
- > **compare_greater**,
- > **compare_greater_or_equal**,
- > **compare_less**,
- > **compare_less_or_equal**.

Cuando hablamos de comparaciones, nos referimos a una evaluación lógica que se realiza por fragmento, donde el valor almacenado en el Stencil Buffer se compara con un valor de referencia definido en el shader. Conceptualmente, este proceso puede representarse de la siguiente manera:

```
if ((StencilReference & StencilReadMask) [compare] (StencilBufferValue &
    StencilReadMask))
{
    // accept fragment
}
else
{
    // reject fragment
}
```

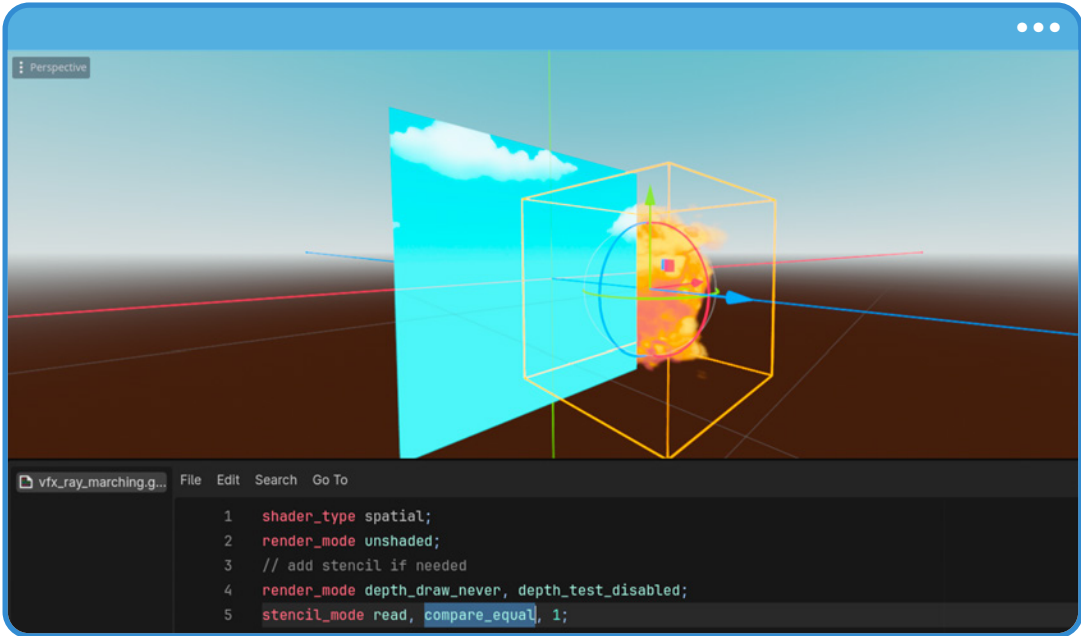
Donde:

- **StencilReference** se refiere al valor que queremos comprobar durante la comparación.
- **StencilReadMask** permite limitar qué bits del valor de Stencil participan en la comparación
- **[compare]** representa el operador de comparación seleccionado, es decir, **compare_equal** en este caso.
- **StencilBufferValue** corresponde al valor que ya existe en el Buffer, escrito previamente por otro objeto, en nuestro ejercicio es 1.

Nota

El funcionamiento del Stencil Test en Godot sigue el mismo modelo que el descrito en LearnOpenGL para APIs gráficas modernas: se compara un valor de referencia con el valor almacenado en el Stencil Buffer mediante una función de comparación y una máscara de bits, aceptando o rechazando el fragmento según el resultado de esa evaluación. Puedes profundizar más en el siguiente enlace: <https://learnopengl.com/Advanced-OpenGL/Stencil-testing>

Por ejemplo, si regresamos al shader **vfx_ray_marching** y cambiamos su tipo de comparación a **compare_not_equal**, la nube se renderizará únicamente cuando se encuentre fuera del área definida por **portal_mesh**.



(4.5.k Se ha modificado temporalmente el tipo de comparación)

Otra manera de rechazar fragmentos consiste en utilizar la instrucción **discard** dentro del fragment shader. A diferencia del Stencil Test, que rechaza fragmentos mediante una comparación con valores del Stencil Buffer, **discard** permite descartar fragmentos de forma explícita en función de cualquier condición que definamos (por ejemplo, el valor de una textura).

Cuando trabajamos con Stencil Buffer, debemos tener presente que los materiales que dependen de transparencias (por ejemplo, aquellos que escriben **ALPHA**) pueden cambiar de render pass y alterar el orden de renderizado. Por esta razón, si nuestro objetivo es crear una máscara con recortes controlados, resulta más estable basarnos en **discard** (o en una máscara binaria) que depender únicamente del canal alfa en el shader que escribe Stencil.

Para demostrarlo, implementaremos una textura en el shader **stencil_write**, tal como se muestra a continuación:


```

1  shader_type spatial;
2  render_mode unshaded;
3  stencil_mode write, 1;
4
5  uniform sampler2D _BaseTex : source_color;
6
7  void fragment()
8  {
9      vec4 base_tex = texture(_BaseTex, UV);
10 }

```

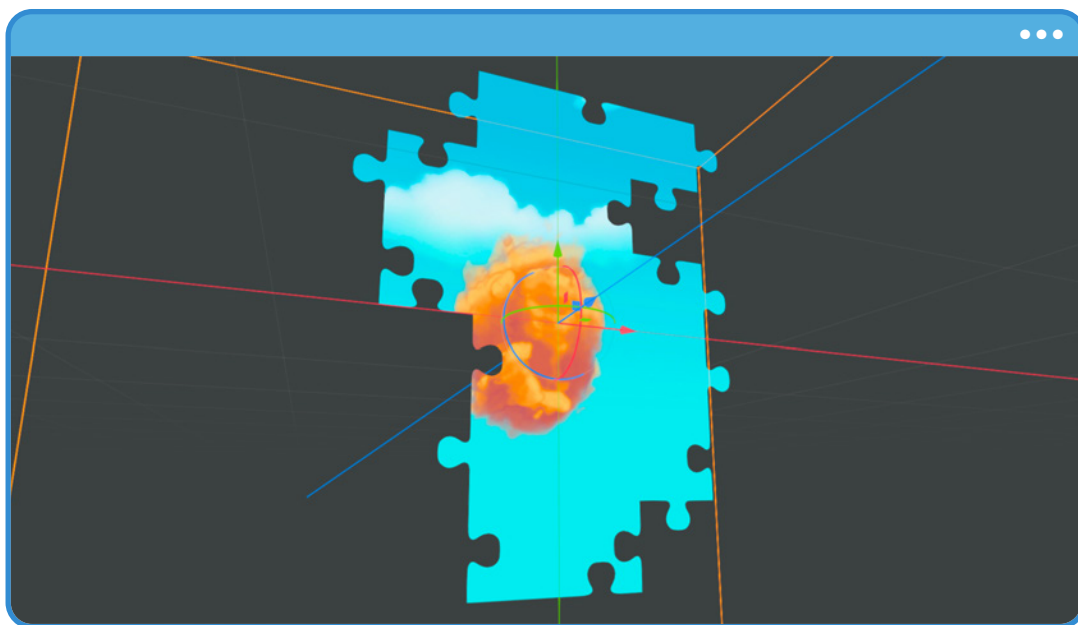
Como podemos observar, hemos incorporado una textura denominada **_BaseTex**. Si utilizamos una textura en escala de grises a modo de máscara (blanco y negro), podemos emplear el canal R de **base_tex** para descartar fragmentos según el valor del texel. Por ejemplo, si queremos conservar únicamente los texels completamente blancos, podemos descartar todo lo demás de la siguiente manera:

```

void fragment()
{
    vec4 base_tex = texture(_BaseTex, UV);
    if(base_tex.r < 1.0) discard;
}

```

Finalmente, si regresamos al material **stencil_write** y configuramos la textura **portal_door_tex** en la propiedad **Base Tex**, obtendremos el siguiente resultado visual:



(4.5.I Se han descartado algunos fragmentos del **portal_mesh**)

Resumen del capítulo.

En este capítulo se consolidaron los conocimientos necesarios para desarrollar VFX avanzados y efectos de post-processing, comprendiendo cómo los shaders pueden utilizarse para modificar la imagen final renderizada y crear efectos visuales complejos a nivel de pantalla y de escena.

Se aprendió a implementar post-procesado mediante la creación de un efecto estilizado inspirado en la estética Game Boy, lo que permitió entender cómo manipular el color, la resolución y el muestreo de la imagen final. A partir de este enfoque, se adquirió experiencia en portar efectos desde Shadertoy, adaptando shaders externos al flujo de trabajo y a las convenciones del motor.

El capítulo también abordó el uso de transparencia basada en distancia, comprendiendo cómo controlar la visibilidad de los fragmentos en función de su relación espacial con la cámara u otros elementos de la escena. Posteriormente, se introdujo el Ray Marching como técnica de renderizado volumétrico, aplicándolo a la implementación de nubes mediante texturas 3D, lo que permitió explorar el uso de volúmenes y muestreo tridimensional dentro del shader.

Finalmente, se adquirieron los fundamentos del Stencil Buffer, entendiendo su funcionamiento dentro del pipeline de renderizado y su utilidad para el control preciso del renderizado por fragmentos. En este contexto, se implementaron dos shaders independientes, **stencil_read** y **stencil_write**, estableciendo una base sólida para la creación de efectos como máscaras, portales y composiciones avanzadas en tiempo real.

Glosario.

.gdshader: Extensión de archivo utilizada en Godot para escribir programas de sombreado.

Albedo: El color base de la superficie de un objeto, utilizado para determinar su apariencia principal.

Anisotropía (o Reflexión Anisotrópica): Modelo de reflexión donde el brillo se deforma de manera direccional, útil para materiales como cabello o metales cepillados.

Ashikhmin-Shirley: Modelo avanzado de especularidad anisotrópica para simular superficies con propiedades direccionales.

Billboard: Efecto visual donde un objeto (generalmente un *QuadMesh*) siempre apunta hacia la cámara.

Blending: Proceso de mezcla de fragmentos para manejar transparencias y efectos de post-procesado.

Blinn-Phong: Modelo de iluminación que añade reflejos especulares utilizando un vector intermedio llamado *halfway vector*.

Built-ins: Variables y matrices internas proporcionadas automáticamente por el motor Godot para facilitar el desarrollo de shaders.

Clipping: Etapa del pipeline de renderizado donde se descartan los fragmentos que quedan fuera del área de visualización.

Cuaterniones (Quaternions): Estructura matemática utilizada para representar rotaciones espaciales evitando problemas como el bloqueo de ejes (*Gimbal Lock*).

Depth Buffer (o Z-Buffer): Almacén de información de profundidad que permite determinar qué objetos están delante de otros.

Depth Testing: Mecanismo que compara la profundidad de cada fragmento para decidir su visibilidad en la escena.

Dot Product (o Producto Punto): Operación matemática que determina la relación angular entre dos vectores; fundamental para el cálculo de iluminación.

Espacio de Objeto (o Local Space): Espacio local del modelo donde los vértices están definidos en relación con su propio pivote.

Espacio de Pantalla (Screen Space): Espacio de coordenadas donde el objeto es finalmente rasterizado y asignado a píxeles en la pantalla.

Espacio de Tangente (Tangent Space): Sistema de coordenadas local a la superficie de un modelo, esencial para la aplicación de mapas de normales.

Espacio de Vista (View Space): Espacio de coordenadas cuya referencia absoluta es la cámara misma.

Espacio Global (World Space): Sistema de referencia absoluto que define la posición de los objetos dentro de la escena de Godot.

Espacio UV: Coordenadas normalizadas utilizadas para mapear texturas bidimensionales sobre la geometría de un objeto.

Forward+: Renderizador de Godot que utiliza un enfoque clusterizado para optimizar el cálculo de múltiples luces dinámicas.

Fragment Shader (etapa de fragmento): Fase final del renderizado encargada de procesar el color de cada píxel visible.

GPU (Graphics Processing Unit): Procesador especializado en el cálculo paralelo masivo donde se ejecutan los shaders.

Input Assembler: Componente del pipeline encargado de organizar los datos geométricos (vértices, normales, UVs) para ser procesados.

Interpolación Lineal (LERP / Mix): Función matemática que permite mezclar suavemente dos valores basándose en un factor de influencia.

Lambertiano (modelo): Modelo de reflexión difusa que calcula la iluminación basándose en el ángulo entre la normal y la dirección de la luz.

Light Shader (método light): Etapa opcional del shader que se ejecuta por cada fuente de luz que afecta a un píxel.

Mesh: Recurso de datos que contiene la geometría basada en vértices, dividida en superficies.

Matrices: Estructuras numéricas organizadas en filas y columnas utilizadas para transformaciones espaciales como traslación, rotación y escalado.

Normal: Vector unitario que indica la dirección profesional perpendicular a una superficie, esencial para la iluminación.

Normal Map (Mapa de normales): Textura que simula relieves detallados en una superficie sin aumentar la carga poligonal.

PBR (Physically Based Rendering): Modelo de renderizado que busca simular el comportamiento de la luz en el mundo real de manera físicamente precisa.

Post-procesado (Post-processing): Efectos visuales aplicados sobre la imagen final de la pantalla, como filtros de color o pixelado.

Ray Marching: Técnica volumétrica que permite renderizar figuras o efectos (como nubes) recorriendo el espacio paso a paso a lo largo de un rayo.

Render Mode: Directivas globales que definen comportamientos del shader, como *unshaded* o `skip_vertex_transform`.

Rendering Pipeline: Proceso en cadena donde los datos geométricos se transforman paso a paso hasta convertirse en objetos visibles en pantalla.

Rim Effect (o Efecto Rim/Fresnel): Efecto que genera una silueta o halo luminoso en los bordes de un objeto dependiendo del ángulo de visión.

SDF (Signed Distance Fields): Técnica matemática que define la distancia más corta desde un punto a la superficie de una forma, fundamental en *Ray Marching*.

Shader: Programa especializado que se ejecuta en la GPU para controlar el renderizado de gráficos.

ShaderInclude: Archivos con extensión .gdshaderinc que permiten centralizar y reutilizar funciones en múltiples shaders.

Smoothstep: Función que genera una transición suave entre dos límites definidos, ideal para estilos artísticos o *toon*.

Spatial: Tipo de shader de Godot diseñado específicamente para trabajar con objetos y entornos tridimensionales.

sRGB: Estándar de espacio de color diseñado para representar y reproducir colores de forma coherente en pantallas y dispositivos.

Stencil Buffer: Herramienta utilizada para controlar qué partes de la escena se dibujan o descartan mediante el uso de máscaras.

Tangente: Vector que, junto con la bitangente, ayuda a definir la orientación local en cada vértice de la superficie.

Textura: Conjunto de datos bidimensionales (píxeles) organizados para ser aplicados como imágenes sobre la superficie de un objeto.

UV Coordinates (Coordenadas UV): Propiedades del mesh que permiten el mapeo preciso de texturas sobre su geometría.

Vertex (Vértice): Punto fundamental en el espacio que, junto con otros, define la forma geométrica de un mesh.

Vertex Shader (etapa de vértices): Fase del shader responsable de la manipulación y transformación de los vértices antes del rasterizado.

Agradecimientos.



Austin Hackett | Hang Yeh | Sergiox | Stephen Rice | Herman Garling Coll | Matthew
 | Ninanico | Neil Palmere | Shiena Jp | Bohdan "teo" Popov | Numa Schilling | Tm
 | Grzegorz Drabikowski | Sean Laplante | Max | Carlos Eduardo Boaro | Sergey
 Yaroshuk | Vojtěch Lacina | João Marinho | Jacek Adamus | Francis Jasmin |
 Antoine Luciani | Sander Vanhove | Tahldon | Raveen Rajadorai | Richard Cobbett
 | Adrian Jenkins | Vlad Scp | Hilton Perantunes | Pokemonbrycedixon | John Mandish | Alex |
 Daniel Stoica | Ricardo Pitta | Patientlion | Claudio Cth | Swagman | Christian Koch | Francisco
 Pereira Alvarado | Kyleharrison | Xxradox | Algio | Daniel Castellanos | Carlos Montes | Thomas
 Detoy | Combat Lobster | Adrian Rivero | Ewald Schulte | Punk Flag | Miri N | Gripfastgameworks
 | Laurenz | Gabriel Malinowski | Cameron Fernandez | Jes | Adamnakonieczny | James |
 Ethan Lucasawesome | Shahid Akhter | Benjamin Russell | John Pitchers | Jonne | Yun Wang
 | Mazhiwei | Timurariman | M Gewiss | Chunck Trafagander | Dmytro Hryppa | Patrick Joos
 | Joseph Torbett | Jason Peterson | Mako | Norbert Haacks | Mika | Sean McIntyre | Collin
 Myhre | Dottorreed | Morpheus Tsai | Aiastesla | Fabian Ruch | Jan Nothacker | Rémy Tauziac |
 Martijn Van Halderen | Yon Montoto | Walpurgis Time | Keating | Andrey Valkov | Thsage | Eric
 Wong | Nils Nordmark | Nikita Blizniuk | Joseph Deluca | Francesco Michellini | Christopher Hall
 | Jaycoleman | Madlion | Graham Reid | D-hill | Marcanthonycacho | Thomas



Jahtani | Tskarhik | Mutrz | Vasil Genov | Manraaj Nijjar | Kovidomi | Robles | Jesse
 Virtanen | Domingo Mesa-maliniak | Frederic Batardy | Matoo | Jiří "venty" Michel
 | Goldennick | Ginton | 박용민 | Hi | María Luisa Carrión | Andrew Meyer | M | Trent L |
 Mark H | Crapule | Dorian Nicou | Sam | Suchaaver Chahal | Victor | Andrew Herch
 | Boti Beto | Raymond Gibbons | Phil | Thomas Hunt | Constantin Balan | Brandon
 Grenier | Abyssallure | Katia Mccarthy | Joakim Karlsson | Dmitrii | Markus Michel | Robert Allen
 | Ross | Mr Thomas Graveline | Stephen James | Tiago Ferreira | Anderson Borba | Andreas
 Gillberg | Morgan Wesemann | Janusz Tarasewicz | Ben Thedragon | ガンズターン公式 | Saul |
 Flyn | Hellno | Karthik Nallan | Rosciuc Bogdan | Big Quail | Matt | Justo Delgado Baudí | Roose
 Alexandre | Mangirdas Balciunas | Jettelly | Asylbekz | Roman Koziolok | Alvaro Larre Borges |
 Creta Park | V | Nikola Stanojević | This Is Benny | Santiago | Alvaro G Lorenzo | Zachnorthcott
 | Orion Ironheart | Brandon Farrell | Nicolas | Douglas Ávila | Mª Dolores Lebrecht | Bernard
 Cloutier | Isaac Iverson | Matheus V | Christophe Brenon | Dave | Vtalyh | Jacopo Galati | K G H
 | Sam Van Berlo | Joshua | Šarūnas Ramonas | Tamego | Emerson Santos | Ashely | Zoe Firi |
 Liyizhanguk | Aleksander G Solheim | Ian | Michael Belknap | Mattias | Fu Ruru | Karim Matrah |
 Zhou Kaiwen | Quentin Delvallet | Mmdrewind



Emmanuel | Someone | Maxie | Patrick Exner | Ivan Ho | Jesse Fletcher | Chris Morgan | Carlos Eduardo Pérez Villanueva | William Brendaw | Swampnrd | Shader Snake | Sơn Nguyễn Minh | Gerwyn Jones | Seth Foznot | Harry Cassell | Deear Art | Cillian Clifford | Sabre | M Fatati | Ardscott | Guilherme Cattani | Albert Miro Montalban | Georges | Kev Zettler | Carsten Bloecher | Clément Vayer | Jose Ignacio Palacios Ortega | Mcspidey | François De La Taste | Paul | Ross Solomon | Gabo Salcedo | Caigan | Brad Svercl | Jing Yi Chong | Mykola Morozov | Kalsarin | Zewy | Phoen Leo | Sebastiankmilo | Hipton | Bawlsy | Tim Arlow | Ffdd | Maksim Loboda | Colin | Lisandro Lorea | Björn Wilke | Robert Mahon | Dave Lecompte | Pawel Antoniuk | Carlos Aguirre Vozmediano | Elliott Chillag | Kyle Hessel | Jose Angel Canabal Delgado | Benjamin Oesterle | Ngo Viet Luan | Meleg Zoltán | Michael Jones | Kevin Krause | Daniel Cavazos | Travis Womack | Ryan Hinds | Dewey Mowris | No Yes | Nigel Che | Bass | Juanye | Britt Selvitelle | Tor | Noah | Stuart Carnie | Kamwade | Smedstad | Anton | Js | Mads Sønderstrup | Steffenschmidt | Tor-arne Nordmo | Алексей Акулов | Bioplant | Lucas Felix | Natevc | Nathan | Kangweon Jeong | Mei Yamasaki | Meownoi A | Nathanrvaughn | Monica | Lighthalzenxii | Nike | Mapinher | Xavier Shaver | Jack Lueyo | James Clancey | Thomas Guyamier | Alfred Reinold Baudisch | Max | Default | Ben Mckenna



Ondřej Kadlec | Tate Sliwa | Dt | Marino | Calvin M Evans | Alen Lapidis | Thoughton | Mariano | Daquan Johnson | Eli Greenwald | Adam Gulacsi | Nea Pocem | Liming Liu | Michael Birtwistle | Tobias Brandner | Richard Pearson | Dimitri Bauer | Paulo Poiati | Edwin | Chauncey Hoover | João Luís Reis | Amy Haber | Corentin Lallier | Alvinanda Purnomo | Kerry K | Ana Greengrass | Juno Nupta | Israel | Aaron B | Kris | Gabe Volpe | Keita Fukuchi | Clémentine Ducournau | Gustavo León | Haydenvanearden | Jdon Leiningner | Bohdan | Develop | Caitlin Cooke | Alexandre Schenk-lessard | Gräodopão | Terencep Dixon | Anatol Bogun | Zammecas | Benji Lising | Kevin Abellan | Ryan T | William | John Daffue | Simone Sorrentino | Craig Spivack | Pavol Gocik | Clpereira | Alejandro Vera | Yb Park | Pyrat | Acorzomate | Oniric View SI | Tulasi Rao | Patricioland | Xanroticon | Nicolas Delbaer | Jordan | Thom | Findemor | Amasten Ameziani | Patrick | Kacey Walsh | Nathakorn Thongtrairat | Christian Snodgrass | Emmanuel Bouis | Pokkefe | Mike Klingbiel | Drew Herbert | Ross Rothenstine | Rhenn | Talal Almardoud | Vinicius Hoyer | Matti Pohjanvirta | Tuomas Ilomäki | Elias Manzaneda Santamaria | Daniel Canham | Liam Cornbill | Daniël Michael Kracht | Eliud De Leon | Sean Welgemoed | Pangake | Jules Fijolek | Krystof Klestil | Luc Magitem | Thijs Tak | Adam Spivey | David Kol | Glenn Barker | Adrián | Gianluca Pumoni | Mal | João Récio | Jacob Lerche | Daniel R Martinez



John Pimentel | Kyle Young | Lysandre Marti | Matthew Makary | Craig Kerwin |
 Sam Winfield | Alec White | Corentin Lecroq | Richard | Alex | Taylor Segura Vindas
 | Erik | Ashley Moon | Simke Nys | Spencer Hoban | Nichas Brennan | Efrian Vega
 Jr | Sam | Gato | Thomas D | April Clark | Jakub | Colin | Lockyer Road | William
 Hoobler | Kenneth Holston | Brian | Carter Layton | Martin Voth | Brett Buckner | Joba López |
 Llynx | Panupat Chongstitwattana | Bryan English | William N Kentris | Penny!! | Minoqi | Anton
 Petersson | Aaron J Schneider | Nate Moore | Colin Miller | Edgar | Sam Blackman | Frank
 Gusche | Gillian Monte | Eric Alvarez | Viktor Sladek | Ian | Paul Cloete | Ben | Ppsi | William
 Beatty | Photonphighter | Arman Frasier | Farhad Yusufi | Lee | Andrew Lee | Codebycandle |
 Łukasz K | Alfie | Jeff Zimmer | Game Log | Aaron Imrie | James Karavakis | Dylan Glow | Miles
 Harris | Kris | Reavenheart | Benjamin Schmitt | Frodo | Tom Kertels | Connor R | Nathan | Gary
 Johnson | Kein Hilario | Blake Rain | Bryant Flores | Joseph Sugabo | Eyal Assaf | Lewis Simpson
 | Christopher Holt | Bluen | Jayden Jacksteit | Alan Jimenez | Noah Blake | Icoza | Elizabeth |
 Alexandra Bradford | Agustin Adducci | Tim | Xuxing | Marcial Santallory | Austin Grimm | Rob
 Brown | Odin Ugelstad | Henry Schwenk | James G Pearson | Marco | Danthecardboardman |
 Aaron Wu



Gerben Van Der Heijden | Nil Vilas | King Artorias Arthur | Marcos | Andrew Moffat |
 Jun Yeo | Matthew Swee Chuan Tan | Bruce Leidl | Mailjib Jb | Ryan Shetley | J Chris
 Druce | Mārtiņš Prelgauskis | Umut Ulutas | Niki | Hyou Sasaki | William Sattanuparp
 | Darrin Henein | Sean Pace | David Betancourt | Gabriel Martinez | Miroslav Edler
 | Angelina | Zane | James Mcvitty | Eric Roberts | James Jones | Robert Georges |
 Terry Wu | Eldaville | Coryo Moore | Joshua Tucker | William Kruspe | Dizzy Moth | Peter Chantler
 | Nural | Gustavo Lopes Domaradzki | Quock | Alec Gall | Steven Lenaerts | Mateus S Pereira
 | Alex | Andrew Brizendine | Daniel Holmen | Garrett Bare | Kevin Crowell | Spaghetmenot |
 Michael Simon | Yuval Dekel | Alan K | Aidan Davey | King Artorias Arthur | Dominic Perno |
 Anthony Beninati | Sammi Tu | James Steele | Pepijn Stuurwold Van Walsum | Holger Schmidt
 | Oliver Gutiérrez | Nova Obrien | Fredrik Hamstad | Jonathan | Thomas M Kidder | Kai | Gk
 | Ben Revai | Daniel Leiva | Miles Welsh | Cannon Pharaoh McClendon | Emanuel Lamela |
 Jānis Alunāns | Michael Radke | Ryan Woods | Torgabor | Reid Hannaford | Mike Barbee |
 Aria Ramshaw | Benjamin Paschke | Mathias | Charles Engel | Benjamin Flanagan | Jettelly
 Publishing | Antonio Colon | Sac Alanpereira | Patrick Grant | Marquis Jackson | Miguel Angel
 Mendez Cruz | Scottie Doria | Ordinary Fox | Daniel Ridao | Jorn Van Denbussche | Shelly Barnes
 | Celia Tran | Rem | George | Aleksander | Dimitry Maizik | Dave | RYANIMATES | Annabella Rosen
 | Colton Wolkins



Daniel Rodriguez Aires | Matteo Kramer-tamburrino | Albin Lundahl | Luis Puentes
 | Andrew Cotter | Dylan Goncalves Martins | Arroyojavier Sanandres | Xury Greer |
 Jjoltton | Adam | Daniel Carswell | Deear Art | Shawn Dhawan | Daniel Gerep | Steve
 Versace | David Dam | Bdilmore | Dima | Josuprasi | Yaramirez | Michael Frederic
 | Ruslan Horiuchkin | Nicholas Vosburgh | Chen | Mateusz | Flapp | Komatsuki | Auro | The
 Zefan | Michail Maridakis | Sebastian Rangger | Henry Liu | Brandonrconyers | Andreas Becker
 | André B Luz | Logan Croley | Daniel Mutis | Ulric Boel | Vitalik | Tom Brewer | Darmaz | Keagen
 Bouska | Chance Mcdonald | Bidda | Pedro Perez | Antonio Contreras Arzate | Dan Crowe |
 Jeong Hoon Shin | Benjamin F Hajas | Javis Jones | Kateryna | Seongho-son | Alessandro Pilati
 | Brian Huqueriza | Hugo Toti | Ronald Casili | Sdc | Joe | Jordan Faas-bush | Eli Makaiwi | Henry
 Audubon | Zoey Lome | Jaakko Ojalehto | Matt Odette | James Asumendi | Trevor Harron |
 Alexander Abshagen | Maxim Oblogin | Tresceneti | Asher | Aaron | David Tag | Jake Elliott |
 Tim | Tobias Cunningham | Ryan | Robert Fraser | Kristian Gibson | Konstantin Kudinov | Lizzy
 Moyes | Charlotte Jones | Sethcrawford | Carlosgolfer | Ryan Rennan | Roman | Jean-francois
 Segretain | Joao Filho | Flowulf | Ben Boyter | Pearshaped | Jacob Yero | Felipe Fausto | Joaquin
 Muñiz | Mel Irizarry | George Thompson | Hakan | Gerard Parareda | Roger Bujan | Eric | Jilly



Jubimage | Luis Alfredo Figueroa Bracamontes | Chris | Alessandro Talloru | Adam
 Gibson | Revo Wu | Aziz Abidi | Lennysioll | Maxim Jourenko | Tobias Holewa | Rxwp |
 Darren Larose | Sebastian Karlsen | Pendant Fills | Damian Stähr | Niklas | Xuwaters
 | Christian Ahlers | James Coleman | Ryan Moos | Tomáš Adámek | Adrianguazon
 | Cagri Benli | Darren | Guy | Mateusz Wolos | Kristofer | Jonathanrivasmencia
 | Joelfreemant | Eric Persson | Saul | Lxyhaqs | Ericpdss | Tấn Phát Huỳnh | Guma | János
 Harsányi | Mr Connor S Adams | Florian Stumpf | Wyatt | Adam Helešić | Jakob Sorenson | Nick
 | Hannah Petherick | Erik Loide | Bradley | Artbarte | Christophe Delahaye | Cheng Bowen |
 Jeremy Kamrath | Amanda Koh | Philip Ewert | Michael Heywood | Tralexium | Maxfield Hewitt | J
 G | Chong Ren Li | Max Gratzl | Aaron | Ronald Gibson | Niall Brady | Sam Morgan | David Stewart
 | Marcel Gentner | Martin Prazak | Josh Bousfield | Pietro Maggi | Adrian Hernik | Sebastian |
 Jonathan Scheer | Alexander Velchev | Kai-fabian Schönauer | Mattia Belletti | Izaac Jordan |
 Kyle Donaldson | Daniel Maiorano | Jesse Schramm | Jonas Lund | Peter Massarello | David
 Carr | David Dunham | Mario Schilling | Jw | Danni | Antonio Sobrinho | Eric | Kaptain Radish | Z
 | Andrew Robyn | Andicus | Jacob | Alek D Flener-satre | Joseph Turnquist | Patrick D Rupp |
 Davide | Bandit | Matt | Wataru Ikeda | Andi Doty | Ryan | Joshua Rehling



Nathan Van Fleet | James Walter | Chris Mcpherson | Pixel Pilgrim Studios | Marcell
 Kovacs | Roro | Dominykas Djačenko | Alex Robbins | Ng Game Junk | Arturs Cernuha

| Iramis Valentin | Niko Van Wonterghem | Michael Andrew Revit | Garrett Drinkard
 | Andrew Pachuillo | Drak | Kerry Shawcross | Vincent Michael Mooney | Brandon
 Wolff | John Laschober | Trevan Haskell | Andrew Greenwood | Dani Fey | Alec
 Burmeister | Kieran Lane | Seonhee Lim | Javier Teodoro De Molina Mejías | Keith Crabtree
 | Damu | Raffaele Picca | Anthony | Luis Ernesto Torres Santibañez | Adam Webber | Daniel
 | Mfernandez | Ryan | Miranda Schweitzer | Taylor Bray | Mark Hubczenko | Fathony Teguh
 Irawan | Renju Mathew | Alexander Cruz | Tyler Smith | Brad | Austin Baldwin | Jacobarkadie
 | Kevin Bergstrom | Alex | Daniel | Matthew Xiong | Quinn Collins | Abbey Howard | Michael
 Hrabánek | Vegard | Harrison Grant | Adrien Baud | Sa Sa | Raf Van Baelen | Santitham |
 Marianne Stalter | Terrence Coy | Jodie | Nicholas Jainschigg | Aerton | Bee | Trevor Starick |
 Michael | Matej Šulek | River Vondi | Jacob | Sam Shockey | Florencia Sanchez | Ezequiel Selva |
 Cody Keller | David Yaeger | Pewbop | Gabriel Benjamin Valdez De León | Ask Skivdal | Gautam
 Dey | Vinicius Nakamura | Semyon | Bas | Felix Bytow | Adam Reid | Clement | Kirill | Bawm
 | Hai Bo Wang | Anthony Leblanc | Duncan | Étienne Guerette | Wesley “Sky” Nworisa | Luca
 Vazzano | R Wijdenes | Tyler Hamilton | Ben Witte | Morchaint | Elliott | Alex Wang | Ethan



Jochen Weidner | George Dimopoulos | Christopher Margroff | Milan Koyš | Alexan-
 dria Lee | Sterling | Samir Alajmovic | David Devcic | Shane Gadsby | Murilo Gama |
 Max Daniels | Dylmm | James Minshull | Cory Kennedy-darby | Jeff T Klouda | Amit
 Feldman | Florian Frankengerger | Stefan Maier | Barbara Zacharewicz | Pablo
 Mansanet | Francis D Cleary | Joseph Krueger | Morgan Hezon | Benjamin Reber |
 Jonathan Tippet | Danny Eisenga | Michael Laplante | Matthew Arabo | Ryan Reed | Ludger
 Meyer-wuelfing | Raji | Thefre | Amani A | Iván D | Adrian | Max Olin | Ruben Rodriguez Torres |
 Eidel Odiseo Giménez | Kay | Daniel Steven Sarmiento Santos | Kevin Do | Anthony Tarr | Adam
 Slavik | Alberto Flor | Jared Moore | Hendrik Poettker | Eric Lugn | Zack | Warp Vessel | Tommi
 Shelton | Earl Braxton Mckenna | Adrien Farfals | Jonathan Cater | Luke | Matteo Stefanini |
 Oschijs | Keo Daniel Bun | Kevin Young | Hardtrip | Luc-frederic Langis | Pj Palomaki | Laurent
 Tourte | Geowarin | Barmy | Khairi Harris | Nick Strasky | Brent Lovatt | Alexander Graham |
 M Koray Duman | Houman Jafarnia | Cameron Oehler | Cesar | Justin Doornbos | Tj Morse
 | Jeavh | Street Nw | Pete | Ross | Sophie Freeman | Gerrit Schimpf | Brody Wilton | Deniz |
 Matthew Laurenson | Nicholas Venditti | Lucas Long | Jardian Halliday | Ahmad Takhimi Bin
 Ahmad Mahayuddin | Joe Bechtel | Delbert Vick | Stefan Larsen | Jollymeatball | Kelita Nolan |
 Lily Jin | Terence Steabner | Daniel Hall | Elia Théo | Matthew Brennan | Razvan | Ditherdream |
 Sjoerd Van Kampen



**Jettelly te desea éxito
en tu carrera profesional.**